

Contents

Lab 2: Getting Started with CubeIDE & STM32 GPIO Peripheral Programming.....	1
2a. LED Toggle	1
2b. User Button	6
Lab 3 : Configuration of UART in polling method to send data from AIoT Board to PC using printf statement	11
Lab 4 : IoT Sensor Initialization: Development of simple programs to interface following sensors with the AIoT board	19
4a. MPU6050 Sensor	19
4b. Audio Sensor(adc with polling)	30
4c. BH1750 Light Intensity Sensor.....	36
4d. HTU21D Temperature & Humidity Sensor	46
4e. Air Quality Sensor	59
Lab 5: Wi-Fi Initialization for IoT Projects.....	67
Lab 6: Sending a Single String Message via MQTT to the Cloud.....	80
Lab 7: Local IoT Monitoring and Dashboard Creation using Node-RED for Temperature and Humidity Data	89
Lab 9 a) : Design & implement a data logger to send audio sensor data from STM32 to NanoEdge AI Studio on PC.	98
Lab 9 b) Design of an ML based audio classification to identify & classify different sounds & deploy the classifier model on the AIoT board.....	108
Lab 9 c) Audio Scene Classification: Deployment of an ML based audio classification to identify & classify different sounds into microcontroller	117
Lab 10: IoT Integration for Real-Time Audio Classification and Cloud Data Transfer	132
Lab 11 a) Design & Development of MPU6050 sensor datalogger for vibration data.....	143
Lab 11 b) Anomaly Detection: Use NanoEdge AI to find out the best anomaly detection model for a given sensor dataset (vibrational analysis)	153
Lab 11c) :Anomaly Detection: Deployment of an ML based Anomaly Detection model into microcontroller using MPU 6050 Sensor	166
Lab 13: Local IoT Monitoring and Dashboard Creation using Node-RED for Real-Time Anomaly Detection.....	180

Lab 2: Getting Started with CubeIDE & STM32 GPIO Peripheral Programming.

2a. LED Toggle

Lab Overview:

The main objective of this experiment is to understand the basics of GPIOs, how a pin can be configured as an output or an input pin etc. By understanding these, we will configure the PB14 pin of the microcontroller as a GPIO Output pin, because the LD2, which is in-built on the microcontroller, is connected to this pin. Then with the help of code we will change the state of the LED.

Objective of the Lab:

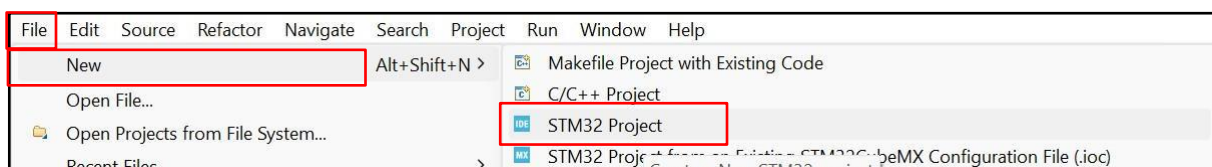
The objective of this experiment is to understand the basics of GPIOs and configure the PB14 pin of the microcontroller as an output to control the state of the built-in LD2 LED.

Experimental Setup:

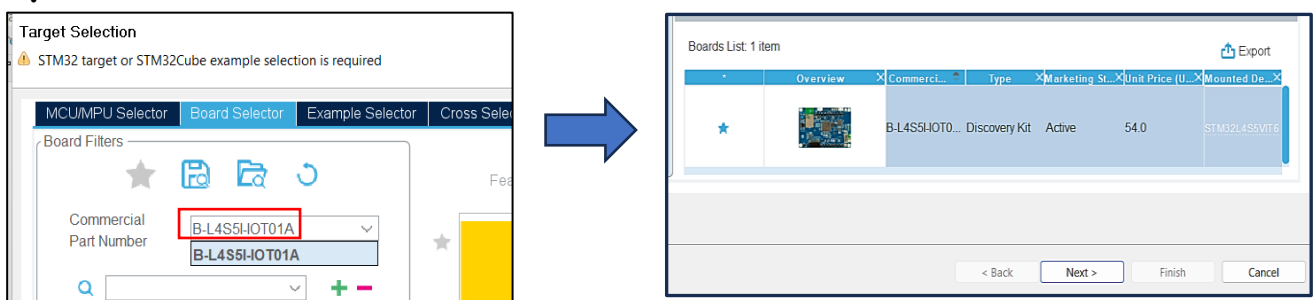
1. STM32 Cube IDE software.
2. STM32 Microcontroller.
3. USB Cable for the microcontroller.
4. PC or Laptop.

Steps Involved:

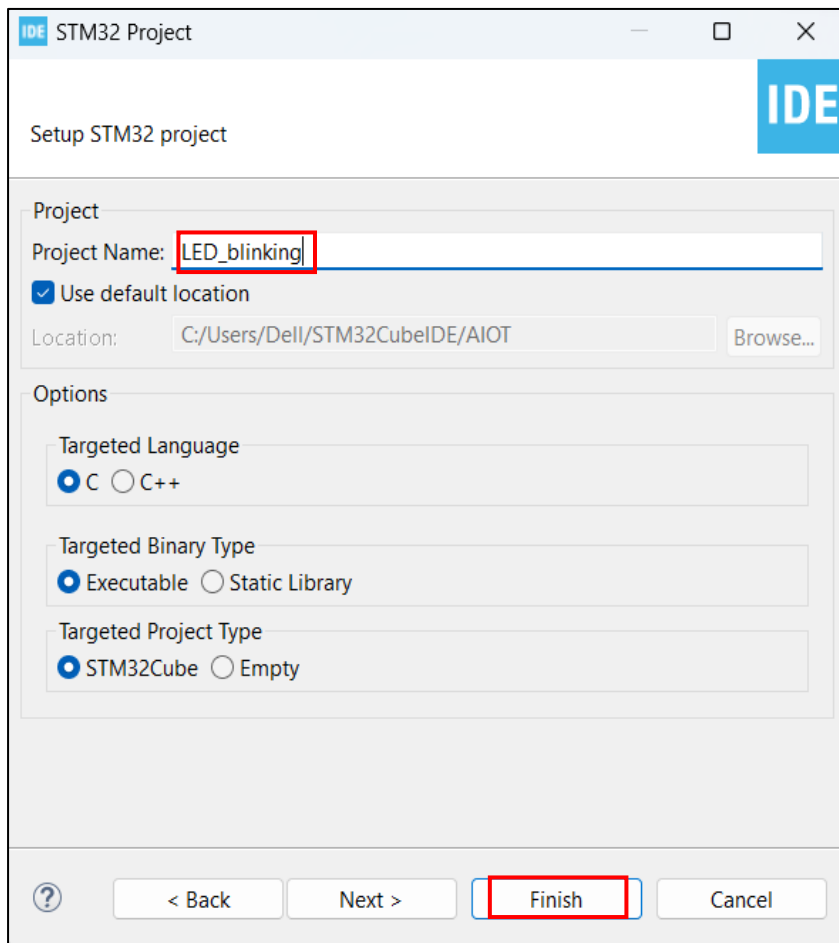
1. Click on **File**→**New**→**STM32 Project** to start your project on Cube IDE.



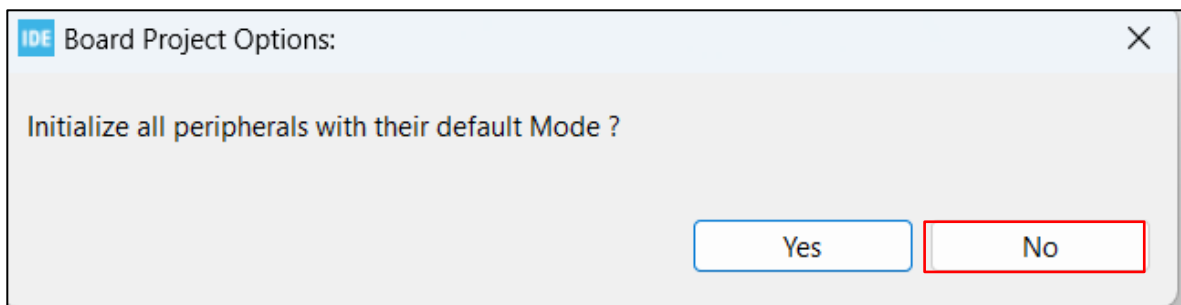
2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with. After this on the right-hand side of the window, under **Board List** you will see the board you have selected. Click on the board and then click on to **Next**.



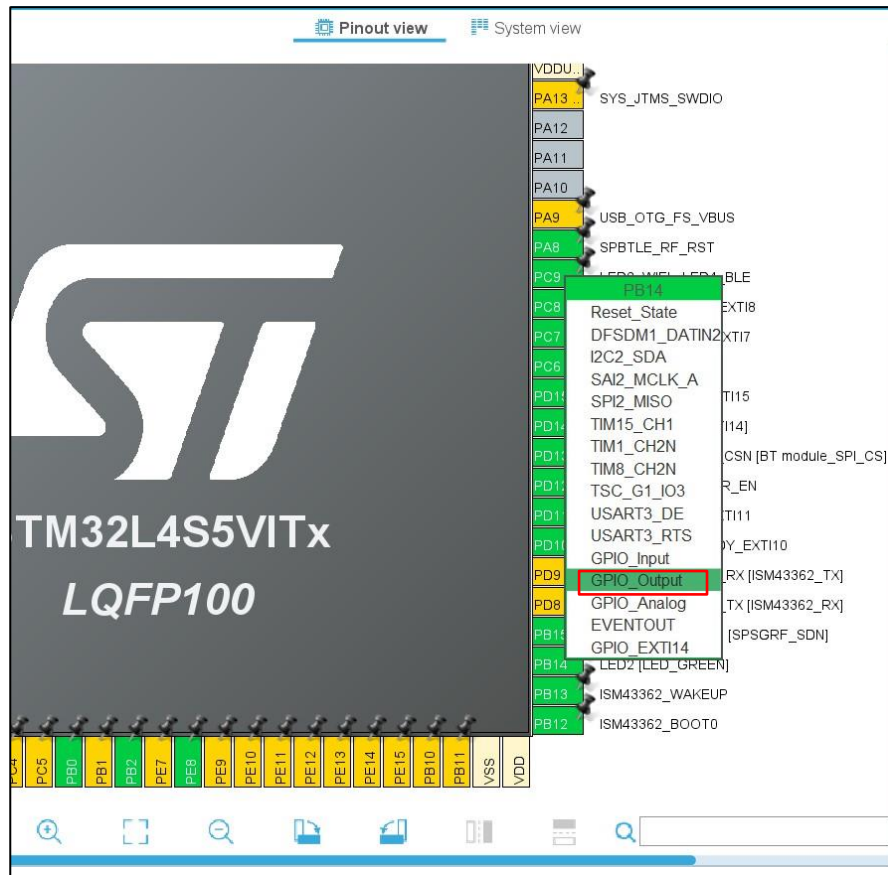
Give your project a name, the rest of the things will remain as default. Click on Finish.



3. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



4. In the **Pinout & Configuration** tab, click on **PB14** pin and select it as **GPIO_OUTPUT**, if it is not selected by default.



6. Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE used HAL libraries. Below are the code snippets, put your code in the appropriate places in the **main.c** file. You can blink an LED with two different coding methods. Two separate code snippets are provided for these two methods.

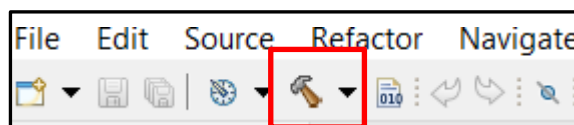
```

86  /* Initialize all configured peripherals */
87  MX_GPIO_Init();
88  /* USER CODE BEGIN 2 */
89
90  /* USER CODE END 2 */
91
92  /* Infinite loop */
93  /* USER CODE BEGIN WHILE */
94  while (1)
95  {
96      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
97      HAL_Delay(100);
98      /* USER CODE END WHILE */
99
100     /* USER CODE BEGIN 3 */
101  }
102     /* USER CODE END 3 */
103 }
104

```

NOTE: HAL (Hardware Abstraction Layer) libraries in STM32CubeIDE provide high-level functions to interface with the hardware peripherals of STM32 microcontrollers. These libraries simplify hardware configuration, initialization, and interaction by abstracting low-level register access and providing easy-to-use API calls.

7. Now click on the build symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.



CDT Build Console [LED_Blink]

```

Finished building: LED_Blink.list

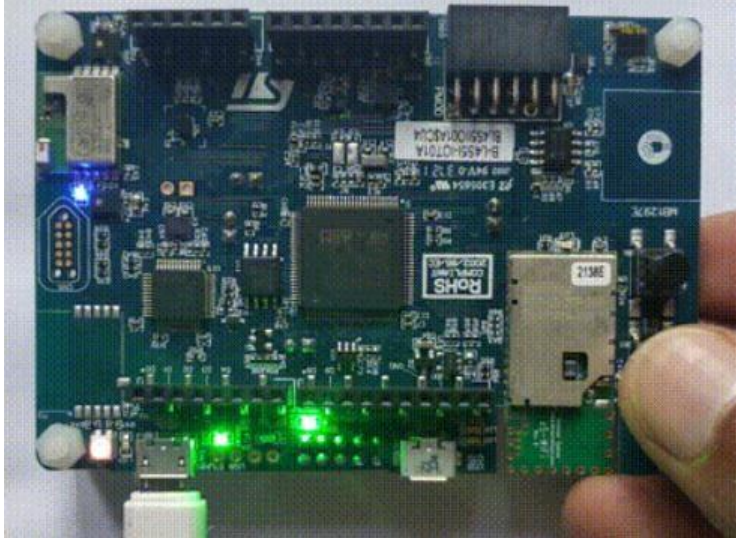
19:41:55 Build Finished. 0 errors, 0 warnings. (took 1s.369ms)

```

8. Next connect your STM32 board to your PC and click on the **Debug** icon to start the

Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.

9. In debugging mode click on the **Resume** icon  to **run** your code. You should be able to see the led on-board your STM32 board blinking.



10. To move out of the debugging mode, click on the **Terminate** icon . You will be moved out of the debugging mode. Congratulations you have just completed your first project on STM32 Cube IDE!

Task:

Toggle the LED by adjusting the delay. Experiment with both increasing and decreasing the delay to observe changes in the blinking rate.

2b. User Button

Lab Objective:

The main objective of this experiment is to understand the basics of GPIOs, how a pin can be configured as an output, input or an interrupt pin etc. By understanding these, we will configure the PB 14 pin of the microcontroller as an GPIO Output pin, because the LD2 which is in-built on the microcontroller is connected to this pin. The blue user push button connected to the PC 13 pin will be configured as GPIO

Note: All important steps and parts are highlighted with a red colour box for the proper understanding of the user. This document is for the use of education purpose only.

EXTI. Then with the help of code we will change the state of the LED with the push of the user push button.

Lab Overview:

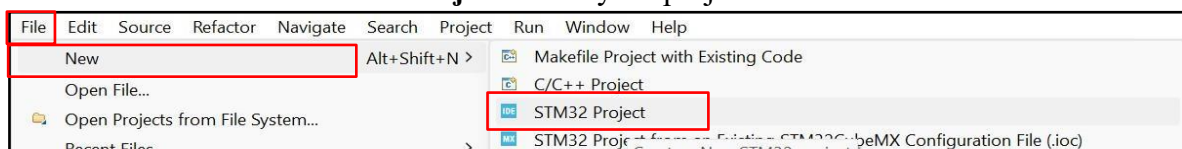
This lab explores the basics of GPIOs by configuring the PB14 pin as an output to control the built-in LD2 LED and the PC13 pin as an input to detect the user push button. The goal is to toggle the LED's state based on the button press, demonstrating GPIO input and output functionality. The experiment will be implemented using simple code to manage the interactions between the button and the LED.

Requirements:

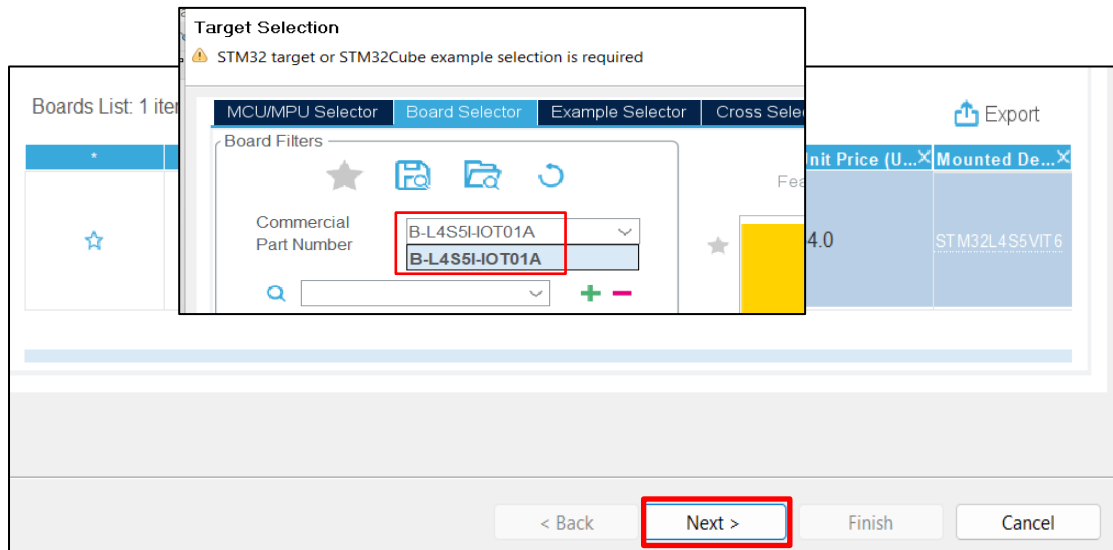
- STM32 Cube IDE software.
- BL4S5I-IOT1A Microcontroller.
- Micro USB Cable for the microcontroller.
- PC or Laptop

Steps Involved:

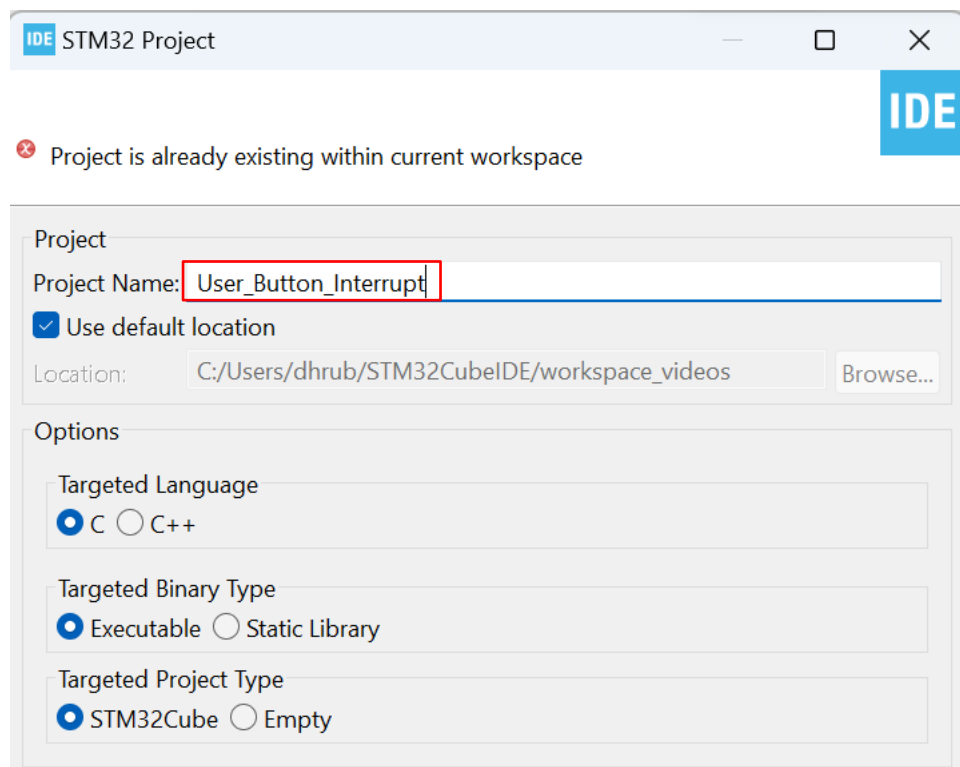
1. Click on **File→New→STM32 Project** to start your project on Cube IDE.



2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.
3. After this on the right-hand side of the window, under **Board List** you will see the board you

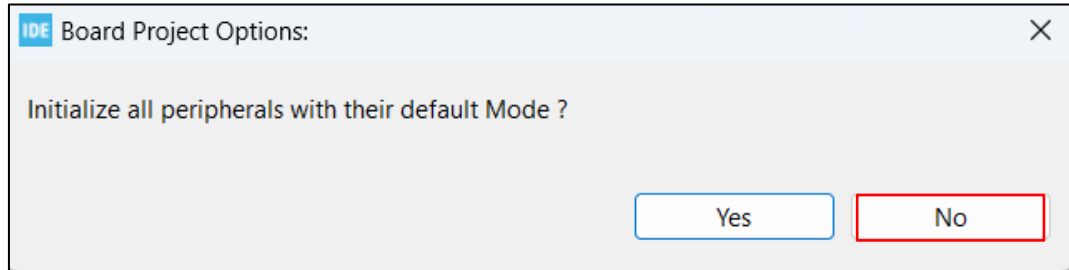


have selected. Click on the board and then click on **Next**.

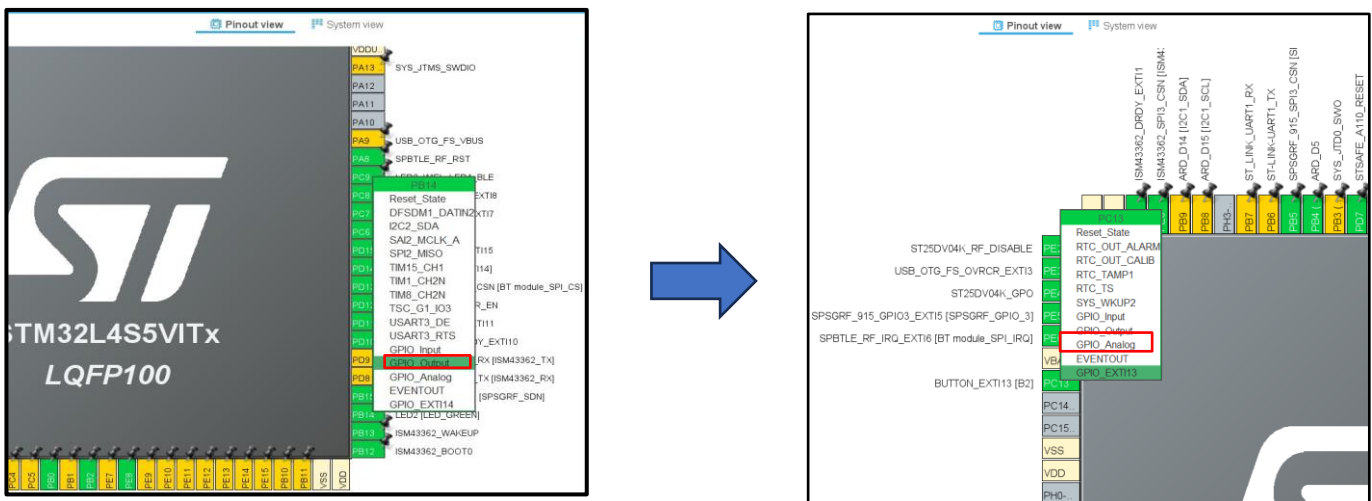


Give your project a name, rest of the things will remain by default as it is for now. Click on **Next**.

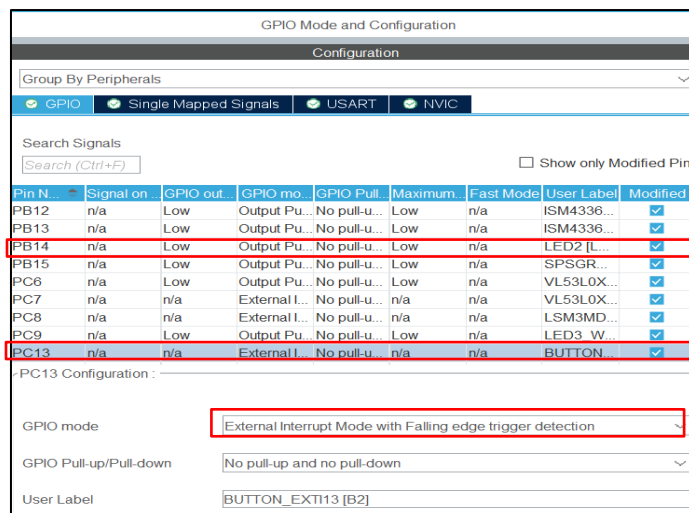
4. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



5. In the **Pinout & Configuration** tab, click on **PB14** pin (LED 2 is connected to this pin) and select it as **GPIO_OUTPUT**, if it is not selected by default. Next click on **PC13** pin (User Button is connected to this pin) and select it as **GPIO_EXTI13**.



6. Next on the left-hand side under **Categories** → **System Core**, select **GPIO** to configure the GPIO

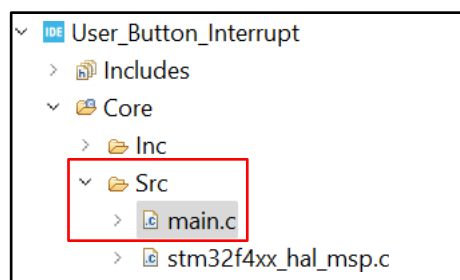


functions. In the GPIO tab select the boxes in **PB14** and **PC13**. Next click on **PC13** and in **GPIO mode** select **External Interrupt Mode with Falling edge trigger detection** from the list of options as shown below. Let everything else remain as default for now.

- Next go to **NVIC** settings and enable the **EXTI line [15:10] interrupts** line.

 SYS	 USART	 NVIC	
 GPIO	 Single Mapped Signals		 RCC
NVIC Interrupt Table		Enabled	Preemption Priority
EXTI line[15:10] interrupts			0

- Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example I have named my project as **User_Button_Interrupt**) **User_Button_Interrupt**→**Core**→**Src**→**main.c** (double click to load the code).



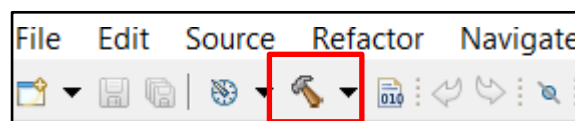
- Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE used HAL libraries. Below are the code snippets, put your code in the appropriate places in the **main.c** file.

```

57 /* Private user code -----
58 /* USER CODE BEGIN 0 */
59 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
60 {
61     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
62 }
63 /* USER CODE END 0 */

```

- Now click on the build symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.



11. Next connect your STM32 board to your PC and click on the **Debug**  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.
12. In debugging mode click on the **Resume** icon  to run your code. You should be able to see the led on-board your STM32 changing with every press of the **User Button**.
13. To move out of the debugging mode, click on the **Terminate**  icon. You will be moved out of the debugging mode.

Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is for the use of education purpose only.

Lab 3 : Configuration of UART in polling method to send data from AIoT Board to PC using printf statement

Lab Objective:

This experiment features the Serial Communication peripheral also known as UART. This lab forms the basis of designing dataloggers. Here, we learn the technique to send text or numeric data from the STM32 board to the PC.

Lab Overview:

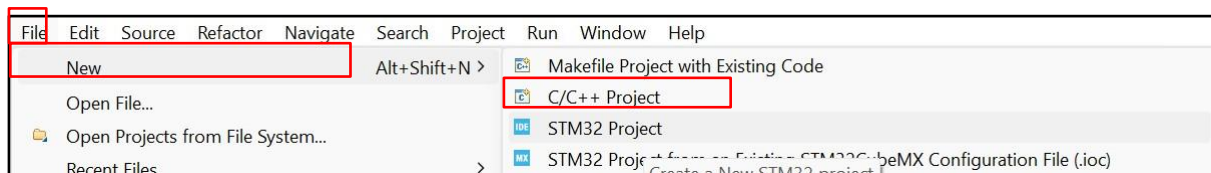
This experiment features the Serial Communication peripheral also known as UART. This lab forms the basis of designing dataloggers. Here, we learn the technique to send text or numeric data from the AIoT board to the PC.

Requirements:

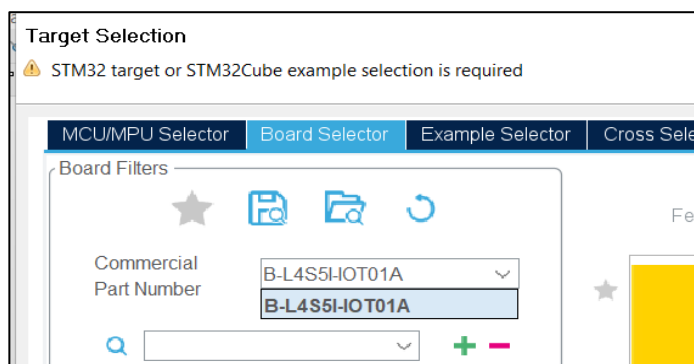
1. STM32 Cube IDE software.
2. BL4S51-IOT1A Microcontroller.
3. Micro USB Cable for the microcontroller.
4. PC or Laptop.

Steps Involved:

1. Click on **File→New→STM32 Project** to start your project on Cube IDE.

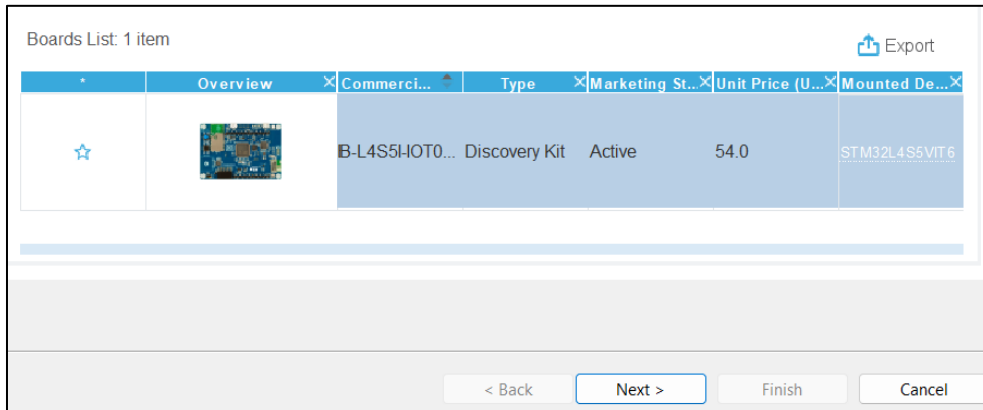


2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.

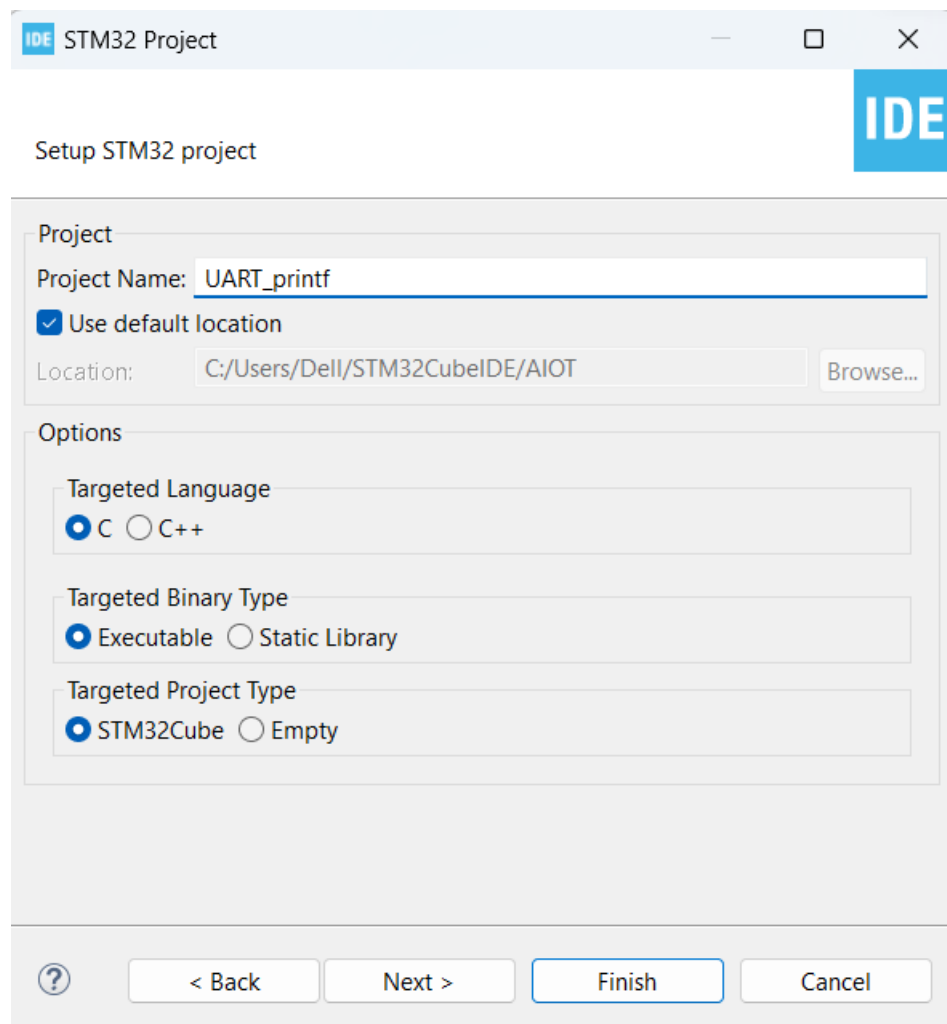


3. After this on the right-hand side of the window, under **Board List** you will see the board you have

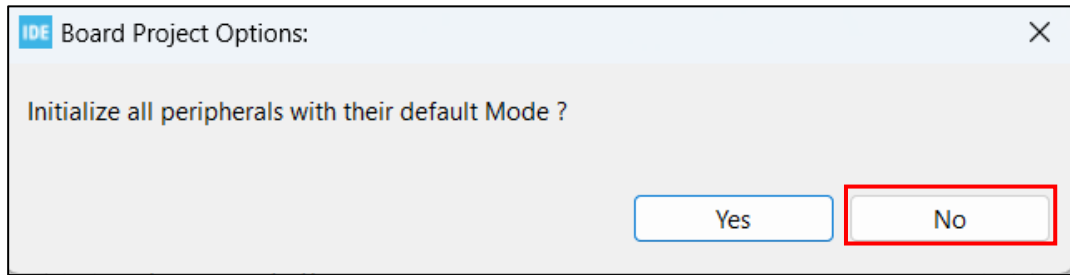
selected. Click on the board and then click on **Next**.



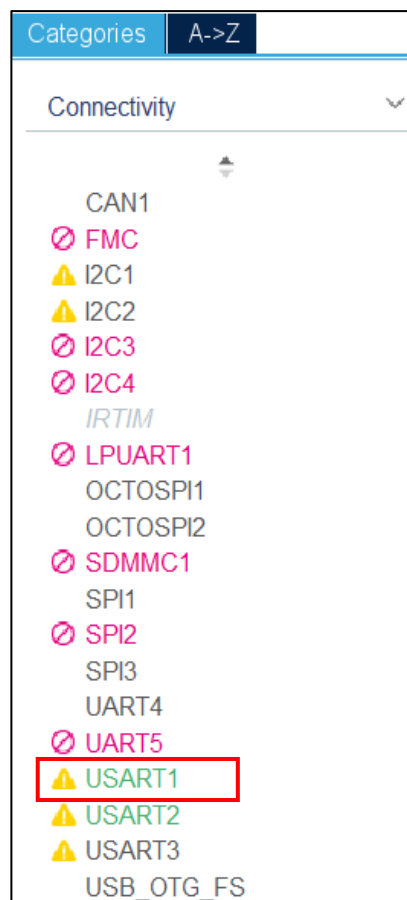
4. In the next window you need to give your project a name, rest of the things will remain by default as it is for now. Click on **Next**.



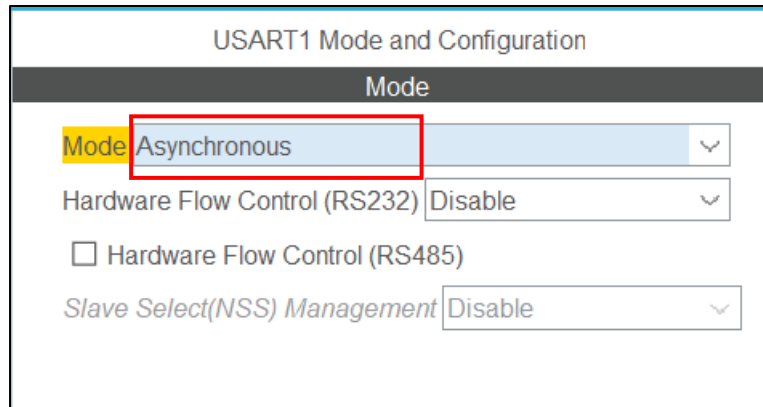
5. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on No.



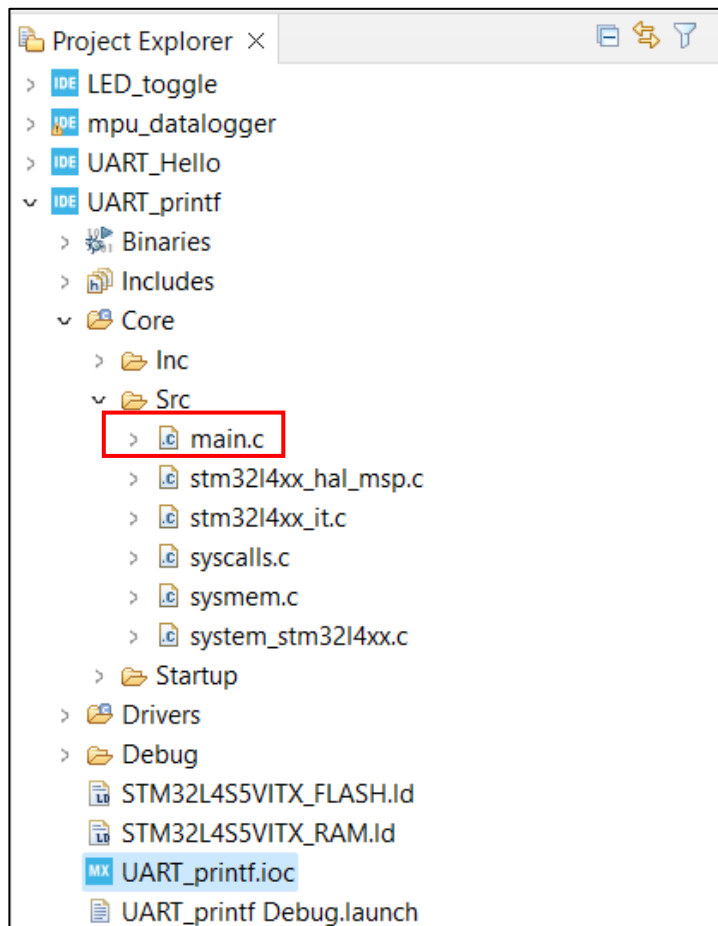
6. In the **Pinout& Configuration** tab, go to Categories Connectivity select UART1.



7. Next go to **UART 1 Mode and Configuration** and select the Mode as Asynchronous.



8. Press **Ctrl + S** to generate your code or you can go to **Project → Generate Code**. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example I have named my project as (UART_printf) **UART_printf→Core→Src→main.c** (double click to load the code).



9. Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE uses HAL libraries. Below is the code snippets, please put your code in the appropriate places in the **main.c** file.

```

*/
/* USER CODE END Header */
/* Includes -----
#include "main.h"

/* Private includes -----
/* USER CODE BEGIN Includes */
#include "stdio.h"
#include "string.h"
/* USER CODE END Includes */

```

```

43 /* Private variables -----
44 UART_HandleTypeDef huart1;
45 UART_HandleTypeDef huart2;
46
47 /* USER CODE BEGIN PV */
48 int value1=95;
49 float value2=89.99;
50 /* USER CODE END PV */
51
52 /* Private function prototypes -----
53 void SystemClock_Config(void);
54 static void MX_GPIO_Init(void);
55 static void MX_USART2_UART_Init(void);
56 static void MX_USART1_UART_Init(void);
57 /* USER CODE BEGIN PFP */
58 int __io_putchar(int);
59 /* USER CODE END PFP */

```

```

462 /* USER CODE BEGIN 4 */
463 int __io_putchar(int ch)
464 {
465     HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
466     return ch;
467 }
468 /* USER CODE END 4 */
469

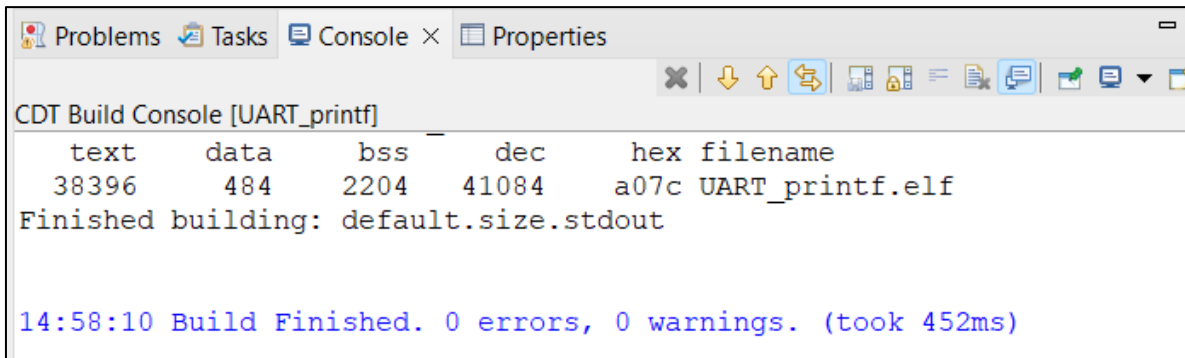
```

```

102 while (1)
103 {
104     printf("the integer value is = %d\r\n",value1);
105     printf("the float value is = %0.2f",value2);
106     printf("\r\n");
107     HAL_Delay(100);
108     /* USER CODE END WHILE */
109
110     /* USER CODE BEGIN 3 */
111 }
112 /* USER CODE END 3 */
113 }
114

```

10. Now click on the build  on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

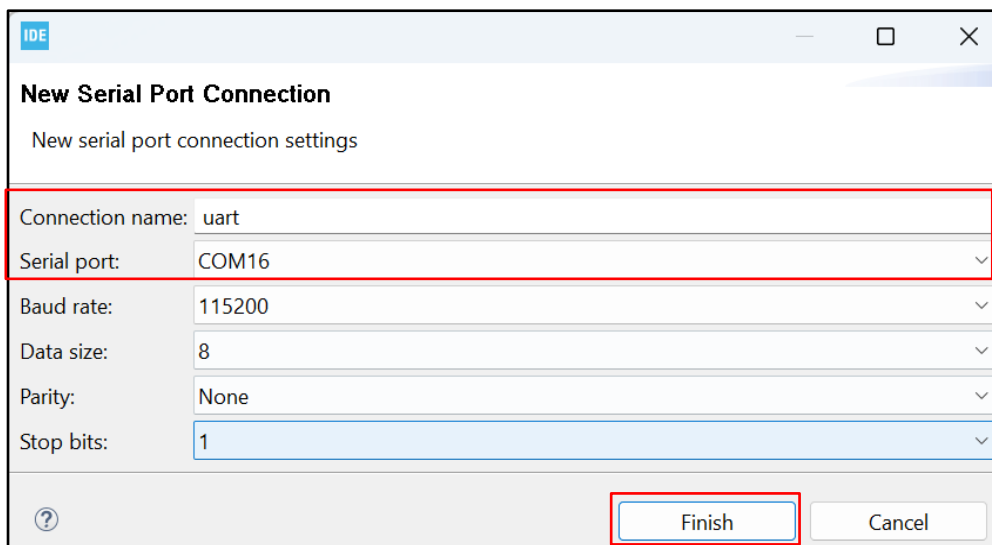
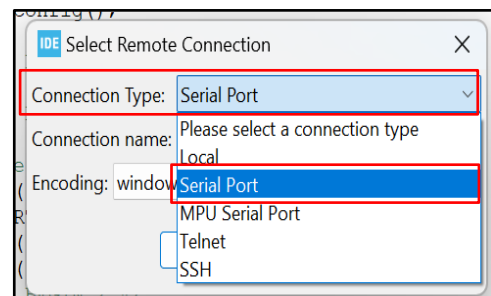
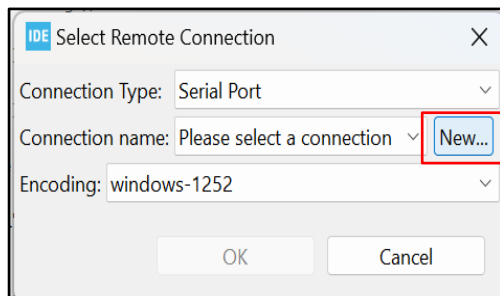
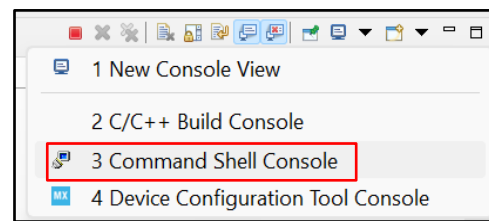
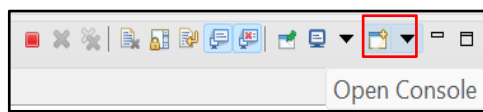


text	data	bss	dec	hex	filename
38396	484	2204	41084	a07c	UART_printf.elf

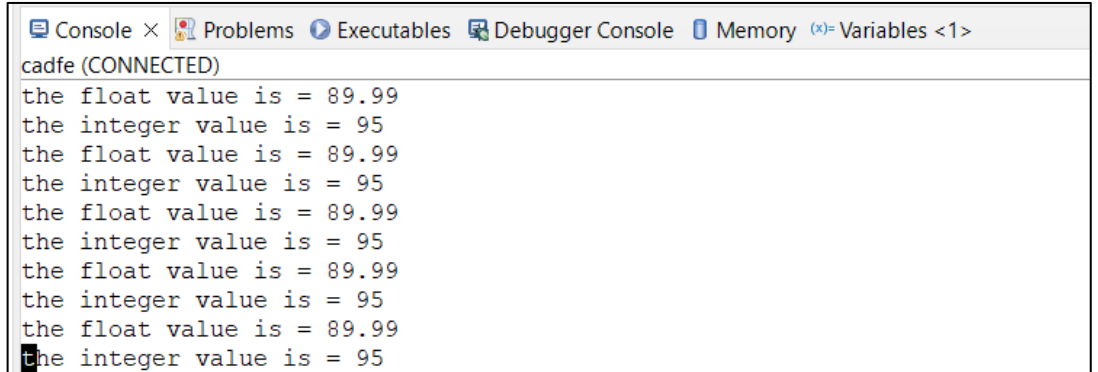
Finished building: default.size.stdout

14:58:10 Build Finished. 0 errors, 0 warnings. (took 452ms)

11. Next connect your STM32 board with your audio sensor connect to it to your PC and click on the **Debug** icon  at the Debugging process. An **Edit configuration** window will open, click on **OK**, without making any changes.
12. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.



13. Click on the **Resume** icon  to run your code. You should be able to see results continuously displayed on the console.



14. Before moving out of the debugging mode, click on **Disconnect** and close the console then click on the **Terminate** icon . You will be moved out of the debugging mode.



Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is for the use of education purpose only.

Lab 4 : IoT Sensor Initialization: Development of simple programs to interface following sensors with the AIoT board

4a. MPU6050 Sensor

Objective of the Lab

Interface the MPU6050 motion Sensor with the BL4S5IIOT01A board, set up I2C communication, retrieve accelerometer and gyroscope data, and display readings on the serial console.

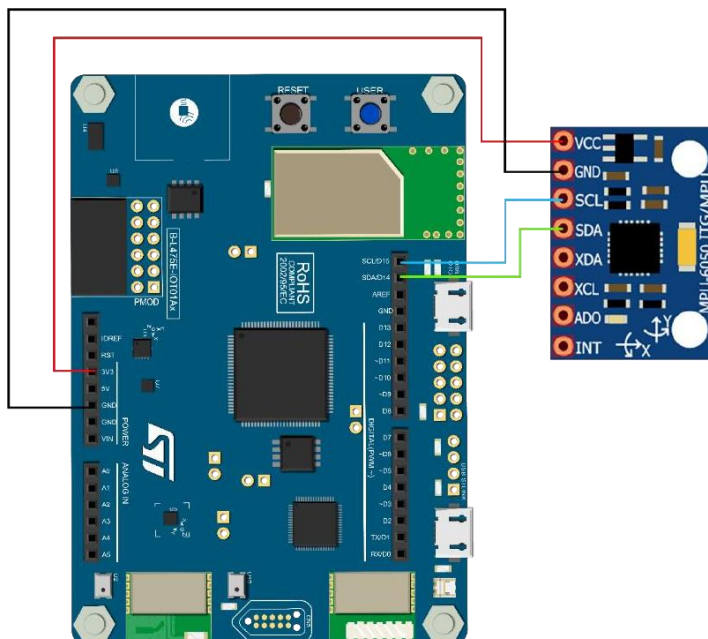
Lab overview

The objective of this experiment is to interface a 6-axis motion sensor (MPU6050).and observe the accelerometer and gyroscope data in serial console..

Requirements:

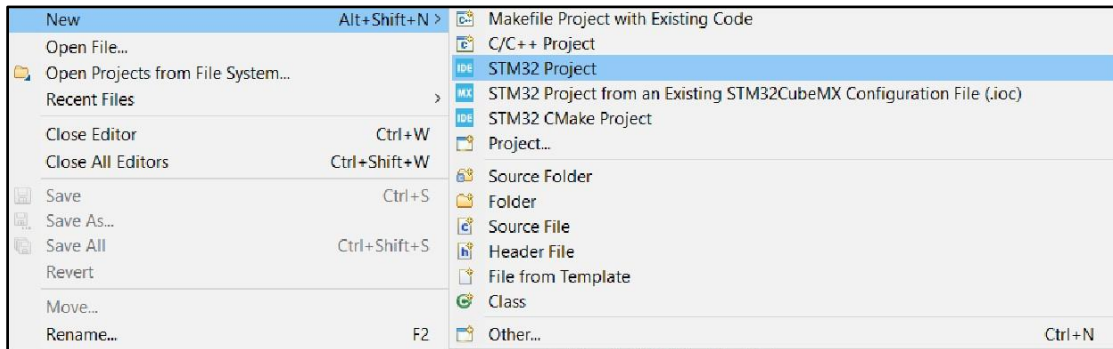
1. STM32 Cube IDE software.
2. MPU6050 sensor (I2C).
3. STM32 Microcontroller.
4. USB Cable for the microcontroller.
5. Jumper Wires.
6. Laptop or PC

Connection Diagram:

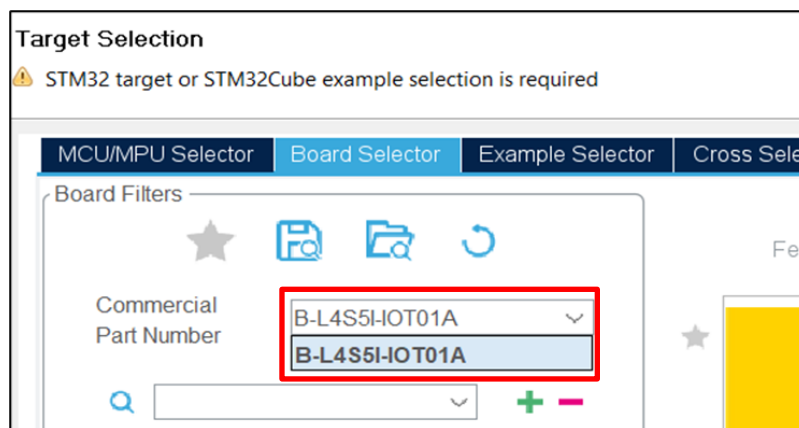


Steps Involved:

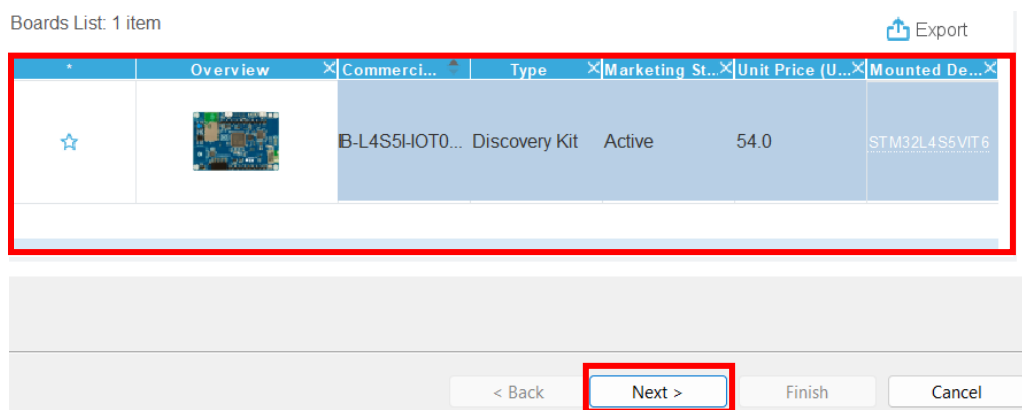
1. Click on **File→New→STM32 Project** to start your project on Cube IDE.

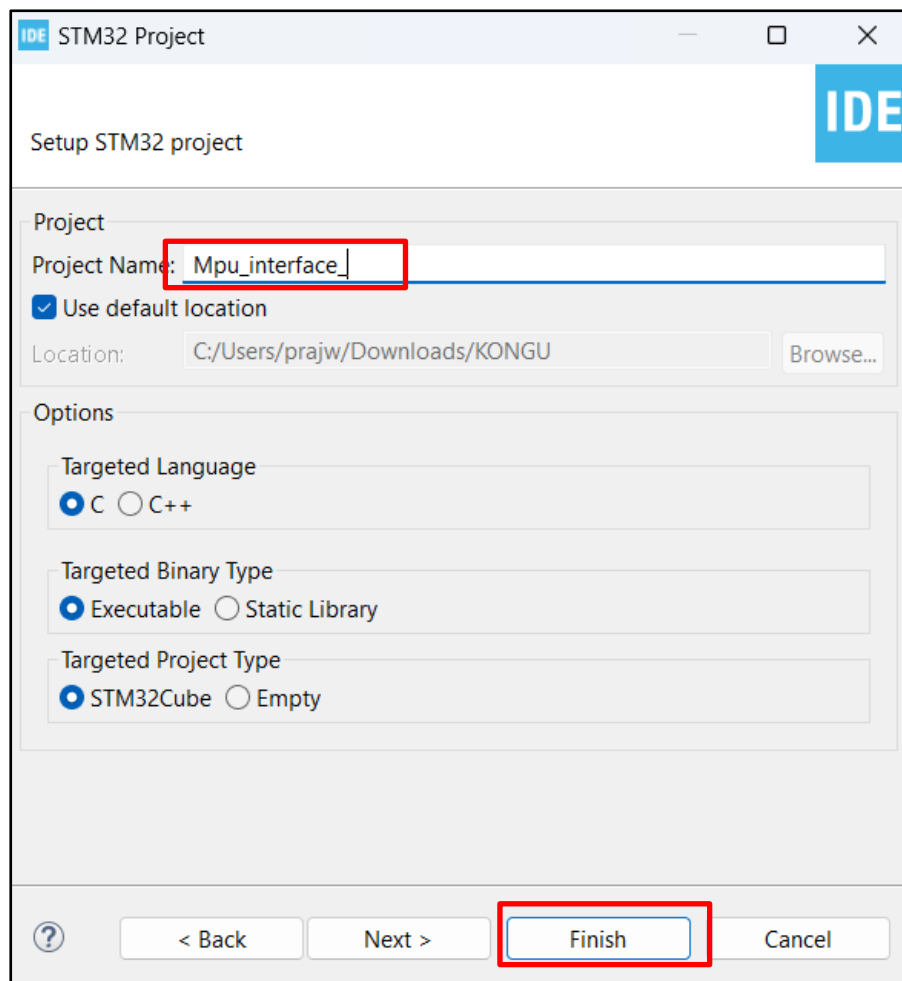


2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.
(NB: If you are having Nucleo-F401RE, you have to select the said Commercial Part Number)

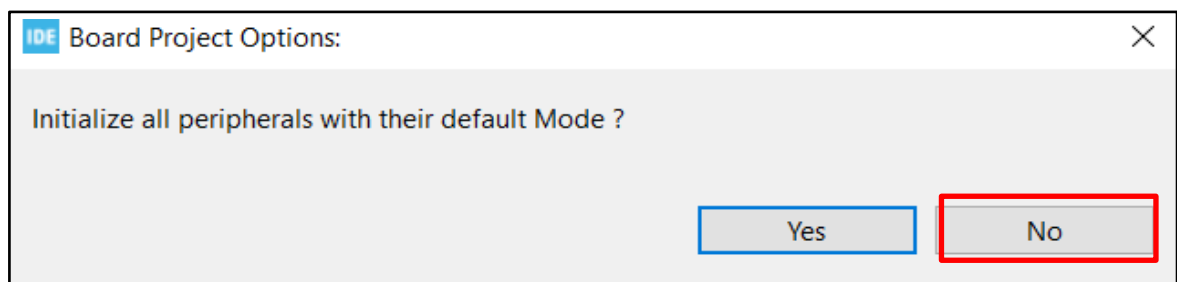


3. After this on the right-hand side of the window, under **Board List** you will see board you have selected. Click on the board and then click on **Next**. In the next window give your project a name, rest of the things will remain by default as it is for now. Click on **Finish**.

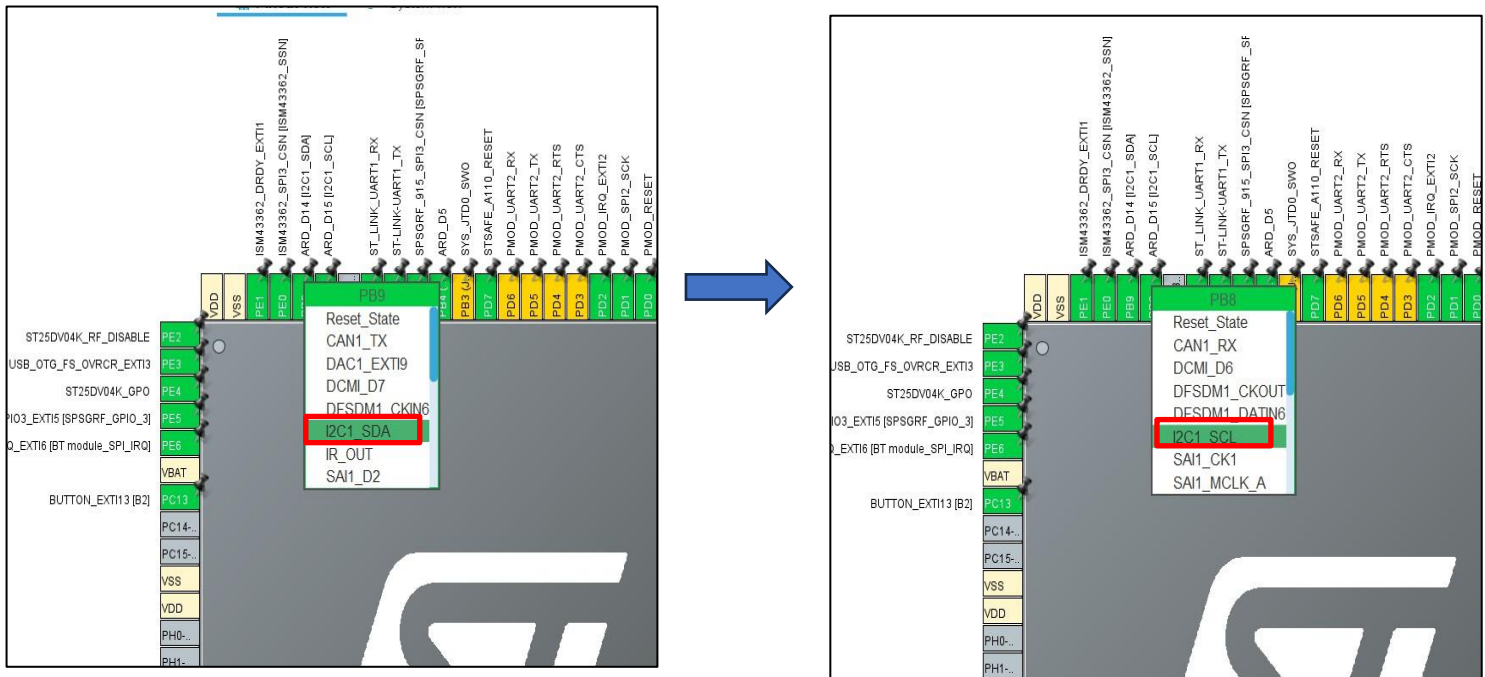




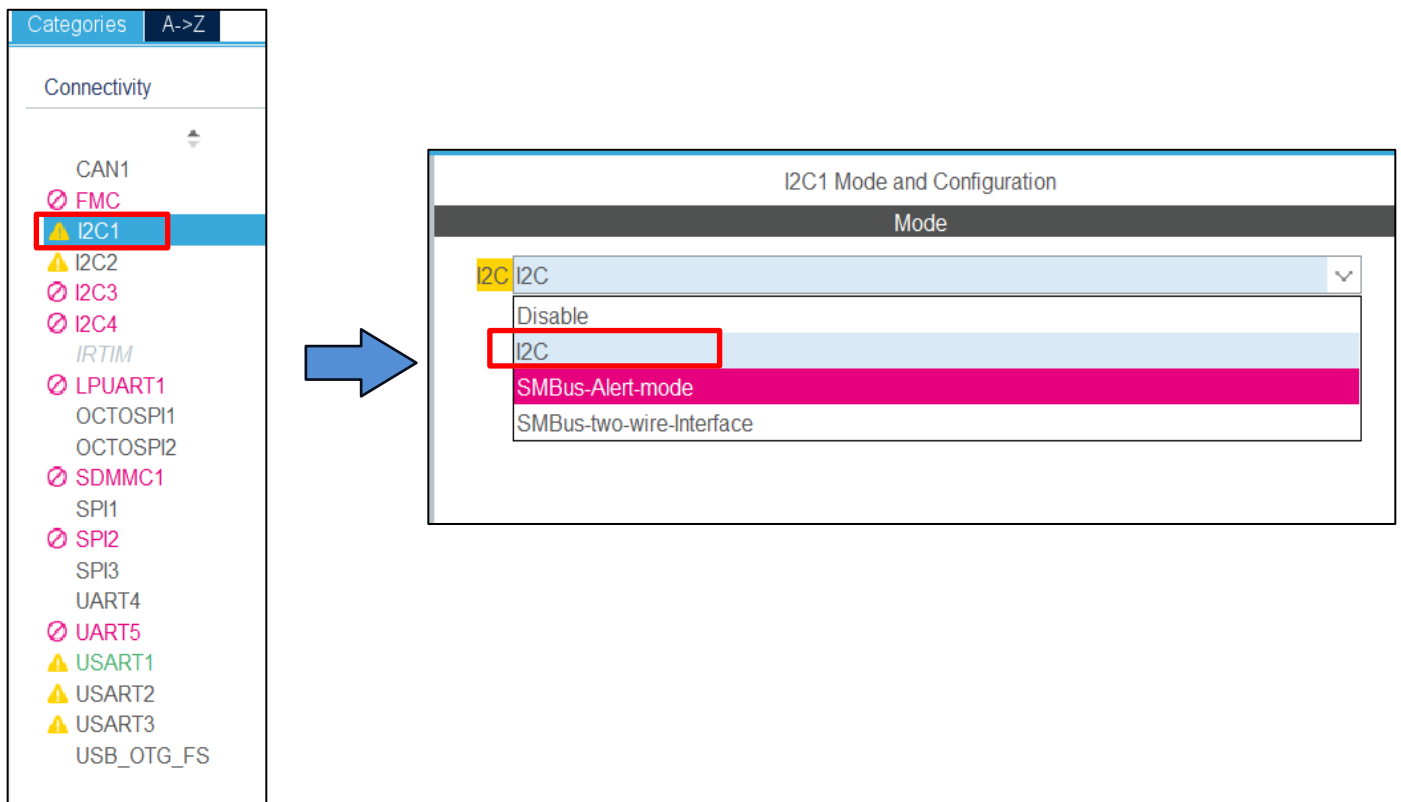
4. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



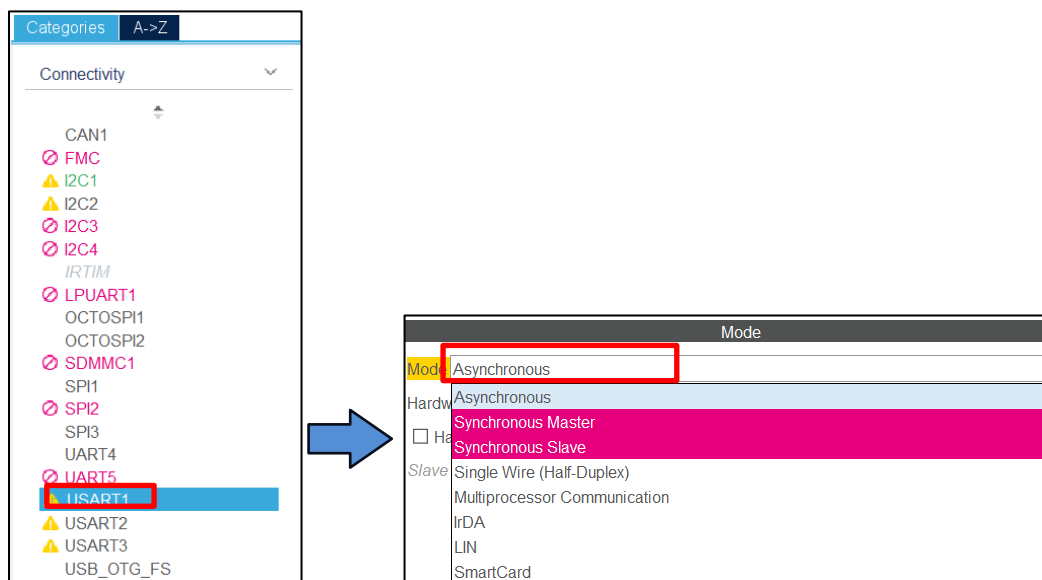
- In the **Pinout & Configuration** tab, click on **PB8** pin and select it as an **I2C1_SCL** and **PB9** pin as an **I2C1_SDA**.



- Next on the left-hand side under **Categories** → **Connectivity**, select **I2C1** and enable it.

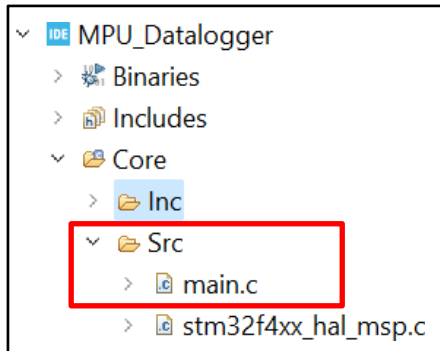


7. Next on the left-hand side under **Categories** → **Connectivity**, select USART1 and enable asynchronous mode.

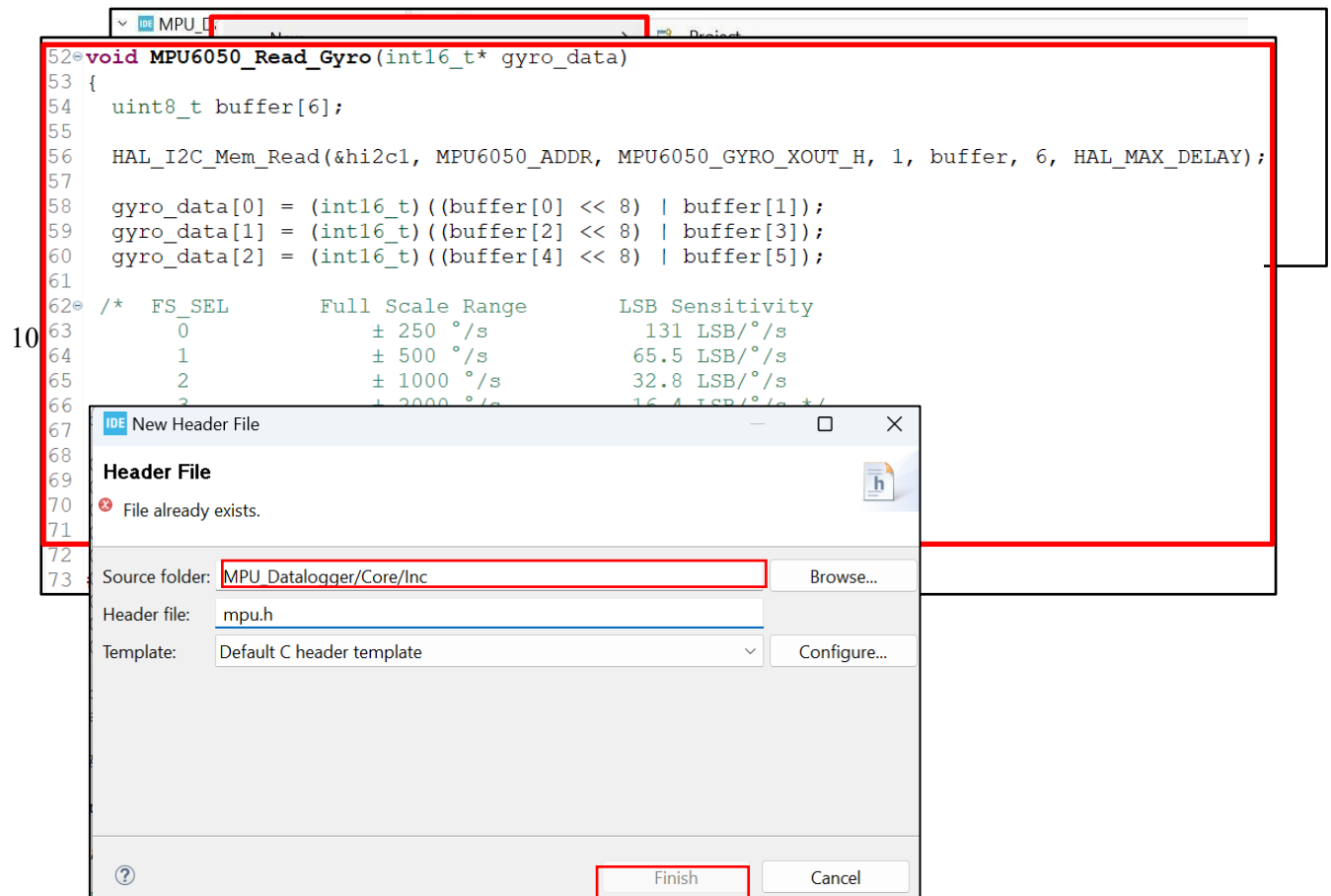


8. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example, I have named my project as (MPU_Datalogger))

MPU_Datalogger→Core →Src→main.c (double click to load the code).



9. Now open your project tree MPU_Datalogger→Core →Inc. Right click on your Inc folder and create a new **Header File**. Name the Header File as **mpu.h** and select on **Finish**.



```

2 #ifndef INC_MPU_H_
3 #define INC_MPU_H_
4
5 extern I2C_HandleTypeDef hi2c1;
6
7 #define MPU6050_ADDR      0xD0
8 #define MPU6050_PWR_MGMT_1 0x6B
9 #define MPU6050_ACCEL_XOUT_H 0x3B
0 #define MPU6050_ACCEL_YOUT_H 0x3D
1 #define MPU6050_ACCEL_ZOUT_H 0x3F
2 #define MPU6050_TEMP_OUT_H 0x41
3 #define MPU6050_GYRO_XOUT_H 0x43
4 #define MPU6050_GYRO_YOUT_H 0x45
5 #define MPU6050_GYRO_ZOUT_H 0x47
6
7 int16_t accel_data[3];
8 int16_t gyro_data[3];
9
10 float Ax, Ay, Az, Gx, Gy, Gz;
11
12 void MPU6050_Init(void)
13 {
14     uint8_t data;
15
16     // Wake up MPU6050
17     data = 0x00;
18     HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, MPU6050_PWR_MGMT_1, 1, &data, 1, HAL_MAX_DELAY);
19 }
20
21 void MPU6050_Read_Accel(int16_t* accel_data)
22 {
23     uint8_t buffer[6];
24
25     HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, MPU6050_ACCEL_XOUT_H, 1, buffer, 6, HAL_MAX_DELAY);
26
27     accel_data[0] = (int16_t)((buffer[0] << 8) | buffer[1]);
28     accel_data[1] = (int16_t)((buffer[2] << 8) | buffer[3]);
29     accel_data[2] = (int16_t)((buffer[4] << 8) | buffer[5]);
30
31     /*
32     AFS_SEL      Full Scale Range      LSB Sensitivity
33     0            ±2g                    16384 LSB/g
34     1            ±4g                    8192 LSB/g
35     2            ±8g                    4096 LSB/g
36     3            ±16g                   2048 LSB/g */
37
38     Ax = accel_data[0]/2048.0;
39     Ay = accel_data[1]/2048.0;
40     Az = accel_data[2]/2048.0;
41 }

```

11. Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE uses HAL libraries. Below are the code snippets, please put your code in the appropriate places in the **main.c** file.

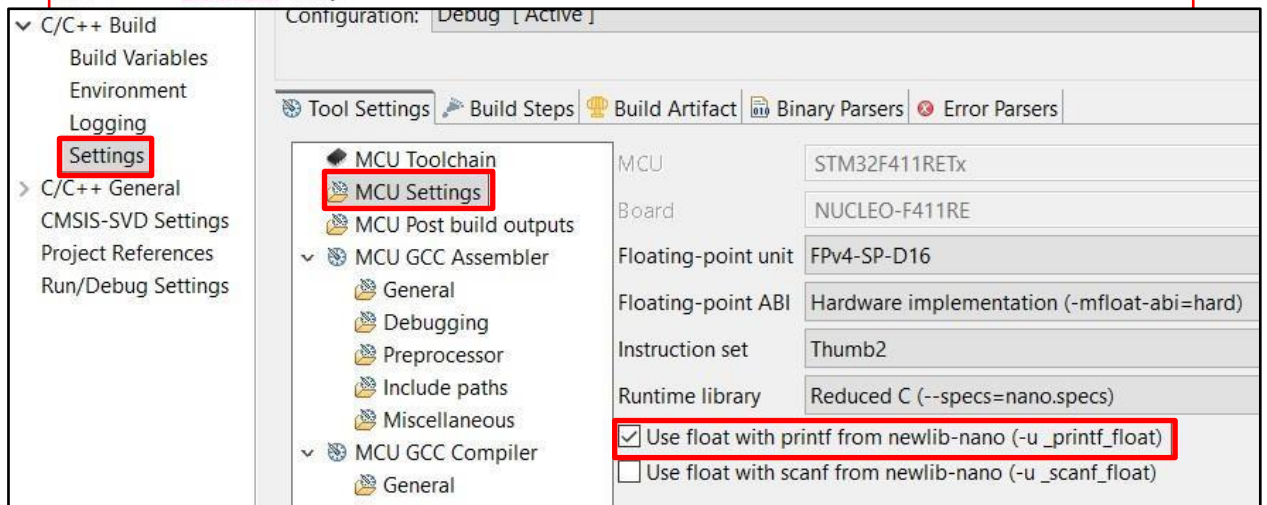
```
22 /* Private includes -----
23 /* USER CODE BEGIN Includes */
24 #include "mpu.h"
25 #include "stdio.h"
26 #include "string.h"
27 /* USER CODE END Includes */
```

```
94 /* Initialize all configured peripherals */
95 MX_GPIO_Init();
96 MX_I2C1_Init();
97 MX_USART1_UART_Init();
98 /* USER CODE BEGIN 2 */
99 MPU6050_Init();
100 HAL_Delay(1000);
101
102 /* USER CODE END 2 */
```

```
105 while (1)
106 {
107     MPU6050_Read_Accel(accel_data);
108     MPU6050_Read_Gyro(gyro_data);
109     printf("Accelerometer Data: %.2f %.2f %.2f \r\n", Ax, Ay, Az);
110     HAL_Delay(300);
111     printf("Gyroscope Data: %.2f %.2f %.2f \r\n", Gx, Gy, Gz);
112     HAL_Delay(300);
113     /* USER CODE END WHILE */
```

12. Right click on the Audio_Classification project and select **Properties**. Go to **C/C++ Build**

```
284 int __io_putchar(int ch) {
285     HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
286     return ch;
}
```



→Settings. Next select **MCU Settings** and enable the option **Use float with printf from newlib- nano (-u _printf_float)**.

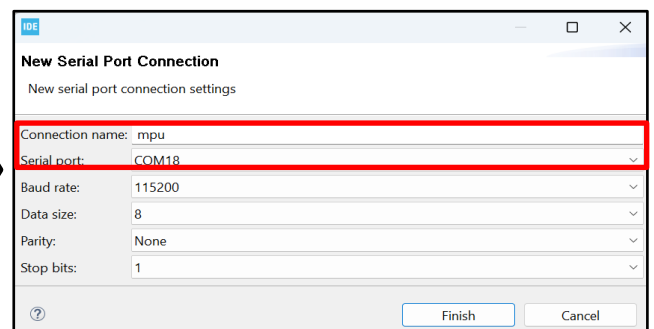
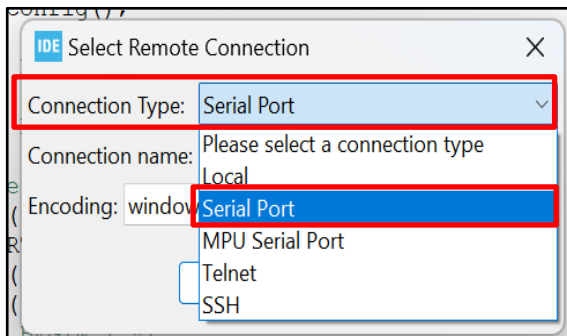
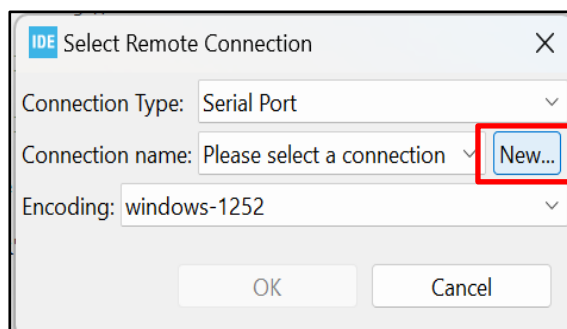
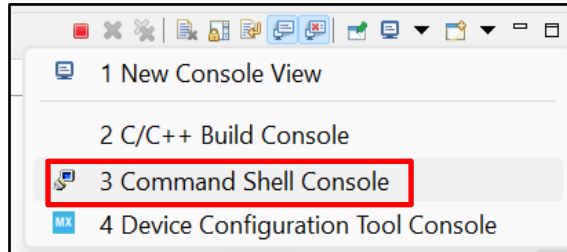
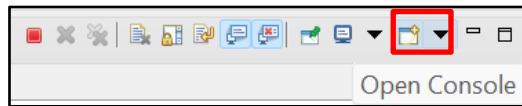
13. Now click on the build  symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

CDT Build Console [MPU_Datalogger]					
Text	Data	BSS	dec	hex	Filename
28440	484	4836	33760	83e0	MPU_Datalogger.elf
Finished building: default.size.stdout					
13:43:39 Build Finished. 0 errors, 0 warnings. (took 891ms)					

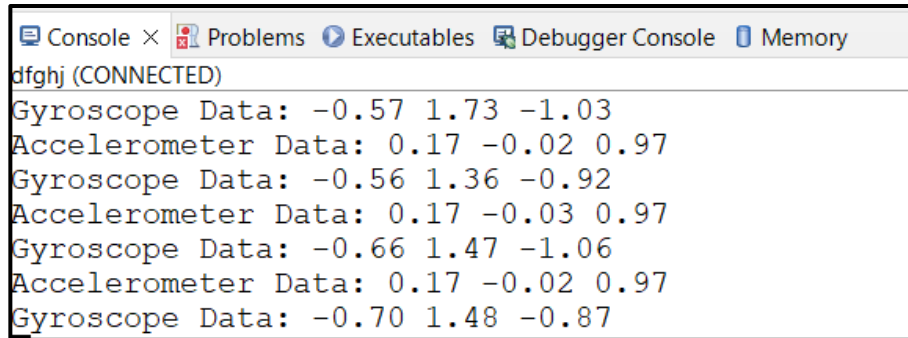
14. Next connect your STM32 board with your audio sensor connect to it to your PC and click on the

Debug  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.

15. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.



16. Click on the **Resume** icon  to run your code. You should be able to see the values of `ofmpu6050sensor` in the **Console**.



```
dfghj (CONNECTED)
Gyroscope Data: -0.57 1.73 -1.03
Accelerometer Data: 0.17 -0.02 0.97
Gyroscope Data: -0.56 1.36 -0.92
Accelerometer Data: 0.17 -0.03 0.97
Gyroscope Data: -0.66 1.47 -1.06
Accelerometer Data: 0.17 -0.02 0.97
Gyroscope Data: -0.70 1.48 -0.87
```

17. Before moving out of the debugging mode, click on **Disconnect**  and close the console then click on the **Terminate**  icon. You will be moved out of the debugging mode.

Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is for the use of education purpose only.

4b. Audio Sensor(adc with polling)

Lab Overview:

In this lab, we will connect an audio sensor to the BL4S51-IOT1A microcontroller and set up the ADC in interrupt mode. Each time the ADC finishes a conversion, it will trigger an interrupt, and the converted value will be stored in a variable for further use.

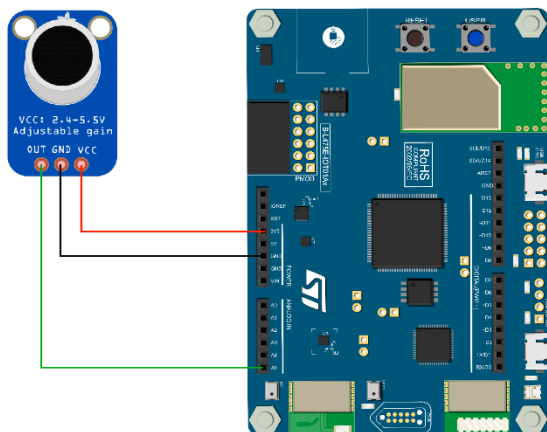
Lab Objective:

The objective of this experiment is to interface an audio sensor to an BL4S51-IOT1A microcontroller and configure the ADC in interrupt mode. Every time the ADC completes a conversion, the ADC will generate an internal interrupt. Once the interrupt occurs, the converted value will be collected from the ADC data registers and stored into a variable.

Requirements:

1. STM32 Cube IDE software.
2. Audio Sensor (Analog).
3. BL4S51-IOT1A Microcontroller.
4. Micro USB Cable for the microcontroller.
5. Jumper Wires.
6. PC or Laptop.

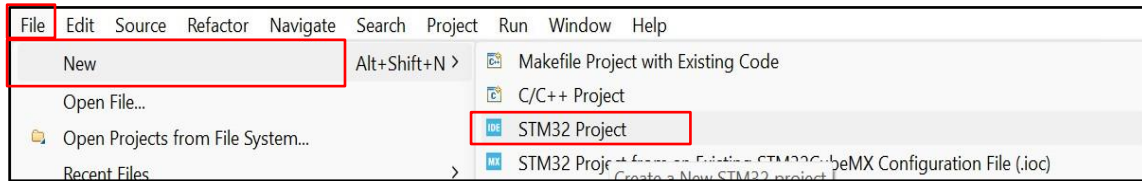
Experimental Setup:



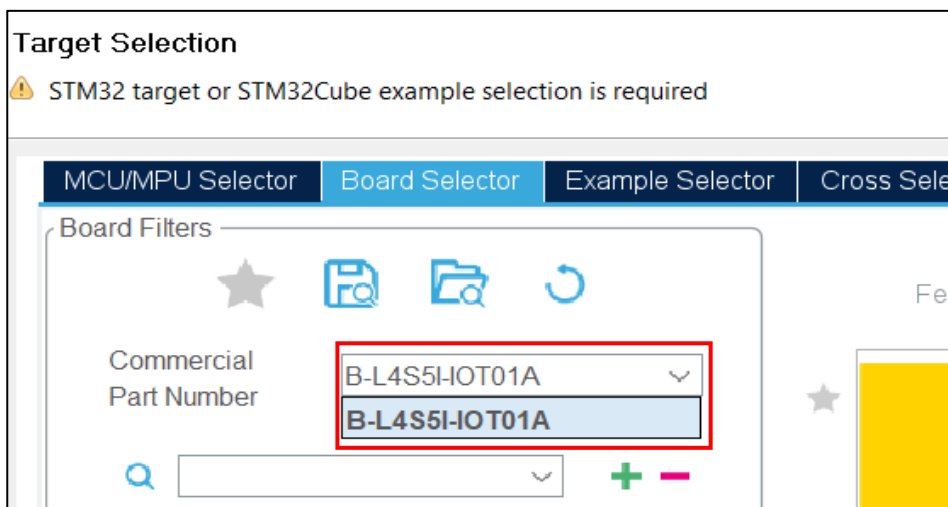
NOTE: Analog-to-Digital Converter (ADC) converts continuous analog signals into discrete digital values that can be processed by a microcontroller. It allows the microcontroller to read and interpret real-world analog signals, such as audio, temperature or light levels.

Procedure:

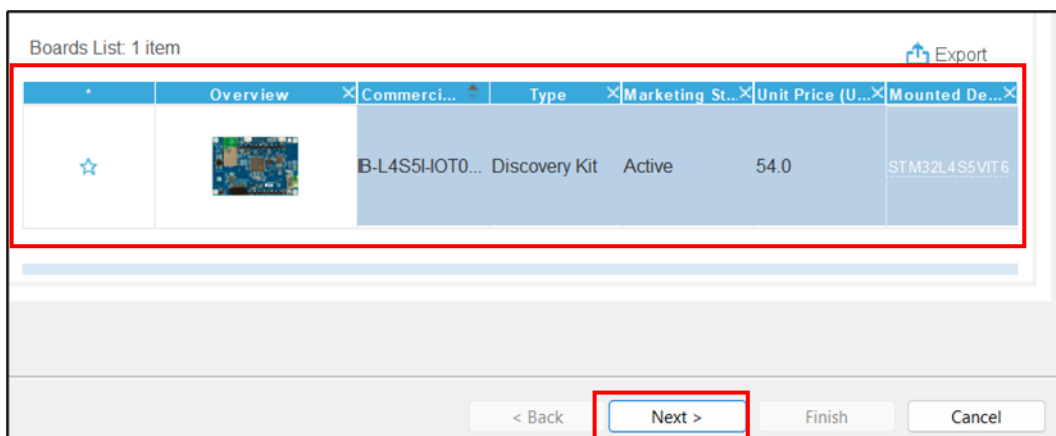
1. Click on **File**→**New**→**STM32 Project** to start your project on Cube IDE



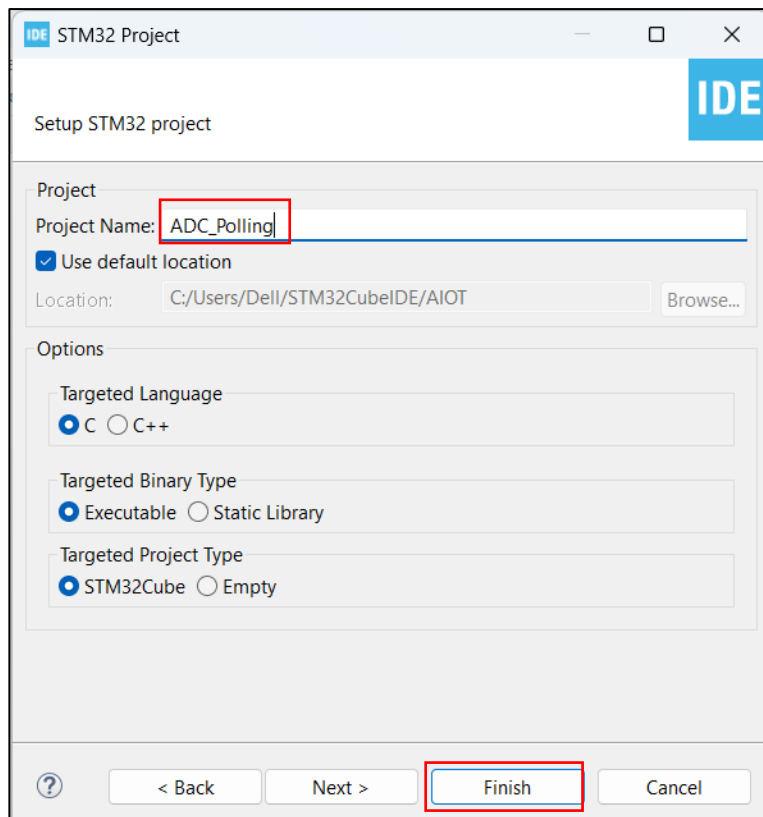
2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.



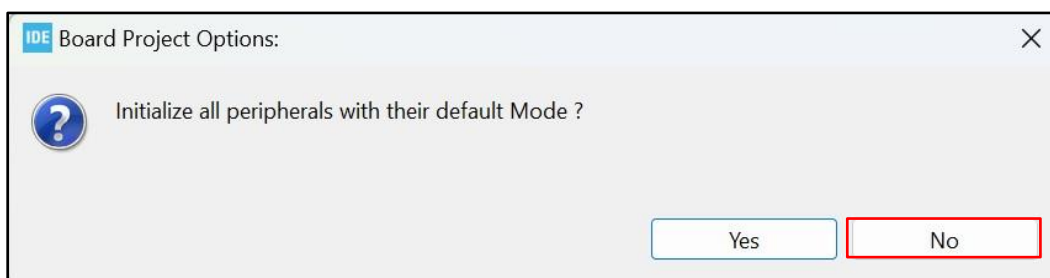
3. After this on the right-hand side of the window, under **Board List** you will see the board you have selected. Click on the board and then click on **Next**.



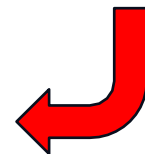
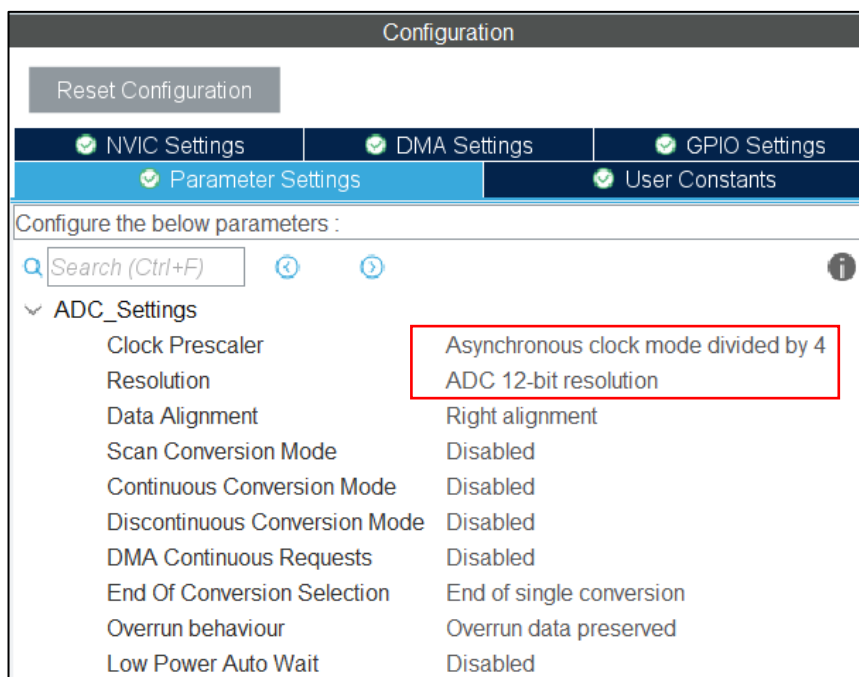
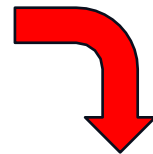
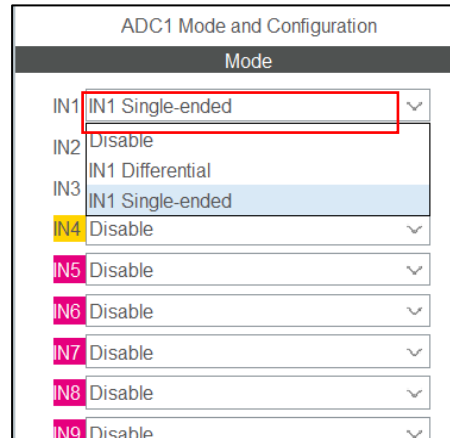
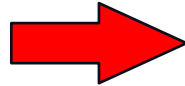
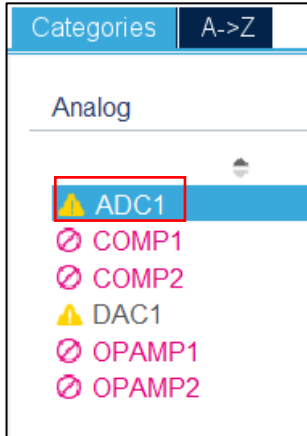
4. In the next window you need to give your project a name, rest of the things will remain by default as it is for now. Click on **Next**.



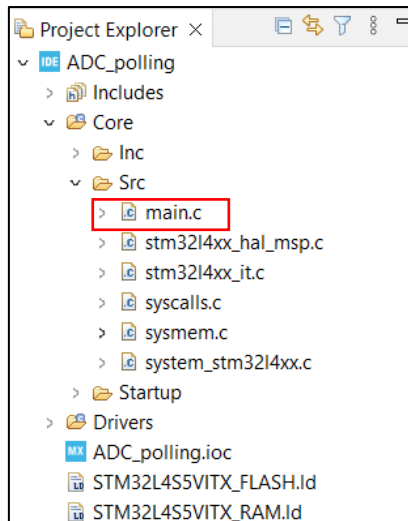
5. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



6. Next on the left-hand side **Categories** → **Analog** select **ADC1** then select **IN1** as IN1 Single-ended under **Mode**. Under Configuration select **Parameter settings** under ADC_Settings select the clock prescaler to by **Asynchronous clock mode divided by 4** and then change the resolution to **ADC 12-bit resolution**.



7. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example I have named my project as (ADC_Interrupt)
ADC_Interrupt→Core →Src→main.c (double click to load the code).



8. Cube IDE automatically generates the initialization code based on the configurations you have done. Cube IDE used HAL libraries. Below is the code snippets, please put your code in the appropriate places in the **main.c** file.

```
23 /* USER CODE BEGIN Includes */
24 #include "stdio.h"
25 #include "string.h"
26 /* USER CODE END Includes */
```

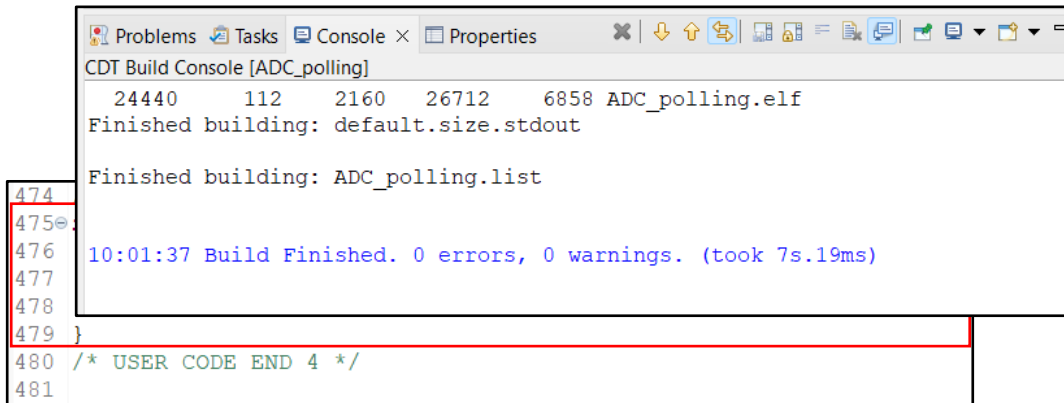
```
47 /* USER CODE BEGIN PV */
48 int value = 0;
49 /* USER CODE END PV */
```

```
57 /* USER CODE BEGIN PFP */
58 int __io_putchar(int);
59 /* USER CODE END PFP */
```

```
101 /* USER CODE BEGIN WHILE */
102 while (1)
103 {
104     Value=HAL_ADC_GetValue(&hadc1);
105     printf("the value is= %d",Value);
106     printf("\r\n");
107     HAL_ADC_Start(&hadc1);
108     HAL_Delay(100);
109
110     /* USER CODE END WHILE */
```

NOTE: entire main.c file is available in the attached code folder, you can copy it from that directory

9. Now click on the   symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

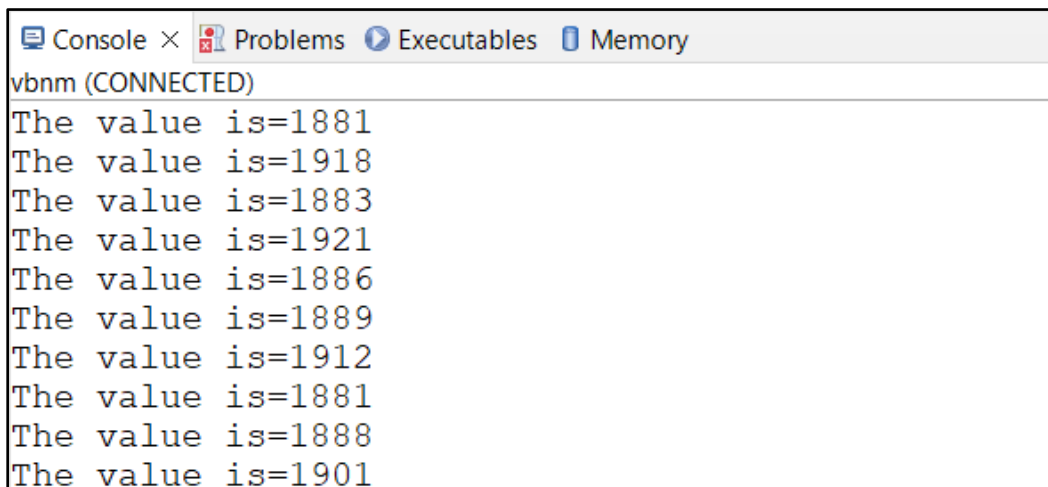


```

CDT Build Console [ADC_polling]
24440      112      2160      26712      6858 ADC_polling.elf
Finished building: default.size.stdout
Finished building: ADC_polling.list
474
475
476 10:01:37 Build Finished. 0 errors, 0 warnings. (took 7s.19ms)
477
478
479 }
480 /* USER CODE END 4 */
481

```

10. Next connect your STM32 board with your audio sensor connect to it to your PC and click on the **Debug**  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.
11. Click on the **Resume** icon  to run your code. You should be able to see the value of audio sensor.



```

vbnm (CONNECTED)
The value is=1881
The value is=1918
The value is=1883
The value is=1921
The value is=1886
The value is=1889
The value is=1912
The value is=1881
The value is=1888
The value is=1901

```

12. To move out of the debugging mode, click on the **Terminate** icon.  You will be moved out of the debugging mode.

Note: All important steps and parts are highlighted with a red colour box for the proper understanding of the user. This document is for the use of education purpose only.

4c. BH1750 Light Intensity Sensor

Lab Overview

In this lab, we will collect samples of light data in a buffer format by interfacing a Light (BH1750) sensor on the BL4S5IIOT01A AIoT board

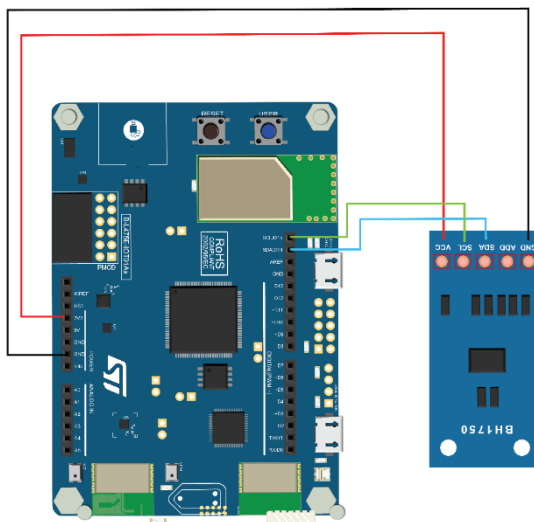
Objective of the Lab

Interface the BH1750 Light Sensor with the BL4S5IIOT01A board, set up I2C communication, retrieve sensor data, and display readings on the serial console.

Requirements:

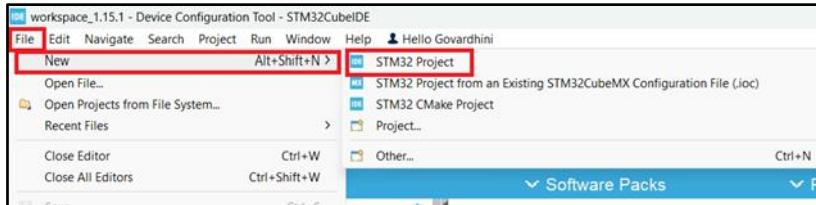
1. STM32 Cube IDE software.
2. Light Sensor (BH1750).
3. BL4S5IOT01A AIoT board.
4. USB Cable for the microcontroller.
5. Jumper Wires.

Experimental Setup



Procedure:

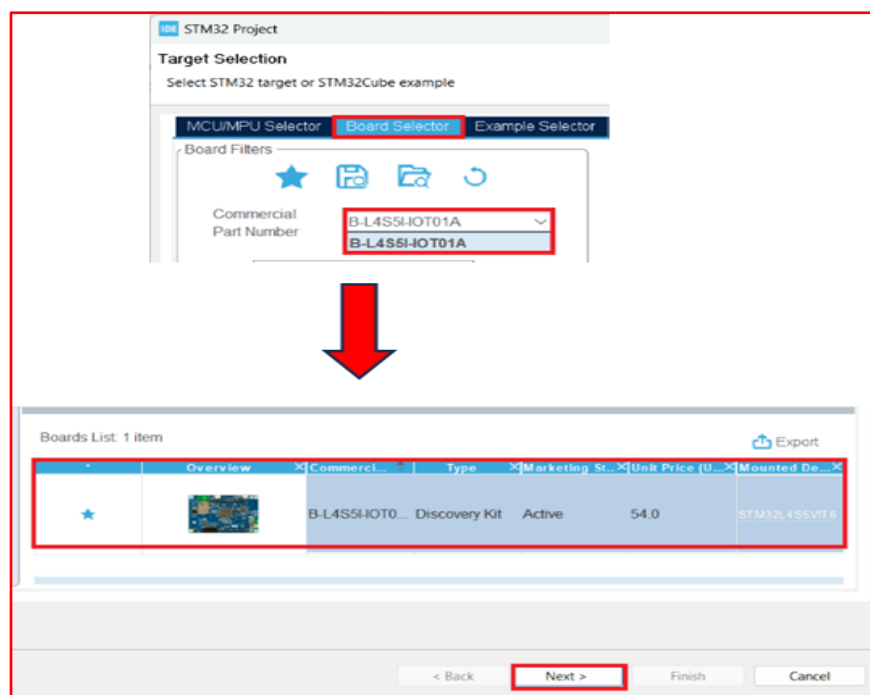
1. Click on **File**→**New**→**STM32 Project** to start your project on Cube IDE



2. Target Hardware Selection.

- **Steps:**
 - a. In the Board Selector, type the board part number B-L4S5I-IOT01A in the Commercial Part Number field, select the board, and click Next.
- **Purpose:**

selecting the specific hardware platform (the B-L4S5I-IOT01A board) to tailor the project settings to the correct board specifications.
- **Screenshot:**



3. Project Name and Settings.

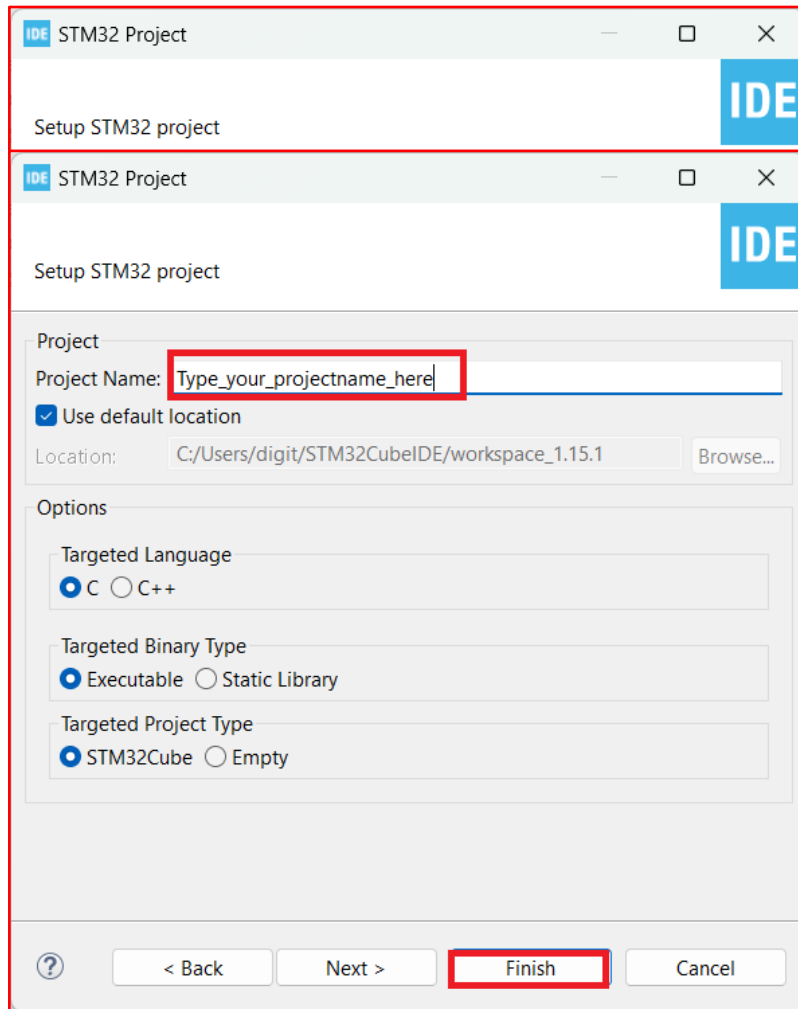
- **Steps:**
 - a. In the Project Name tab, give a name for your project.

b. Click Finish.

- **Purpose:**

This step assigns a unique name to your project and finalizes the initial setup, ensuring that the project files are organized and easily identifiable.

- **Screenshot:**



4. Disable Default Mode

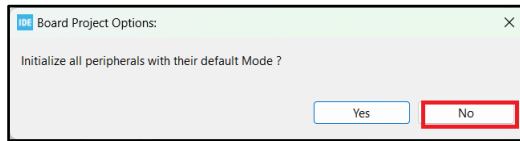
- **Steps:**

a. When prompted, select No to disable the default mode.

- **Purpose:**

By disabling the default mode, you are ensuring that the STM32CubeMX doesn't automatically configure peripherals for default settings.

- **Screenshot:**



5. Open the .ioc File

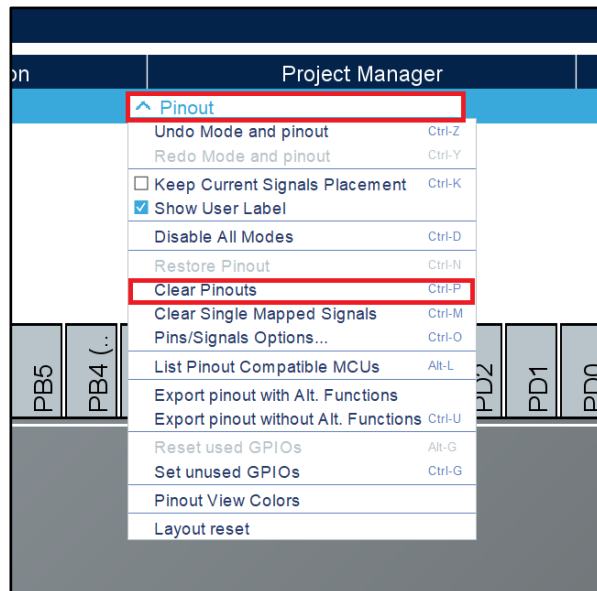
- **Steps:**

- The .ioc file for your project will open automatically.
- Go to the Pinout tab and click on Clear Pinout.

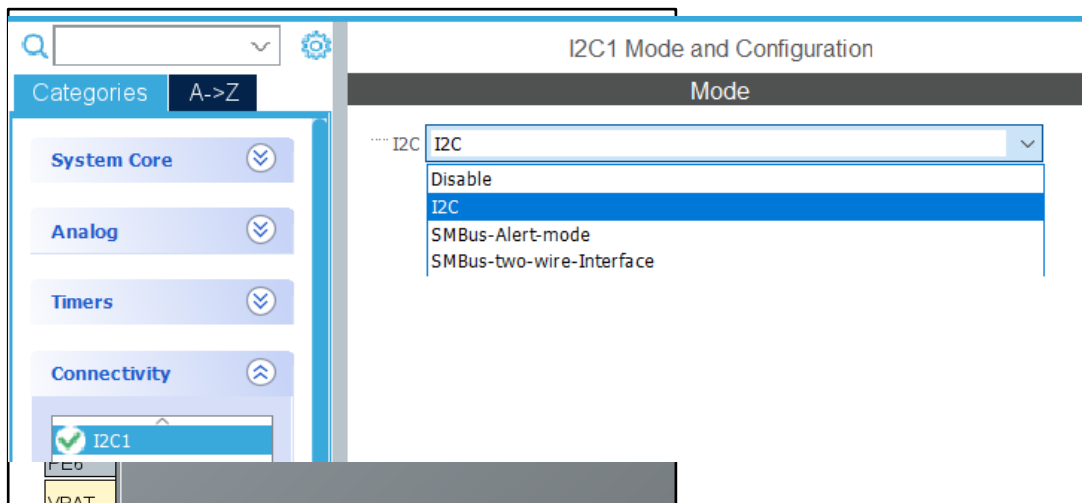
- **Purpose:**

The .ioc file contains all the configuration settings for your project. Opening it allows you to configure pins, peripherals, and middleware required for the project. This step clears the default pinout configuration in STM32CubeMX, providing a clean slate to set up custom pins and other peripherals required for your project.

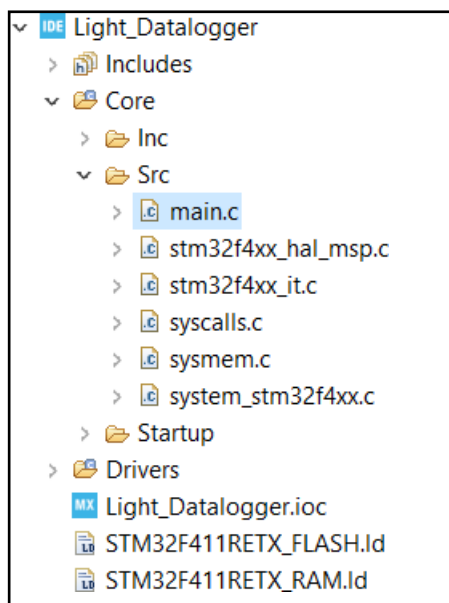
- **Screenshot:**



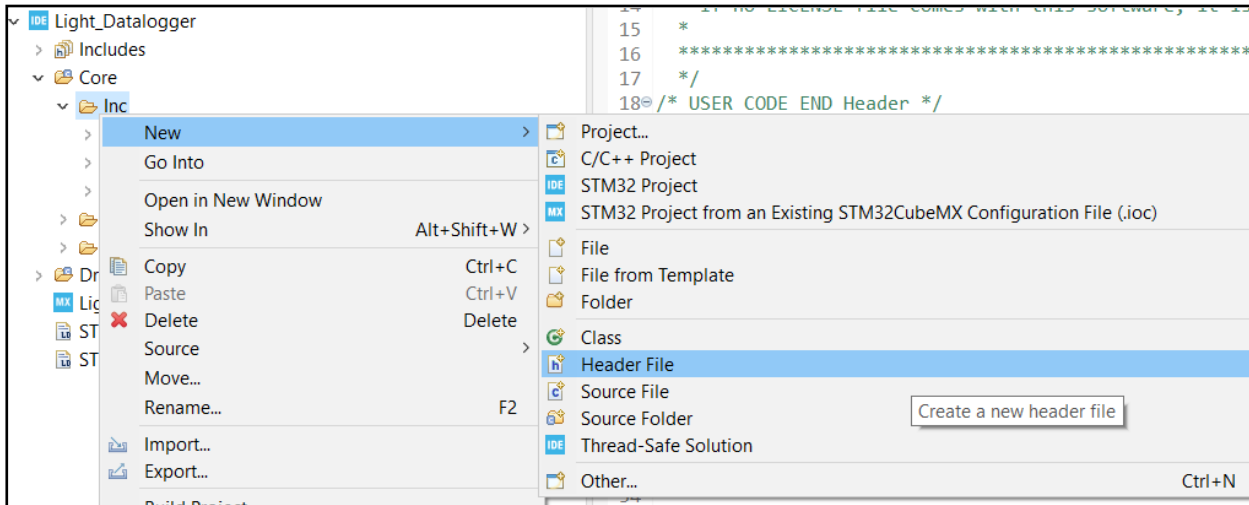
6. Enable **PB8 (I2C1_SCL)** and **PB9 (I2C1_SDA)** for I2C communication, and configure PB6 and PB7 for USART communication between the BL4S5IIOT01A board and the BH1750 light sensor.
7. Next on the left-hand side under **Categories** → **Connectivity**, select I2C1 and enable it.



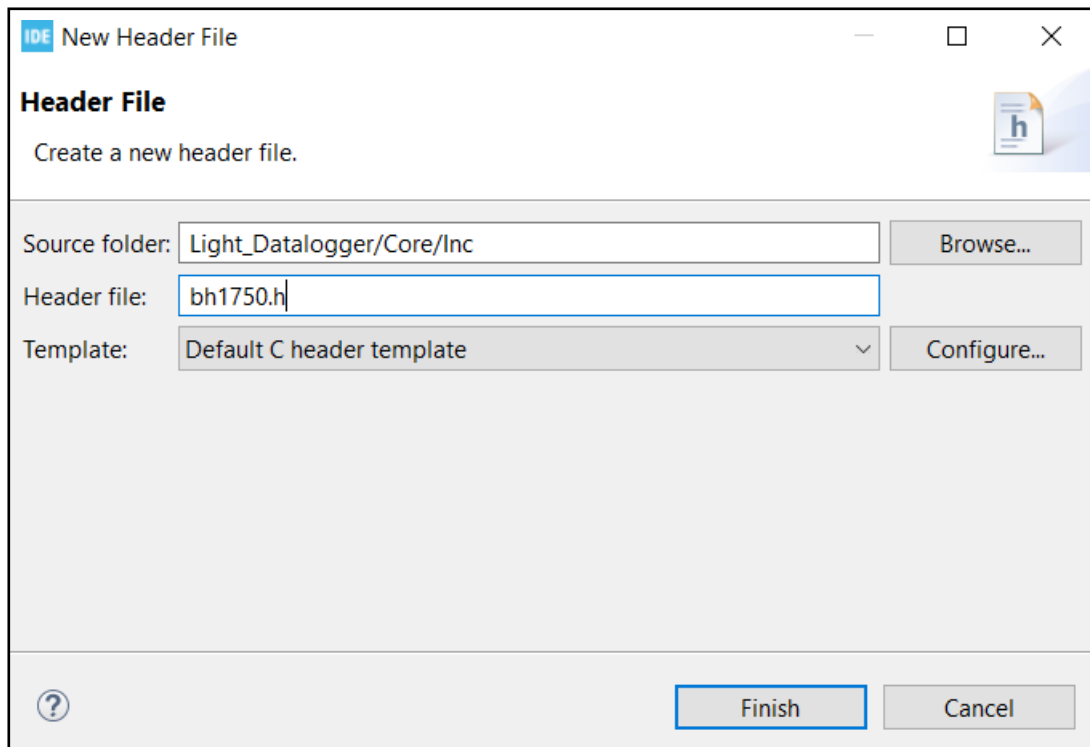
8. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example, I have named my project as (Light_Datalogger) **Light_Datalogger** → **Core** → **Src** → **main.c** (double click to load the code).



9. Now open your project tree **Light_Datalogger**→**Core** →**Inc**. Right click on your **Inc** folder and create a new **Header File**.



10. Name the Header File as **bh1750.h** and select on **Finish**.



11. Below are the code snippets, please put your code in the appropriate places in the **bh1750.h** file.

```

2  #ifndef INC_BH1750_H_
3  #define INC_BH1750_H_
4
5  #include "stdio.h"
6
7  // BH1750 I2C Address
8  #define BH1750_ADDR 0x23 // BH1750 I2C address
9
10
11 // Function prototypes
12 void BH1750_Init(I2C_HandleTypeDef *hi2c);
13 float BH1750_ReadLux(I2C_HandleTypeDef *hi2c);
14
15 // Error Status
16 HAL_StatusTypeDef Transmit_Err, Receive_Err;
17
18
19 // BH1750 initialization
20 void BH1750_Init(I2C_HandleTypeDef *hi2c) {
21     uint8_t cmd[] = {0x10}; // Power on
22     Transmit_Err = HAL_I2C_Master_Transmit(hi2c, BH1750_ADDR << 1, cmd, sizeof(cmd), HAL_MAX_DELAY);
23     if(Transmit_Err != HAL_ERROR){
24         printf("\r\n");
25         printf("BH1750 has been initialized");
26         printf("\r\n");
27     }
28 }
29
30 // Reading Light Intensity from BH1750 sensor
31 float BH1750_ReadLux(I2C_HandleTypeDef *hi2c) {
32     uint8_t data[2];
33     HAL_I2C_Master_Receive(hi2c, BH1750_ADDR << 1, data, sizeof(data), HAL_MAX_DELAY);
34
35     uint16_t lux = (data[0] << 8) | data[1];
36     return (float)lux / 1.2;
37 }
38 #endif

```

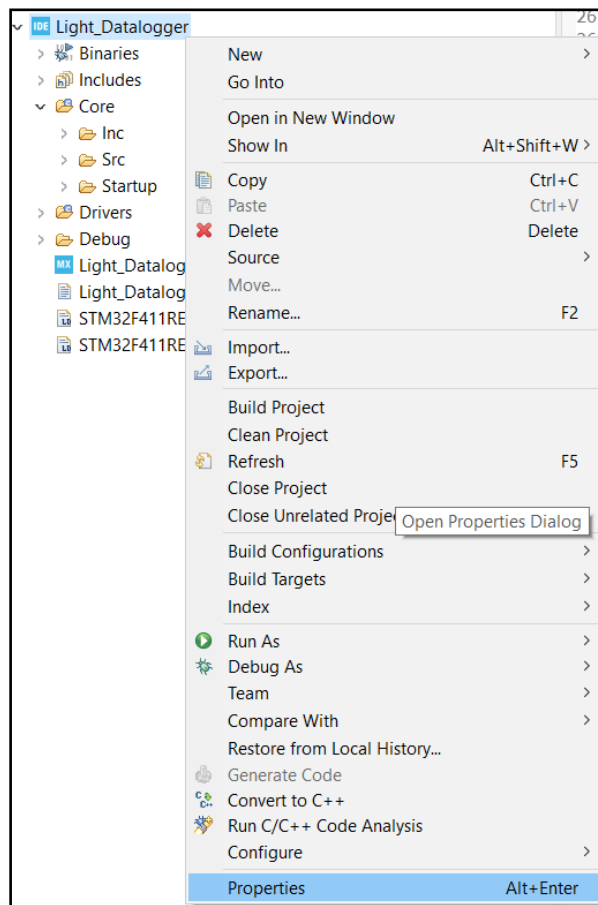
12. Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE uses HAL libraries. Below is the code snippets, please put your code in the appropriate places in the **main.c** file.

```

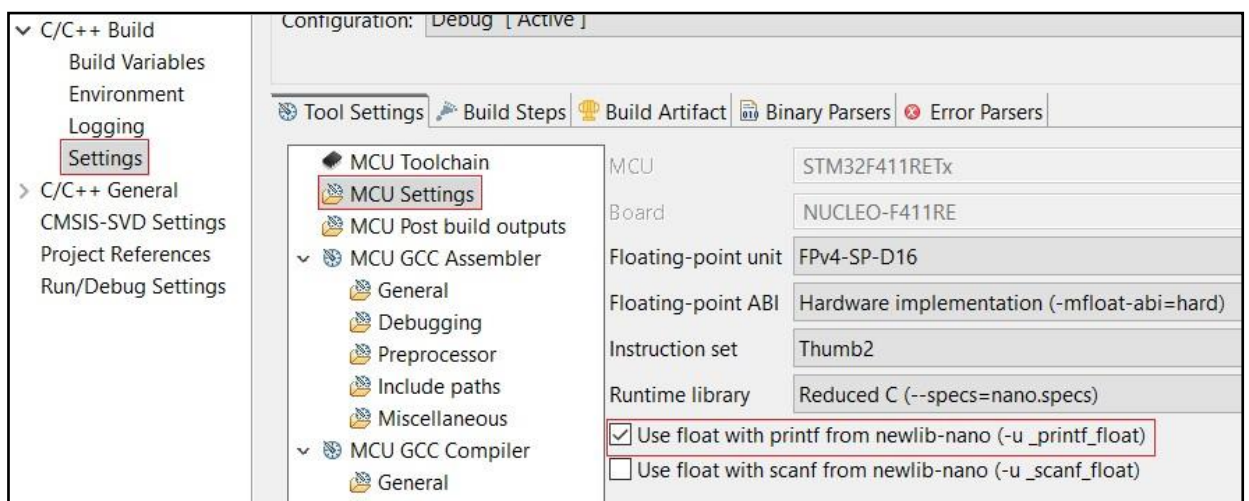
19 /* Includes -----
20 #include "main.h"
21
22/* Private includes -----
23 /* USER CODE BEGIN Includes */
24 #include "bh1750.h"
25 #include "stdio.h"
26 #include "string.h"
27 /* USER CODE END Includes */
28
29/* Private typedef -----
30 /* USER CODE BEGIN PTD */
31
32 /* USER CODE END PTD */
33
34/* Private define -----
35 /* USER CODE BEGIN PD */
36
37 /* USER CODE END PD */
38
39/* Private macro -----
40 /* USER CODE BEGIN PM */
41 #define DATA_INPUT_USER 1
42 #define AXIS_NUMBER 1
43 /* USER CODE END PM */
44
45 /* Private variables -----
46 I2C_HandleTypeDef hi2c1;
47
48 UART_HandleTypeDef huart1;
49
50 /* USER CODE BEGIN PV */
51 float light;
52 float light_buffer[DATA_INPUT_USER * AXIS_NUMBER] = {0};
53 /* USER CODE END PV */
54
55
56
57
58
59
60 /* USER CODE BEGIN PFP */
61 void fill_light_buffer();
62 void Log();
63 /* USER CODE END PFP */
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100 /* USER CODE BEGIN 2 */
101
102 BH1750_Init(&hi2c1);
103
104 /* USER CODE END 2 */
105
106 /* Infinite loop */
107 /* USER CODE BEGIN WHILE */
108 while (1)
109 {
110     Log();
111
112     /* USER CODE END WHILE */
113
114     /* USER CODE BEGIN 3 */
115 }
116 /* USER CODE END 3 */
117 }
118
119
120
121
122
123
124
125
126 /* USER CODE BEGIN 4 */
127 void fill_light_buffer() {
128     for(int i = 0; i < DATA_INPUT_USER; i++){
129         light = BH1750_ReadLux(&hi2c1);
130         light_buffer[AXIS_NUMBER * i] = light;
131         HAL_Delay(3);
132     }
133 }
134
135 void Log() {
136     fill_light_buffer();
137     for(int i = 0; i < DATA_INPUT_USER; i++) {
138         printf("%.2f", light_buffer[AXIS_NUMBER * i]);
139         printf(" ");
140     }
141     printf("\r\n");
142 }
143
144 int __io_putchar(int ch){
145     HAL_UART_Transmit(&huart1, (uint8_t *) &ch, 1, HAL_MAX_DELAY);
146     return ch;
147 }
148 /* USER CODE END 4 */

```

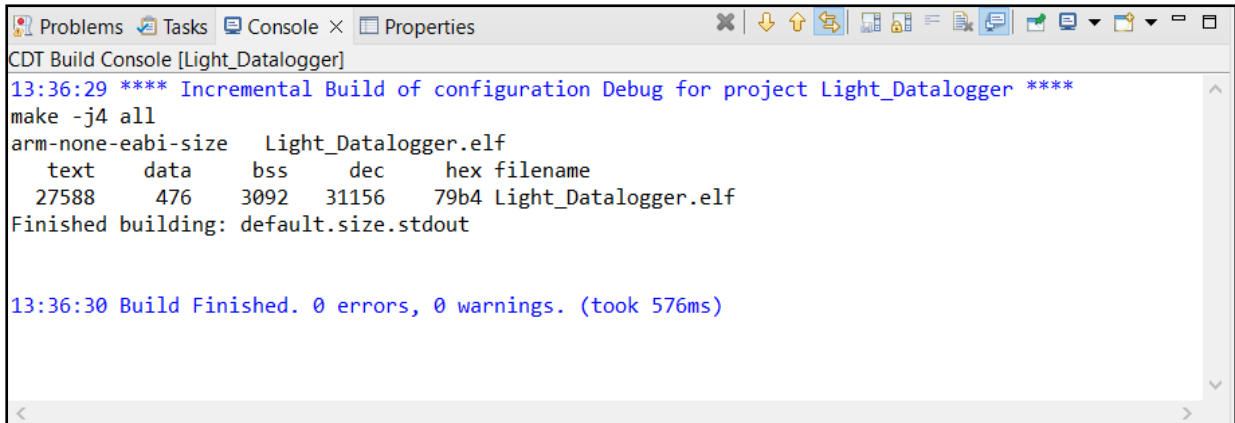

13. Now right click on your project tree, go to **Properties**.



14. After that select **Settings**→**MCU Settings** and enable the float printf.



15. Now click on the build symbol  on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.



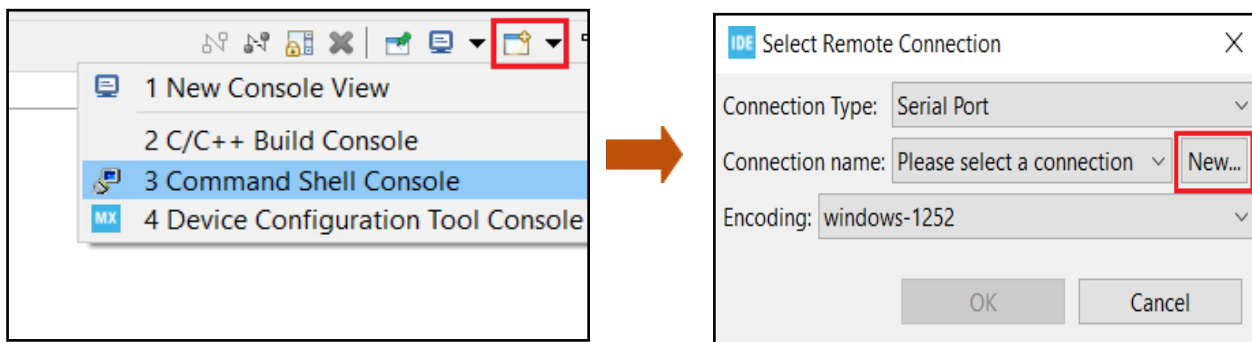
```

CDT Build Console [Light_Datalogger]
13:36:29 **** Incremental Build of configuration Debug for project Light_Datalogger ****
make -j4 all
arm-none-eabi-size  Light_Datalogger.elf
  text    data    bss     dec     hex filename
 27588    476    3092    31156   79b4 Light_Datalogger.elf
Finished building: default.size.stdout

13:36:30 Build Finished. 0 errors, 0 warnings. (took 576ms)

```

16. Next connect your STM32 board with your audio sensor connect to it to your PC and click on the **Debug**  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes
17. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.



18. Click on the **Resume** icon  to run your code. You should be able to see the value of the light sensor in the **Console**

```

Console x Problems
jhg (CONNECTED)
963.33
963.33
963.33
963.33
963.33
963.33
963.33
963.33
963.33
963.33
963.33
963.33

```

Flashlight

```

Console x Problem
jhg (CONNECTED)
25.83
25.83
25.83
25.83
25.83
25.83
25.83
25.83
25.83
25.83
25.83
25.83
25.83

```

Day Light

```

Problems Tasks Console
Serial_Monitor (CONNECTED)
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33
3.33

```

Dark Space

19. Before moving out of the debugging mode, click on **Disconnect**  and close the console then click  on the **Terminate**  icon. You will be moved out of the debugging mode.

Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is for the use of education purpose only.

4d. HTU21D Temperature & Humidity Sensor

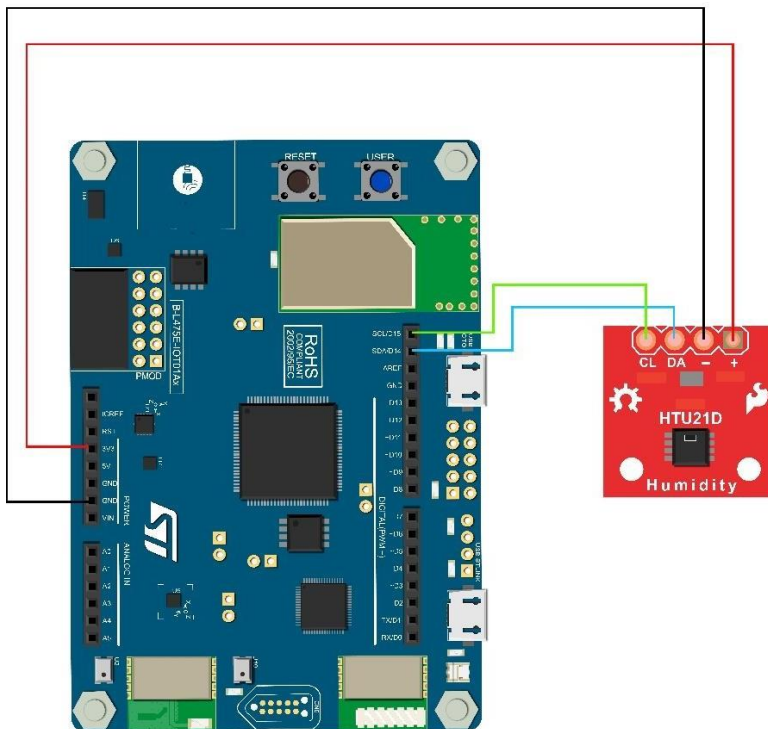
Lab Overview

In this lab, we will interface temperature & humidity (HTU21) sensor on the BL4S5IIOT01A AIoT board.

Objective of the Lab

To interface the temperature & humidity sensor with the BL4S5IIOT01A board, configure I2C communication, acquire sensor data, and display the readings in the serial console.

Experimental Setup



Steps Involved:

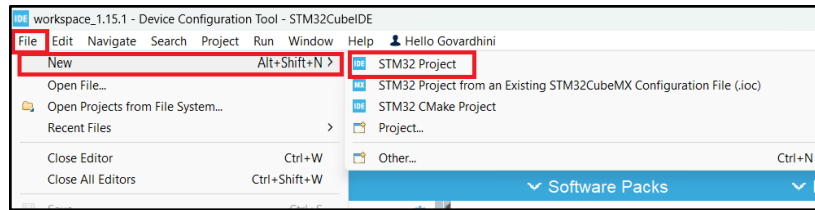
I. Steps Involved for Creating New STM32 project

1. Start a New STM32 Project

- **Steps:**
 - a. Go to File, click on New, and select STM32 Project.
- **Purpose:**

This step allows you to create a new STM32 project using STM32CubeMX for your application.

- **Screenshot:**



2. Target Hardware Selection.

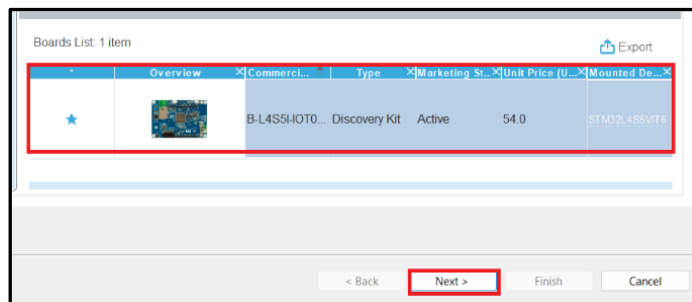
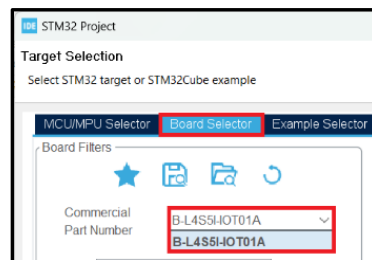
- **Steps:**

- In the Board Selector, type the board part number B-L4S5I-IOT01A in the Commercial Part Number field, select the board, and click Next.

- **Purpose:**

selecting the specific hardware platform (the B-L4S5I-IOT01A board) to tailor the project settings to the correct board specifications.

- **Screenshot:**



3. Project Name and Settings.

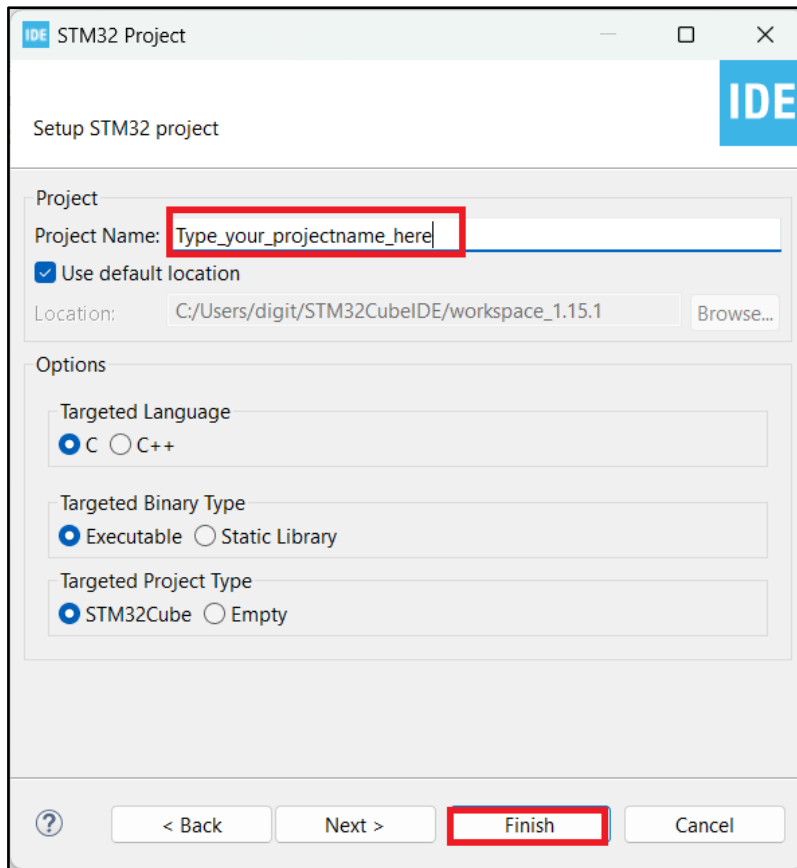
- **Steps:**

- In the Project Name tab, give a name for your project.
- Click Finish.

- **Purpose:**

This step assigns a unique name to your project and finalizes the initial setup, ensuring that the project files are organized and easily identifiable.

- **Screenshot:**



4. Disable Default Mode

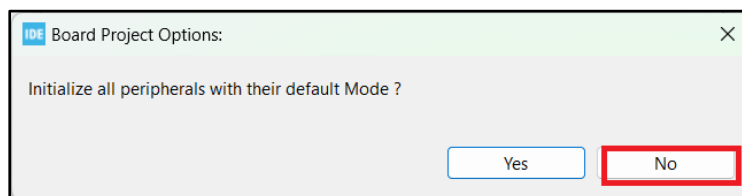
- **Steps:**

- When prompted, select No to disable the default mode.

- **Purpose:**

By disabling the default mode, you are ensuring that the STM32CubeMX doesn't automatically configure peripherals for default settings, allowing you to manually configure the necessary settings for the Wi-Fi module initialization.

- **Screenshot:**



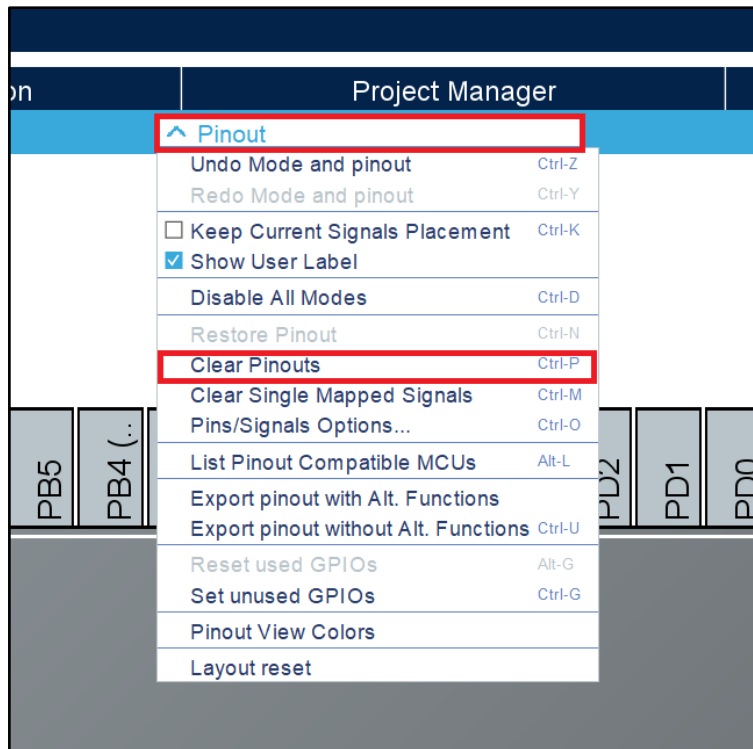
5. Open the .ioc File

- **Steps:**

- The .ioc file for your project will open automatically.
- Go to the Pinout tab and click on Clear Pinout.

- **Purpose:**

The .ioc file contains all the configuration settings for your project. Opening it allows you to configure pins, peripherals, and middleware, such as the Wi-Fi settings required for the project. This step clears the default pinout configuration in STM32CubeMX, providing a clean slate to set up custom pins for the Wi-Fi module and other peripherals required for your project.



- **Screenshot:**

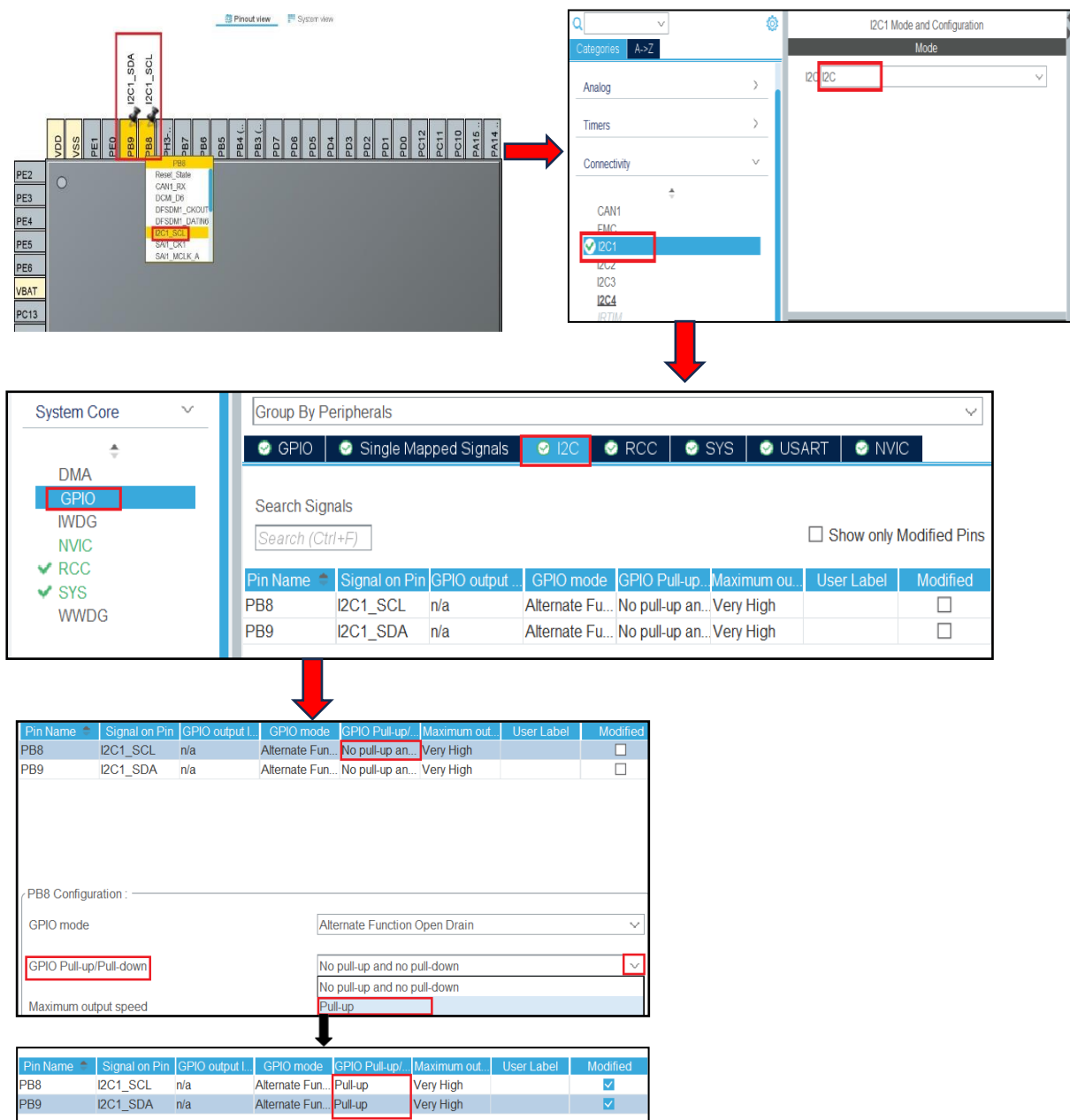
6. Enable I2C1_SCL(PB8) and I2C1_SDA(PB9)

- **Steps:**

- Set **PB8** Pin as I2C1_SCL and **PB9** Pin as I2C1_SDA
- Shift to the **Categories** tab, go to **Connectivity**, and select **I2C1** to enable it.
- After selecting I2C option, shift to the **Categories** tab, go to **System Core**, select **GPIO** and change the GPIO pin **PB8** and **PB9** as **Pull-up**

- **Purpose:** This pin facilitates the use of the I2C communication protocol to read sensor data.

- **Screenshot:**



The screenshots illustrate the steps to configure I2C pins (PB8 and PB9) for I2C1 communication. The first screenshot shows the pinout view with PB8 assigned to I2C1_SDA and PB9 to I2C1_SCL. The second screenshot shows the I2C1 Mode and Configuration window with the Mode set to I2C. The third screenshot shows the System Core peripherals list with I2C selected. The fourth screenshot shows the detailed pin configuration for PB8 and PB9, where the GPIO Pull-up/Pull-down is set to Pull-up.

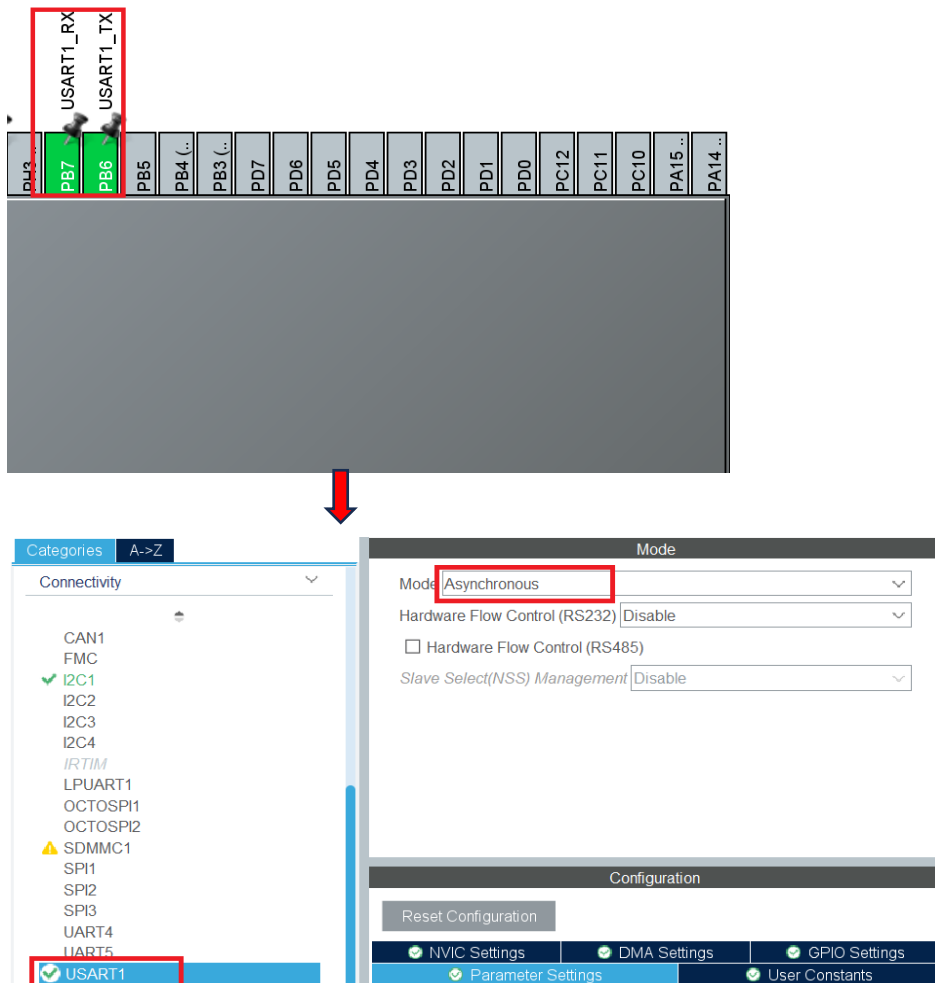
Pin Name	Signal on Pin	GPIO output	GPIO mode	GPIO Pull-up/	Maximum ou...	User Label	Modified
PB8	I2C1_SCL	n/a	Alternate Fu...	No pull-up an...	Very High		☐
PB9	I2C1_SDA	n/a	Alternate Fu...	No pull-up an...	Very High		☐

Pin Name	Signal on Pin	GPIO output	GPIO mode	GPIO Pull-up/	Maximum out...	User Label	Modified
PB8	I2C1_SCL	n/a	Alternate Fun...	Pull-up	Very High		☒
PB9	I2C1_SDA	n/a	Alternate Fun...	Pull-up	Very High		☒

7. Enable USART1 Pins (PB7 and PB8 for TX and RX)

- **Steps:**
 - a. Set **PB7** (USART1_RX) and **PB6** (USART1_TX) for UART communication in the **Pinout & Configuration** tab.
 - b. Shift to the **Categories** tab, go to **Connectivity**, select **USART1**, and set the **Mode** to **Asynchronous**.
- **Purpose:** These pins are used for serial communication between the board and the Wi-Fi module. Configuring USART1 in **Asynchronous** mode allows debugging and status messaging.

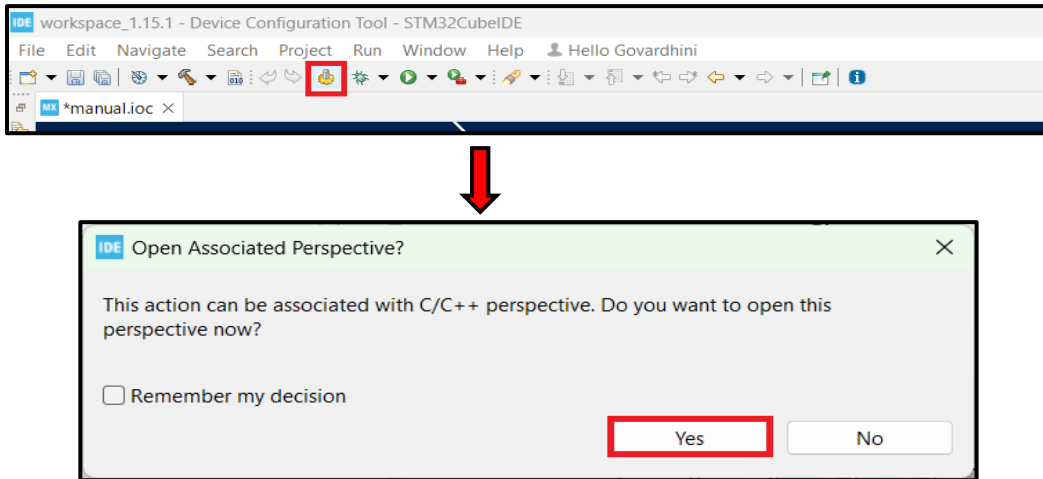
- **Screenshot:**



8. Generate Code

- **Steps:**
 - a. After completing the pin configurations, go to the **Project** tab in STM32CubeMX and click on **Generate Code**.
- **Purpose:** This step generates the code with all the necessary configurations for peripheral initialization and pin setup in the **.ioc** and **main.c** files, ensuring that the project is ready for further coding.

- **Screenshot:**



II. Steps Involved for Interfacing Temperature & Humidity Sensor in STM32CubeIDE

1. Create Folders in the project.

- **Steps:**

- a. Open your project tree **Interfacing_HTU21DF_Sensor**. Right click on your **Inc** folder and paste **Header File** and name as **HTU21.h**
- b. Right click on the **Src** folder and create a new **Source File** and name it as **HTU21.c**

NB: The code for **HTU21.h** and **HTU21.c** will be there in **folder** section provided along with the manuals.

- **Purpose:** The purpose of these steps is to set up the project structure for interfacing with the temperature & humidity sensor. By creating a header files and source files within the **project** directory, developers can organize code specific to the sensor.

Source File (HTU21.c):

```
1
2 #include <HTU21.h>
3
4 void HTU21_Reset(){
5     uint8_t Data=HTU21_RESET;
6     HAL_Delay(15);
7     HAL_I2C_Master_Transmit(&hi2c1, HTU21_I2CADDR, &Data, 1, 1000);
8     HAL_Delay(15);
9 }
10
11 char HTU21_Init(){
12     uint8_t check;
13     uint8_t Data=HTU21_READREG;
14
15     HAL_I2C_Master_Transmit(&hi2c1, HTU21_I2CADDR, &Data, 1, 1000);
16     HAL_I2C_Master_Receive(&hi2c1, HTU21_I2CADDR, &check, 1, 1000);
17     return check;
18 }
19
20 float HTU21_GetTemp()
21 {
22     uint8_t Temp_Data[2];
23     uint8_t Data=HTU21_READTEMP;
24
25     HAL_I2C_Master_Transmit(&hi2c1, HTU21_I2CADDR, &Data, 1, 1000);
26     HAL_I2C_Master_Receive(&hi2c1, HTU21_I2CADDR, Temp_Data, 2, 1000);
27
28     uint16_t Raw_Temp =(uint16_t) (Temp_Data[0] << 8 | (Temp_Data[1]& 0b11111100));
29
30     float temp = Raw_Temp;
31     temp *= 175.72f;
32     temp /= 65536.0f;
33     temp -= 46.85f;
34     return temp;
35 }
36
37 float HTU21_GetHum()
38 {
39     uint8_t Hum_Data[2];
40     uint8_t Data=HTU21_READHUM;
41
42     HAL_I2C_Master_Transmit(&hi2c1, HTU21_I2CADDR, &Data, 1, 1000);
43     HAL_I2C_Master_Receive(&hi2c1, HTU21_I2CADDR, Hum_Data, 2, 1000);
44
45     uint16_t Raw_Hum =(uint16_t) (Hum_Data[0] << 8 | (Hum_Data[1]& 0b11110000));
46
47     float hum = Raw_Hum;
48     hum *= 175.72f;
49     hum /= 65536.0f;
50     hum -= 46.85f;
51     return hum;
52
53
54
55 }
```

Main.c file:

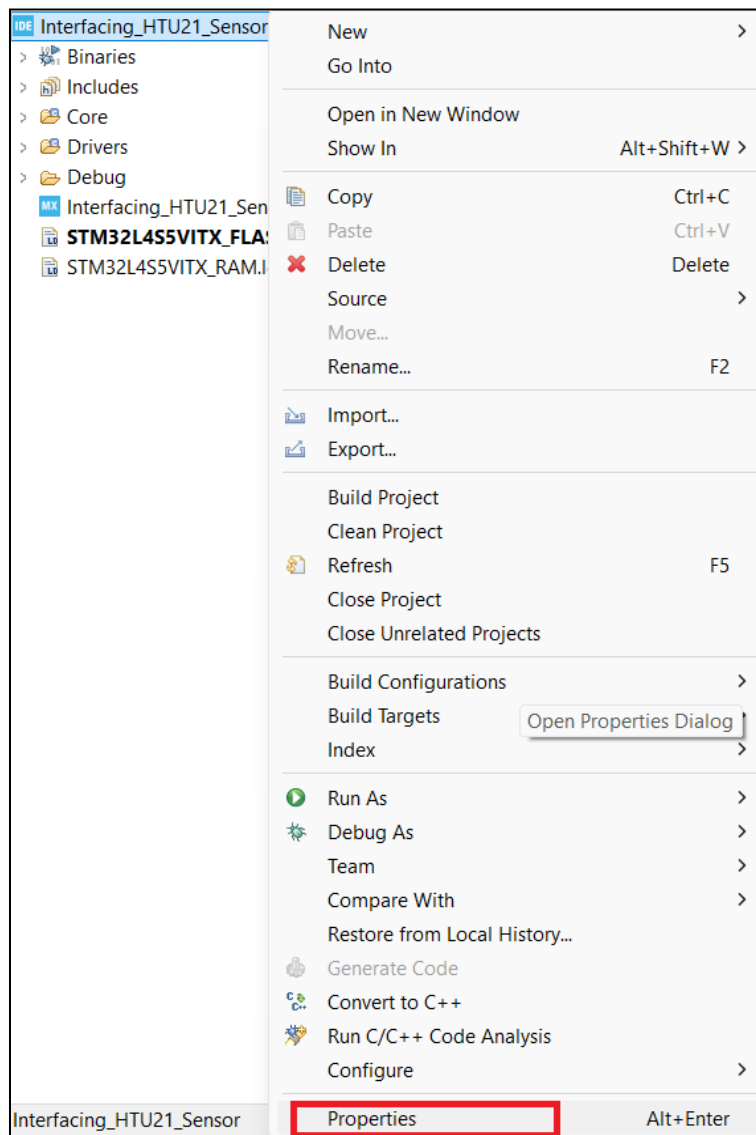
```

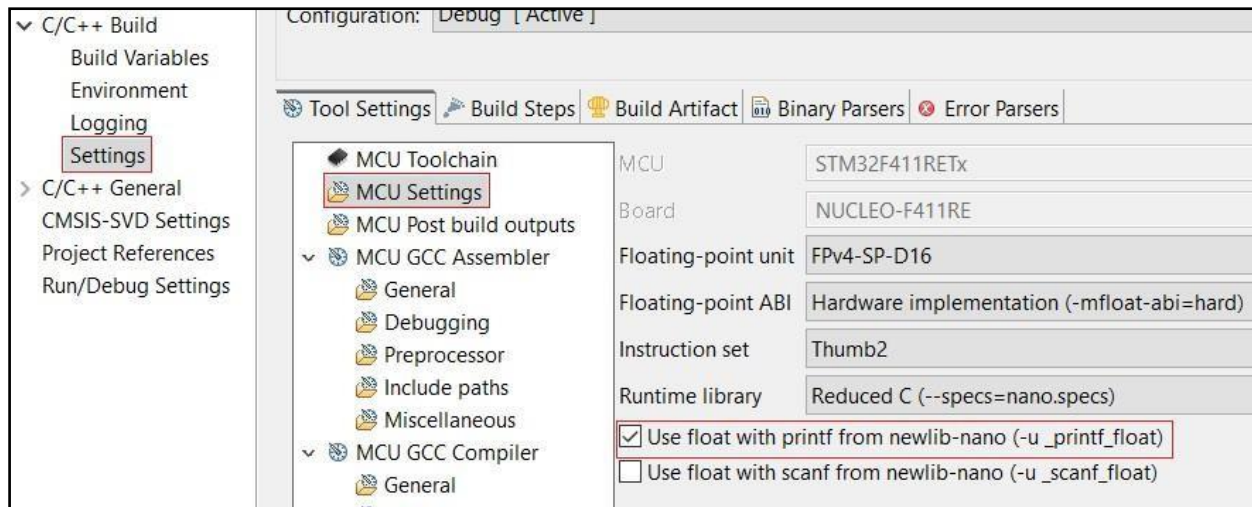
18 /* USER CODE END Header */
19 /* Includes ----- */
20 #include "main.h"
21
22 /* Private includes ----- */
23 /* USER CODE BEGIN Includes */
24 #include "stdio.h"
25 #include "string.h"
26 #include "HTU21.h"
27 /* USER CODE END Includes */
28
29 /* Private typedef ----- */
30 /* USER CODE BEGIN PTD */
31
32 /* USER CODE END PTD */
33
34 /* Initialize all configured peripherals */
35 MX_GPIO_Init();
36 MX_USART2_UART_Init();
37 MX_I2C1_Init();
38 /* USER CODE BEGIN 2 */
39 HAL_Delay(1000);
40 HTU21_Reset();
41
42 while (HTU21_Init() != 0x2) {
43     printf("Initializing HTU21 Sensor...\r\n");
44     HAL_Delay(100);
45 }
46 printf("HTU21 Sensor Initialization Success!\r\n");
47 /* USER CODE END 2 */
48
49 /* Infinite loop */
50 /* USER CODE BEGIN WHILE */
51 while (1)
52 {
53     float HTU21_Temp_Value = HTU21_GetTemp();
54     printf("Temperature = %.2f\r\n", HTU21_Temp_Value);
55     HAL_Delay(300);
56
57     float HTU21_Hum_Value = HTU21_GetHum();
58     printf("Humidity = %.2f\r\n", HTU21_Hum_Value);
59     HAL_Delay(300);
60 }
61 /* USER CODE END WHILE */
62
63 /* USER CODE BEGIN 3 */
64
65 /* USER CODE END 3 */

```

2. Configure settings in Properties.

- **Steps:**
 - Right click on your project tree, go to **Properties**.
 - Select **Settings - MCU Settings** and enable the **float printf**.
 - Click on  the build symbol on the top left corner on your Cube IDE.
- **Purpose:** The step involves configuring the necessary settings in the properties, building the project, and verifying if there are any errors.
- **Screenshot:**





```
make -j4 all
arm-none-eabi-size  Interfacing_HTU21DF_Sensor.elf
  text    data    bss     dec     hex filename
 27612    476    2060    30148    75c4 Interfacing_HTU21DF_Sensor.elf
Finished building: default.size.stdout
```

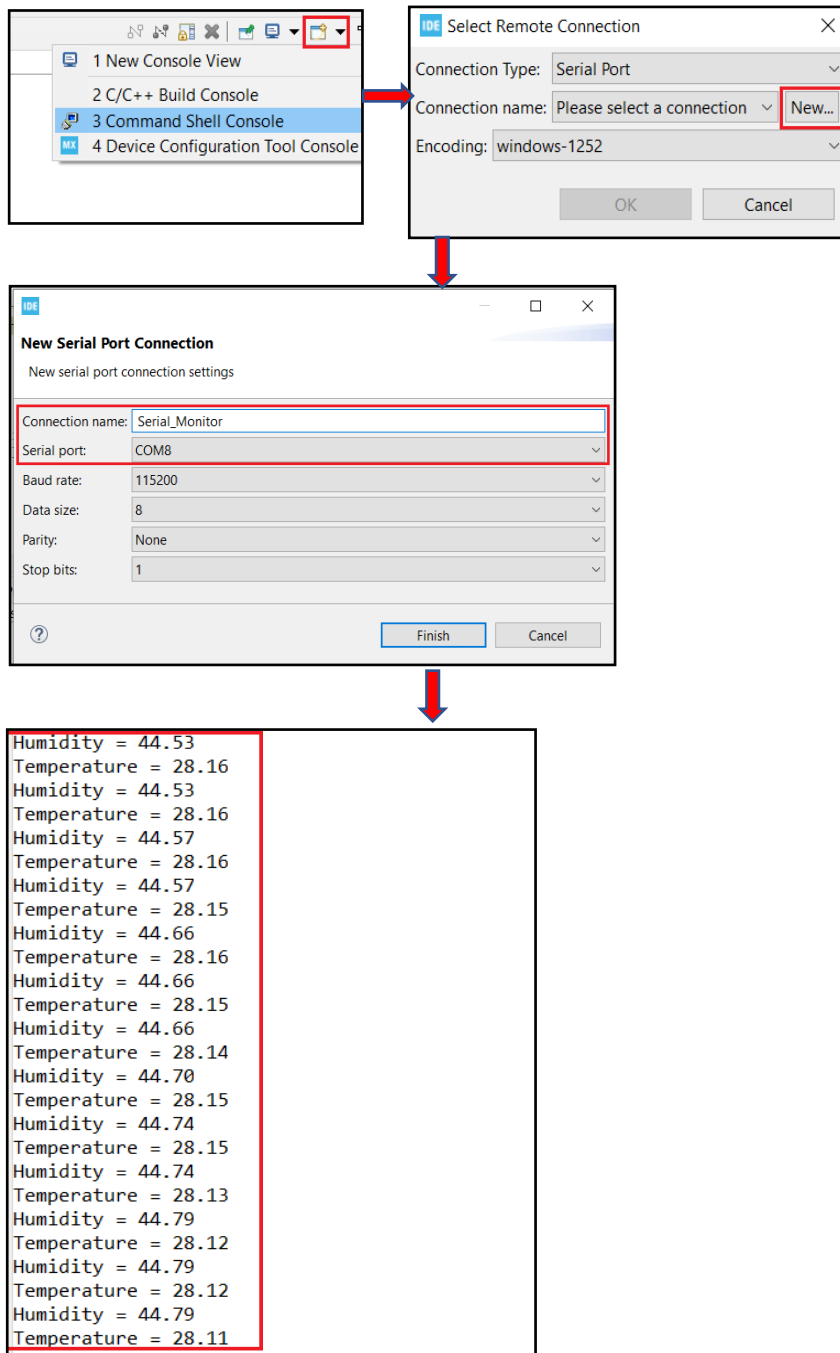
14:33:44 Build Finished. 0 errors, 0 warnings. (took 736ms)

3. Generate Code

- Steps:

- After adding the necessary files, go to the **Project** tab in STM32CubeMX and click on **Generate Code** again.
- Ensure that your board is connected to the PC via ST-Link.
- Run the project by selecting **Debug** and then clicking **Run Debug**. Once it shows "Download Verified Successfully," proceed to create a serial monitor to view the data by selecting **Command Shell Console**.
- When you select **Command Shell Console**, choose **Serial Port** as the connection type. Click on **New**, provide an appropriate name for the console, and select the correct port. You can check the port associated with ST-Link by going to **Device Manager**, expanding the **Ports** section, and identifying the port connected to ST-Link.
- Click **Finish**, then click **OK** to open the serial monitor. Next, in the navigation bar, select the **Resume** option. After resuming, the output will be displayed in the serial console. Ensure that the hotspot is turned on for the device with the configuration you provided in the previous step.

- **Purpose:** This step generates the project files with all the added configurations, ensuring the project is fully initialized with the correct settings. It integrates all changes, including added files, into the project and makes the code ready for further development and compilation.
- **Screenshot:**



Final Observation:

- **Accurate Sensor Readings:** The sensor provided reliable and precise measurements.

- **Successful I2C Communication:** The I2C protocol facilitated efficient data transfer.
- **Effective Code Implementation:** The code was well-structured and optimized for performance.
- **Clear Data Display:** The sensor data was accurately displaying on the serial console.

4e. Air Quality Sensor

Lab Overview:

In this lab, we will connect air quality sensor to the BL4S51-IOT1A microcontroller. Each time the ADC finishes a conversion, it will trigger an interrupt, and the converted value will be stored in a variable for further use.

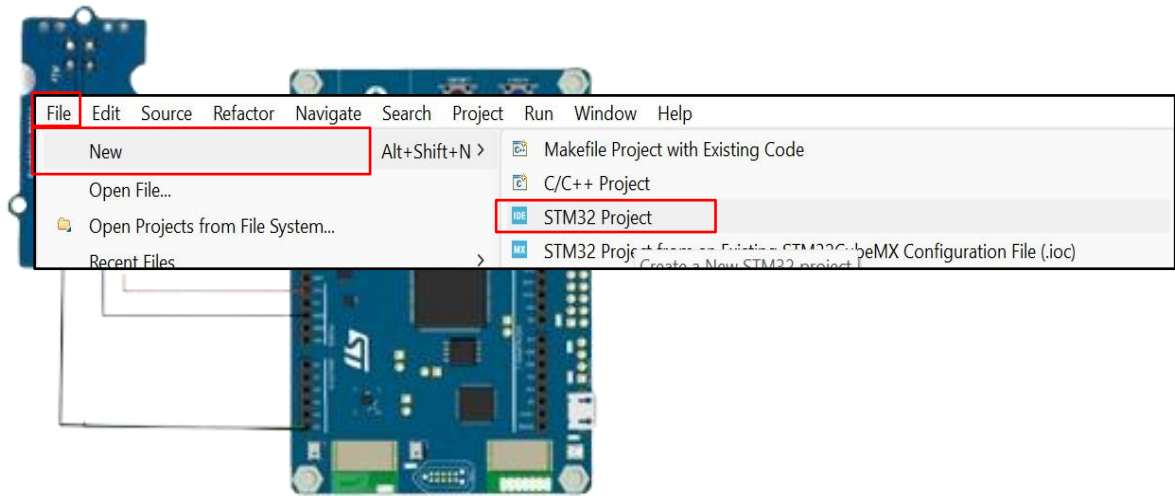
Lab Objective:

The objective of this experiment is to interface an air quality sensor to an BL4S51-IOT1A microcontroller and configure the ADC in interrupt mode. Every time the ADC completes a conversion, the ADC will generate an internal interrupt, the converted value will be collected from the ADC data registers and stored into a variable.

Requirements:

1. STM32 Cube IDE software.
2. Air Quality sensor (Analog V1.3).
3. BL4S51-IOT1A Microcontroller.
4. Micro USB Cable for the microcontroller.
5. Jumper Wires.
6. PC or Laptop.

Experimental Setup:

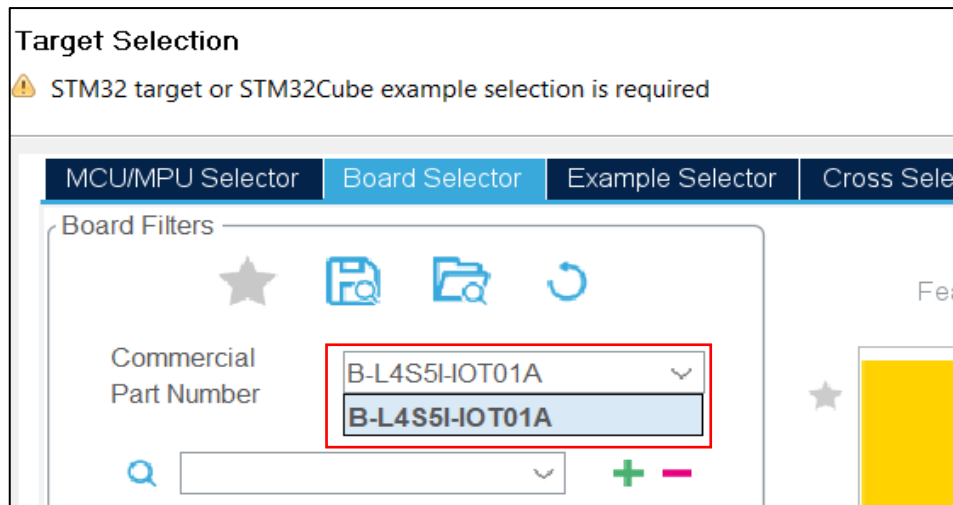


NOTE: Analog-to-Digital Converter (ADC) converts continuous analog signals into discrete digital values that can be processed by a microcontroller. It allows the microcontroller to read and interpret real-world analog signals, such as audio, temperature or light levels

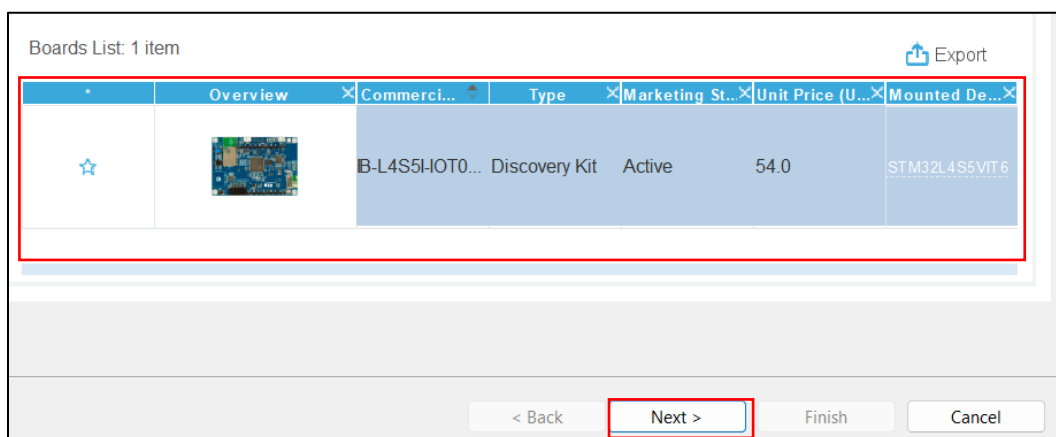
Steps Involved:

1. Click on **File**→**New**→**STM32 Project** to start your project on Cube IDE.

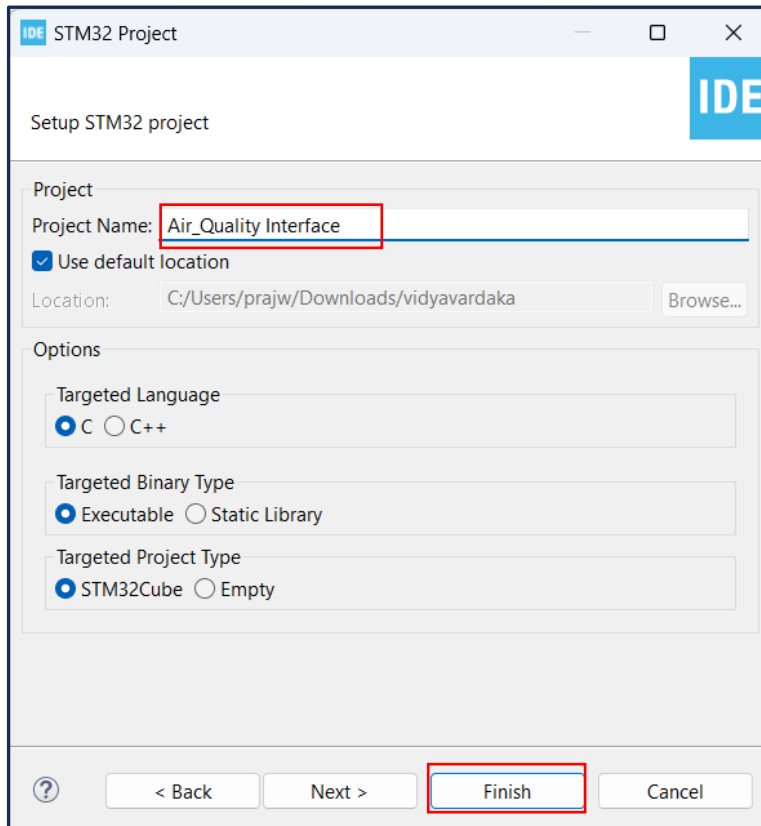
2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.



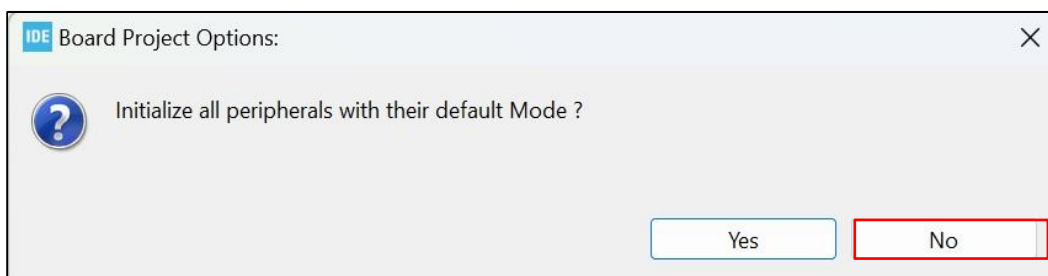
3. After this on the right-hand side of the window, under **Board List** you will see the board you have selected. Click on the board and then click on **Next**.



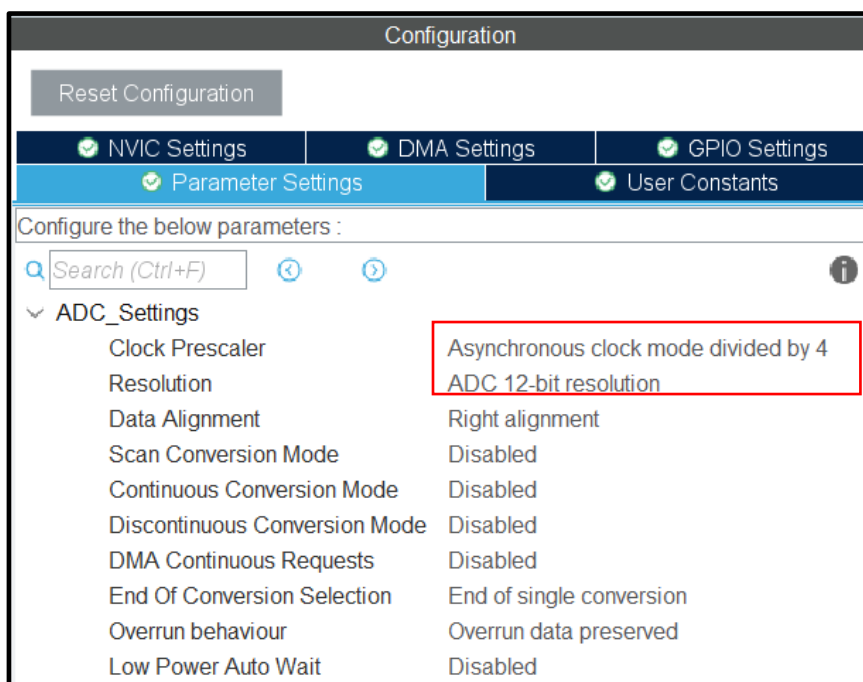
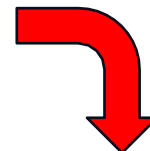
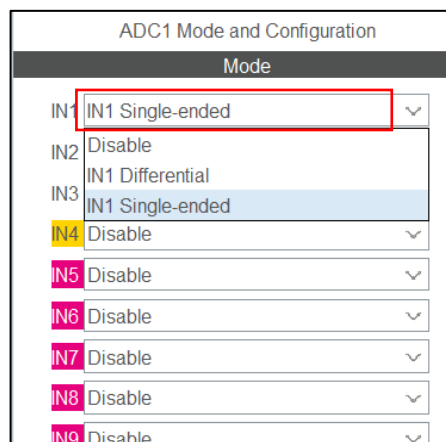
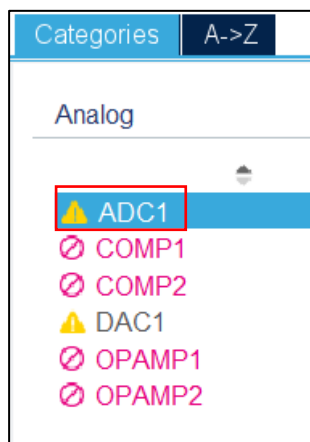
4. In the next window you need to give your project a name, rest of the things will remain by default as it is for now. Click on **Next**.



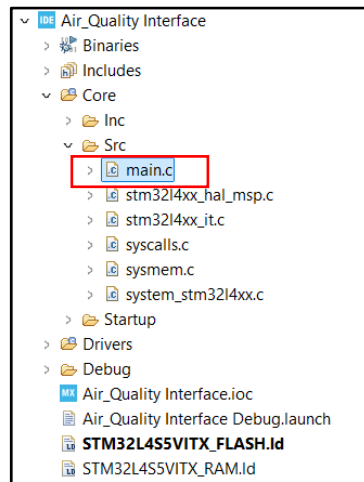
5. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



6. Next on the left-hand side **Categories** → **Analog** select **ADC1** then select **IN1** as IN1 Single-ended under **Mode**. Under Configuration select **Parameter settings** under ADC_Settings select the clock prescaler to by **Asynchronous clock mode divided by 4** and then change the resolution to **ADC 12-bit resolution**.



7. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example I have named my project as (ADC_Interrupt) **ADC_Polling**→**Core** →**Src**→**main.c** (double click to load the code).



8. Cube IDE automatically generates the initialization code based on the configurations you have done. Cube IDE used HAL libraries. Below is the code snippets, please put your code in the appropriate places in the **main.c** file.

```
23 /* USER CODE BEGIN Includes */
24 #include "stdio.h"
25 #include "string.h"
26 /* USER CODE END Includes */
```

```
47 /* USER CODE BEGIN PV */
48 int value = 0;
49 /* USER CODE END PV */
```

```
57 /* USER CODE BEGIN PFP */
58 int __io_putchar(int);
59 /* USER CODE END PFP */
```

```
101 /* USER CODE BEGIN WHILE */
102 while (1)
103 {
104     Value=HAL_ADC_GetValue(&hadc1);
105     printf("the value is= %d",Value);
106     printf("\r\n");
107     HAL_ADC_Start(&hadc1);
108     HAL_Delay(100);
109 }
110 /* USER CODE END WHILE */
```

NOTE: entire main.c file is available in the attached code folder, you can copy it from that directory

9. Now click on the  symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

```
474 /* USER CODE BEGIN 4 */
475 int __io_putchar(int ch)
476 {
477     HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
478     return ch;
479 }
480 /* USER CODE END 4 */
481
```

Problems Tasks Console × Properties
CDT Build Console [ADC_polling]
24440 112 2160 26712 6858 ADC_polling.elf
Finished building: default.size.stdout
Finished building: ADC_polling.list
10:01:37 Build Finished. 0 errors, 0 warnings. (took 7s.19ms)

10. Next connect your STM32 board with your air quality sensor connect to it to your PC and click on the **Debug**  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.
11. Click on the **Resume**  icon to run your code. You should be able to see the value of air quality sensor sensor.

Console × Problems Executables Memory
vbnm (CONNECTED)
The value is=493
The value is=896
The value is=857
The value is=447
The value is=524
The value is=911
The value is=826
The value is=435
The value is=556
The value is=892

12. To move out of the debugging mode, click on the **Terminate**  icon. You will be moved out of the debugging mode.

Note: All important steps and parts are highlighted with a red colour box for the proper understanding of the user. This document is for the use of education purpose only.

Lab 5: Wi-Fi Initialization for IoT Projects

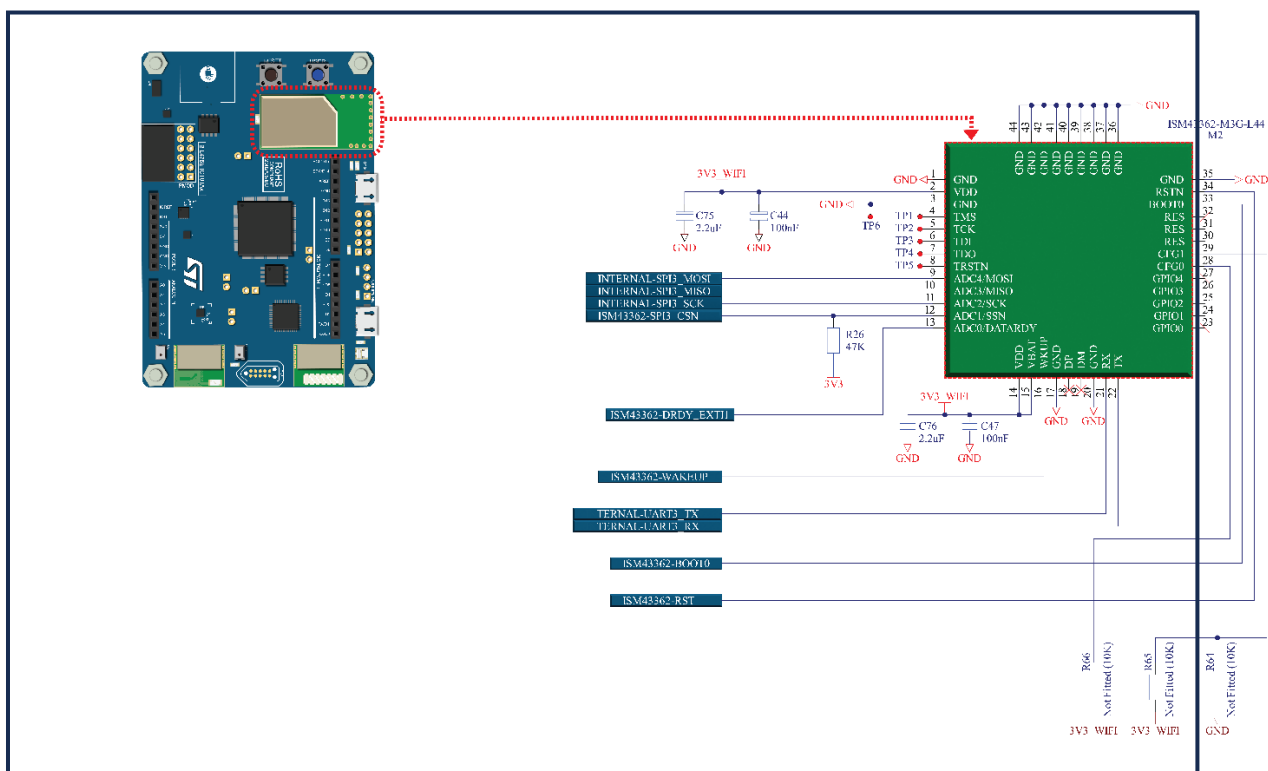
Lab Overview

To enable cloud communication, the IoT device needs to connect to a network. In this lab, we will initialize the Wi-Fi module on the BL4S5IIOT01A AIoT board to connect it to a local Wi-Fi network.

Objective of the Lab

To learn how to initialize the Wi-Fi module, configure network credentials, and establish a connection between the IoT device and a local Wi-Fi network.

Experimental Setup



Steps Involved:

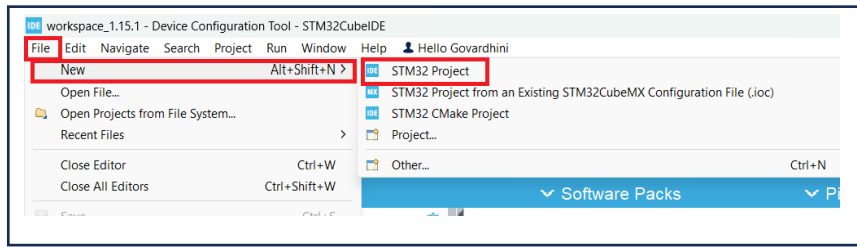
I. Steps Involved for Creating New STM32 project

1. Start a New STM32 Project

- **Steps:**
 - b. Go to File, click on New, and select STM32 Project.
- **Purpose:**

This step allows you to create a new STM32 project using STM32CubeMX for your application.

- **Screenshot:**



2. Target Hardware Selection.

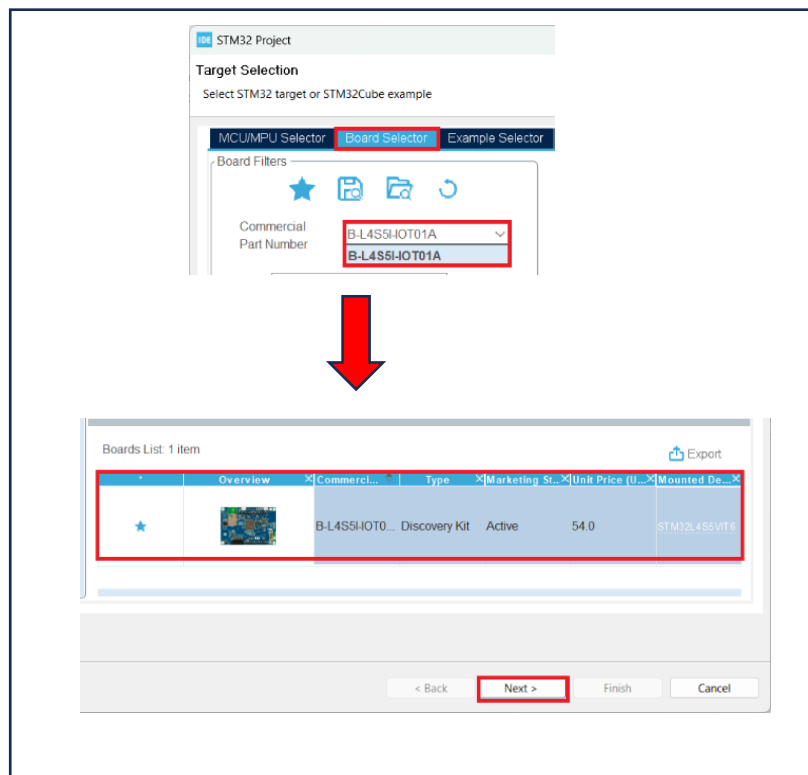
- **Steps:**

- In the Board Selector, type the board part number B-L4S5I-IOT01A in the Commercial Part Number field, select the board, and click Next.

- **Purpose:**

selecting the specific hardware platform (the B-L4S5I-IOT01A board) to tailor the project settings to the correct board specifications.

- **Screenshot:**



3. Project Name and Settings.

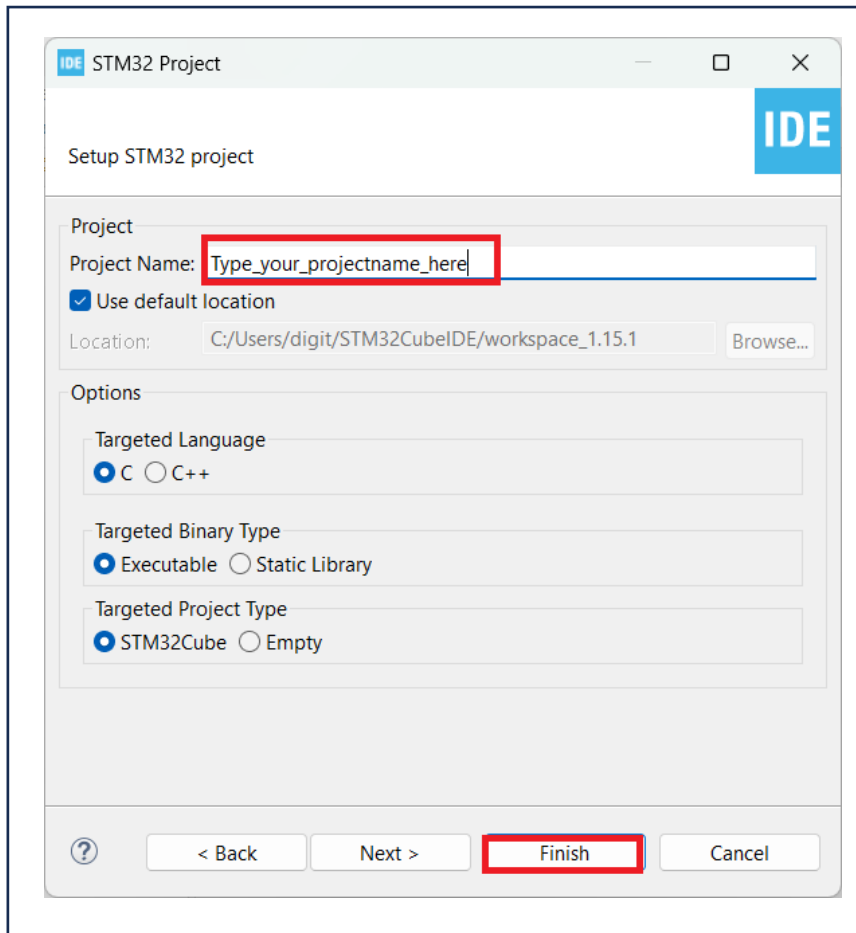
- **Steps:**

- In the Project Name tab, give a name for your project.
- Click Finish.

- **Purpose:**

This step assigns a unique name to your project and finalizes the initial setup, ensuring that the project files are organized and easily identifiable.

- **Screenshot:**



4. Disable Default Mode

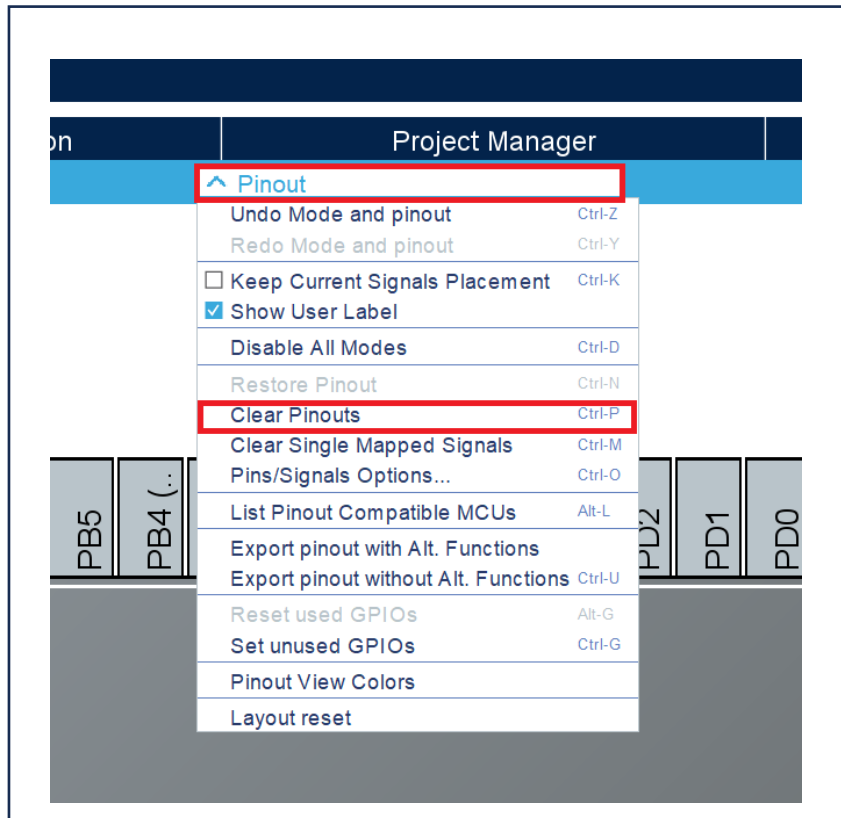
- **Steps:**

- c. When prompted, select No to disable the default mode.

- **Purpose:**

By disabling the default mode, you are ensuring that the STM32CubeMX doesn't automatically configure peripherals for default settings, allowing you to manually configure the necessary settings for the Wi-Fi module initialization.

- **Screenshot:**

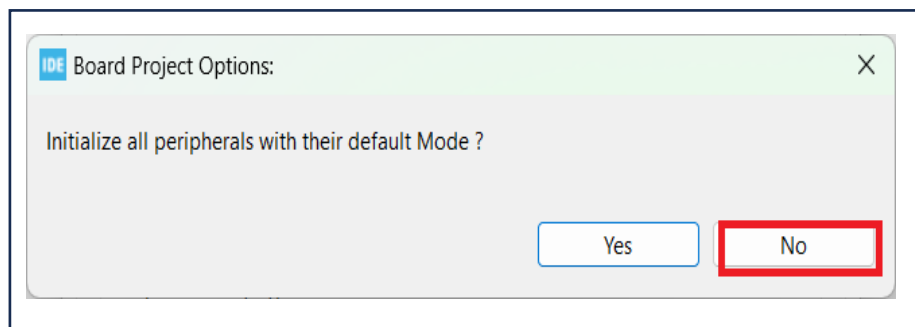


5. Open the .ioc File

- **Steps:**
 - The .ioc file for your project will open automatically.
 - Go to the Pinout tab and click on Clear Pinout.
- **Purpose:**

The .ioc file contains all the configuration settings for your project. Opening it allows you to configure pins, peripherals, and middleware, such as the Wi-Fi settings required for the project. This step clears the default pinout configuration in STM32CubeMX, providing a clean slate to set up custom pins for the Wi-Fi module and other peripherals required for your project.

- **Screenshot:**



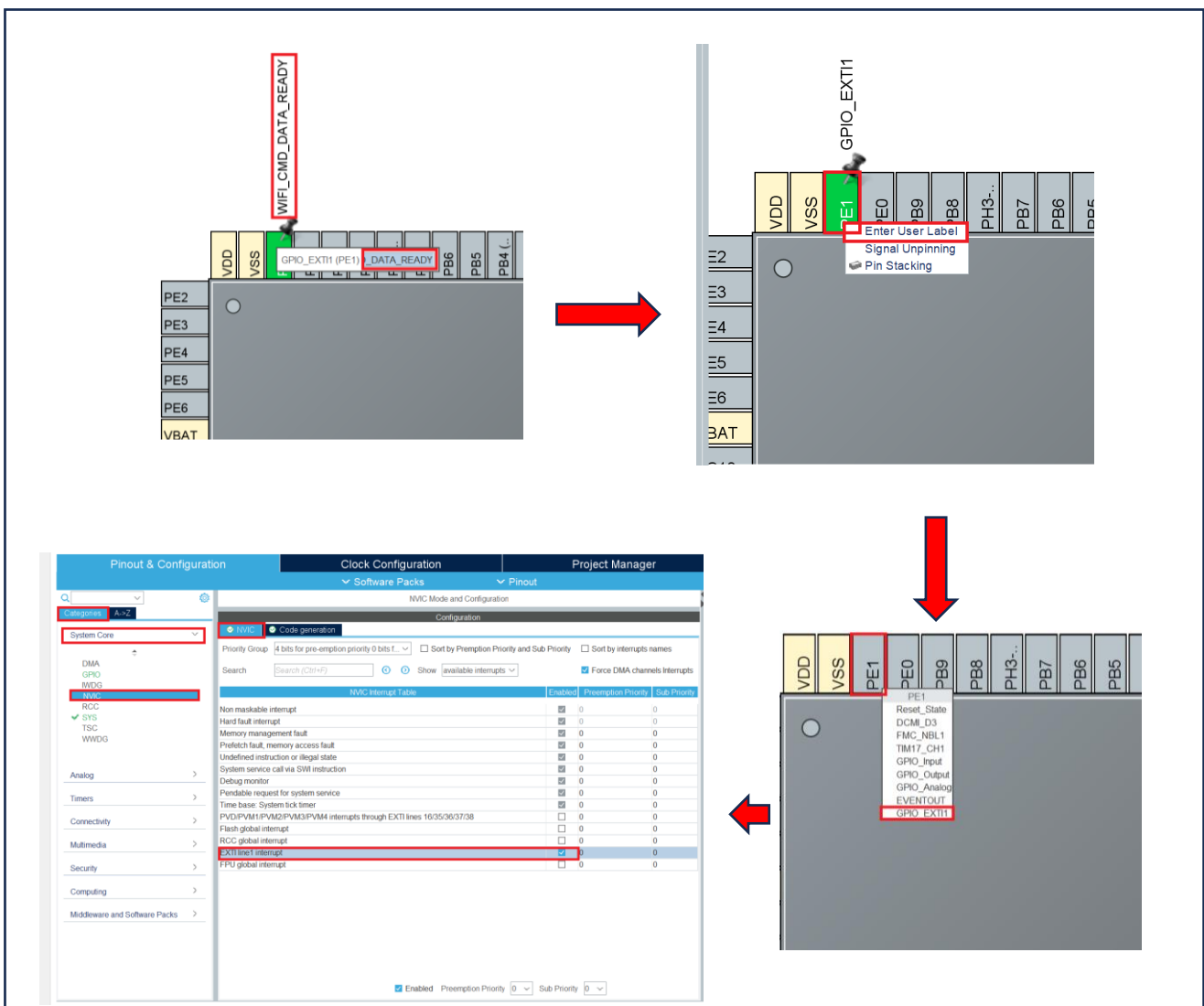
6. Enable WIFI_CMD_DATA_READY Pin (PE1)

- Steps:

- Set **PE1** as **GPIO_EXTI1** in the **Pinout & Configuration** tab and rename it as **WIFI_CMD_DATA_READY**.
- Shift to the **Categories** tab, go to **System Core**, and select **NVIC** to enable **EXTI Line 1 Interrupt**.

- Purpose:** This pin is used to signal when the Wi-Fi module's CMD/Data is ready for communication. Enabling the interrupt ensures that we can handle state changes efficiently.

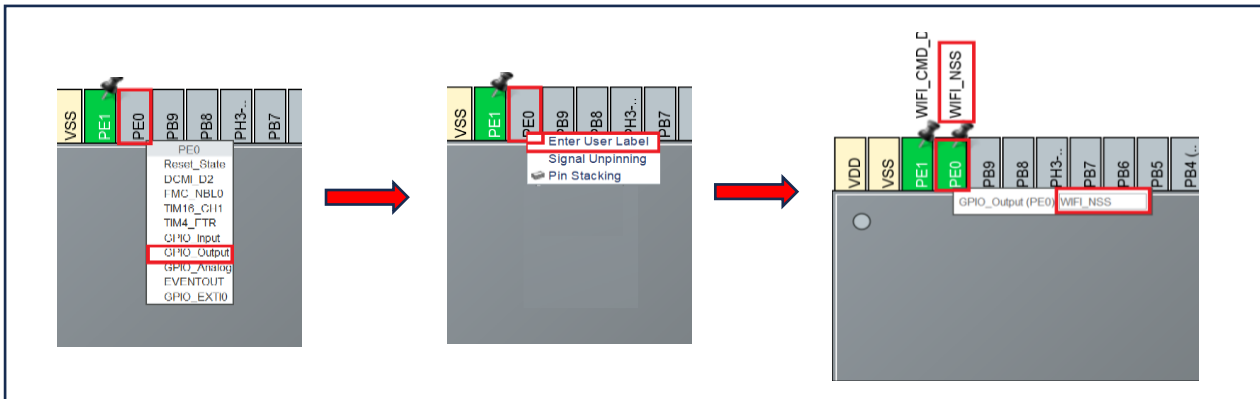
- Screenshot:



II. Steps Involved for Wi-Fi Initialization in .ioc File.

1. Enable WIFI_NSS Pin (PE0)

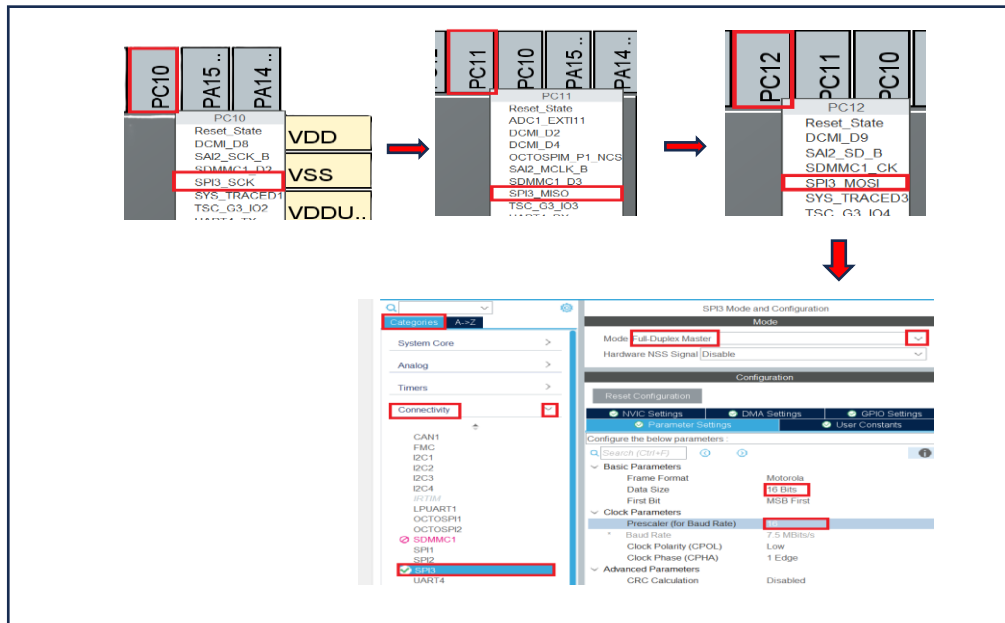
- **Steps:**
 - a. Set **PE0** as **GPIO Output** in the **Pinout & Configuration** tab.
- **Purpose:** This pin is used as **Chip Select (CS)** for SPI communication, enabling communication with the Wi-Fi module during data transfer.
- **Screenshot:**



2. Enable SPI3 Pins (PC12, PC11, PC10 for MOSI, MISO, and SCK)

- **Steps:**
 - a. Set **PC12** (SPI3_MOSI), **PC11** (SPI3_MISO), and **PC10** (SPI3_SCK) for SPI communication in the **Pinout & Configuration** tab.
 - b. Shift to the **Categories** tab, go to **Connectivity**, and select **SPI3**. Configure the **Mode** to **Full-Duplex Master**.
 - c. In the **SPI3 Configuration** settings, change:
 - **Data Size** to **16 bits** (instead of the default 4 bits).
 - **Prescaler** to **16** (to set the clock speed appropriately).
- **Purpose:** These pins enable the SPI communication interface between the board and the Wi-Fi module for data transmission. Setting the data size to 16 bits and adjusting the prescaler ensures compatibility with the Wi-Fi module.

- **Screenshot:**



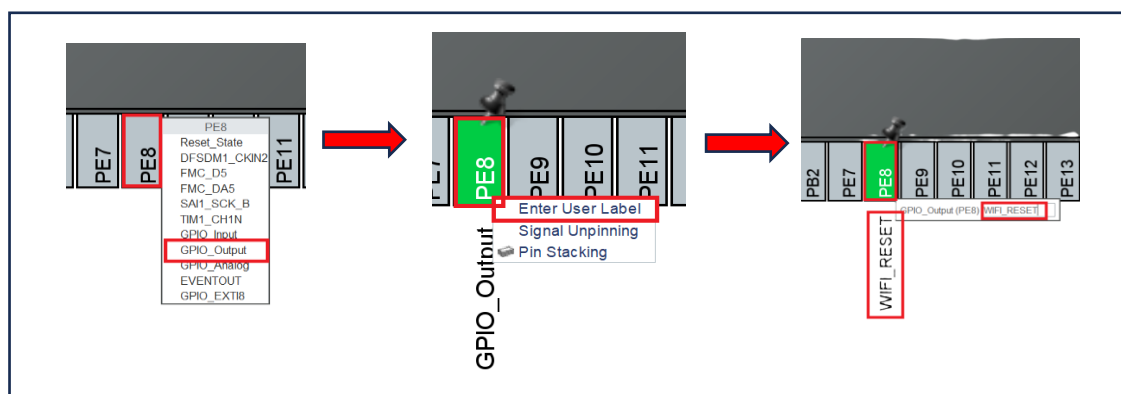
3. Enable WIFI_RESET Pin (PE8)

- **Steps:**

- Set PE8 as **GPIO Output** in the **Pinout & Configuration** tab.

- **Purpose:** This pin is used to reset the Wi-Fi module, ensuring that it starts in a known good state before communication begins.

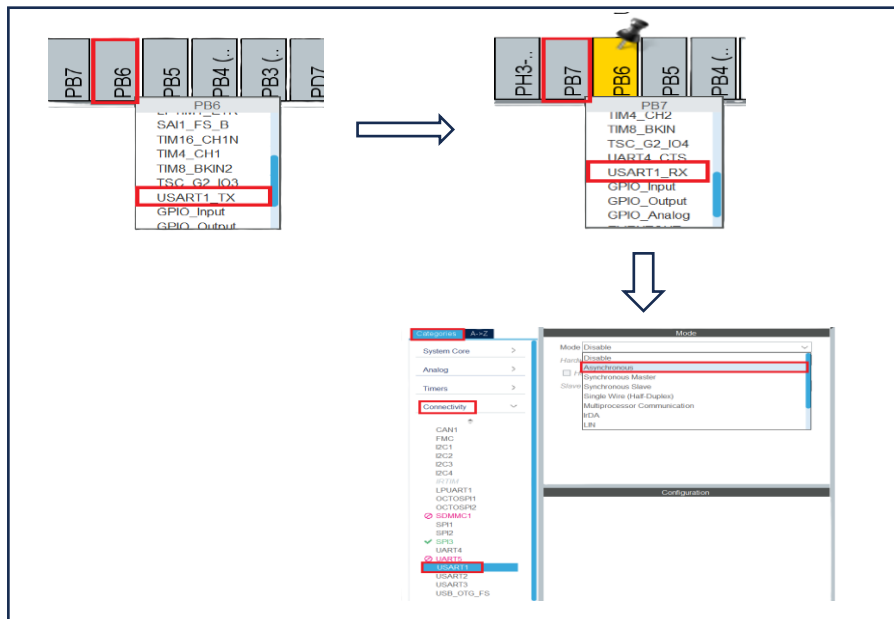
- **Screenshot:**



4. Enable USART1 Pins (PB7 and PB8 for TX and RX)

- **Steps:**

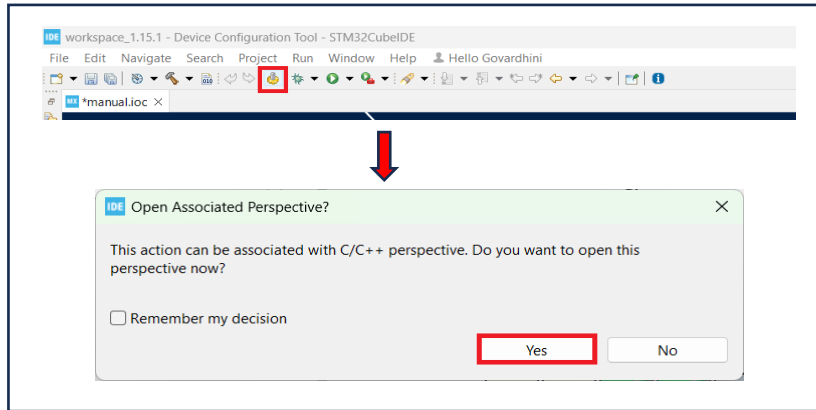
- a. Set **PB7** (USART1_RX) and **PB6** (USART1_TX) for UART communication in the **Pinout & Configuration** tab.
 - b. Shift to the **Categories** tab, go to **Connectivity**, select **USART1**, and set the **Mode** to **Asynchronous**.
- **Purpose:** These pins are used for serial communication between the board and the Wi-Fi module. Configuring USART1 in **Asynchronous** mode allows debugging and status messaging.
 - **Screenshot:**



5. Generate Code

- **Steps:**
 - a. After completing the pin configurations, go to the **Project** tab in STM32CubeMX and click on **Generate Code**.
- **Purpose:** This step generates the code with all the necessary configurations for peripheral initialization and pin setup in the **.ioc** and **main.c** files, ensuring that the project is ready for further coding.

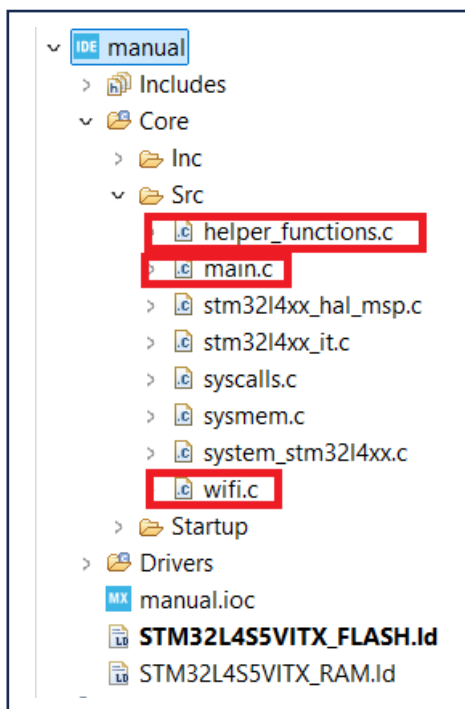
- **Screenshot:**



III. Steps Involved for Wi-Fi Initialization in STM32CubeMX

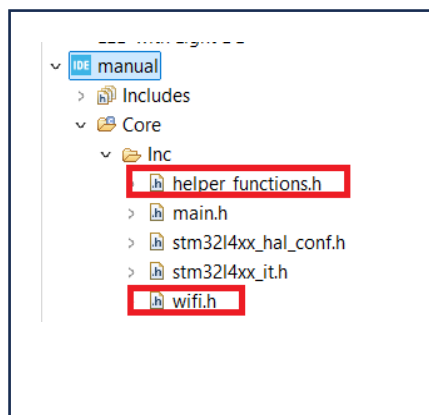
1. Add Files in Src Folder

- **Steps:**
 - After generating the initial code, go to the **Src** folder located inside the **Core** folder of your project.
 - Copy the provided **main.c**, **wifi.c**, and **helper_function.c** files into this folder, replacing the previously generated **main.c** file.
- **Purpose:** This step ensures that the correct files with the necessary logic for the Wi-Fi module initialization, communication, and helper functions are included in the project. By adding these files, the Wi-Fi module functionality and other supporting code are integrated into the project.
- **Screenshot:**



13. Add Files in Inc Folder

- **Steps:**
 - a. Go to the **Inc** folder inside the **Core** folder of your project.
 - b. Copy the provided **wifi.h** and **helper_function.h** files into this folder.
- **Purpose:** This step makes sure that the header files containing declarations for the Wi-Fi module functions and helper functions are available in the project. These header files provide the necessary interfaces to the corresponding .c files, allowing the functions to be used in **main.c** and other parts of the project.



- **Screenshot:**

14. Set SSID and Passphrase

- **Steps:**
 - a. Open the main.c file located in the Src folder of your project.
 - b. Locate the following lines: `char ssid[] = " YourSSID "; char passphrase[] = " YourPassword ";`
 - c. Set the `ssid[]` variable to the name of the Wi-Fi network you want the module to connect to (e.g., "YourSSID").
 - d. Set the `passphrase[]` variable to the Wi-Fi network's password (e.g., "YourPassword").
- **Purpose:** This step configures the Wi-Fi module with the network credentials it will use to connect to the desired Wi-Fi network. The SSID represents the network name, and the passphrase is the security key required to authenticate and join the network.

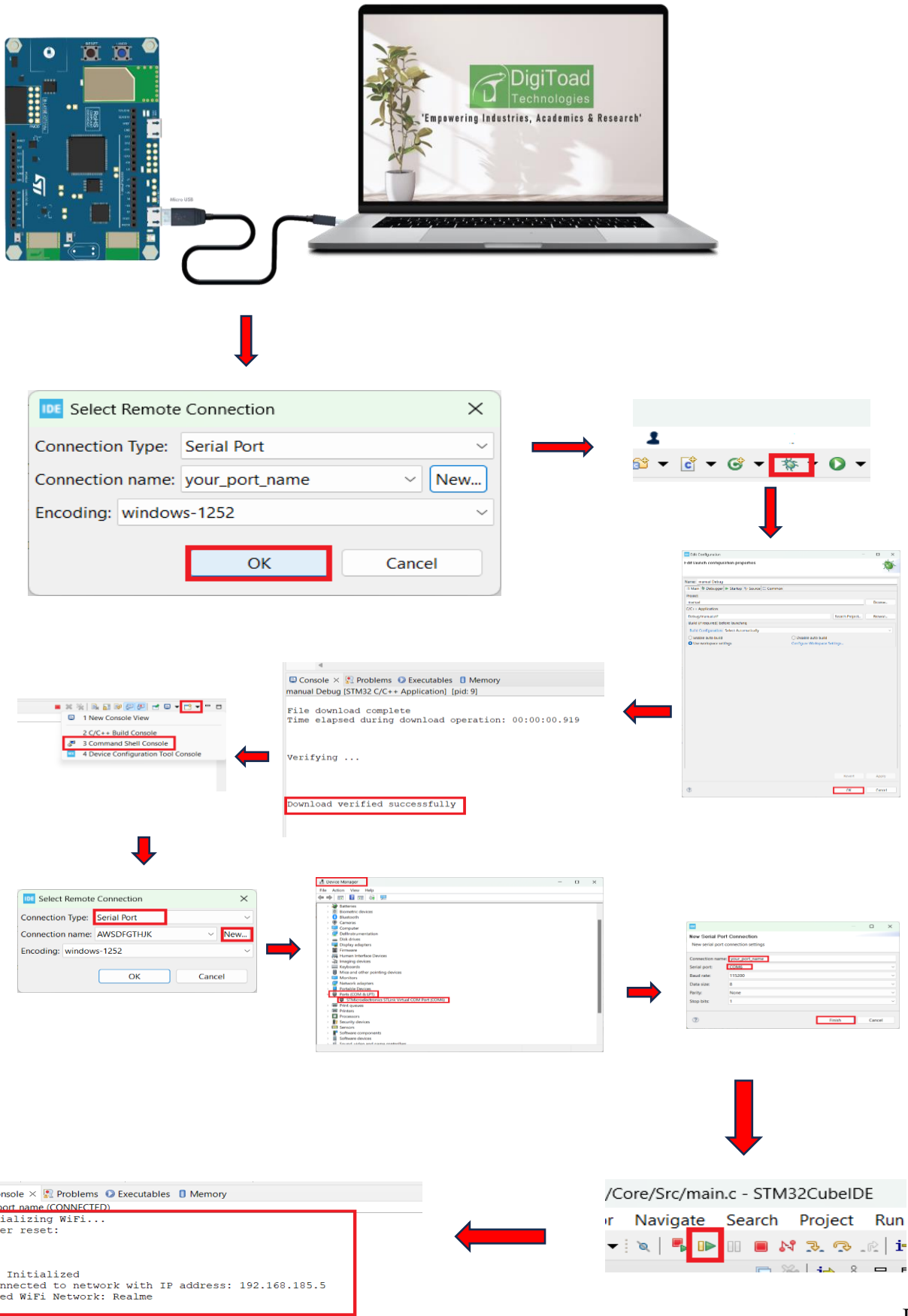
- **Screenshot:**

```
50
51 /* USER CODE BEGIN PV */
52 WIFI_HandleTypeDef hwifi;
53 char ssid[] = "Your Hotspot Name";
54 char passphrase[] = "Your Password";
55 __IO FlagStatus cmdDataReady = 0;
56 /* USER CODE END PV */
```

15. Generate Code

- **Steps:**
 - a. After adding the necessary files, go to the **Project** tab in STM32CubeMX and click on **Generate Code** again.
 - b. Ensure that your board is connected to the PC via ST-Link.
 - c. Run the project by selecting **Debug** and then clicking **Run Debug**. Once it shows "Download Verified Successfully," proceed to create a serial monitor to view the data by selecting **Command Shell Console**.
 - d. When you select **Command Shell Console**, choose **Serial Port** as the connection type. Click on **New**, provide an appropriate name for the console, and select the correct port. You can check the port associated with ST-Link by going to **Device Manager**, expanding the **Ports** section, and identifying the port connected to ST-Link.
 - e. Click **Finish**, then click **OK** to open the serial monitor. Next, in the navigation bar, select the **Resume** option. After resuming, the output will be displayed in the serial console. Ensure that the hotspot is turned on for the device with the configuration you provided in the previous step.
- **Purpose:** This step generates the project files with all the added configurations, ensuring the project is fully initialized with the correct settings. It integrates all changes, including added files, into the project and makes the code ready for further development and compilation.

• Screenshot:



Final Observation:

- The **Wi-Fi module** was successfully initialized and connected to the network.
- After sending the initialization command, the system responded with "**OK**" and the message "**Wi-Fi Initialized**".
- The Wi-Fi module successfully connected to the available Wi-Fi network, obtaining the **IP address** of connected Wi-Fi network
- The system confirmed the successful connection to the Wi-Fi network with the message "**Joined Wi-Fi Network**".

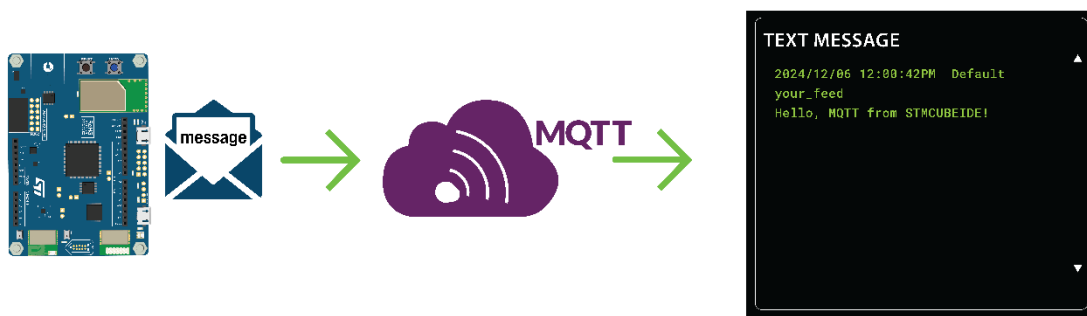
Lab 6: Sending a Single String Message via MQTT to the Cloud

Lab Overview

In this lab, we will use the STM32 board equipped with the Wi-Fi module to send a single string message, such as "Hello, Cloud!", to the cloud via MQTT. The focus is on demonstrating the process of connecting the IoT device to the cloud and transmitting data over the network. This lab builds on the previous Wi-Fi initialization and sensor data collection, allowing students to implement the final communication step with the cloud using MQTT.

Objective of the Lab

The objective of this lab is to learn how to send a single string message from the IoT device to the cloud using MQTT. By completing this lab, students will understand how to establish MQTT communication, configure the necessary credentials, and send data to cloud platforms without using a continuous loop for data transmission.



Experimental Setup

Pre-lab Requirements

1. Before proceeding with this lab, you must complete the **Wi-Fi Initialization Lab** to ensure that the STM32 board is properly connected to the Wi-Fi network and ready for MQTT communication.

Note: Both the Wi-Fi Initialization and MQTT message sending functionalities should be implemented within the same project.

Important Instruction:

For this lab, you should not create separate projects for the **Wi-Fi Initialization** and **MQTT Message Transfer** labs. Instead, integrate both functionalities into a single project. Start with the project from the **Wi-Fi Initialization Lab** and then follow the steps in this manual to implement the functionality for sending a string message via MQTT.

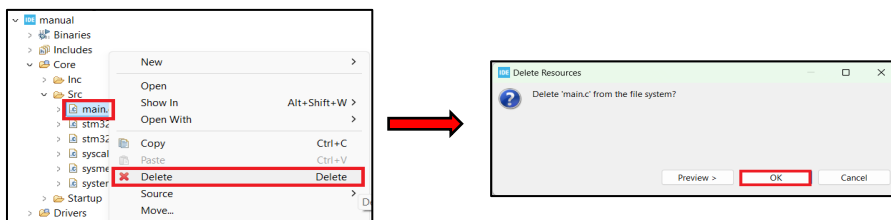
By the end of this lab, you should have a **single project** where:

- The **Wi-Fi module** is initialized and successfully connects to the Wi-Fi network.
- A **single string message** ("Hello, MQTT from STM32!") is sent once from the STM32 board to the cloud via MQTT.
- The cloud platform (like Adafruit.io or another MQTT broker) receives the message and you can monitor it from the cloud.

Steps Involved:

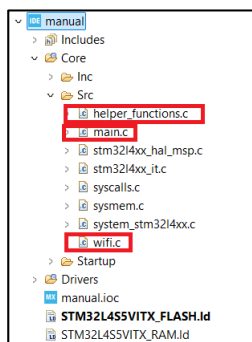
1. Delete Existing main.c and wifi.c Files

- **Steps:**
 - Go to the **Src** folder inside the **Core** folder of your project.
 - If **main.c** and **wifi.c** files already exist, delete them.
- **Purpose:** This step ensures that you start with a clean slate for integrating the new files provided for this lab.
- **Screenshot**



2. Add the main.c and wifi.c Files

- **Steps:**
 - Copy the **main.c** and **wifi.c** files provided for this lab into the **Src** folder.
- **Purpose:** These files contain the necessary logic for Wi-Fi initialization, MQTT setup, and data transmission. **main.c** integrates the sensor reading, MQTT initialization, and communication, while **wifi.c** manages the communication with the Wi-Fi module.
- **Screenshot:**



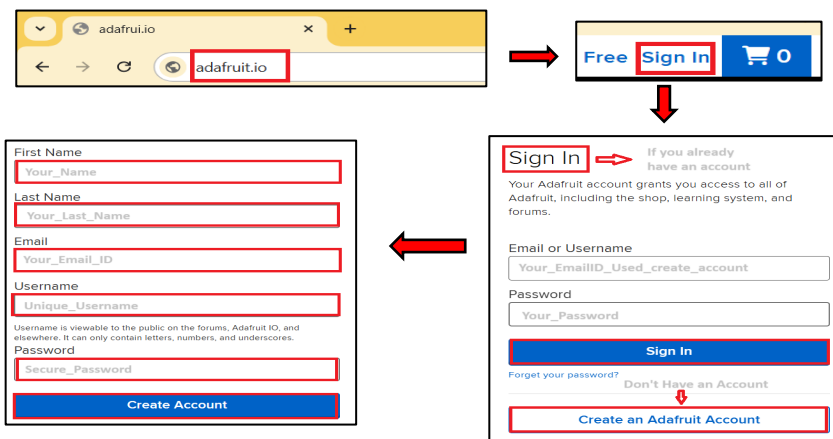
3. Create Adafruit Cloud Account

- **Steps:**

- Type "Adafruit.io" in Chrome search and go to the Adafruit website.
- Click on **Sign In** if you already have an account and log in with your credentials.
- If you do not have an account, click on **Sign Up** and create an account by providing the requested information.
- Once your account is created, you will be able to access the services offered by the Adafruit Cloud.

- **Purpose:** This step sets up your Adafruit account, providing access to Adafruit.io and the cloud services needed for the project.

- **Screenshot:**



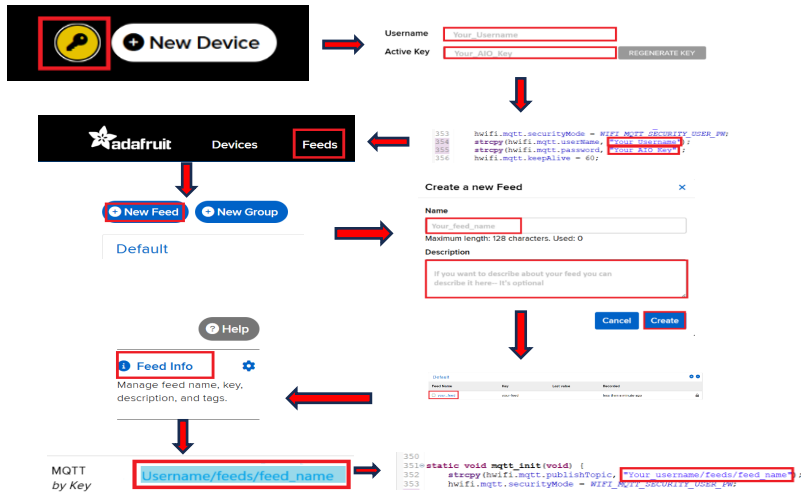
4. Modify MQTT Credentials in main.c

- **Steps:**

- Open **main.c** and navigate to lines 354 and 355.
- Log in to your **Adafruit.io** account and copy the **username** and **AIoT key** (IoT key).
- Replace the **username** at line 354 (`hwifi.mqtt.userName`) and the **AIoT key** at line 355 (`hwifi.mqtt.password`) with the credentials from your **Adafruit.io** account.
- Go to **Adafruit.io > Feeds** and create one new feed to handle text message.
- Copy the **MQTT Key** of feed from the **Feed Info** section.
- Replace the respective feed name in **main.c** for text message.

- **Purpose:** This step connects your IoT device to the correct cloud account and feeds, allowing data to be sent to the cloud. It also ensures proper authentication and secure communication.

- **Screenshot:**



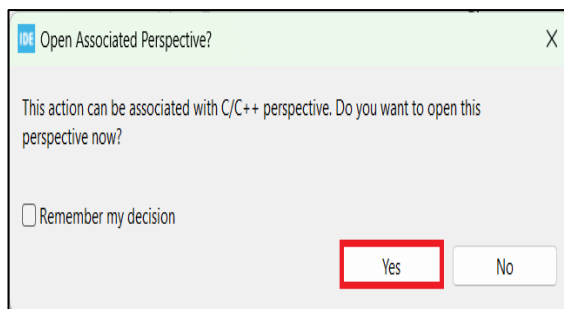
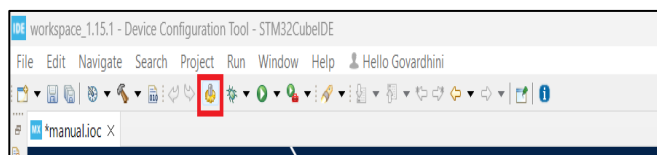
5. Generate Code

- **Steps:**

- After modifying **main.c**, save the changes and click **Generate Code**.
- Build the project to ensure there are no errors in the code.

- **Purpose:** This step generates the updated project code, which includes the newly added MQTT functionality and Wi-Fi setup.

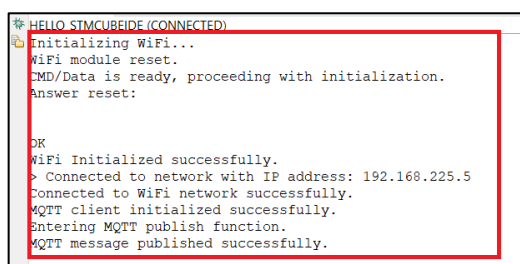
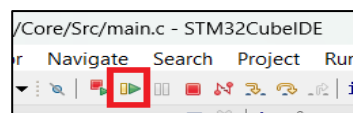
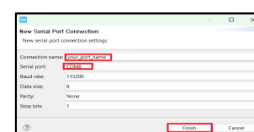
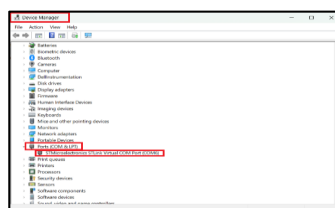
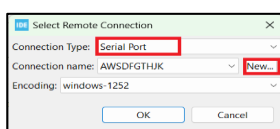
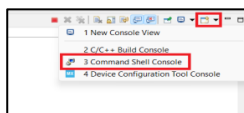
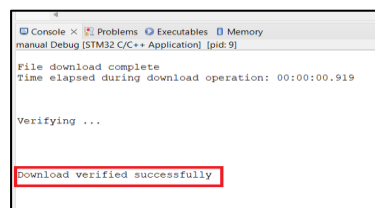
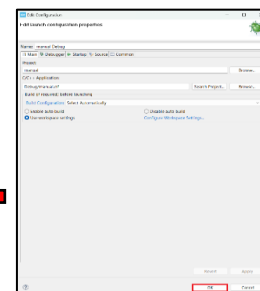
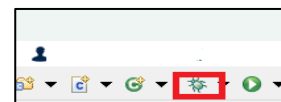
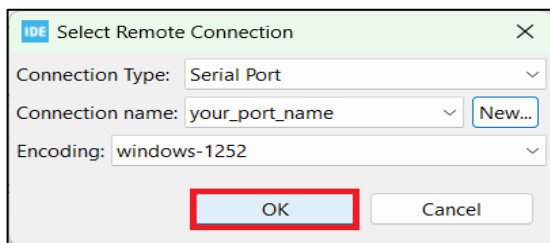
- **Screenshot:**



6. Verify Connection to Cloud

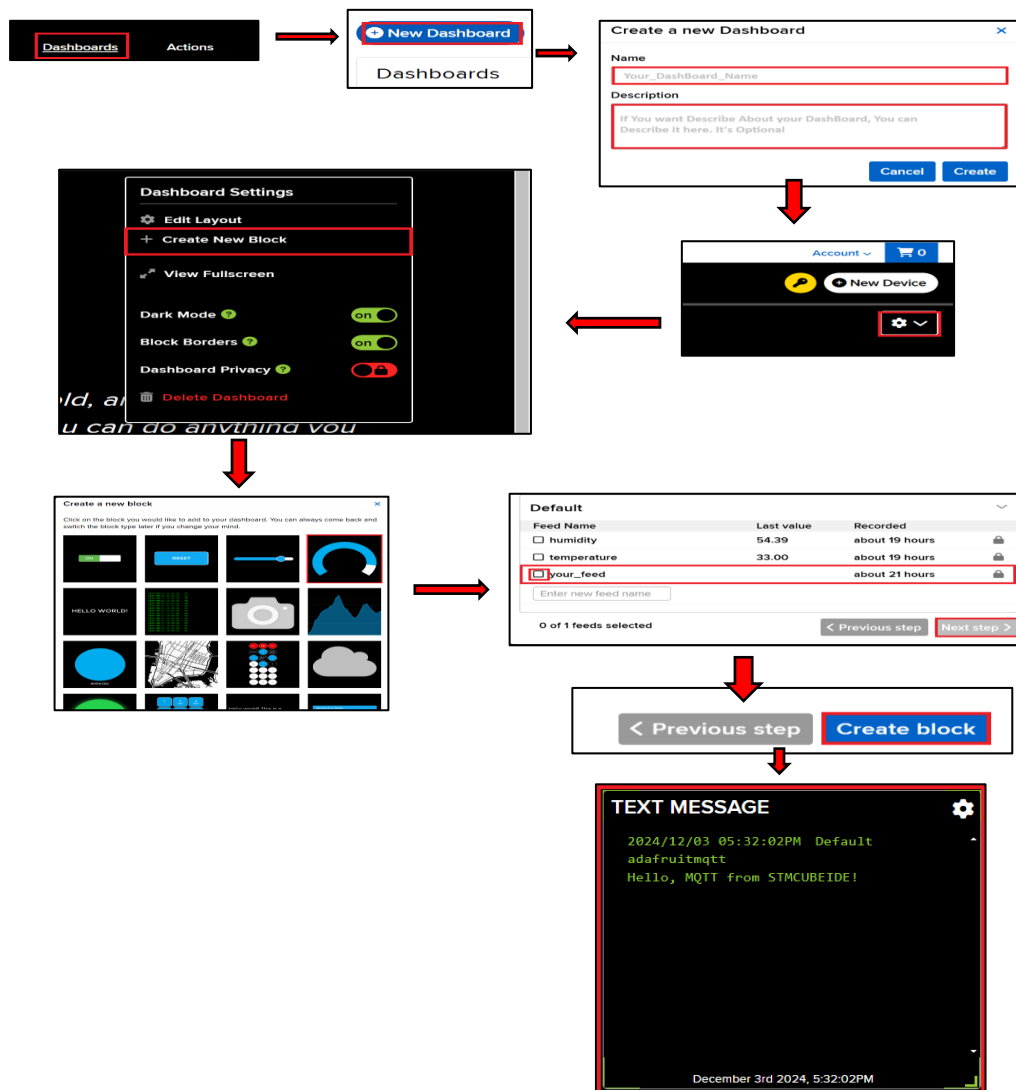
- **Steps:**
 - a. Ensure your **Wi-Fi hotspot** is turned on in your mobile device. Then, make sure that the **SSID** and **password** in your **device configuration** in **main.c** are set to match the credentials of the Wi-Fi network you want the device to connect to.
 - b. Ensure that all **MQTT configurations** in **main.c** are correctly modified for your Adafruit account, and debug the code to verify no errors are present.
 - c. Run the project by selecting **Debug** and then clicking **Run Debug**. Once it shows "Download Verified Successfully," proceed to create a serial monitor to view the data by selecting **Command Shell Console**.
 - d. When you select **Command Shell Console**, choose **Serial Port** as the connection type. Click on **New**, provide an appropriate name for the console, and select the correct **port**. You can check the port associated with **ST-Link** by going to **Device Manager**, expanding the **Ports** section, and identifying the port connected to **ST-Link**.
 - e. Click **Finish**, then click **OK** to open the serial monitor. Next, in the navigation bar, select the **Resume** option. After resuming, the output will be displayed in the serial console. Ensure that the hotspot is turned on for the device with the configuration you provided in the previous step.
- **Purpose:** This step ensures the device is correctly connected to the Wi-Fi network, able to communicate with the MQTT broker, and successfully sending data to **Adafruit.io**. It also verifies that the data is being transmitted and displayed through the serial monitor.

- **Screenshot:**



7. Create and Configure the Adafruit Dashboard

- **Steps:**
 - a. Go to the **Adafruit.io Dashboard** and click **Create New Dashboard**.
 - b. In the dashboard, click the **Settings icon** and select **Edit Block**.
 - c. Choose a visual representation for the text feed.
 - d. Set up the minimum and maximum warning values if required.
 - e. Create the block for the respective text feed.
 - f. Again **Reset** the board and you will be seeing the data getting updated in **feed** and **Dashboard** also.
- **Purpose:** This step allows you to visualize the text data from your IoT device on the **Adafruit.io Dashboard**.
- **Screenshot:**



Final Observation:

- The **Wi-Fi module** was successfully initialized and connected to the **network**.
- The system received an "**OK**" response after sending the **initialization command**, indicating that the Wi-Fi module was reset and ready.
- The device successfully connected to the network with the assigned **IP address: 192.168.225.5**.
- The **MQTT client** was initialized successfully, confirming that the device is ready to communicate with the **MQTT broker**.
- The **MQTT message "Hello, MQTT from STM32!"** was successfully published to the **cloud**.
- The system confirmed that the message was sent without errors, indicating that the **MQTT communication** was successful.

Lab 7: Local IoT Monitoring and Dashboard Creation using Node-RED for Temperature and Humidity Data

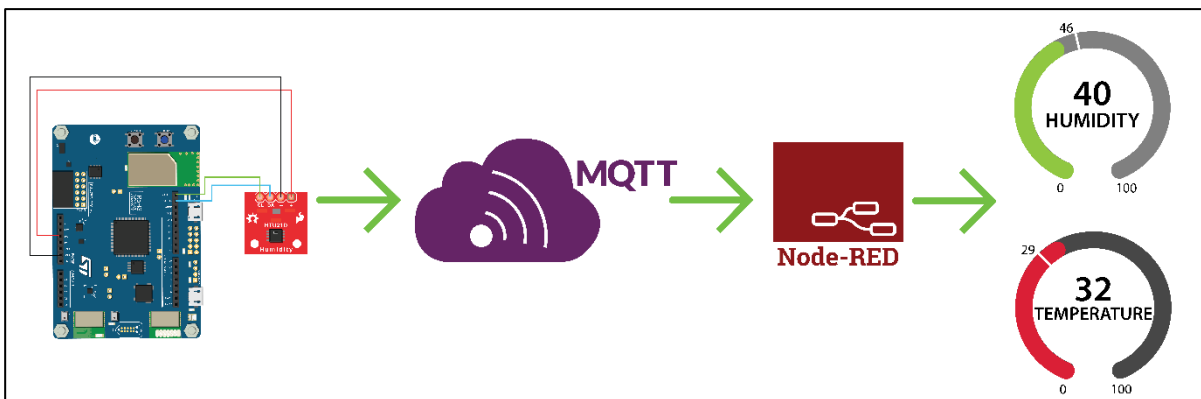
Lab Overview

In this lab, students will collect temperature and humidity data from the HTU21 sensor, send the data to a local MQTT broker, and create a local dashboard using Node-RED for real-time monitoring. The focus of this lab is on utilizing Node-RED for local data visualization, which enables monitoring and managing sensor data on a local PC.

Objective of the Lab

The objective of this lab is to learn how to collect temperature and humidity data from the HTU21 sensor, send it to a local MQTT broker, and use Node-RED to create a local dashboard for real-time data visualization.

Experimental Setup



Pre-lab Requirements

Before proceeding with this lab, you must complete the following steps to ensure all prerequisites are in place for successful completion:

1. **HTU21 Interfacing and Data Collection Lab**

Ensure that you have successfully interfaced the **HTU21 sensor** with the **STM32 board** and are able to collect data from the sensor. This step is essential as it provides the foundational sensor data that will be sent to the cloud in this lab.

2. **Wi-Fi Initialization Lab**

Complete the **Wi-Fi Initialization Lab** to establish Wi-Fi connectivity on your IoT device. This will enable the device to connect to a Wi-Fi network, which is necessary for cloud communication in this lab.

3. **HTU21 Interface and IoT Integration Project**

Integrate the HTU21 sensor data collection and Wi-Fi initialization functionality into a single project.



This is important because the data collected from the HTU21 sensor will be published to the cloud (via MQTT) and later collected and visualized using Node-RED.

Important Instruction:

For this lab, **do not create separate projects** for the HTU21 Interfacing, Wi-Fi Initialization, and Node-RED integration labs. Instead, integrate all functionalities into a single project.

- Start with the project from the **Wi-Fi Initialization Lab**,
- Add the HTU21 sensor interfacing code from the **HTU21 Interface and IoT Integration Project**,
- Finally, execute this lab for integrating MQTT and local data monitoring through Node-RED.

By the end of this lab, you should have a single project where:

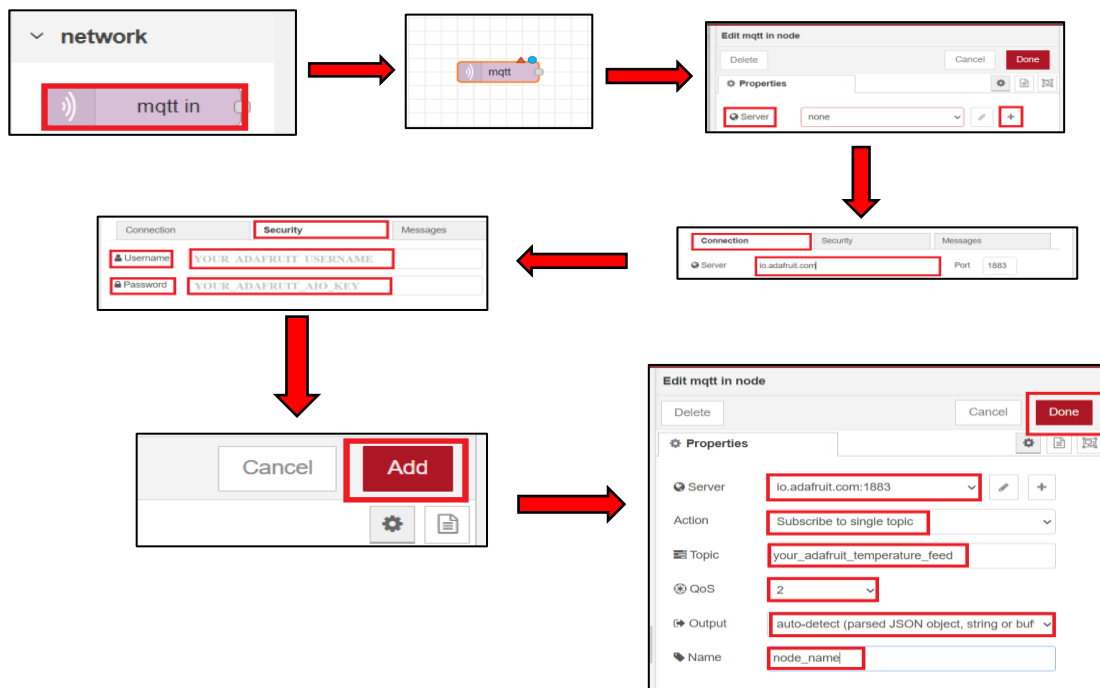
- The HTU21 sensor is properly interfaced and collects data.
- The Wi-Fi module is initialized and connected to the network.
- Data from the HTU21 sensor is sent to Adafruit.io via MQTT for cloud-based visualization, and Node-RED is used to visualize the data locally.

Steps Involved:

1. Add MQTT Node in Node-RED

- **Steps:**
 - a. Go to Command prompt and type **node-red** and click enter so node red will start **running**.
 - b. Once you start running it in Command Prompt you will get an **Local IP Address** copy it and paste it in the browser, the node-red page will be ready.
 - c. Drag the MQTT input node into the Node-RED workspace.
 - d. Double-click on the MQTT node to open the configuration window.
 - e. Click on the **plus** icon near the **Server** field to add a new server.
 - f. In the new window, set the server to **io.adafruit.com** and the port number to **1883**. Leave other configuration settings as default.
 - g. Navigate to the **Security** tab. In the **Username** field, enter your **Adafruit username**, and in the **Password** field, provide your **AIO Key** from your Adafruit account. Then, click **Add**.
 - h. Ensure that the newly created server (io.adafruit.com) is selected in the **Server** dropdown.
 - i. In the **Action** dropdown, select **Subscribe to a single topic**.
 - j. Set the **QoS level** to **2** and select **Auto-detect** for the output format.
 - k. Enter a custom name for the MQTT node in the **Name** field and click **Done**.
- **Purpose:** This step ensures that you start with a clean slate for integrating the new files provided for this lab.
- **Note:** Similarly, repeat the same steps for subscribing to the humidity topic.

- **Screenshot:**

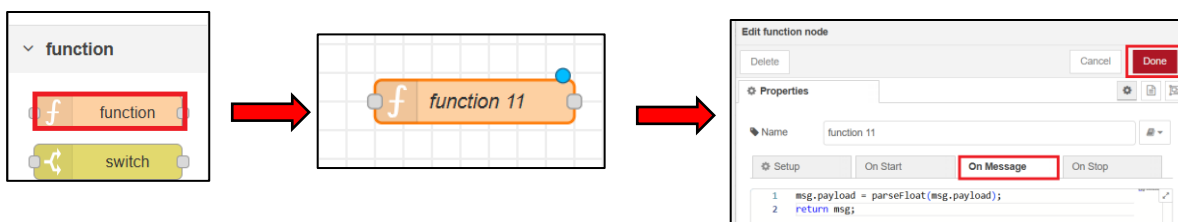


2. Add Function Node to Process Data

Steps:

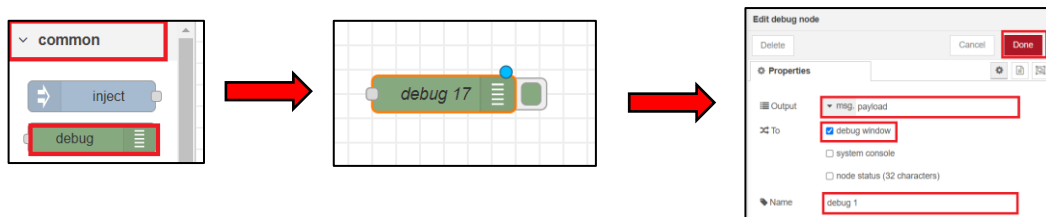
- Drag the **Function** node into the Node-RED workspace.
 - Double-click on the **Function** node to open the configuration window.
 - In the **Name** field, give a custom name for the function (e.g., "Parse Message").
 - Navigate to the **On Message** tab and enter the following code:

```
javascript msg.payload = parseFloat(msg.payload); return msg;
```
- **Purpose:** This step processes the incoming data by parsing the payload (which is a string) into a floating-point number using the **parseFloat()** function. This ensures that the data can be used for further processing or visualization in the flow. The function node modifies the message payload to make it compatible with other operations in the flow.
 - **Note:** Similarly, repeat the same steps for processing the humidity data.
 - **Screenshot:**



3. Add Debug Node to Verify Output

- **Steps:**
 - a. Drag the **Debug** node into the Node-RED workspace.
 - b. Double-click on the **Debug** node to open the configuration window.
 - c. In the **Output** field, select **msg.payload** to display the processed data in the debug window.
 - d. Make sure the **Debug Window** is selected to show the output in the debug pane.
 - e. In the **Name** field, you can provide a custom name for the debug node (e.g., "Display Temperature Data").
 - f. Click **Done** to save the configuration.
- **Purpose:** This step allows you to verify that the incoming temperature data is being received and processed correctly. The data will be shown in the debug window, ensuring the MQTT messages are properly parsed and that the system is working as expected.
- **Note:** Similarly, repeat the same steps to verify the output for the humidity data.
- **Screenshot:**

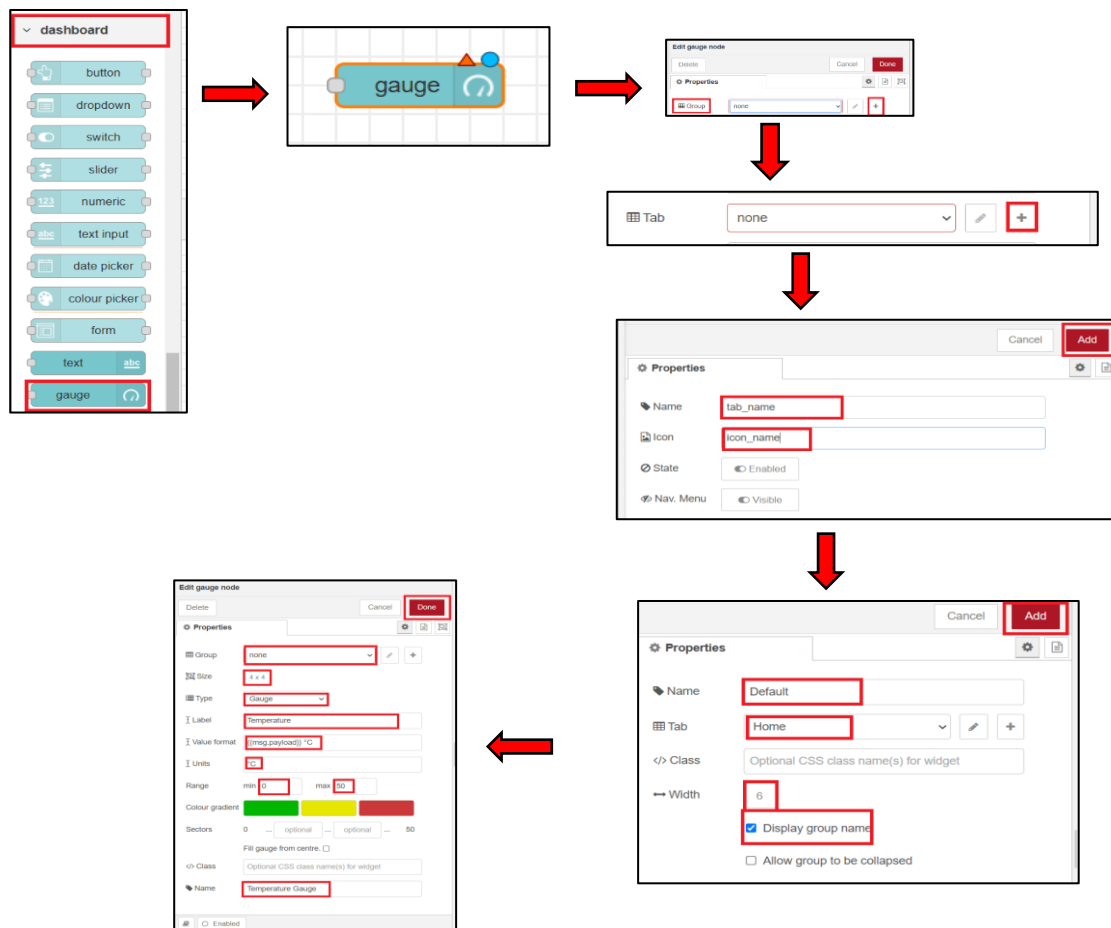


4. Add Gauge Node for Temperature Visualization

Steps:

- a. Before proceeding, ensure you have downloaded the **Dashboard Package** mentioned in the Node-RED installation manual.
- b. Drag the **Gauge** node from the Dashboard section into the Node-RED workspace.
- c. Double-click the **Gauge** node to open the configuration window. In the **Group** field, click the + icon to create a new group.
- d. In the **Group Name** field, enter a name (e.g., "Temperature Data Group"). In the **Tab** field, click the + icon to create a new tab. Add a name and an icon for your tab and click **Add**.
- e. After being redirected to the group configuration page, click **Add** and then **Next**. Ensure that the group you created is selected in the **Group** option of the gauge configuration
- f. In the **Size** section, select the desired gauge size (e.g., **4x4**). In the **Label** field, enter "Temperature". In the **Value Format** field, enter `{{msg.payload}} °C` to display the temperature in Celsius. In the **Unit** field, enter "Celsius".
- g. In the **Range** section, define the minimum and maximum values for your gauge (e.g., 0°C to 50°C)

- **Purpose:** This step configures a **gauge node** in the Node-RED dashboard to visualize the temperature data in real-time, with customizable range and warning highlights.
- **Note:** Similarly, repeat the same steps to create a gauge node for visualizing the humidity data.
- **Screenshot:**



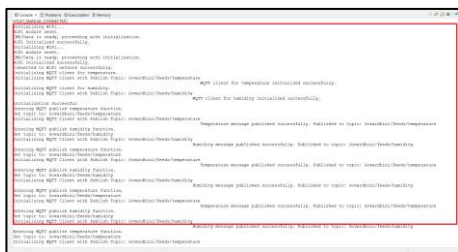
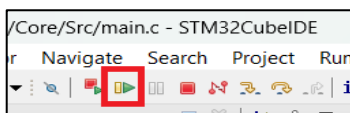
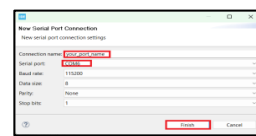
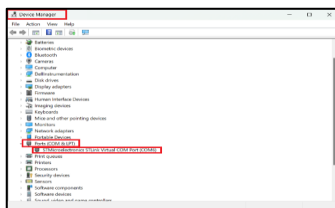
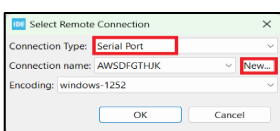
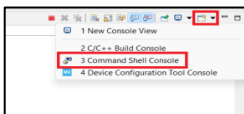
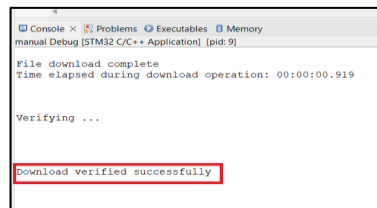
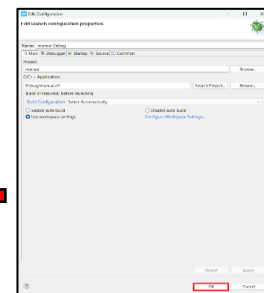
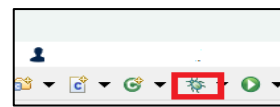
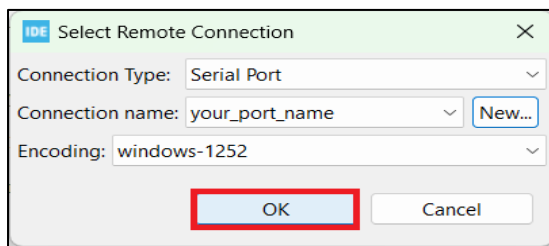
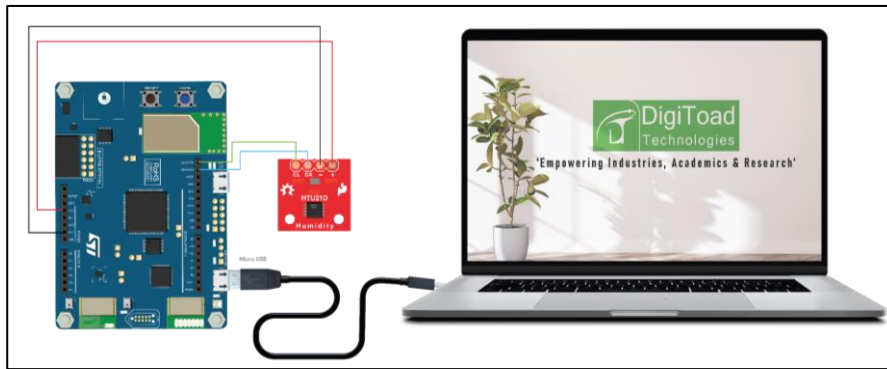
5. Run the Project in STM32CubeIDE

Steps:

- Open the **HTU21 Integrated with IoT Project** in STM32CubeIDE.
- Ensure your Wi-Fi hotspot is turned on in your mobile device. Then, make sure that the SSID and password in your device configuration in main.c are set to match the credentials of the Wi-Fi network you want the device to connect to.
- Ensure that all MQTT configurations in main.c are correctly modified for your Adafruit account, and debug the code to verify no errors are present.
- Run the project by selecting Debug and then clicking Run Debug. Once it shows "Download Verified Successfully," proceed to create a serial monitor to view the data by selecting Command Shell Console.

- e. When you select Command Shell Console, choose Serial Port as the connection type. Click on New, provide an appropriate name for the console, and select the correct port. You can check the port associated with ST-Link by going to Device Manager, expanding the Ports section, and identifying the port connected to ST-Link.
 - f. Click Finish, then click OK to open the serial monitor. Next, in the navigation bar, select the Resume option. After resuming, the output will be displayed in the serial console. Ensure that the hotspot is turned on for the device with the configuration you provided in the previous step.
- **Purpose:** This step initializes the STM32 board, enabling it to collect sensor data from the HTU21 sensor and send it to the **Adafruit MQTT broker**. Running the project ensures that the data from the sensor is transmitted to the cloud via MQTT, allowing Node-RED to retrieve and visualize the data.

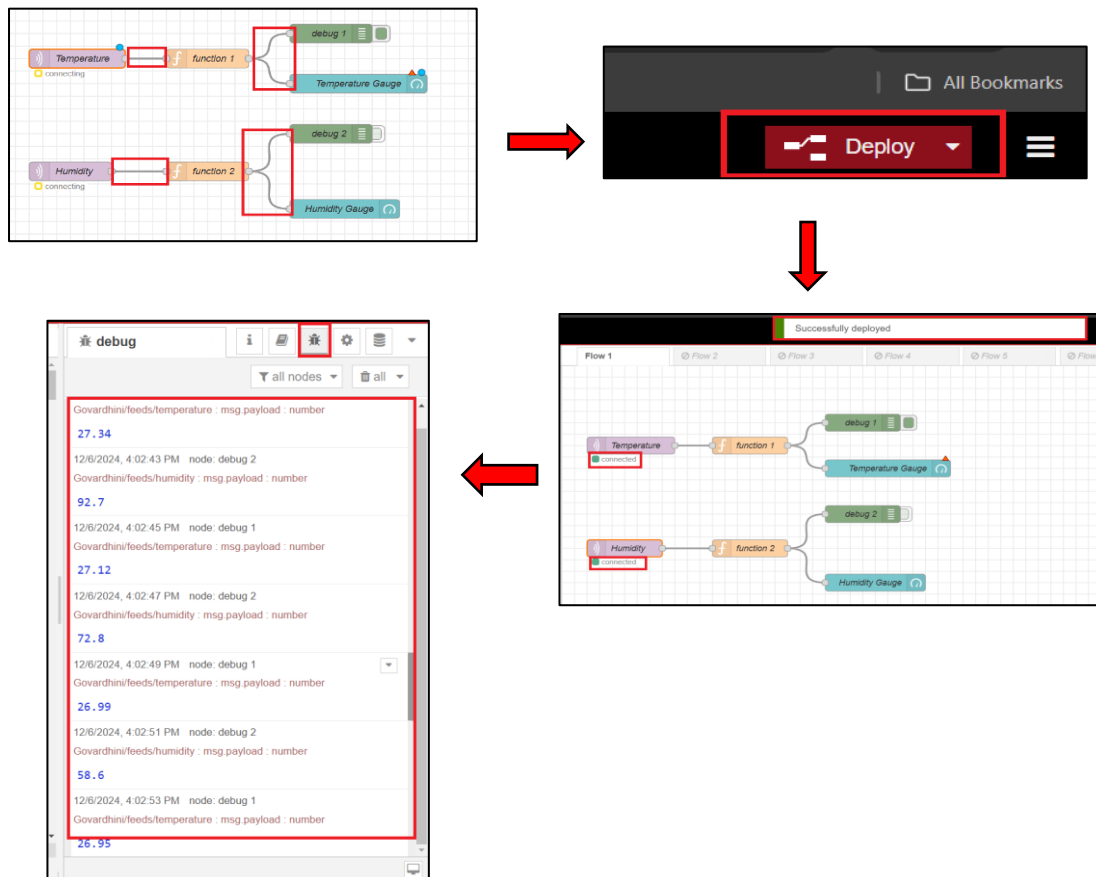
- Screenshot:



6. Deploy and Verify Data in Node-RED

Steps:

- f. Open **Node-RED** and ensure that the MQTT nodes are correctly configured to receive data. Before proceeding, verify that Adafruit feeds are receiving data.
 - g. Once verified, click **Deploy** in Node-RED to deploy the flow.
 - h. Navigate to the **Debug message node**. Once deployed, you will see the status near the MQTT-in node, showing text like "Connected."
 - i. In the **Debug panel**, you should now see the **temperature** and **humidity** data being received and displayed.
- **Purpose:** This step ensures that Node-RED is properly receiving sensor data from the MQTT broker and displaying it in the Debug panel, confirming that the data flow is working as expected.
 - **Screenshot:**



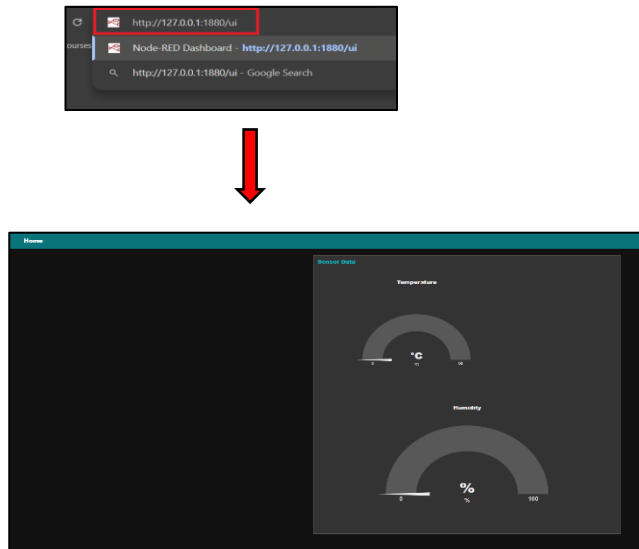
7. Access the Dashboard in Node-RED

Steps:

- g. After deploying the flow, open a new browser tab.
- h. Enter the **local IP** address obtained from the Node-RED environment (e.g., 127.0.0.1:1880).
- i. Append /ui to the IP address (e.g., 127.0.0.1:1880/ui) to access the Node-RED dashboard.

j. You will be able to see the **temperature** and **humidity** data displayed in the dashboard created using the **gauge nodes**.

- **Purpose:** This step allows you to visualize the real-time sensor data on a web-based dashboard, providing an interactive display for monitoring temperature and humidity.
- **Screenshot:**



Final Observation:

- The **STM32 board** successfully connected to the **Wi-Fi network**.
- The **HTU21 sensor** collected temperature and humidity data and sent it to the **Adafruit MQTT broker** via **MQTT**.
- **Node-RED** successfully subscribed to the MQTT topics and received the sensor data from the Adafruit broker.
- The data was correctly displayed in the **Debug panel** of Node-RED.
- The **Node-RED Dashboard** visualized the temperature and humidity data in real-time using the **Gauge nodes**.
- The system confirmed the data was successfully transmitted to the cloud, allowing for local monitoring and visualization of the sensor data.

Lab 9 a) : Design & implement a data logger to send audio sensor data from STM32 to NanoEdge AI Studio on PC.

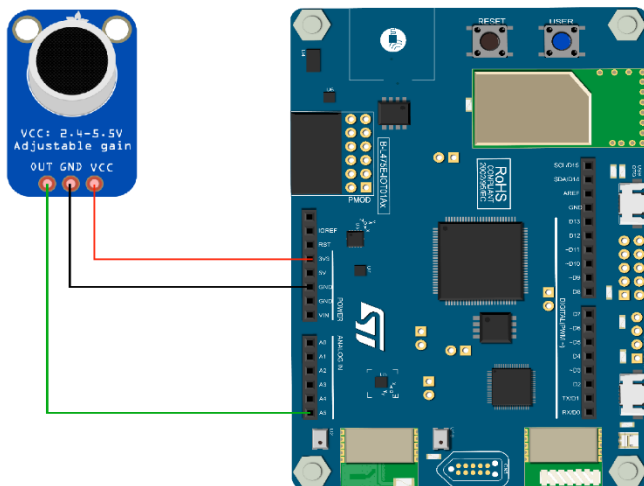
Lab Overview:

In this experiment we design an audio datalogger. This datalogger can collect real time audio samples and store it into a PC as a CSV file. The audio csv files are later used for classification applications

Objective of the Lab:

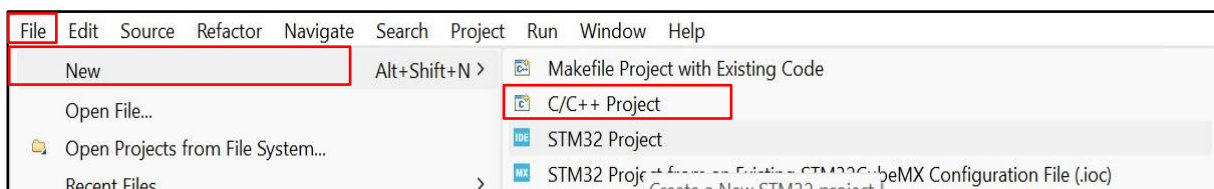
The objective of this experiment is to interface an audio sensor to an STM32 microcontroller and create a datalogger code. This datalogger code will create a buffer where all the audio data sample will be stored, using which we will be able to create datasets of audio samples to build a machine learning model in the NanoEdge AI Studio.

Experimental Setup:

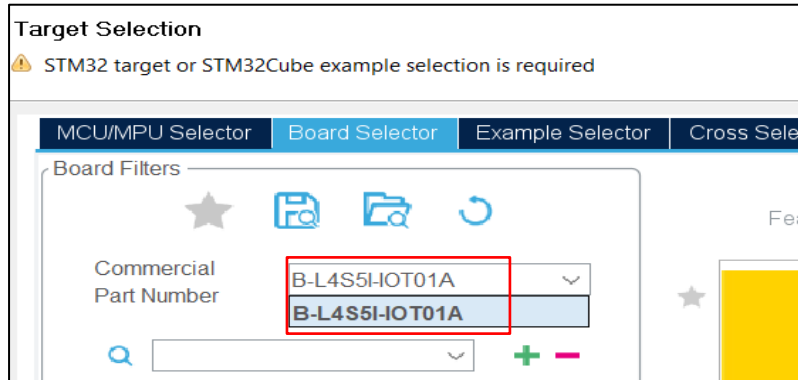


Steps Involved:

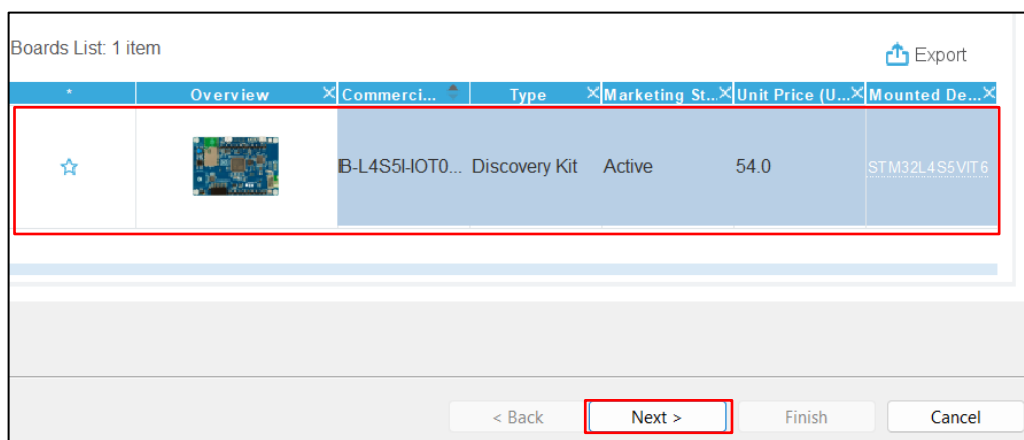
1. Click on **File**→**New**→**STM32 Project** to start your project on Cube IDE.



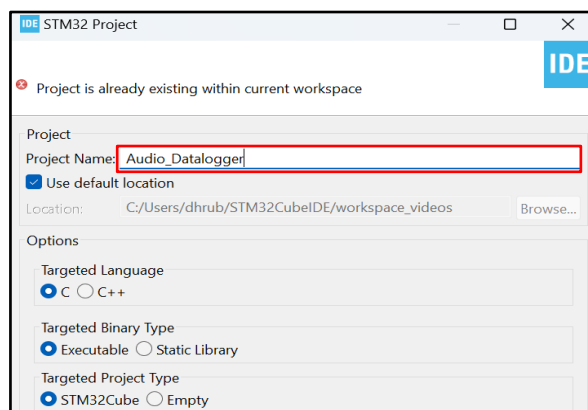
2. **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.



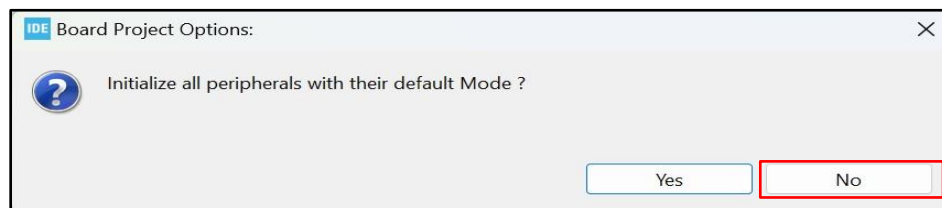
- After this on the right-hand side of the window, under **Board List** you will see the board you have selected. Click on the board and then click on **Next**.



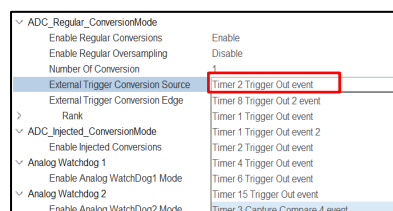
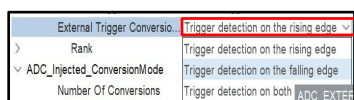
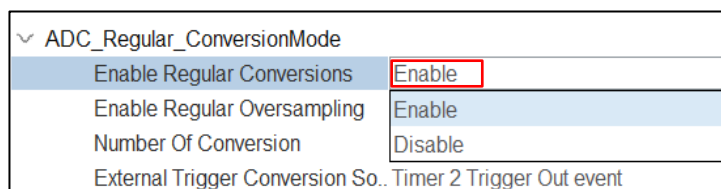
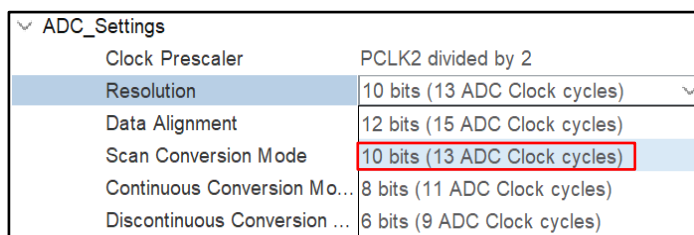
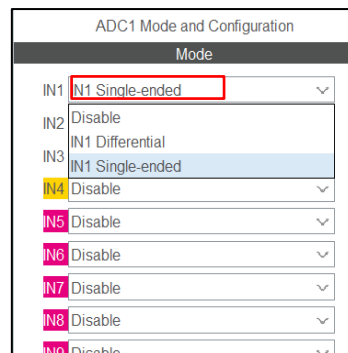
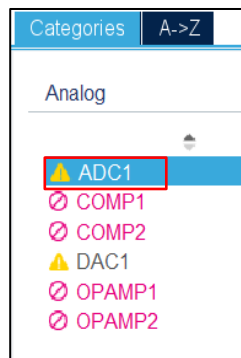
- In the next window you need to give your project a name, rest of the things will remain by default as it is for now. Click on **Next**.



- Cube IDE will ask if you want to initialize all peripherals with their default mode, click on No.



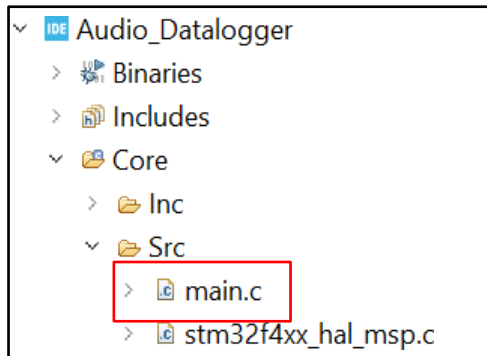
6. Next on the left-hand side **Categories** → **Analog** select **ADC1** then select **IN1** as IN1 Single-ended under **Mode**. Under **Configuration** select **Parameter Settings**, change **Resolution** to 10-bits, then



NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
ADC1 global interrupt	☒	0	0

change the **External Trigger Conversion Source** to **Timer 2 Trigger Out Event** and **External**

9. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example I have named my project as (Audio_Datalogger)**Audio_Datalogger**→**Core** →**Src** →**main.c** (double click to load the code).



10. Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE uses HAL libraries. Below is the code snippets, please put your code in the appropriate places in the **main.c** file.

```

22 /* Private includes -----
23 /* USER CODE BEGIN Includes */
24 #include "stdio.h"
25 #include "string.h"
26 /* USER CODE END Includes */

```

```

34 /* USER CODE BEGIN PD */
35 #define DATA_INPUT_USER 256
36 #define AXIS_NUMBER 1
37 /* USER CODE END PD */

```

```

51 /* USER CODE BEGIN PV */
52 float mic_x = 0.0;
53 float mic_buffer[DATA_INPUT_USER * AXIS_NUMBER] = {0};
54 volatile uint32_t buffer_index = 0;
55 /* USER CODE END PV */

```

```

119 /* USER CODE BEGIN 2 */
120 HAL_ADC_Start_IT(&hadcl);
121 HAL_TIM_Base_Start(&htim2);
122 /* USER CODE END 2 */

```

```

63 /* USER CODE BEGIN PFP */
64 void Log(void);
65 void fill_mic_buffer(void);
66 int __io_putchar(int);
67 /* USER CODE END PFP */

```

```

70/* Private user code -----*/
71/* USER CODE BEGIN 0 */
72void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
73 {
74     if (hadc->Instance == ADC1)
75     {
76         mic = HAL_ADC_GetValue(&hadc1);
77         mic_buffer[buffer_index] = mic;
78
79         buffer_index++;
80         if (buffer_index >= DATA_INPUT_USER)
81         {
82             buffer_index = 0; // Reset index to 0 when it reaches the buffer size
83         }
84     }
85 }
86/* USER CODE END 0 */

```

```

363/* USER CODE BEGIN 4 */
364int __io_putchar(int ch){
365    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 0xFFFF);
366    return ch;
367 }
368void fill_mic_buffer(){
369     for(int i=0; i<DATA_INPUT_USER; i++){
370         mic_buffer[AXIS_NUMBER * i] = mic_x;
371         HAL_Delay(3);
372     }
373 }
374void Log(){
375     fill_mic_buffer();
376     for(int i=0; i<DATA_INPUT_USER; i++){
377         printf("%.2f", mic_buffer[AXIS_NUMBER * i]);
378         printf(" ");
379     }
380     printf("\r\n");
381 }

```

```

139 /* USER CODE BEGIN WHILE */
140 while (1)
141 {
142     Log();
143     /* USER CODE END WHILE */

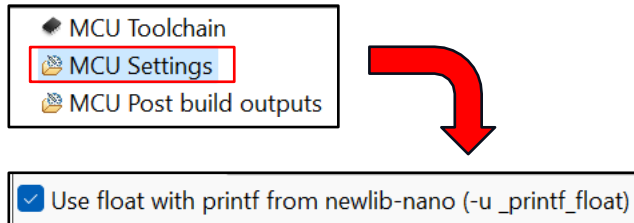
```

```

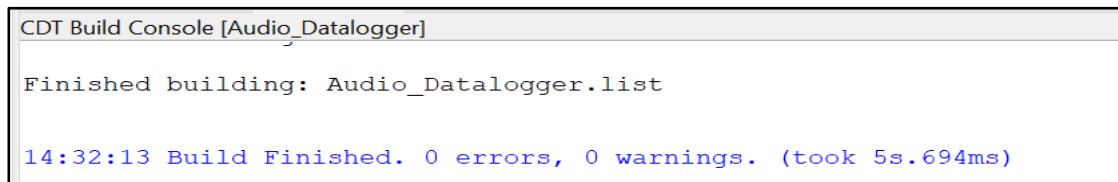
465int __io_putchar(int ch)
466 {
467     HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
468     return ch;
469 }

```

11. Right click on the Audio_Datalogger project and select **Properties**. Go to **C/C++ Build** → **Settings**. Next select **MCU Settings** and enable the option **Use float with printf from newlib-nano (-u _printf_float)**.

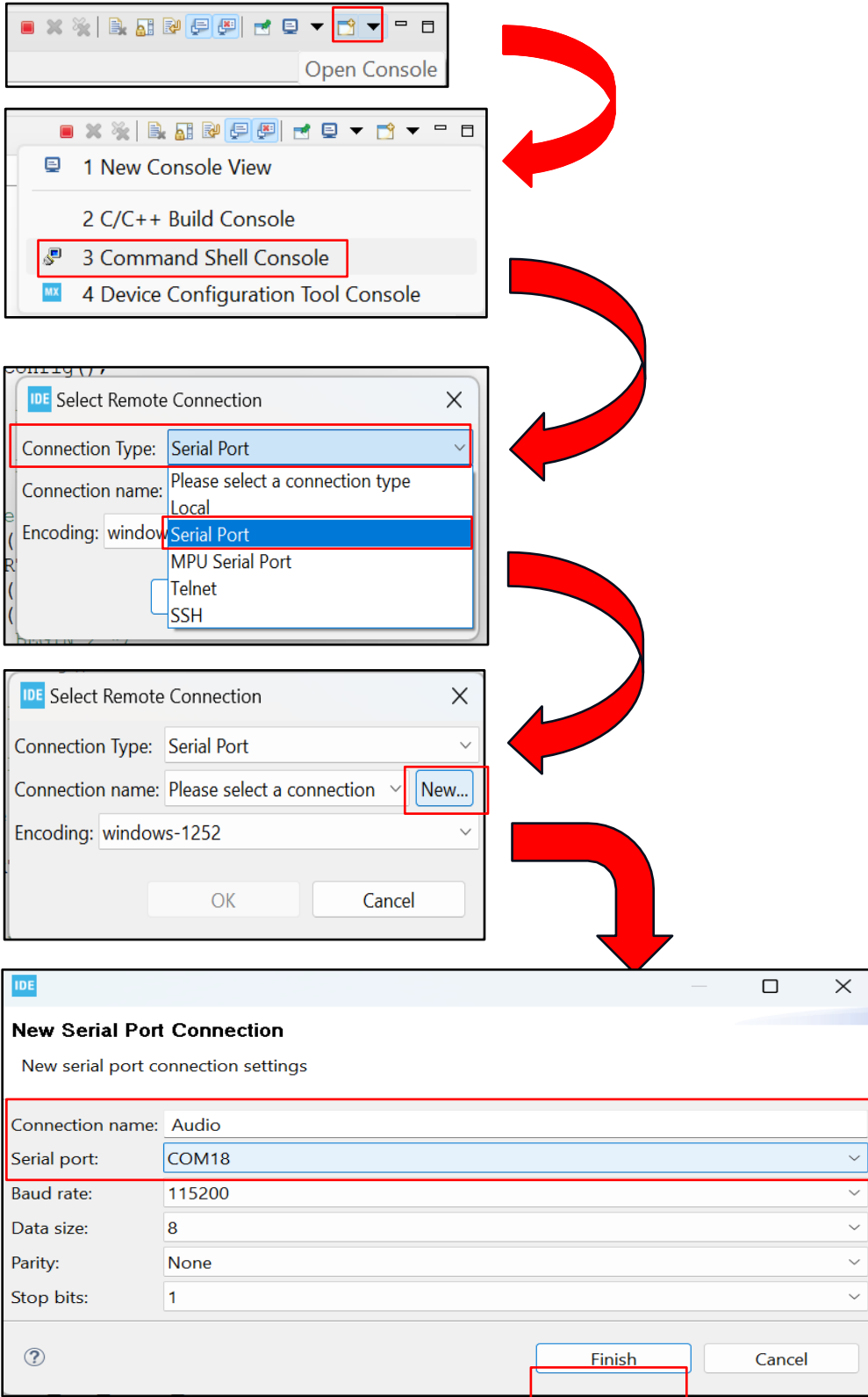


12. Now click on the build  symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.



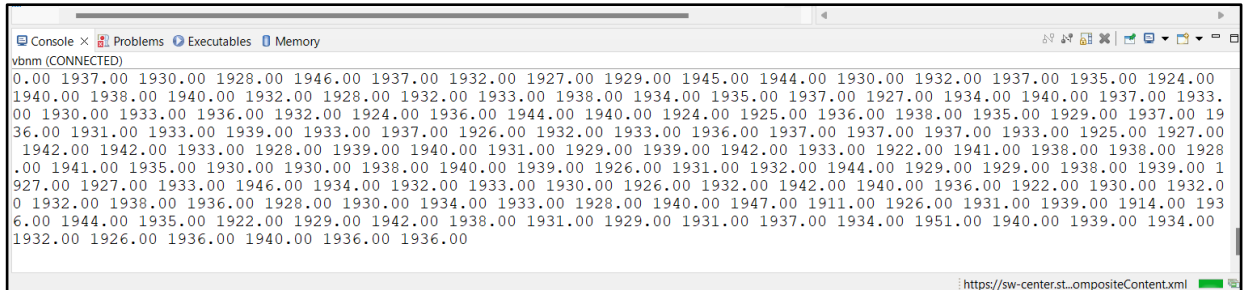
13. Next connect your STM32 board with your audio sensor connect to it to your PC and click on the **Debug**  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.

14. the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.



15. Click on the **Resume** icon  to run your code.

16. Before moving out of the debugging mode, click on **Disconnect**  and close the console then click on the **Terminate** icon . You will be moved out of the debugging mode.



```

vbnm (CONNECTED)
0.00 1937.00 1930.00 1928.00 1946.00 1937.00 1932.00 1927.00 1929.00 1945.00 1944.00 1930.00 1932.00 1937.00 1935.00 1924.00
1940.00 1938.00 1940.00 1932.00 1928.00 1932.00 1933.00 1938.00 1934.00 1935.00 1937.00 1927.00 1934.00 1940.00 1937.00 1933.
00 1930.00 1933.00 1936.00 1932.00 1924.00 1936.00 1944.00 1940.00 1924.00 1925.00 1936.00 1938.00 1935.00 1929.00 1937.00 19
36.00 1931.00 1933.00 1939.00 1933.00 1937.00 1926.00 1932.00 1933.00 1936.00 1937.00 1937.00 1937.00 1933.00 1925.00 1927.00
1942.00 1942.00 1933.00 1928.00 1939.00 1940.00 1931.00 1929.00 1939.00 1942.00 1933.00 1922.00 1941.00 1938.00 1938.00 1928
.00 1941.00 1935.00 1930.00 1930.00 1938.00 1940.00 1939.00 1926.00 1931.00 1932.00 1944.00 1929.00 1929.00 1938.00 1939.00 1
927.00 1927.00 1933.00 1946.00 1934.00 1932.00 1933.00 1930.00 1926.00 1932.00 1942.00 1940.00 1936.00 1922.00 1930.00 1932.0
0 1932.00 1938.00 1936.00 1928.00 1930.00 1934.00 1933.00 1928.00 1940.00 1947.00 1911.00 1926.00 1931.00 1939.00 1914.00 193
6.00 1944.00 1935.00 1922.00 1929.00 1942.00 1938.00 1931.00 1929.00 1931.00 1937.00 1934.00 1951.00 1940.00 1939.00 1934.00
1932.00 1926.00 1936.00 1940.00 1936.00 1936.00

```

NOTE:

- If you are getting any errors in main.c file ,you may refer to the code provided in the folder.
- We will be using this datalogger code to log the data into microcontroller.

Final Observation:

- You can be able to see the value of **audio sensor** in the form of buffer containing **256** samples

Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is for the use of education purpose only.

Lab 9 b) Design of an ML based audio classification to identify & classify different sounds & deploy the classifier model on the AIoT board.

Lab Overview:

In this experiment, we use NanoEdge AI Studio to identify the best classification model to classify different sound signals, E.g: 1. Crying of a baby, 2. Siren of an ambulance, 3. Ambient noise.

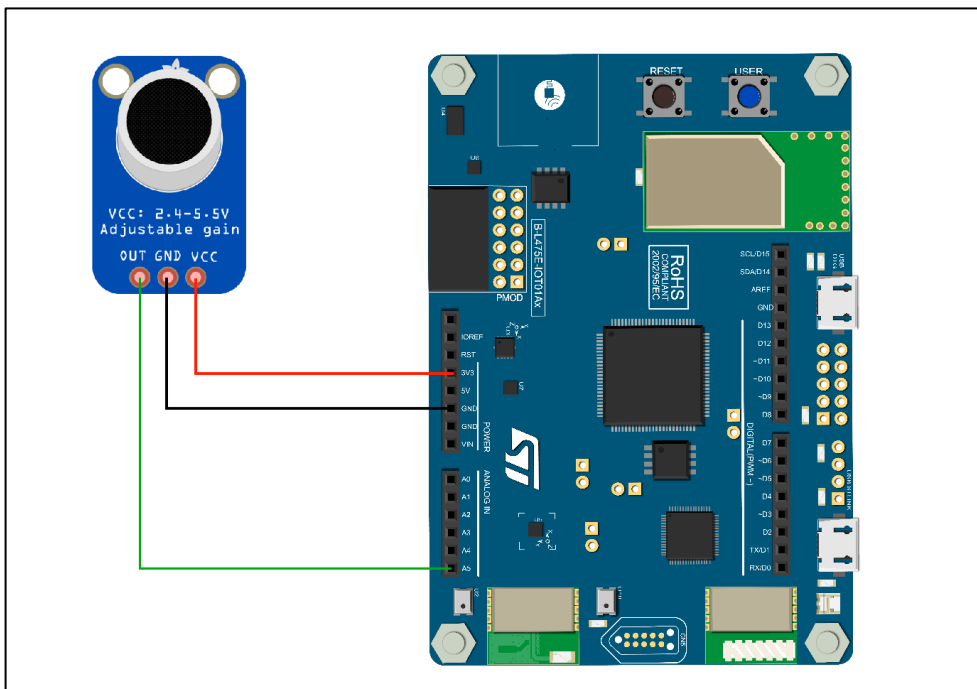
Objective of the Lab:

The objective is to log audio samples on a microcontroller and train a machine learning model on NanoEdge AI Studio to classify different audio types

Experimental Setup:

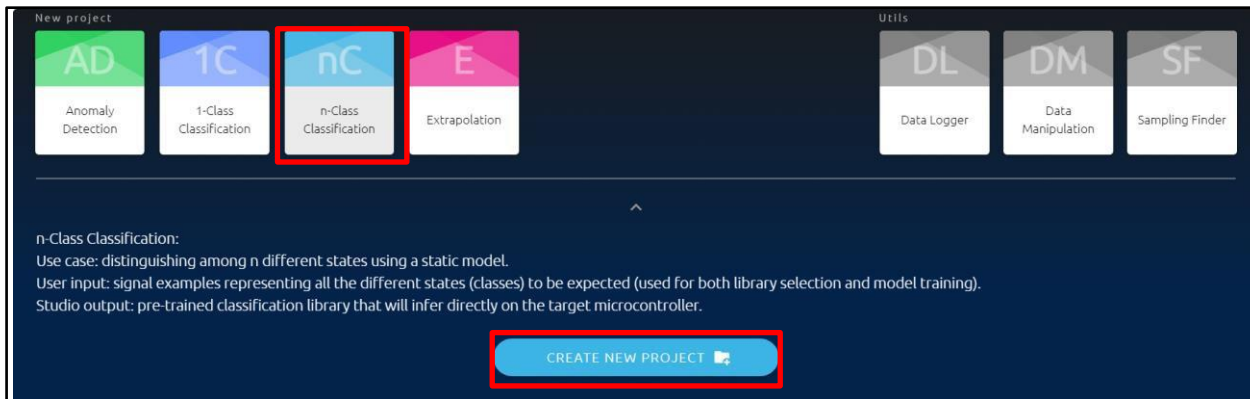
1. NanoEdge AI Studio.
2. Audio Sensor (Analog).
3. STM32 Microcontroller.
4. USB Cable for the microcontroller.
5. Jumper Wires.

Connection Diagram:

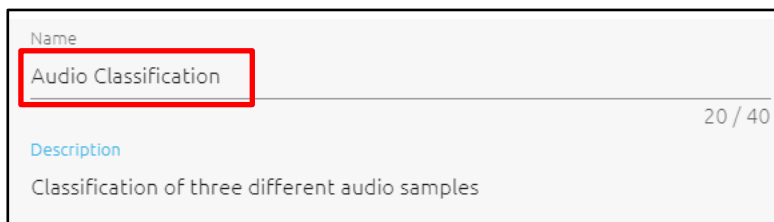


Steps Involved (NanoEdge AI Studio):

1. Open NanoEdge AI Studio.
2. Select **n-Class Classification** project type and select **Create New Project**.



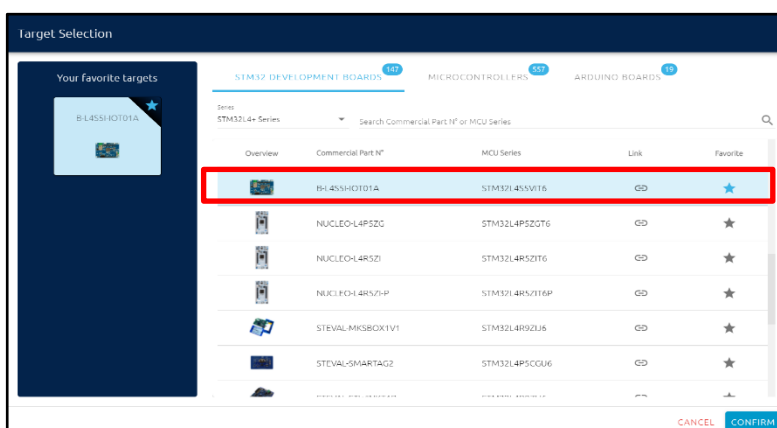
3. A new window will open. On the first step, **Project Settings**, name the project as Audio Classification. Under **Description** type “classifying different audio samples”.



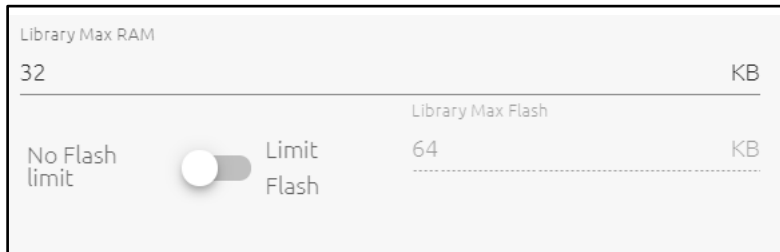
Name
 Audio Classification

Description
 Classification of three different audio samples

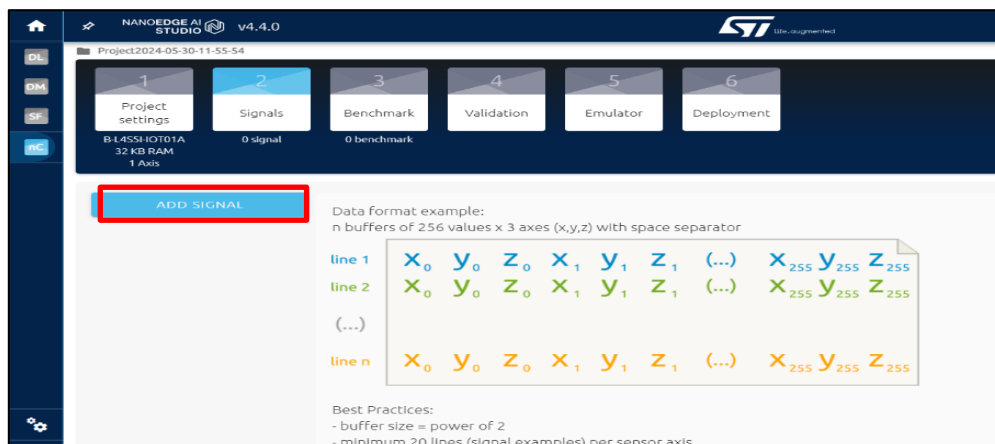
4. In **Target** section, select your proper STM32 Nucleo board i.e B-L45SI-IOT1A board. In **Sensor type** section, select the type of sensor you are working with. For this experiment you have to choose Microphone sensor 1 axis.

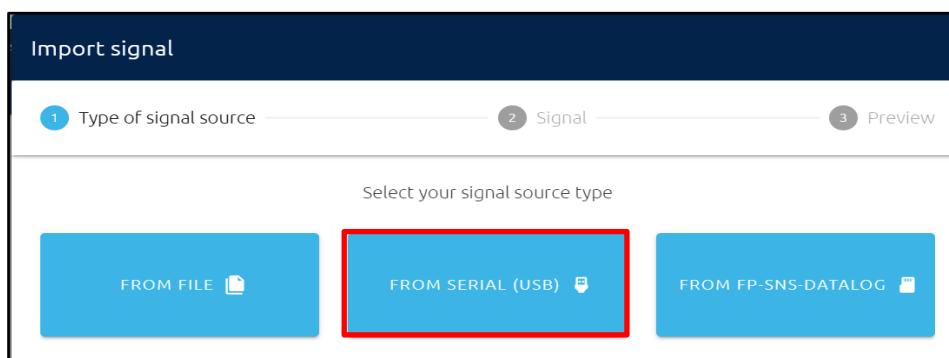


- Let **Max RAM** and **Max Flash** option remain as default. Click on **Save & Next** to move onto the next step.



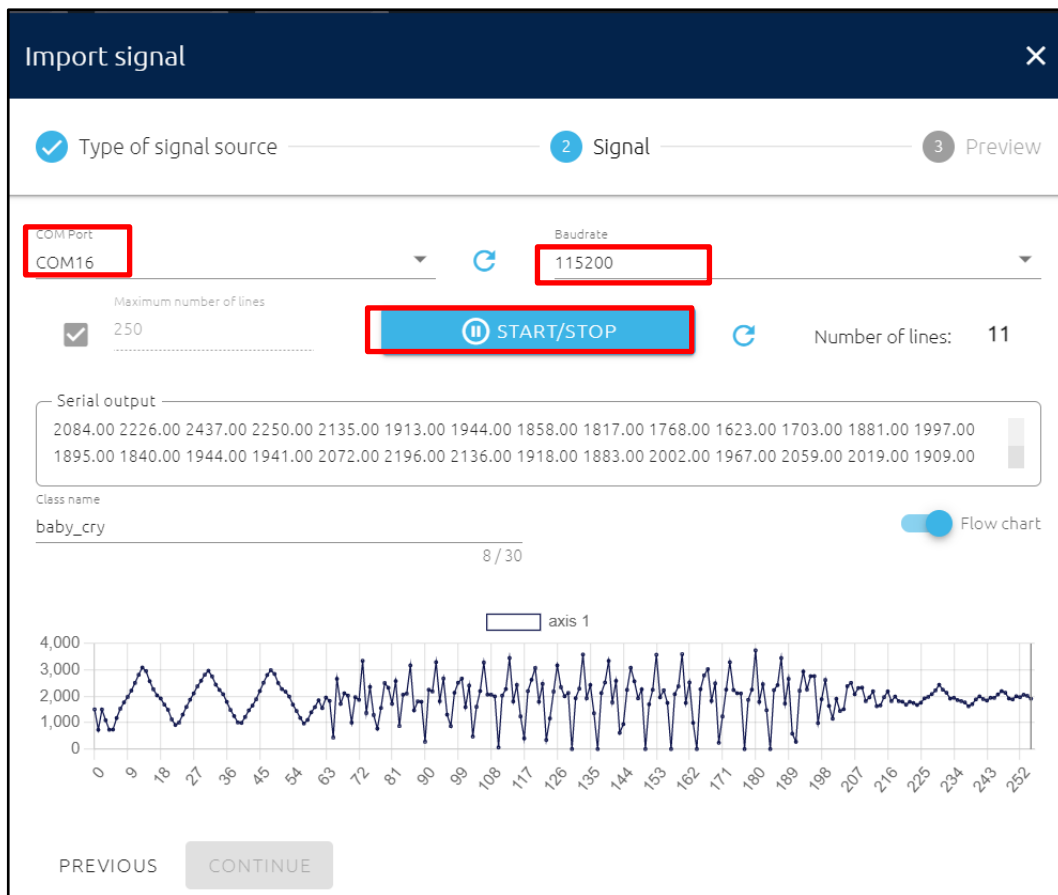
- In the second step on NanoEdge AI Studio-**Signals**, click on **Add Signals**. A window will open with different source options for you to import signals, select **From Serial (USB)** as we are collecting sensor data connected to microcontroller board, which in turn is connected to your PC through a serial USB cable.





NOTE: Datalogger code should be running inside the microcontroller and needs to be disconnected from the serial console of STM32 CubeIDE.

7. Now you can add your signals. In **COM Port** select the correct COM port based on device manager info. Select **Baud rate** as 115200. In **Maximum number of lines** enter the number of lines of data you want to collect. In this case we will collect 250 lines of data.



Import signal

1 Type of signal source 2 Signal 3 Preview

COM Port: COM16 Baudrate: 115200

Maximum number of lines: 250

START/STOP

Number of lines: 11

Serial output

2084.00 2226.00 2437.00 2250.00 2135.00 1913.00 1944.00 1858.00 1817.00 1768.00 1623.00 1703.00 1881.00 1997.00
1895.00 1840.00 1944.00 1941.00 2072.00 2196.00 2136.00 1918.00 1883.00 2002.00 1967.00 2059.00 2019.00 1909.00

Class name: baby_cry

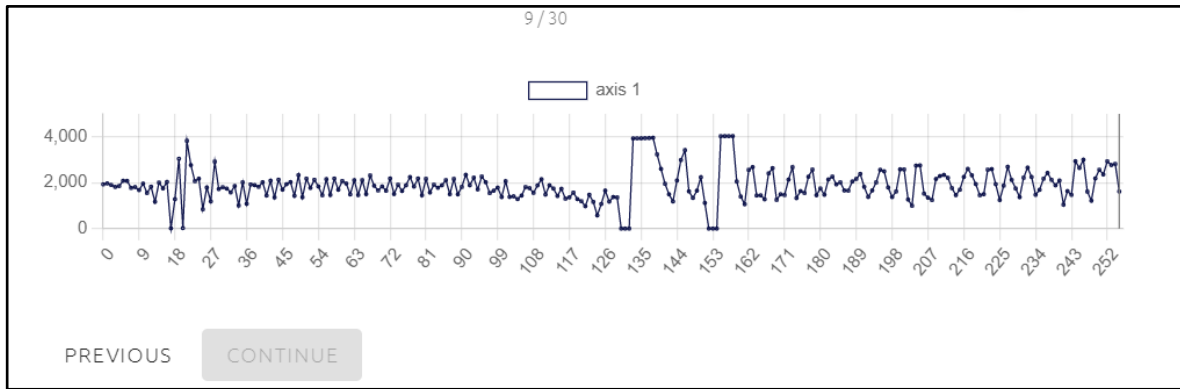
8 / 30

Flow chart

axis 1

PREVIOUS CONTINUE

8. To add signals, first play the sound sample on a device near the microphone sensor and then click on **START/STOP**. Play the audio file in continuous loop until 200 lines of data are collected, then stop the process.



Click on **Continue** to move onto importing the audio sample.

9. In this window, select the **Delimiter** as **Space**. If any line of data has some corrupted values, an error message will be shown under the preview lines. You can select the particular line(s) and delete it. Select **Preview Lines** as 250 to check all the line of data you have collected, and click on **Import**.

Import signal

☒ Type of signal source
 ☒ ¹Signal
 ☒ ³Preview

Delimiter

☐ Comma
 ☐ Tab
 ☐ Semicolon
 ☒ Space
 ☐ Other:

Enter delimiter

DELETE LINES

< Preview of 20 lines >
 

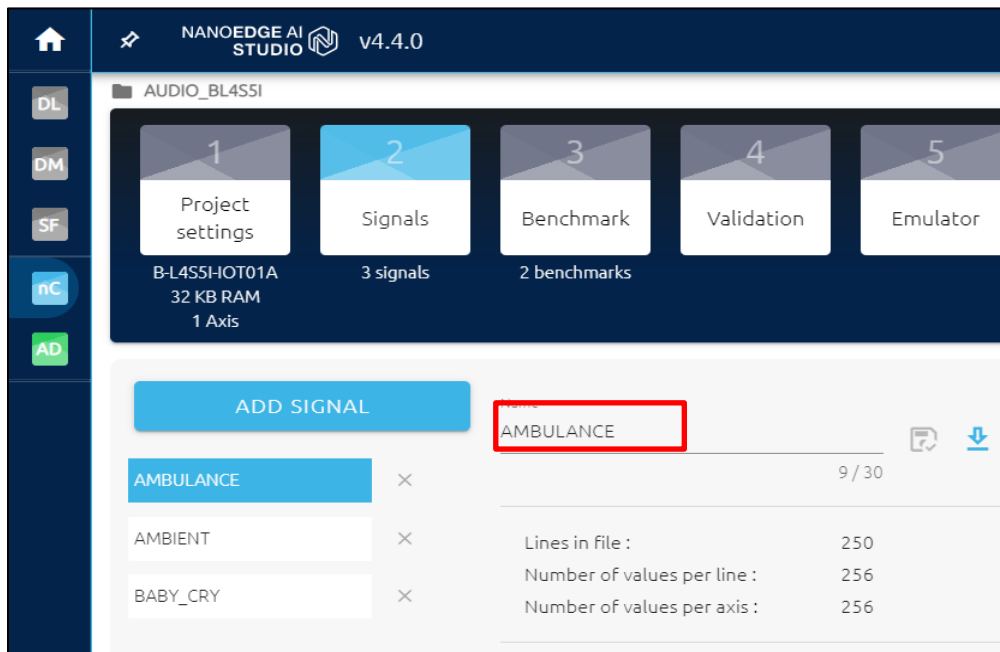
delete	line_index	nb_columns	1	2	3	4	5	6	7	8
☐	1	256	290	333	379	428	483	541	598	650
☐	2	256	546	584	622	654	675	681	680	675
☐	3	256	580	614	406	376	357	363	394	431
☐	4	256	657	620	577	534	497	473	462	462

PREVIOUS

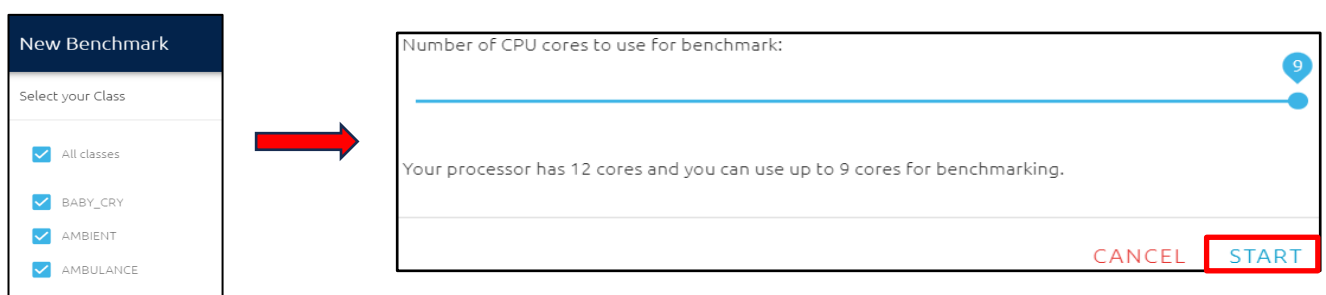
IMPORT

Preview lines
20

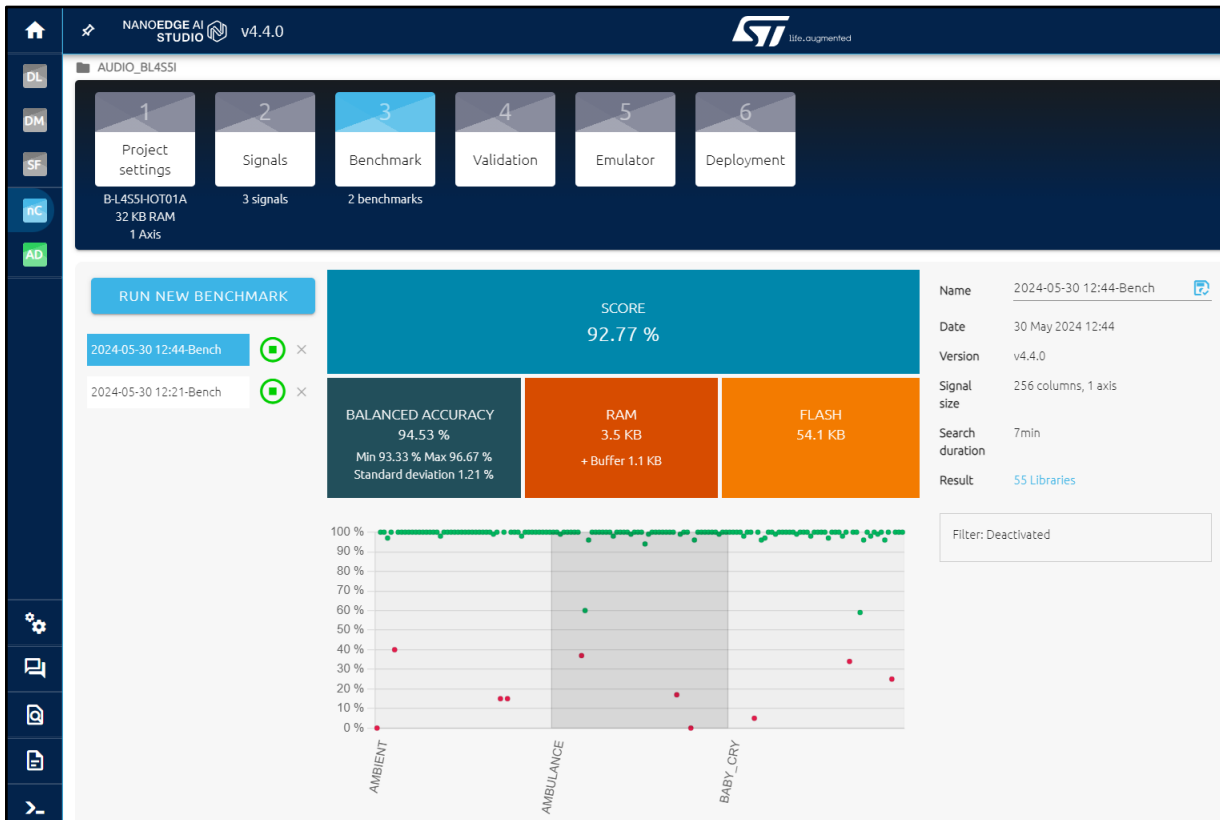
10. In the new window, rename the sample under **Name** (For example, if the audio sample is of a crying baby, name it Baby Cry), and click on the blue save icon to save the renamed sample.



11. In the same manner, collect two more audio samples following the previous steps. Name the three audio samples for example. **Baby Cry, Ambulance** and **Ambient Noise**.
12. In the third step of the NanoEdge AI Studio- **Benchmark**, click on **Run New Benchmark**. Next select **All classes** and click on **Start**. Now the Benchmark process will start and NanoEdge AI Studio will try to find the best library for your use case.



13. After the Benchmark has reached a **Score** of above 90% stop the benchmark process.



14. After the benchmark process is done, go to the next stage, **Emulator**. Click on **Serial Emulator** to move on to test the library you have selected on the NanoEdge AI Studio.

Benchmark

2024-10-24 21:47-Bench

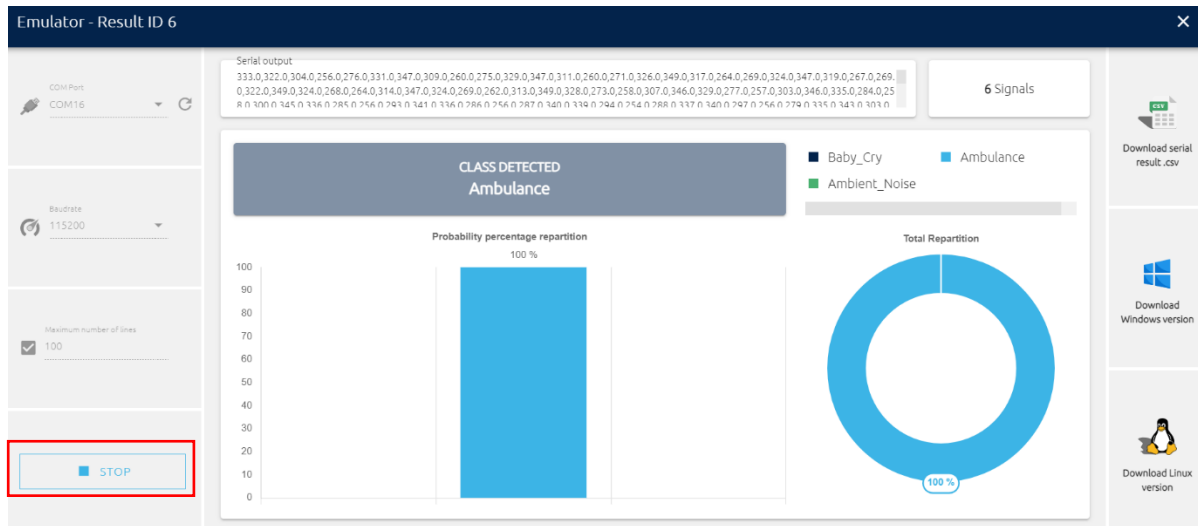
+ NEW EXPERIMENT

Please select at least one library to start a new experiment

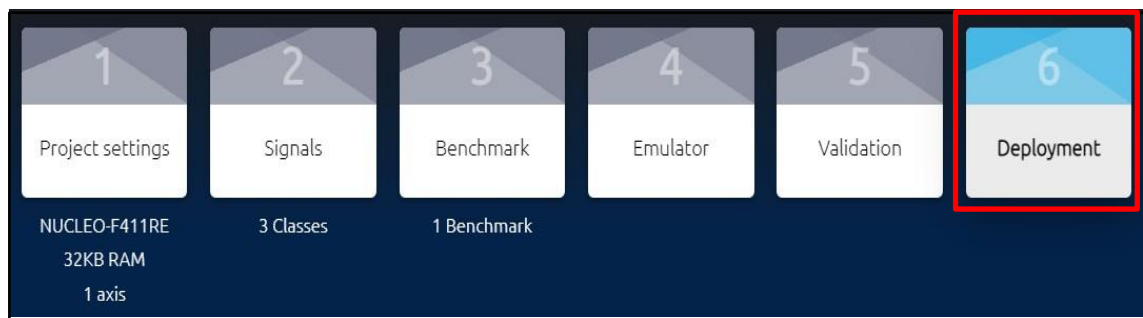
	Result ID	Score	Balanced accuracy	RAM	Flash	Execution time	Model	Report	Serial emulator	Compilation lib
	6	96.33 %	97.67 %	2.4 KB	1.7 KB	1.9 ms	XGB			

NOTE:The XGB (Extreme Gradient Boosting) model in NanoEdge AI Studio is a machine learning algorithm based on gradient boosting techniques, which iteratively builds an ensemble of decision trees to improve prediction accuracy. It is highly efficient and performs well with both classification and regression tasks, making it suitable for edge devices with limited resources. XGB models are known for their speed, scalability, and ability to handle large datasets effectively.

15. In the new window, select **From Serial (USB)**. Select **COM Port** according to info from Device Manager. Select Baud rate as 115200. Play the audio files on a device and click on **START/STOP**. Now you can test your AI library before deploying the model.



16. Finally go to **Deployment** stage, and click on **Compile Library**. Click on **Get Library**, and save your library on the PC.



17. Extract the Zip file , which you have downloaded from NanoEdgeAI studio. Knowledge.h and NanoedgeAI.h are the main header files needed for our further deployment into microcontroller.

Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
docs	File folder					
emulators	File folder					
knowledge	Header file	8 KB	No	8 KB	0%	19-12-2024 11:32
libneai.a	A File	7 KB	No	7 KB	0%	19-12-2024 11:32
metadata	JSON Source File	6 KB	No	6 KB	0%	19-12-2024 11:32
NanoEdgeAI	Header file	4 KB	No	4 KB	0%	19-12-2024 11:32

Observation:

- Model able to classify different sound signals.
- The trained ML model is saved for future deployment to the microcontroller.

Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is meant for the use of education purpose only.

Lab 9 c) Audio Scene Classification: Deployment of an ML based audio classification to identify & classify different sounds into microcontroller

Lab Overview

This lab focuses on interfacing an audio sensor with an STM32 microcontroller and deploying a Machine Learning model, created using NanoEdge AI Studio, onto the microcontroller. The goal is to enable real-time audio classification, allowing the microcontroller to make autonomous decisions based on sensor data.

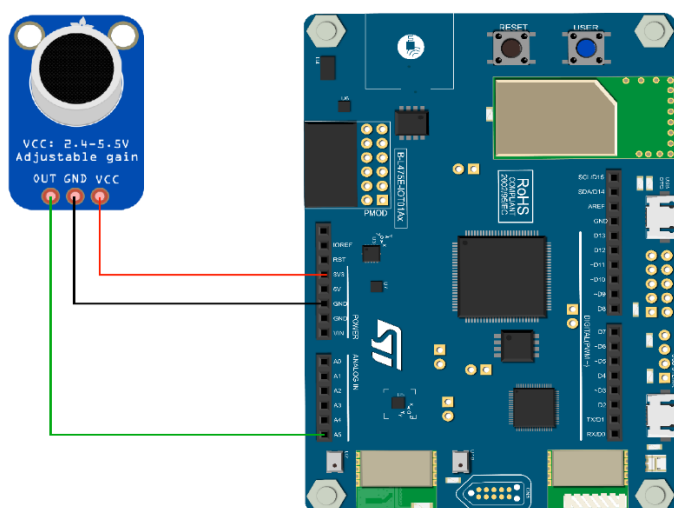
Objective of the Lab:

The objective of this experiment is to interface an audio sensor to an STM32 microcontroller and deploy the Machine Learning model built using the NanoEdge AI Studio into the microcontroller. This will give the microcontroller the ability to make a decision on the device itself based on classification on real time audio sensor data.

Requirements:

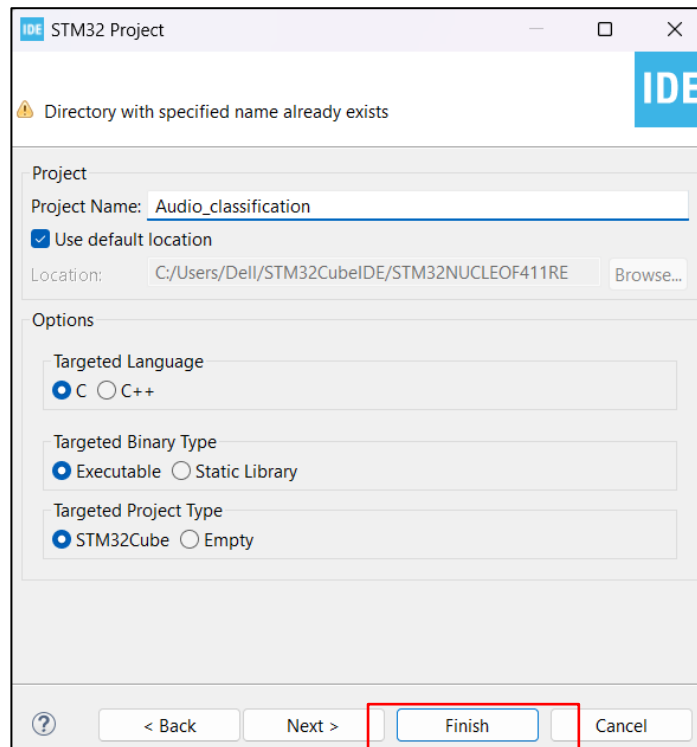
1. STM32 Cube IDE software.
2. Audio Sensor (Analog).
3. STM32 Microcontroller.
4. USB Cable for the microcontroller.
5. Jumper Wires.
6. PC or Laptop.

Connection Diagram:

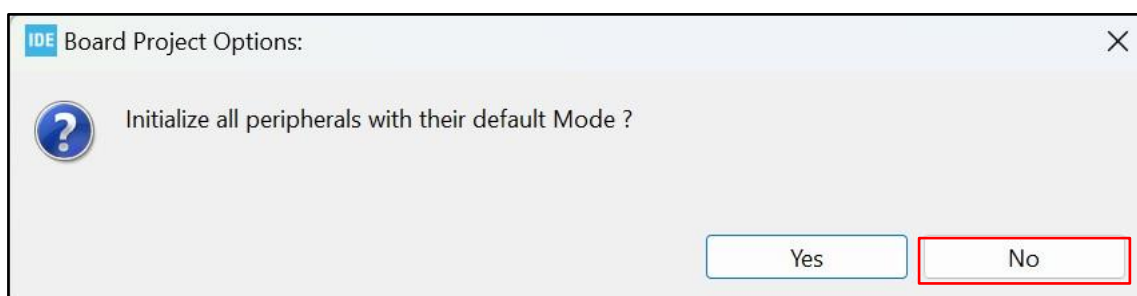


Steps Involved:

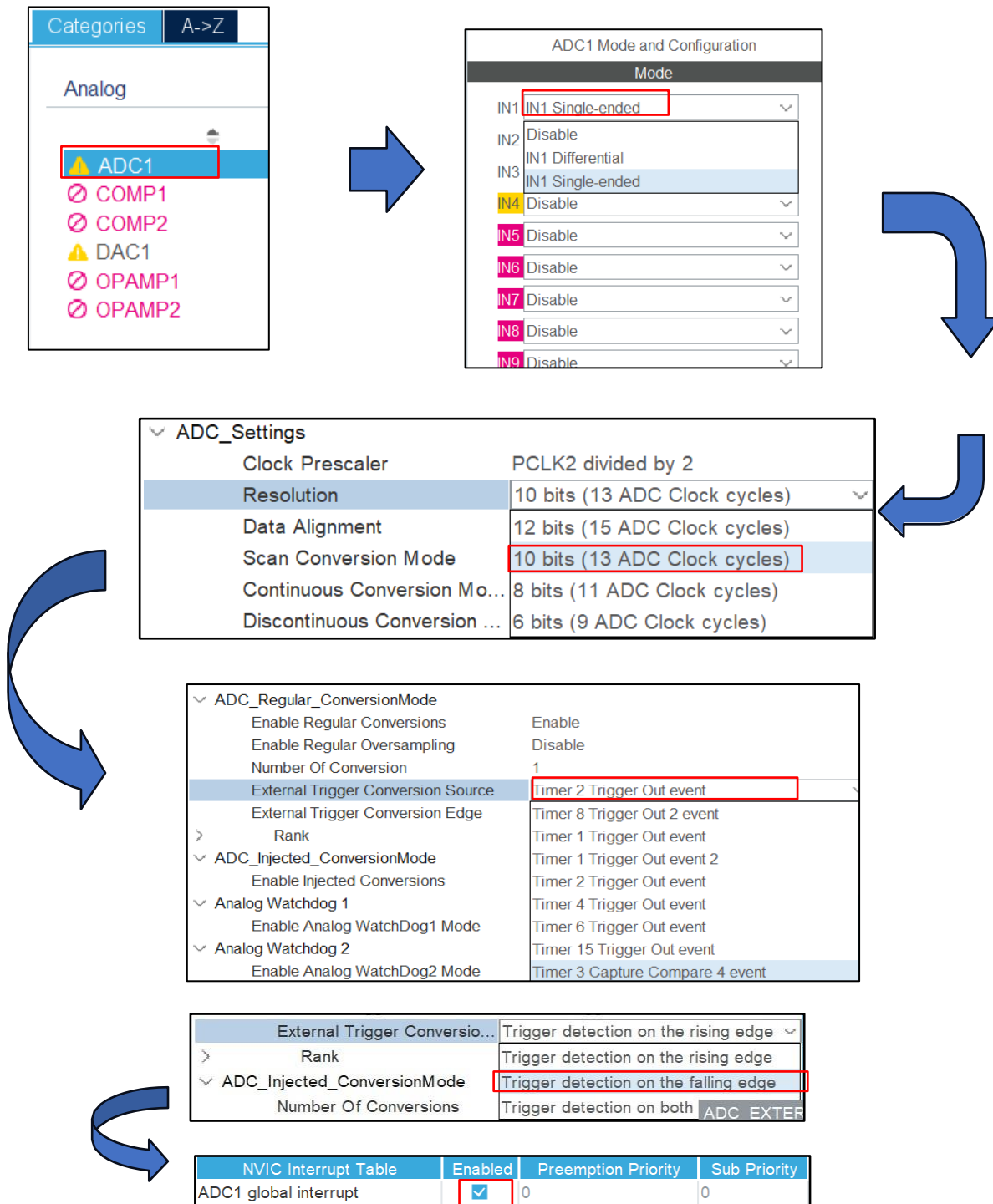
1. Select the microcontroller then name the project Audio_Classification. Follow the steps outlined in Experiment No: 9a to create the STM32 project file and click Finish to generate the project.



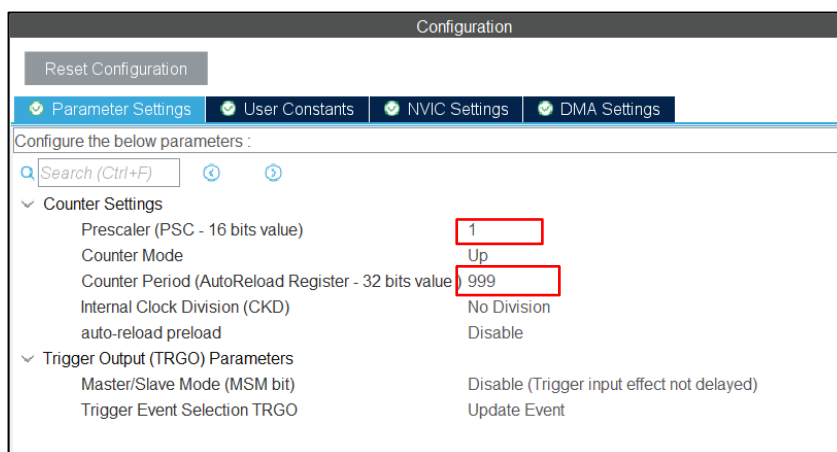
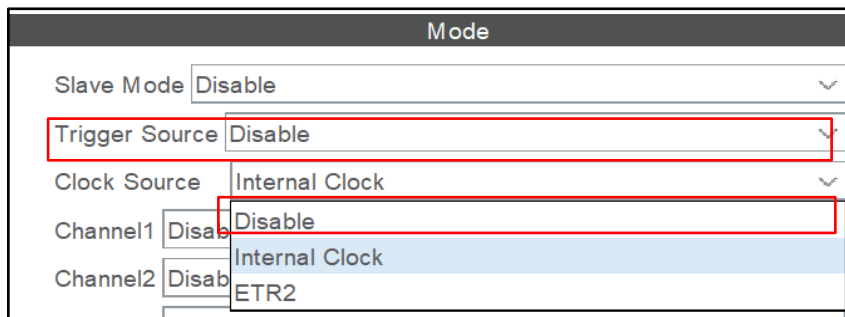
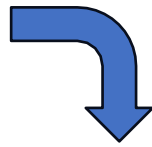
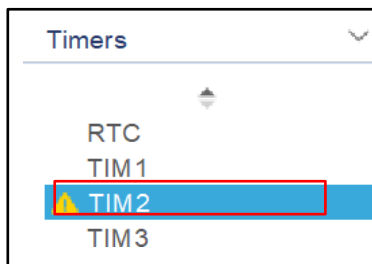
2. Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



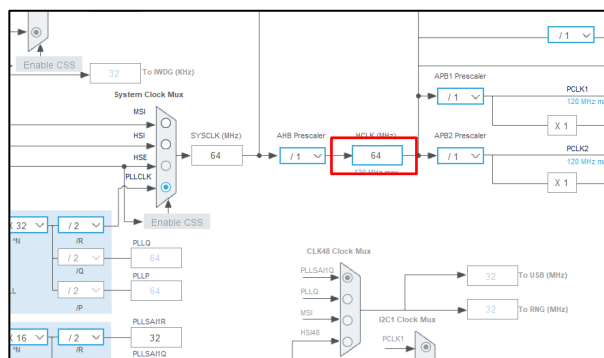
3. Next on the left-hand side **Categories** → **Analog** select **ADC1** then select **IN1** as IN1 Single ended under **Mode**. Under **Configuration** select **Parameter Settings**, change **Resolution** to 10-bits, then change the **External Trigger Conversion Source** to **Timer 2 Trigger Out Event** and **External Trigger Conversion Mode** to **Trigger detection on the falling edge**. Under **NVIC** enable the **ADC1 global interrupt**. Otherwise, you can also go to **System Core** → **NVIC** and enable the same.



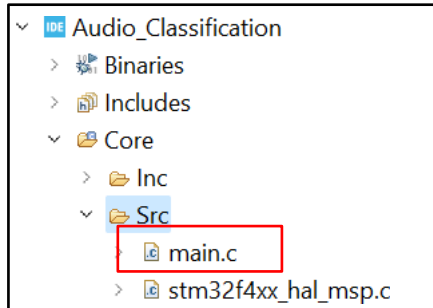
- Next go to Timer, select TIM2 and select Clock Source as Internal Clock. Update the Counter Period value as 999 and change Trigger Event Selection to Update Event.



5. Go to **Clock Configuration**, enter 64 as a value in **HCLK** and press enter to configure the peripheral and timer clocks. In this experiment we are trying to sample the audio signals at 32 KHz.



6. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example I have named my project as (Audio_Classification)**Audio_Classification**→**Core**→**Src**→**main.c** (double click to load the code).



7. Cube IDE automatically generates the initialization code based on the configurations you have done. Cube IDE uses HAL libraries. Below are the code snippets, please put your code in the appropriate places in the main.c file. The highlighted part where the class names are written should be the same as given in the NanoEdgeAI.h file downloaded from NanoEdge AI Studio in the Deployment stage.

```
22/* Private includes -----
23 /* USER CODE BEGIN Includes */
24 #include "stdio.h"
25 #include "string.h"
26 #include "knowledge.h"
27 #include "NanoEdgeAI.h"
28 /* USER CODE END Includes */
```

```
40/* Private macro -----
41 /* USER CODE BEGIN PM */
42 #define DATA_INPUT_USER 256
43 #define AXIS_NUMBER 1
44 #define CONFIRMATIONS NB (uint32_t) (3)
45 /* USER CODE END PM */
```

```
54 /* USER CODE BEGIN PV */
55 volatile uint32_t buffer_index = 0;
56 float mic_x = 0.0;
57 float mic_buffer[DATA_INPUT_USER * AXIS_NUMBER] = {0};
58 float output_class_buffer[CLASS_NUMBER]; // Buffer of cl
59 const char *id2class[CLASS_NUMBER + 1] = { // Buffer for
60     "unknown",
61     "Baby Cry",
62     "Ambulance",
63     "Noise",
64 };
```

```
73 /* USER CODE BEGIN PFP */
74 void Inference(void);
75 void fill_mic_buffer(void);
76 int io_putchar(int);
77 /* USER CODE END PFP */
```

```

111  /* USER CODE BEGIN Init */
112  enum neai_state error_code = neai_classification_init(knowledge);
113  if (error_code != NEAI_OK)
114  {
115      /* This happens if the knowledge does not correspond to the library or
116
117      printf("Knowledge initialization ERROR");
118      printf("%d", error_code);
119  }
120  else
121  {
122      printf("Knowledge initialization done");
123  }
124  /* USER CODE END Init */

```

```

138  /* USER CODE BEGIN 2 */
139  HAL_ADC_Start_IT(&hadcl1);
140  HAL_TIM_Base_Start(&htim2);
141  /* USER CODE END 2 */

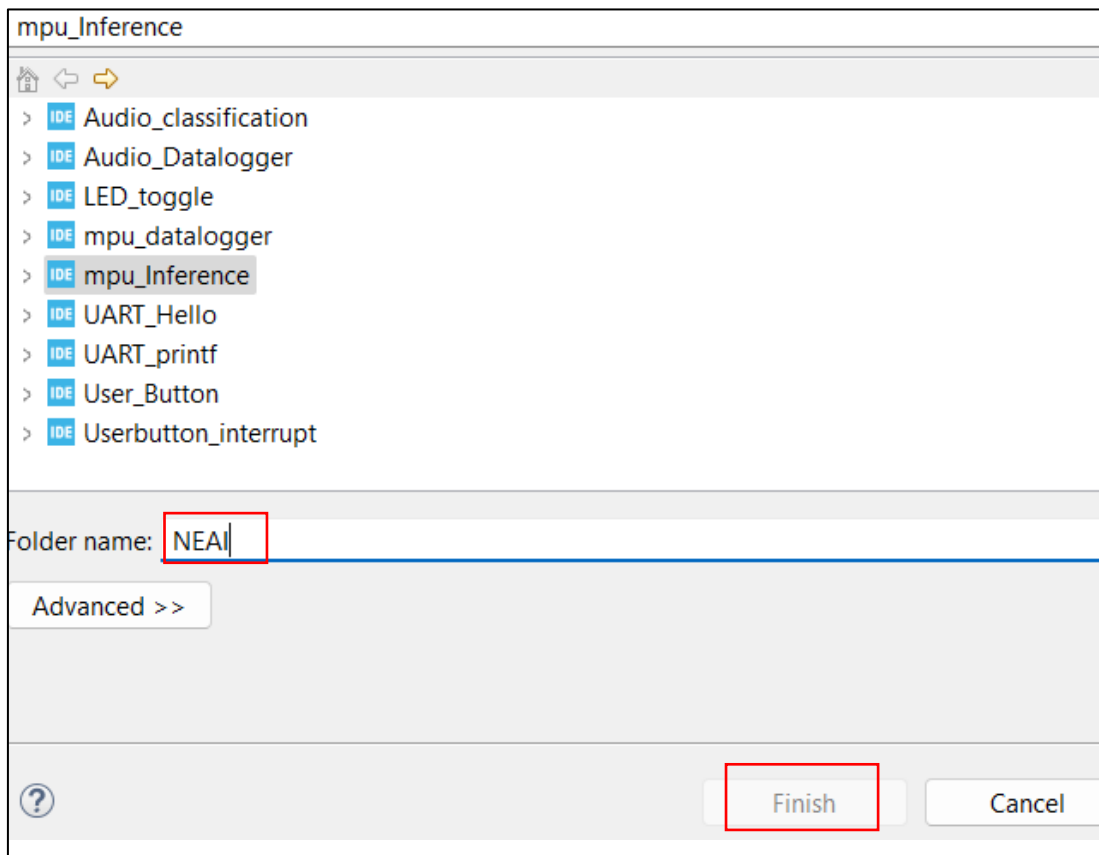
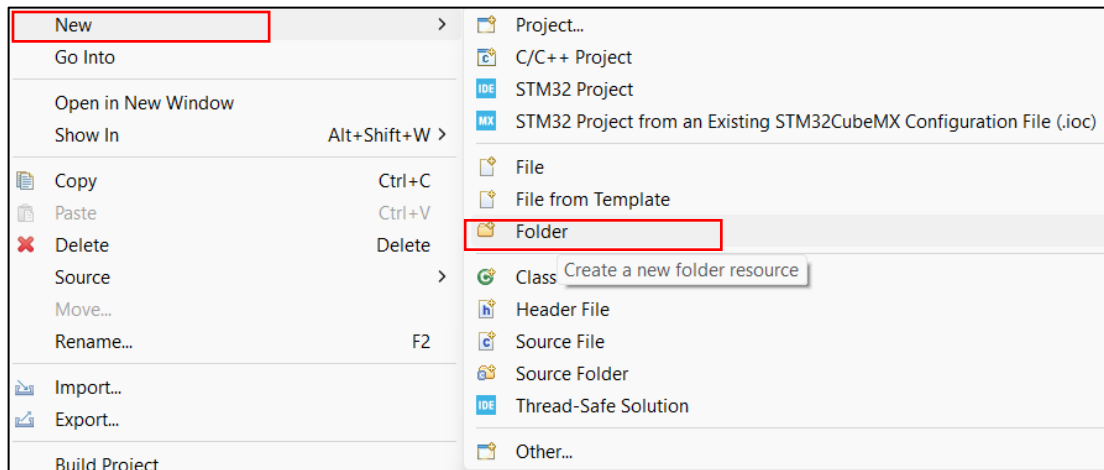
```

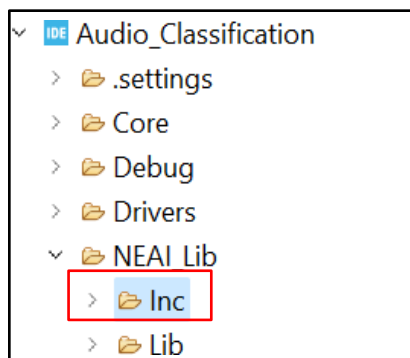
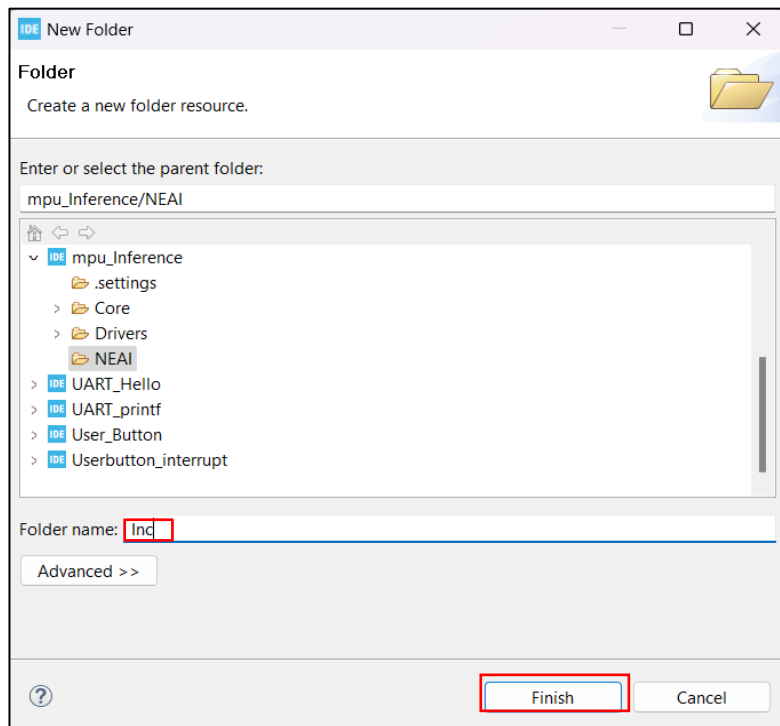
```

557  /* USER CODE BEGIN 4 */
558  void fill_mic_buffer()
559  {
560      for(int i=0; i<DATA_INPUT_USER; i++)
561      {
562          mic_buffer[AXIS_NUMBER*i]=mic;
563          HAL_Delay(3);
564      }
565  }
566
567  void Inference()
568  {
569      uint16_t i, id_class_t0, id_class_tn;
570      fill_mic_buffer();
571      neai_classification(mic_buffer, output_class_buffer, &id_class_t0);
572      for(i=0; i<CONFIRMATIONS_NB-1; i++)
573      {
574          fill_mic_buffer();
575          neai_classification(mic_buffer, output_class_buffer, &id_class_tn);
576          if(id_class_t0 != id_class_tn)
577          {
578              break;
579          }
580          if(id_class_t0 == id_class_tn)
581          {
582              printf("Detected Class:");
583              printf(id2class[id_class_t0]);
584              printf("\r\n");
585          }
586          else
587          {
588              printf("?");
589              printf("\r\n");
590          }
591      }
592  }
593
594  int __io_putchar(int ch){
595      HAL_UART_Transmit(&huart1, (uint8_t *) &ch, 1, HAL_MAX_DELAY);
596      return ch;
597  }
598  /* USER CODE END 4 */

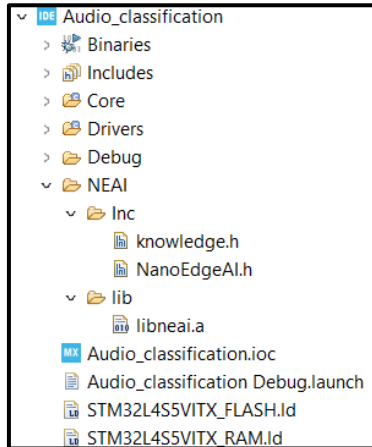
```

8. Right click on the Audio_Classification project and select **New→Folder**. Name the new folder as NEAI and click on finish. You will be able to see a folder named NEAI inside of your Audio_Classification project. Now right click on this newly created NEAI folder and create two separate new folders and name them Inc and Lib.

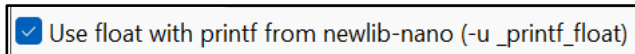
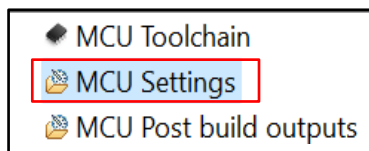




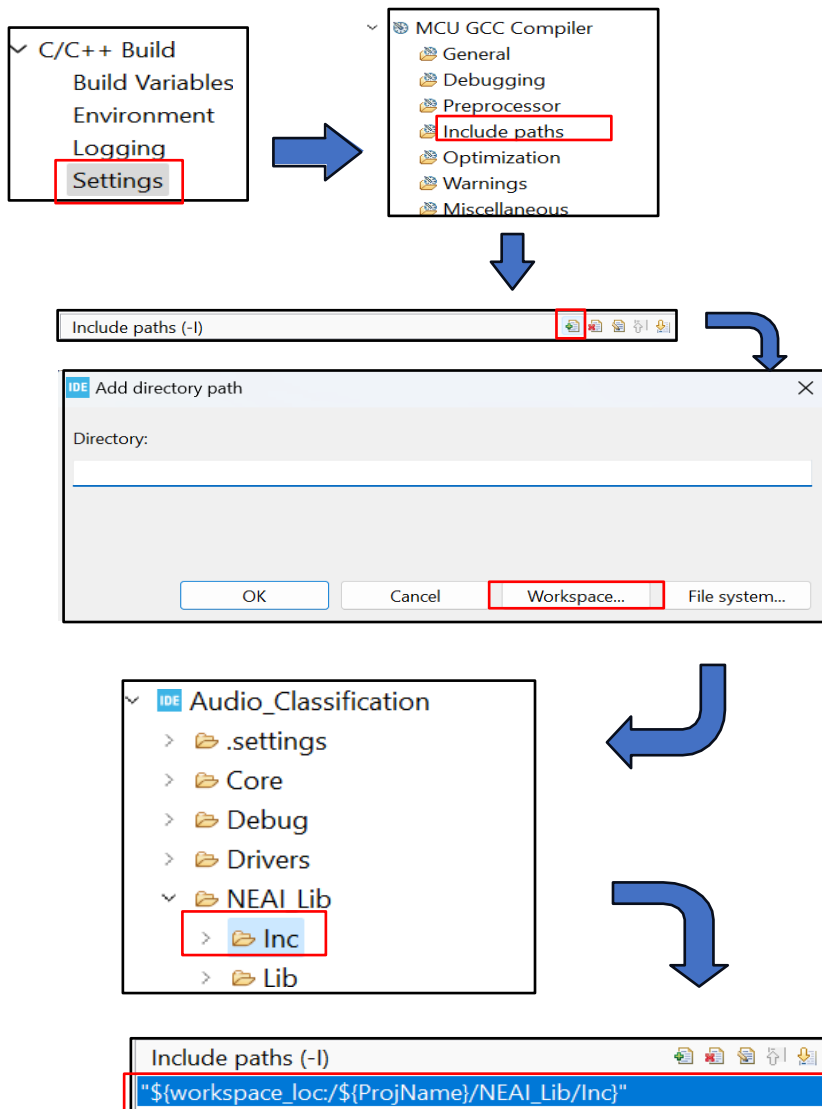
9. Now inside the Inc folder copy and paste the NanoEdgeAI.h and knowledge.h files. Inside the Lib folder copy the libneai.a file. These files are in the folder you downloaded from NanoEdge AI Studio during the deployment phase of Machine Learning model building.
10. Right click on the Audio_Classification project and select **Properties**. Go to **C/C++ Build→Settings**.



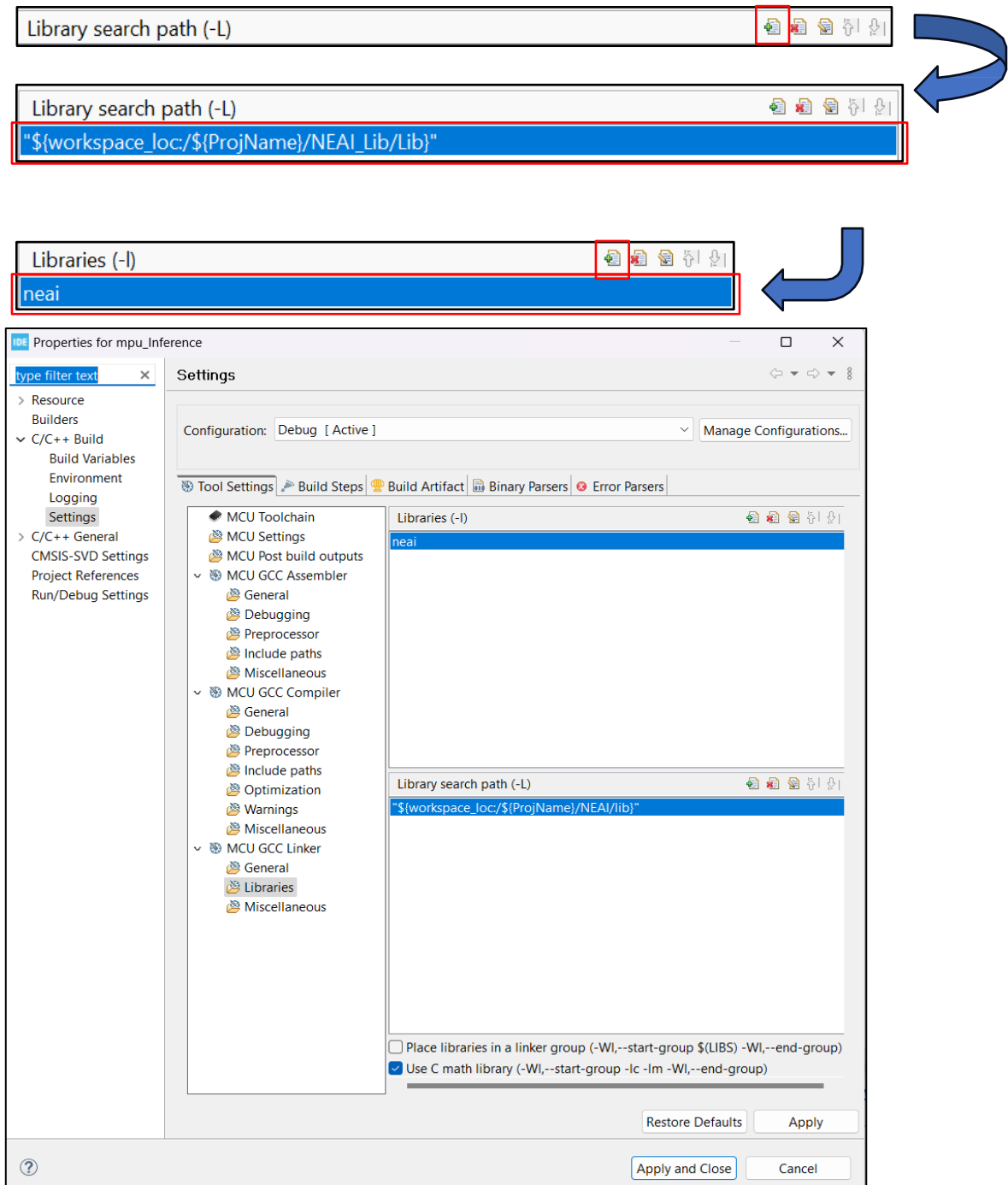
Next select **MCU Settings** and enable the option **Use float with printf from newlib-nano(-u _printf_float)**.



11. Next select **MCU GCC Compiler** → **Include paths** click on **Add** and in the new window select **Workspace** and select the **Inc** folder and click on **Ok**. The path of the **Inc** folder will be now added.



12. Next go to **MCU GCC Linker → Libraries**. In **Library search path (-L)** section, add the path of **Lib** folder similarly to the previous step of adding the **Inc** folder path. Next in the **Libraries(-l)** section click on **Add** and type **neai** then click on **Ok**.

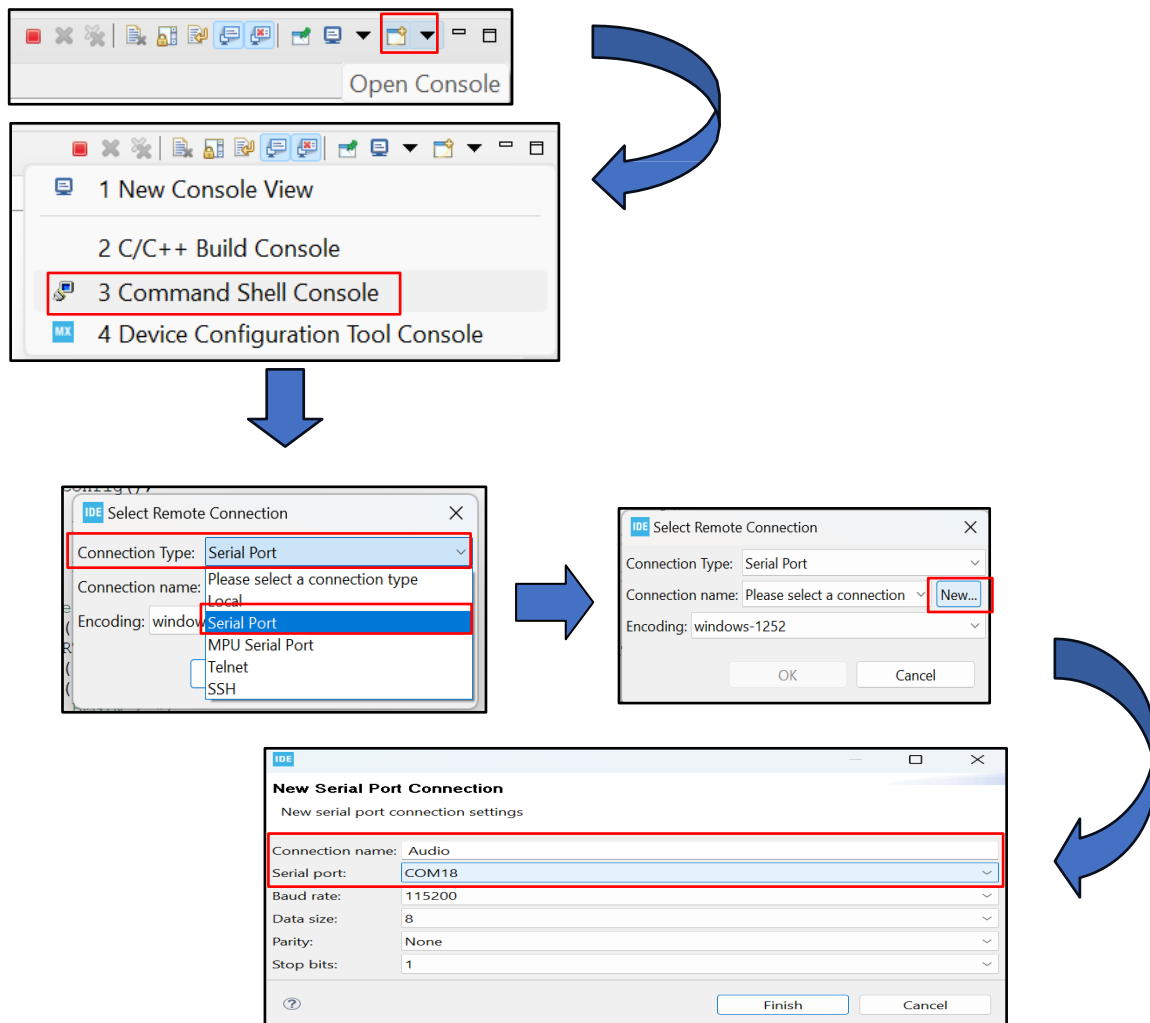


13. Now click on the build  symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

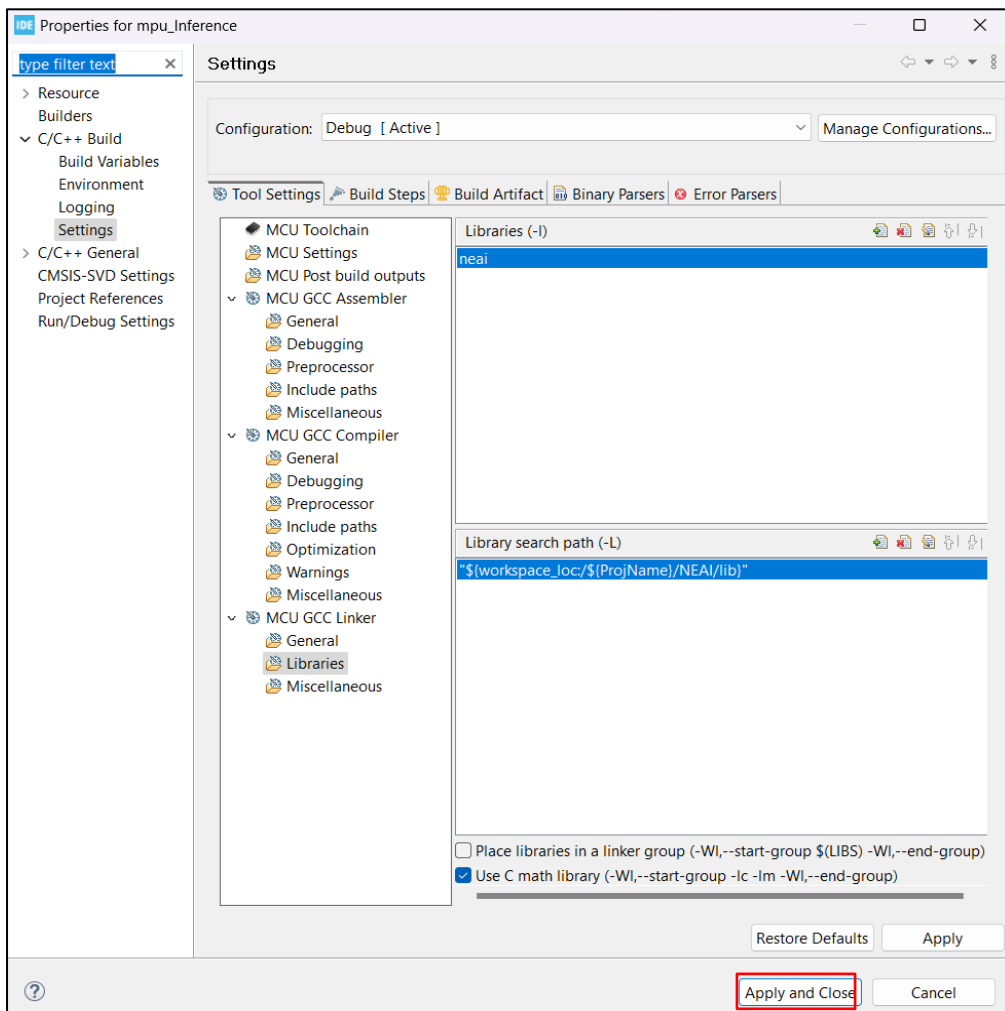
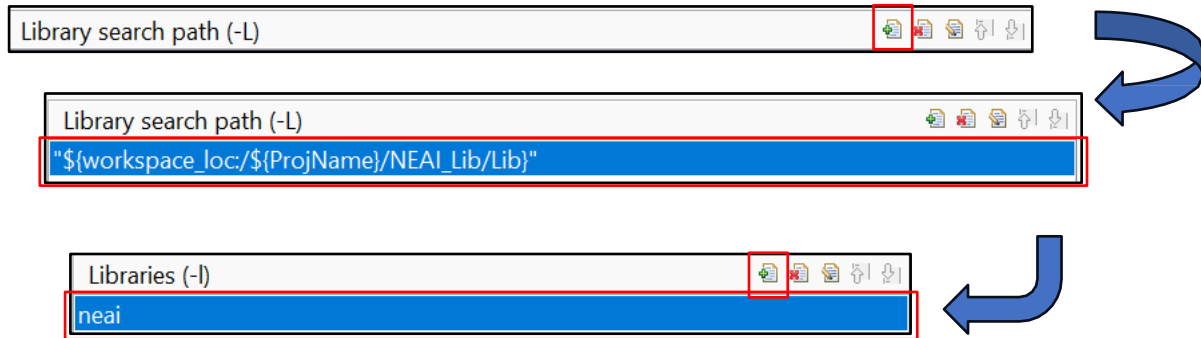
```
CDT Build Console [Audio_Classification]
101992    500    5908 108400    1a770 Audio_Classification.elf
Finished building: default.size.stdout

12:13:48 Build Finished. 0 errors, 0 warnings. (took 1s.416ms)
```

14. Next connect your STM32 board with your audio sensor connect it to your PC and click on the **Debug** icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.
15. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.



16. Next go to **MCU GCC Linker → Libraries**. In **Library search path (-L)** section, add the path of **Lib** folder similarly to the previous step of adding the **Inc** folder path. Next in the **Libraries(-l)** section click on **Add** and type **neai** then click on **Ok**.



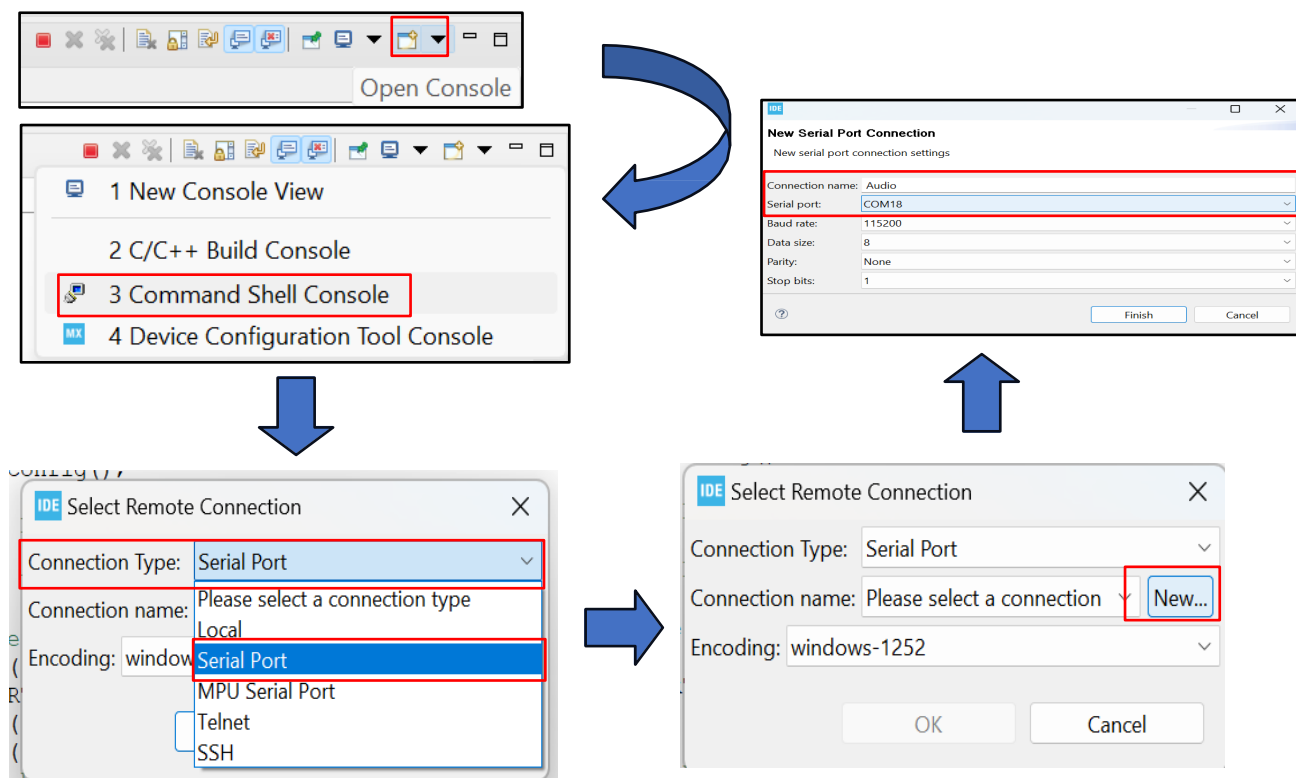
17. Now click on the build  symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

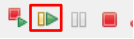
```
CDT Build Console [Audio_Classification]
101992      500      5908  108400   1a770 Audio_Classification.elf
Finished building: default.size.stdout

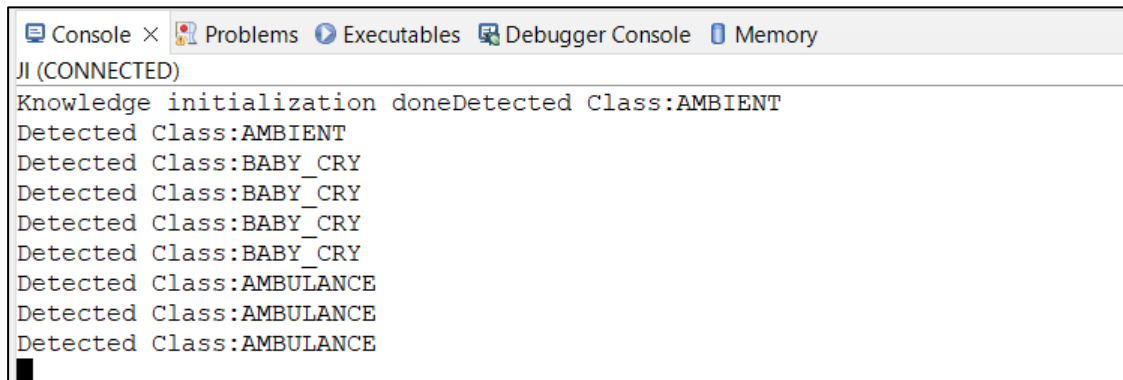
12:13:48 Build Finished. 0 errors, 0 warnings. (took 1s.416ms)
```

18. Next connect your STM32 board with your audio sensor connect it to your PC and click on the **Debug**  icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.

19. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen



20. Click on the **Resume**  icon to run your code. You should be able to see the output as different classes based on your project. Based on the audio sensor data, your model will identify which class the data belongs to.



```
Console x Problems Executables Debugger Console Memory
J1 (CONNECTED)
Knowledge initialization doneDetected Class:AMBIENT
Detected Class:AMBIENT
Detected Class:BABY_CRY
Detected Class:BABY_CRY
Detected Class:BABY_CRY
Detected Class:BABY_CRY
Detected Class:AMBULANCE
Detected Class:AMBULANCE
Detected Class:AMBULANCE
```

21. Before moving out of the debugging mode, click on **Disconnect**  close the console then click on the **Terminate**  icon. You will be moved out of the debugging mode.

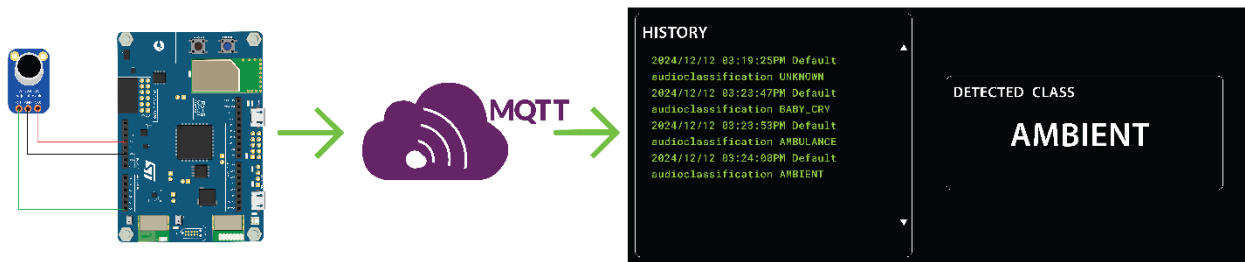
Note: All important steps and parts are highlighted with a red colour box for the proper understanding of the user. This document is for the use of education purpose only.

Lab 10: IoT Integration for Real-Time Audio Classification and Cloud Data Transfer

Lab Overview

In this lab, we will integrate **IoT technology** with the audio classification application that detects **baby cries**, **ambulance sirens**, and **ambient noise**. The focus is on initializing **MQTT communication**, enabling the **STM32 board** to send classification results to the **cloud** for real-time visualization and analysis. This lab builds upon the **Wi-Fi Initialization** and **Audio Classification Application** by adding **cloud integration** to support **remote monitoring**.

Objective of the Lab



The objective of this lab is to learn how to use **MQTT** for transferring **audio classification results** from the **STM32 board** to the **cloud**. Students will gain an understanding of integrating **IoT technology** with **real-time audio classification systems** for **cloud-based visualization and monitoring**.

Experimental Setup

Pre-lab Requirements

Before proceeding with this lab, you must complete the following steps to ensure all prerequisites are in place for successful completion:

1. **Audio Classification Application Lab**

Ensure that you have successfully developed the audio classification application that detects baby cries, ambulance sirens, and ambient noise on the STM32 board. This step provides the foundational classification data that will be sent to the cloud in this lab.

2. Wi-Fi Initialization Lab

Complete the Wi-Fi Initialization Lab to establish Wi-Fi connectivity on your IoT device. This enables the device to connect to a Wi-Fi network, which is necessary for cloud communication in this lab.

Important Instruction:

For this lab, you should not create separate projects for the **Audio Classification** and **Wi-Fi Initialization** labs. Instead, integrate both functionalities into a **single project**. Start with the project from the **Wi-Fi Initialization Lab** and add the **audio classification application code** to it. Follow the steps in this manual to add the required functionality for **detecting audio events** and **sending the classification results to the cloud via MQTT**.

By the end of this lab, you should have a single project where:

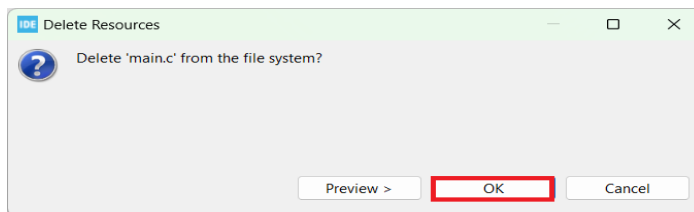
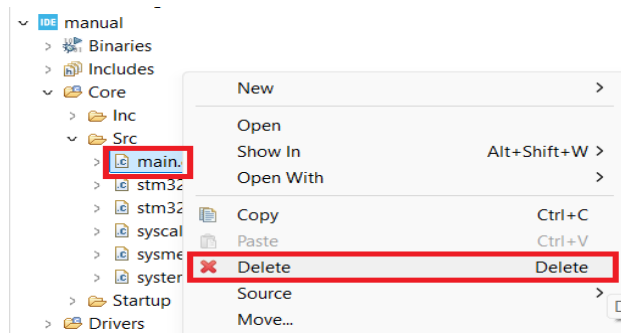
- The **audio classification system** is implemented and detects **baby cries, ambulance sirens, and ambient noise**.
- The **Wi-Fi module** is initialized and connects to the internet.
- Classification results are sent to **Adafruit.io** via **MQTT** for **real-time cloud-based visualization**.

Steps Involved:

1. Delete Existing main.c and wifi.c Files

- **Steps:**
 - a. Go to the **Src** folder inside the **Core** folder of your project.
 - b. If **main.c** and **wifi.c** files already exist, delete them.
- **Purpose:** This step ensures that you start with a clean slate for integrating the new files provided for this lab.

- Screenshot



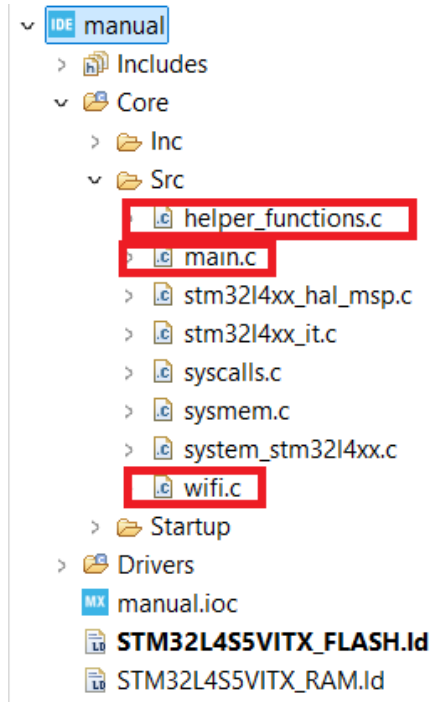
2. Add the main.c and wifi.c Files

- Steps:

a. Copy the **main.c** and **wifi.c** files provided for this lab into the **Src** folder.

- **Purpose:** These files contain the necessary logic for Wi-Fi initialization, MQTT setup, and data transmission. **main.c** integrates the sensor reading, MQTT initialization, and communication, while **wifi.c** manages the communication with the Wi-Fi module.

- **Screenshot:**



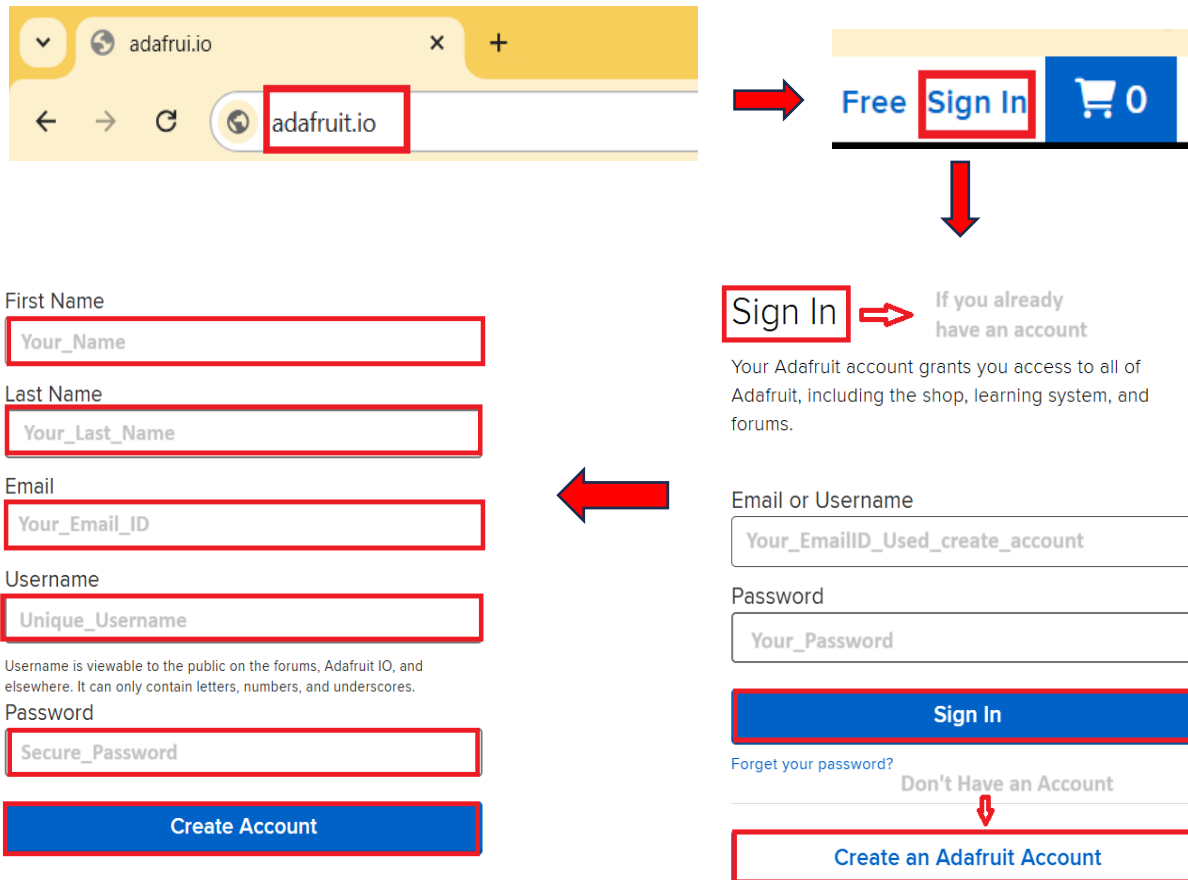
3. Create Adafruit Cloud Account

- **Steps:**

- a. Type "Adafruit.io" in Chrome search and go to the Adafruit website.
- b. Click on **Sign In** if you already have an account and log in with your credentials.
- c. If you do not have an account, click on **Sign Up** and create an account by providing the requested information.
- d. Once your account is created, you will be able to access the services offered by the Adafruit Cloud.

- **Purpose:** This step sets up your Adafruit account, providing access to Adafruit.io and the cloud services needed for the project.

- **Screenshot:**



First Name
Your_Name

Last Name
Your_Last_Name

Email
Your_Email_ID

Username
Unique_Username

Username is viewable to the public on the forums, Adafruit IO, and elsewhere. It can only contain letters, numbers, and underscores.

Password
Secure_Password

Create Account

Sign In

If you already have an account

Your Adafruit account grants you access to all of Adafruit, including the shop, learning system, and forums.

Email or Username
Your_EmailID_Used_create_account

Password
Your_Password

Sign In

Forget your password?

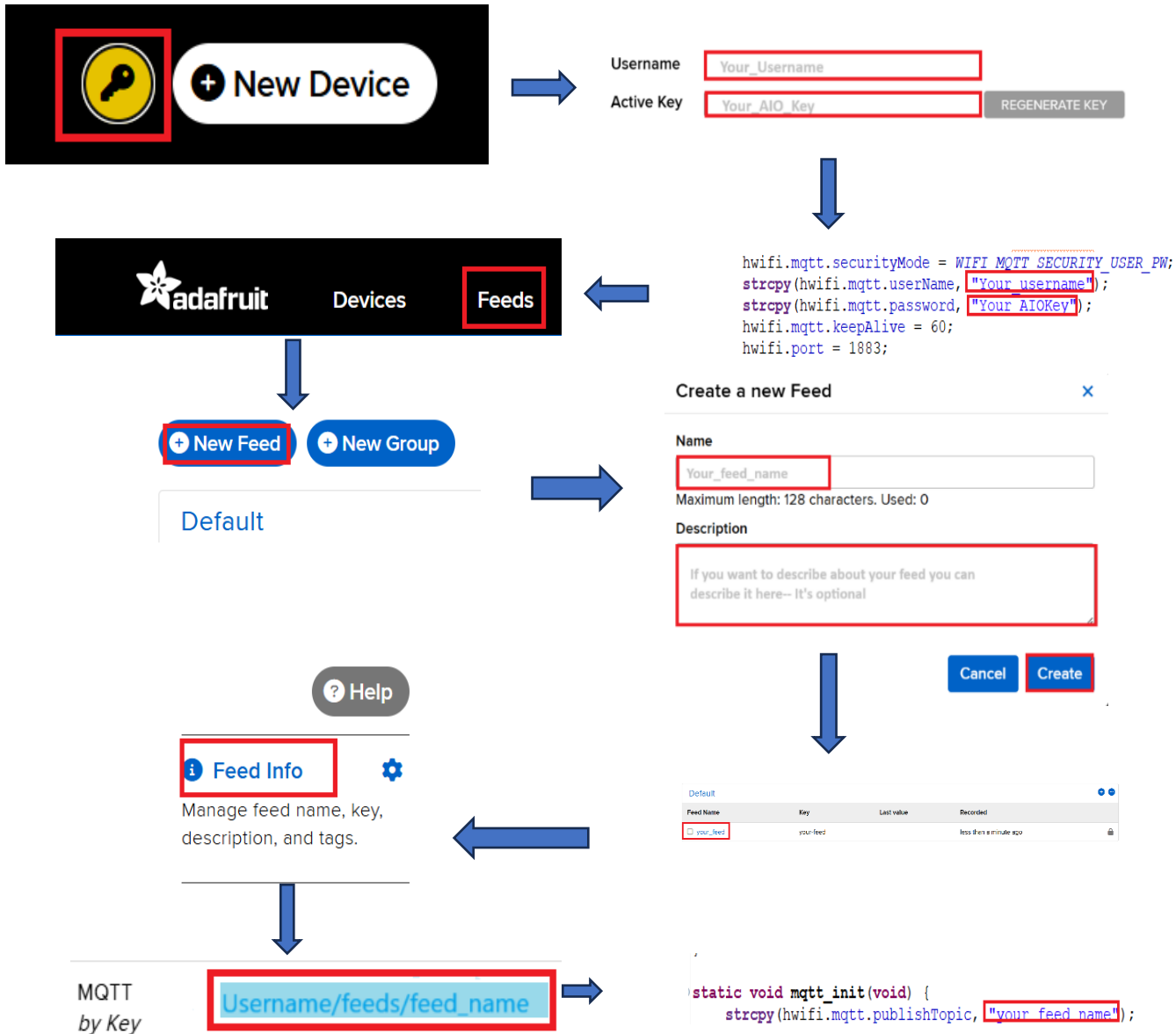
Don't Have an Account

Create an Adafruit Account

4. Modify MQTT Credentials in main.c

- **Steps:**
 - Open **main.c** and navigate to line 548 and 549.
 - Log in to your **Adafruit.io** account and copy the **username** and **AIoT key** (IoT key).
 - Replace the **username** at line 548 (hwifi.mqtt.userName) and the **AIoT key** at line 549 (hwifi.mqtt.password) with the credentials from your **Adafruit.io** account.
 - Go to **Adafruit.io > Feeds** and create one new feed to handle audio_classification.
 - Copy the **MQTT Key** of feed from the **Feed Info** section.
 - Replace the respective feed name in **main.c** for text message.
- **Purpose:** This step connects your IoT device to the correct cloud account and feeds, allowing data to be sent to the cloud. It also ensures proper authentication and secure communication.

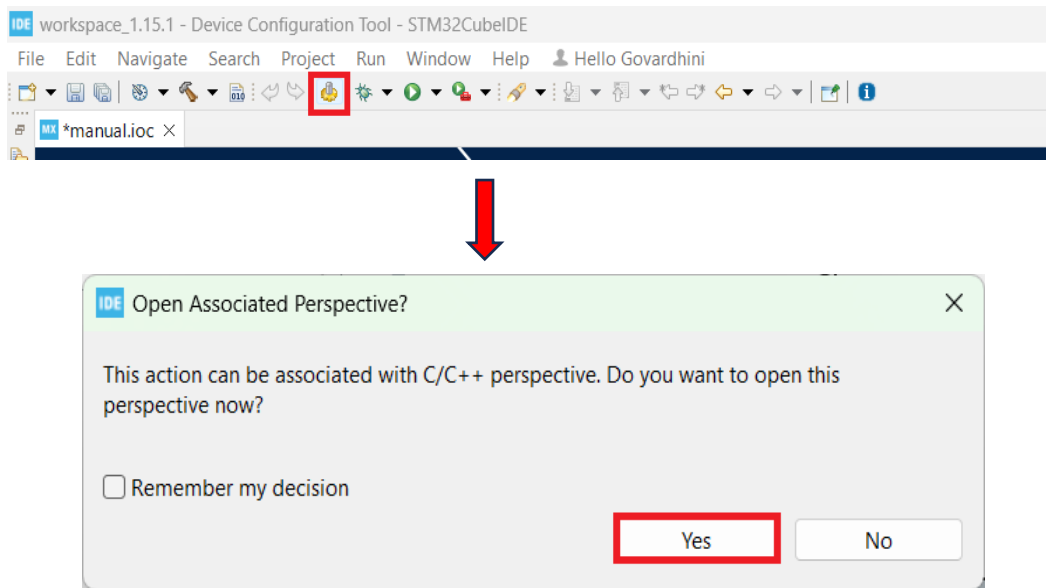
- **Screenshot:**



5. Generate Code

- **Steps:**
 - After modifying **main.c**, save the changes and click **Generate Code**.
 - Build the project to ensure there are no errors in the code.
- **Purpose:** This step generates the updated project code, which includes the newly added MQTT functionality and Wi-Fi setup.

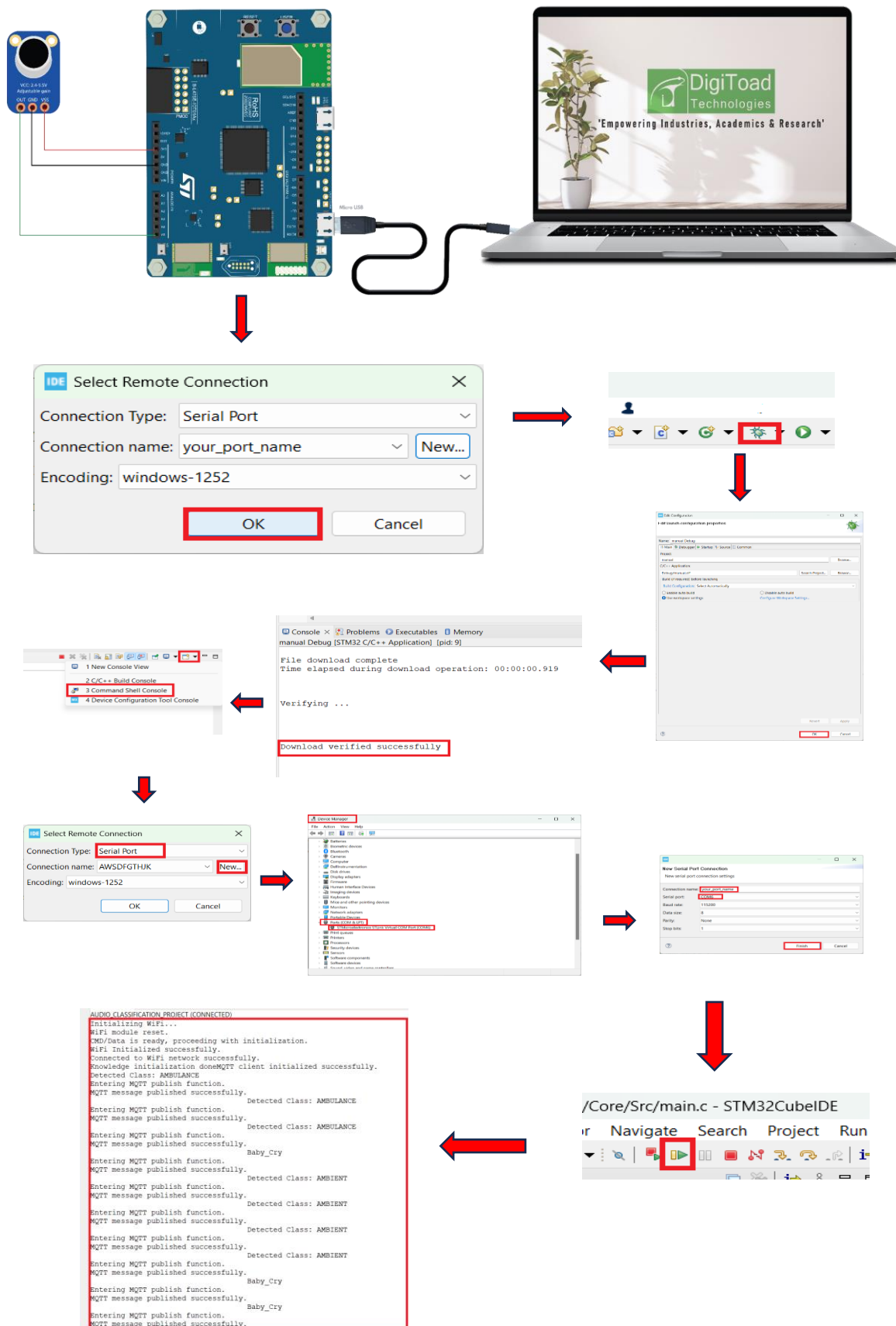
- **Screenshot:**



6. Verify Connection to Cloud

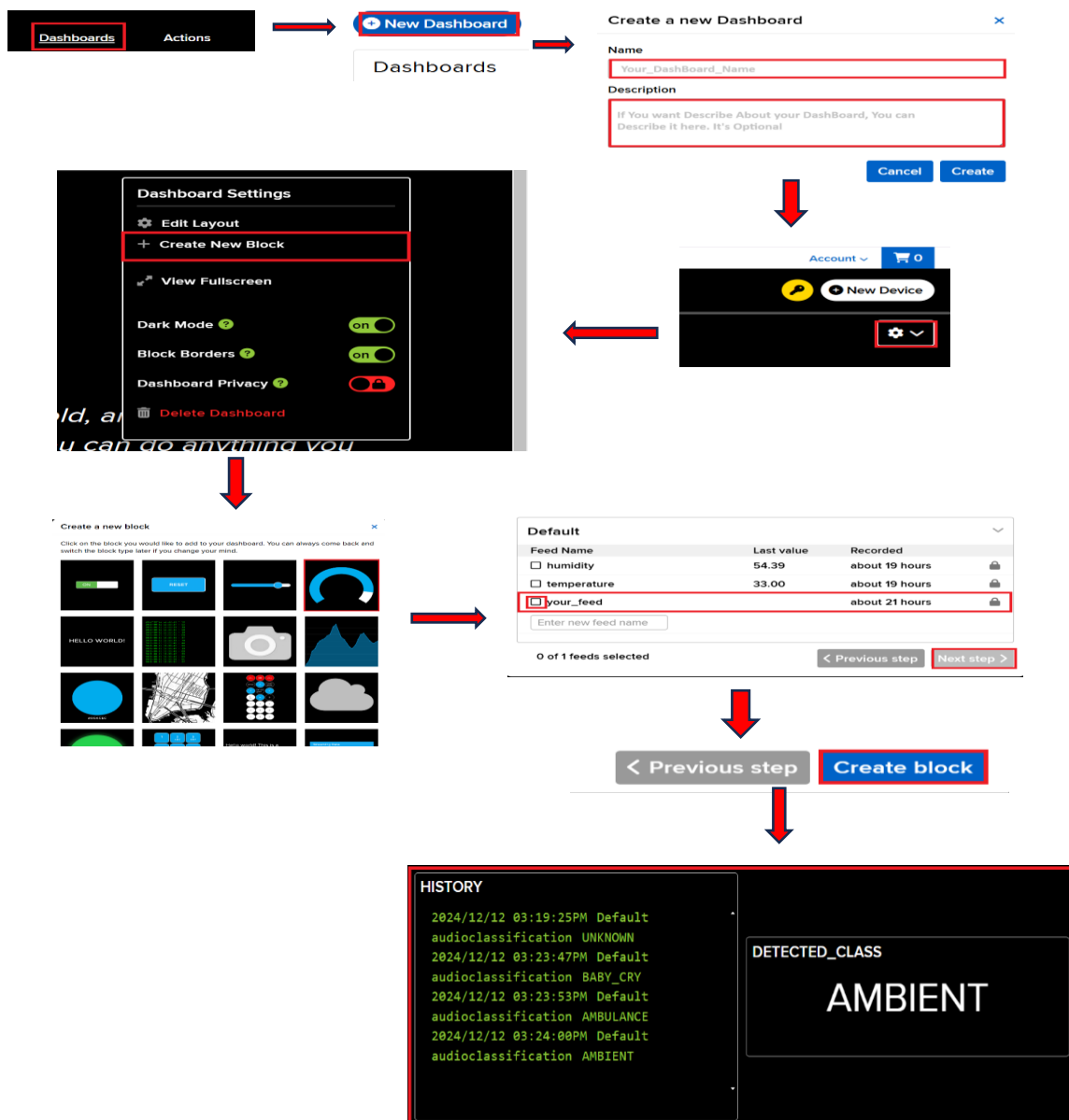
- **Steps:**
 - j. Ensure your **Wi-Fi hotspot** is turned on in your mobile device. Then, make sure that the **SSID** and **password** in your **device configuration** in **main.c** are set to match the credentials of the Wi-Fi network you want the device to connect to.
 - k. Ensure that all **MQTT configurations** in **main.c** are correctly modified for your Adafruit account, and debug the code to verify no errors are present.
 - l. Run the project by selecting **Debug** and then clicking **Run Debug**. Once it shows "Download Verified Successfully," proceed to create a serial monitor to view the data by selecting **Command Shell Console**. When you select **Command Shell Console**, choose **Serial Port** as the connection type. Click on **New**, provide an appropriate name for the console, and select the correct **port**. You can check the port associated with **ST-Link** by going to **Device Manager**, expanding the **Ports** section, and identifying the port connected to **ST-Link**.
 - m. Click **Finish**, then click **OK** to open the serial monitor. Next, in the navigation bar, select the **Resume** option. After resuming, the output will be displayed in the serial console. Ensure that the hotspot is turned on for the device with the configuration you provided in the previous step.
- **Purpose:** This step ensures the device is correctly connected to the Wi-Fi network, able to communicate with the MQTT broker, and successfully sending data to **Adafruit.io**. It also verifies that the data is being transmitted and displayed through the serial monitor.

• Screenshot:



7. Create and Configure the Adafruit Dashboard

- **Steps:**
 - Go to the **Adafruit.io Dashboard** and click **Create New Dashboard**.
 - In the dashboard, click the **Settings icon** and select **Edit Block**.
 - Choose a visual representation for the Audio_classification feed.
 - Create the block for Audio_classification feed.
 - Again **Reset** the board and you will be seeing the data getting updated in **feed** and **Dashboard** also.
- **Purpose:** This step allows you to visualize the Audio_Classification data from your IoT device on the **Adafruit.io Dashboard**. It enables you to monitor the data in real time.
- **Screenshot:**



Final Observation:

- **Wi-Fi Initialization:** The **Wi-Fi module** was successfully **initialized**, and the device **connected to the network** with the IP address of the network to which your board got connected to.
- **MQTT Client Initialization:** The **MQTT client** was successfully **initialized**, enabling **secure communication** between the **STM32 board** and **Adafruit.io**.
- **Audio Classification System:** The **audio classification system** accurately **detected baby cries, ambulance sirens, and ambient noise** using the **NanoEdge AI library**.
- **Real-Time Data Transmission:** **Classification results** (e.g., "BABY_CRY," "AMBULANCE," and "AMBIENT") were successfully **transmitted** to the **Adafruit.io feed** via **MQTT** in **real time**.

Lab 11 a) Design & Development of MPU6050 sensor datalogger for vibration data

Objective of the Lab:

The objective of this experiment is to create a datalogger code for 6-axis motion sensor (MPU6050). The datalogger code will create a buffer where all the motion sensor data sample will be stored, using which we will be able to create datasets of motion samples to build a machine learning model in the NanoEdge AI Studio.

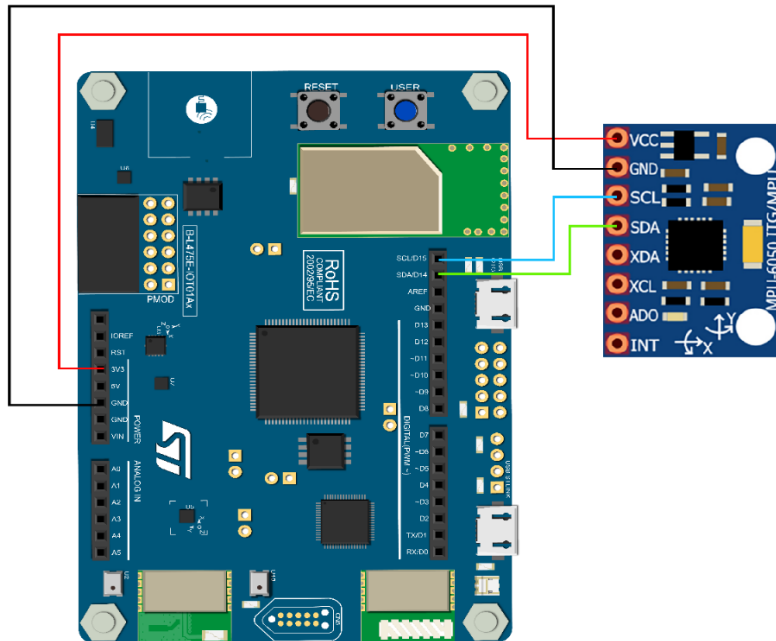
Overview of the Lab:

Vibration sensors are widely used in many applications. Here we learn about reading the MPU6050 and creating the data structure to format the sensor data..

Requirements:

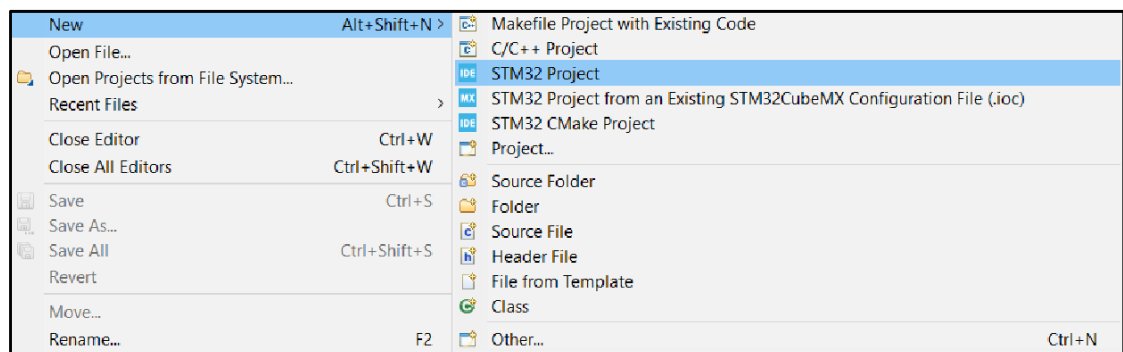
1. STM32 Cube IDE software.
2. MPU6050 sensor (I2C).
3. STM32 Microcontroller.
4. USB Cable for the microcontroller.
5. Jumper Wires.
6. Laptop or PC

Experimental Setup:



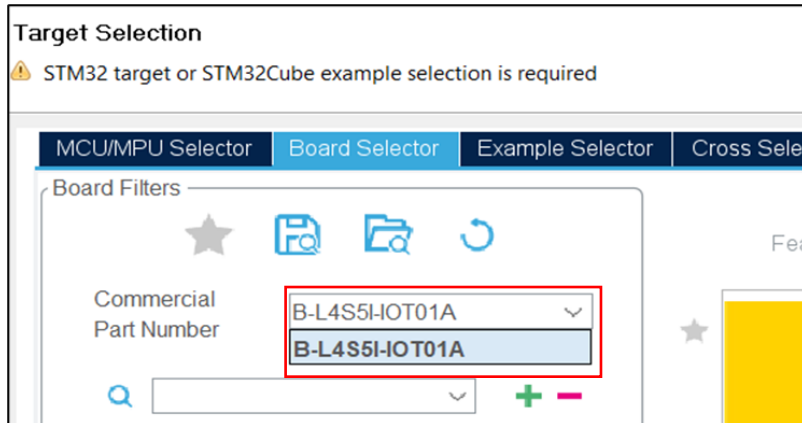
Steps Involved:

1. Click on **File**→**New**→**STM32 Project** to start your project on Cube IDE.

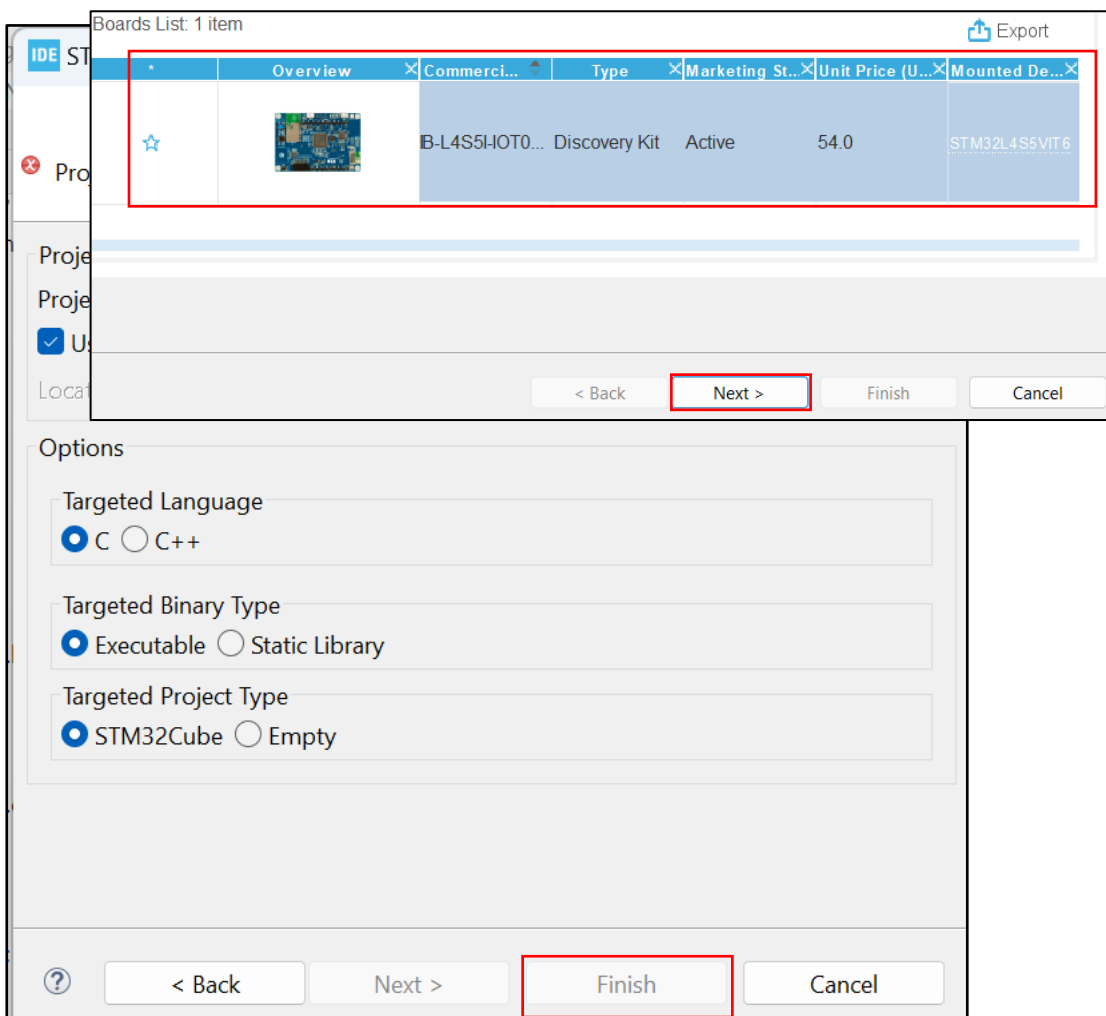


2. A **Target Selection** window will open. Click on **Board Selector**, where you need to select the microcontroller board you are working with.

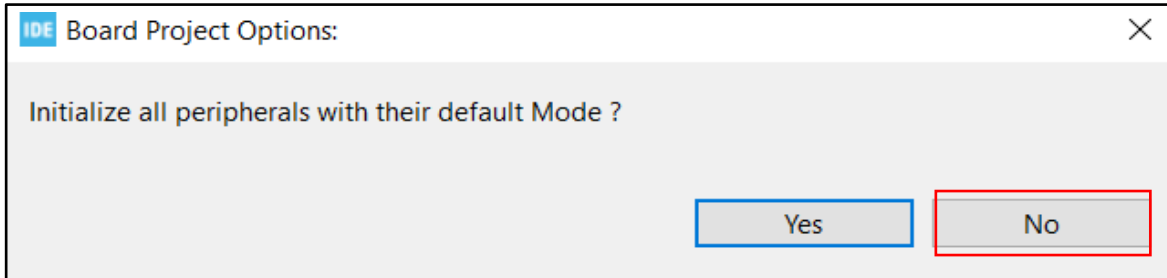
(NB: If you are having Nucleo-F401RE, you have to select the said Commercial Part Number)



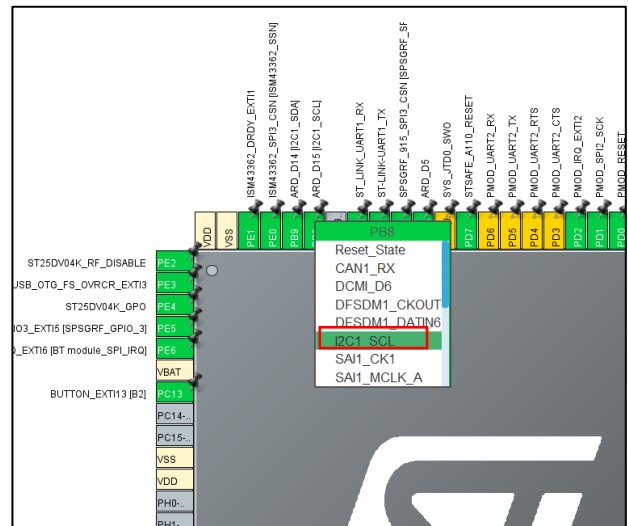
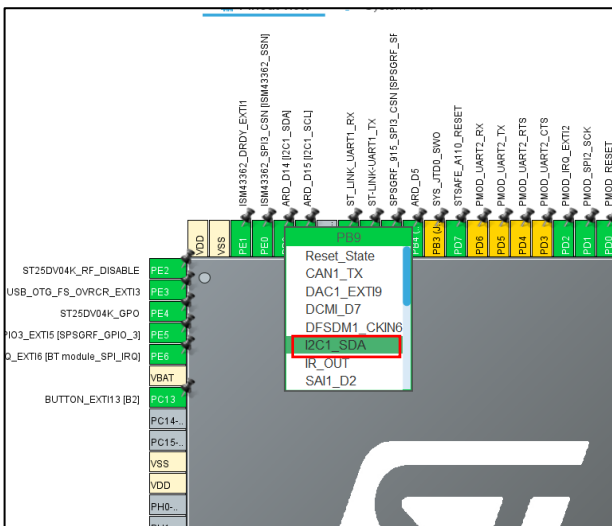
- After this on the right-hand side of the window, under **Board List** you will see the board you have selected. Click on the board and then click on **Next**. In the next window give your project a name, rest of the things will remain by default as it is for now. Click on **Finish**.



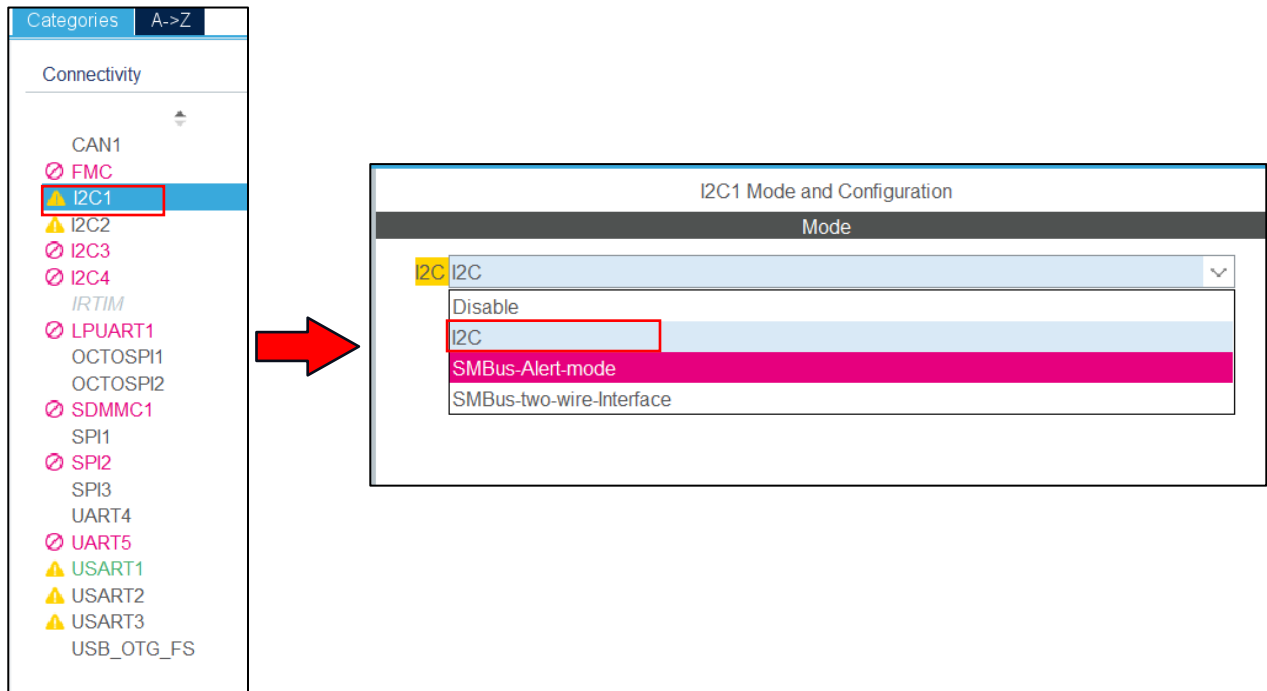
- Cube IDE will ask if you want to initialize all peripherals with their default mode, click on **No**.



- In the **Pinout & Configuration** tab, click on **PB8** pin and select it as an **I2C1_SCL** and **PB9** pin as an **I2C1_SDA**.

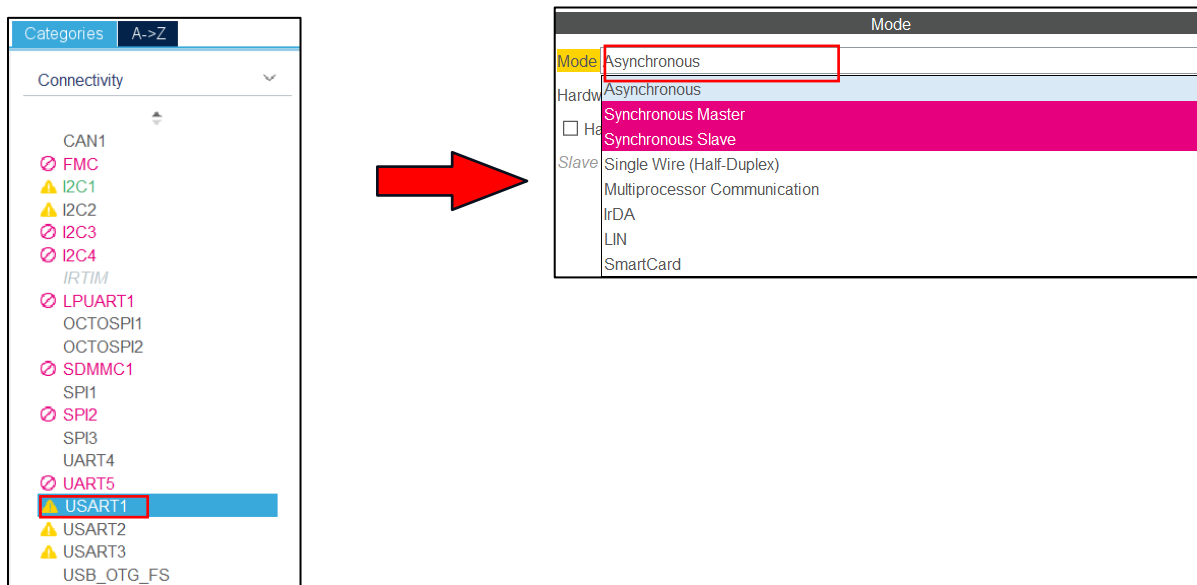


- Next on the left-hand side under **Categories** → **Connectivity**, select **I2C1** and enable it.

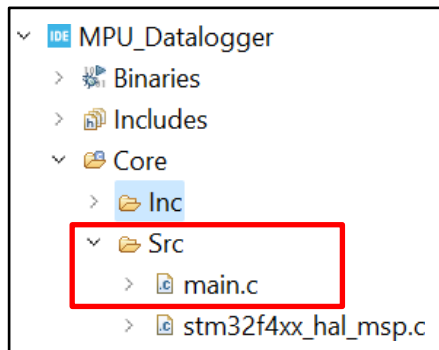


NOTE: I2C (Inter-Integrated Circuit) is a serial communication protocol that allows multiple devices to communicate over two wires: a data line (SDA) and a clock line (SCL). It uses a master-slave architecture, where the master controls the communication and multiple slaves can be addressed by unique addresses. MPU6050 is a 6 axis motion sensor in that we are making use of 3 axis accelerometer data for our project,,data will be aligned as x-y-z format in your serial console.

- Next on the left-hand side under **Categories →Connectivity**, select USART1 and enable asynchronous mode.



8. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example, I have named my project as (MPU_Datalogger) **MPU_Datalogger**→**Core**→**Src**→**main.c** (double click to load the code).



9. Now open your project tree **MPU_Datalogger**→**Core**→**Inc**. Right click on your **Inc** folder and add the header file **mpu.h** (you can copy and paste it from the code folder provided along with)
10. Press on to Device configuration tool to activate your header file into main.c file

11. Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE uses HAL libraries. Below are the code snippets, please put your code in the appropriate places in the **main.c** file.

```

22/* Private includes -----
23/* USER CODE BEGIN Includes */
24#include "mpu.h"
25#include "stdio.h"
26#include "string.h"
27/* USER CODE END Includes */

34/* Private define -----
35/* USER CODE BEGIN PD */
36#define DATA_INPUT_USER 256
37#define AXIS_NUMBER 3
38/* USER CODE END PD */

50/* USER CODE BEGIN PV */
51float mpu_buffer[AXIS_NUMBER * DATA_INPUT_USER] = {0};
52/* USER CODE END PV */

59/* USER CODE BEGIN PFP */
60int __io_putchar(int);
61void fill_mpu_buffer();
62void Log();
63/* USER CODE END PFP */

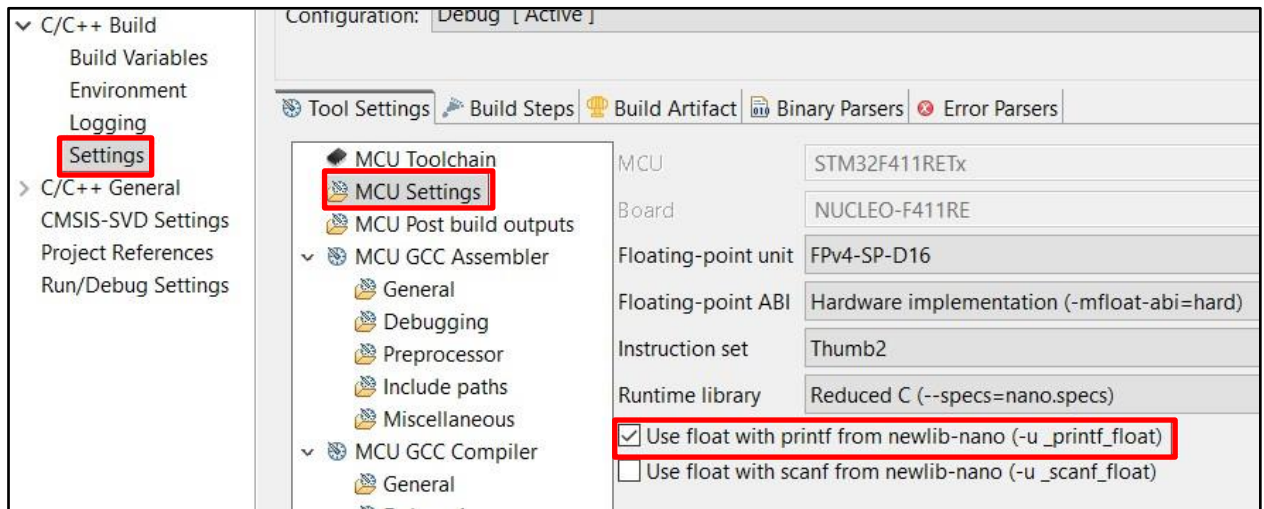
100/* USER CODE BEGIN 2 */
101MPU6050_Init();
102HAL_Delay(1000);
103/* USER CODE END 2 */

106/* USER CODE BEGIN WHILE */
107while (1)
108{
109    Log();
110/* USER CODE END WHILE */

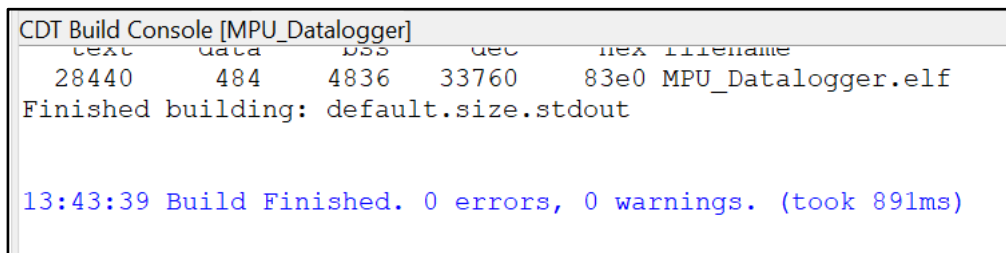
464/* USER CODE BEGIN 4 */
465int __io_putchar(int ch)
466{
467    HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
468    return ch;
469}
470
471void fill_mpu_buffer() {
472    for (int i = 0; i < DATA_INPUT_USER; i++) {
473        MPU6050_Read_Accel(&accel_data);
474        //MPU6050_Read_Gyro();
475        mpu_buffer[AXIS_NUMBER * i] = Ax; // for gyro replace Ax with Gx
476        mpu_buffer[AXIS_NUMBER * i + 1] = Ay; // for gyro replace Ay with Gy
477        mpu_buffer[AXIS_NUMBER * i + 2] = Az; // for gyro replace Az with Gz
478    }
479}
480
481void Log() {
482    fill_mpu_buffer();
483    for (int i = 0; i < DATA_INPUT_USER; i++) {
484        printf("%.2f",mpu_buffer[AXIS_NUMBER * i]);
485        printf(" ");
486        printf("%.2f",mpu_buffer[AXIS_NUMBER * i + 1]);
487        printf(" ");
488        printf("%.2f",mpu_buffer[AXIS_NUMBER * i + 2]);
489        printf(" ");
490    }
491    printf("\r\n");
492    HAL_Delay(100);
493}
494}
495/* USER CODE END 4 */

```

12. Right click on the MPU_datalogger project and select **Properties**. Go to **C/C++ Build** → **Settings**. Next select **MCU Settings** and enable the option **Use float with printf from newlib-nano (-u_printf_float)**.

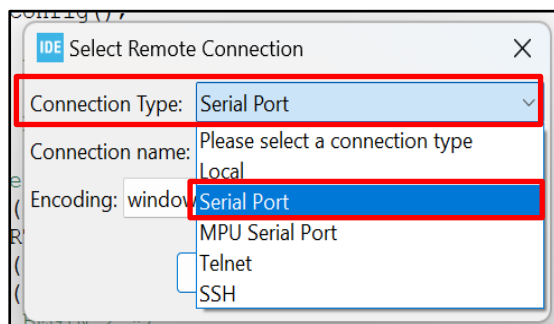
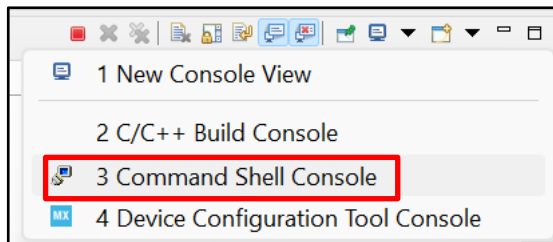
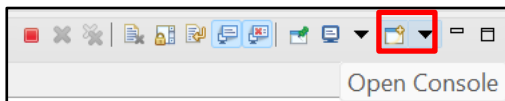


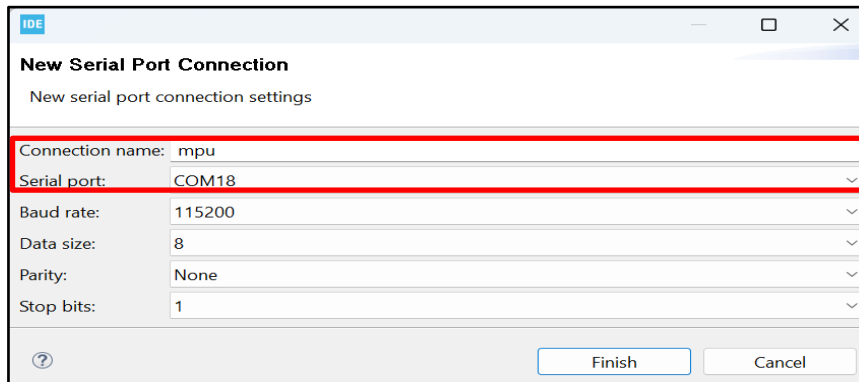
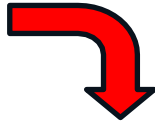
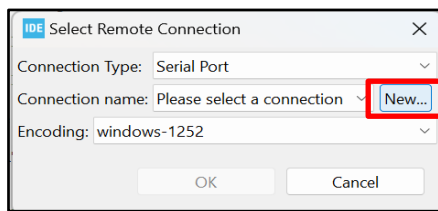
13. Now click on the build  symbol on the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.



14. Next connect your STM32 board with your MPU sensor connect to it to your PC and click on the **Debug**  or the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.

15. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.





16. Click on the **Resume** icon  to run your code. You should be able to see the values of `mpu6050sensor` in the **Console**.

```
mpu (CONNECTED)
7 7.66 -2.10 -0.55 7.67 -2.11 -0.55 7.62 -2.11 -0.55 7.62 -2.12 -0.5
-0.57 7.61 -2.08 -0.62 7.64 -2.03 -0.60 7.62 -2.08 -0.58 7.66 -2.06
-2.06 -0.57 7.67 -2.09 -0.60 7.61 -2.12 -0.58 7.63 -2.09 -0.60 7.62
7.62 -2.07 -0.60 7.62 -2.07 -0.60 7.62 -2.08 -0.61 7.58 -2.09 -0.58
0.60 7.60 -2.12 -0.59 7.67 -2.14 -0.57 7.63 -2.14 -0.57 7.63 -2.13 -
.06 -0.56 7.56 -2.09 -0.60 7.55
```

17. Before moving out of the debugging mode, click on **Disconnect**  and close the console then click on the **Terminate**  icon. You will be moved out of the debugging mode.

Observation:

- You can be able to see the value of mpu sensor in the form of buffer containing 768 samples.

Note: All important steps and parts are highlighted with a red color box for the proper understanding of the user. This document is for the use of education purpose only.

Lab 11 b) Anomaly Detection: Use NanoEdge AI to find out the best anomaly detection model for a given sensor dataset (vibrational analysis)

Lab Overview:

Anomaly Detection technique is used to find irregularities in the data pattern. Here, we collect vibration sensor for different operating conditions of a Machine/Fan and use NanoEdge AI Studio to find Anomaly Detection model to detect the vibration shifts or irregularities. The same model is then deployed on the AIoT board to detect vibration anomalies in real time

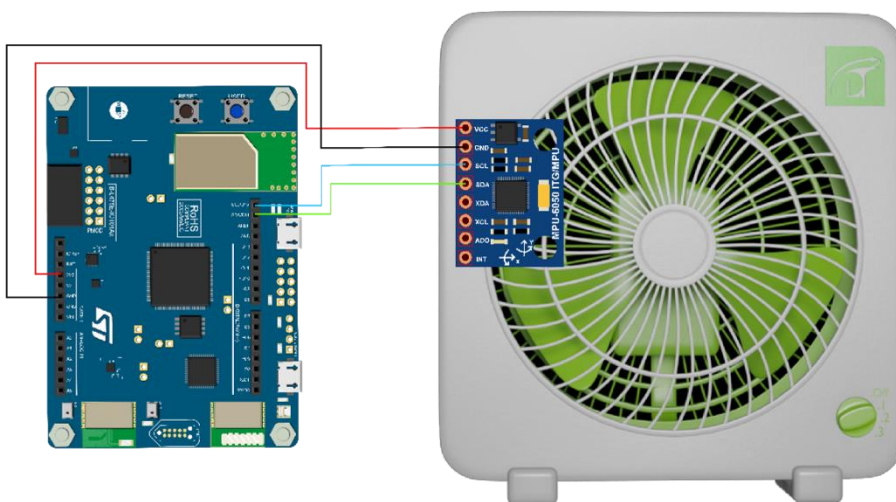
Objective:

This project aims to captures vibration data from the MPU6050 sensor to detect abnormal patterns in machines. The data will be utilized to train a machine learning model in NanoEdge AI Studio after being uploaded to the target microcontroller.

Requirements:

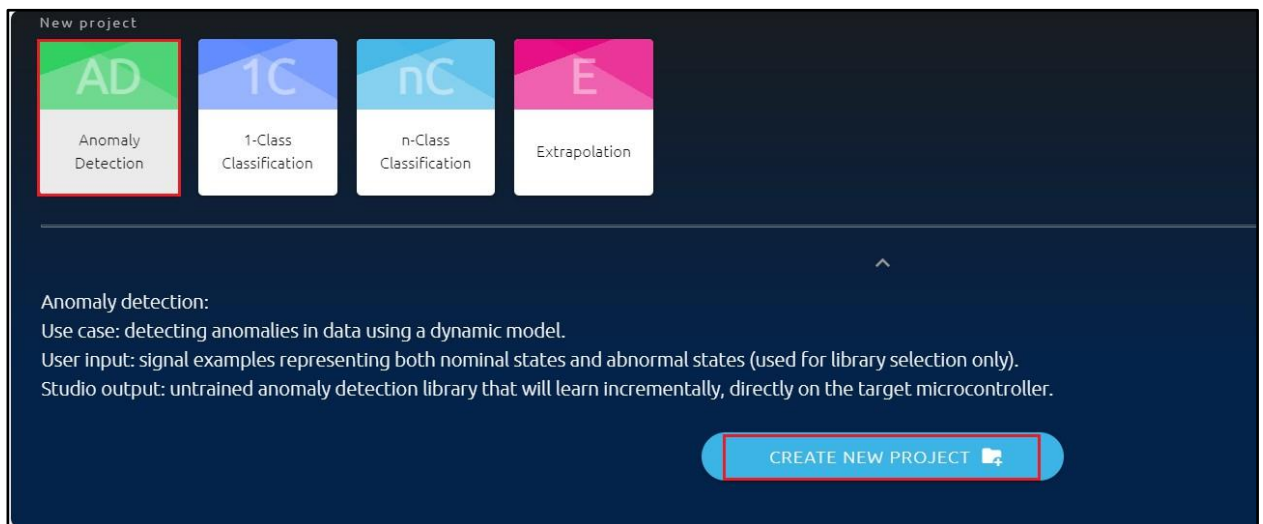
1. NanoEdge AI Studio software.
2. MPU6050 sensor (I2C).
3. STM32 Microcontroller.
4. USB Cable for the microcontroller.
5. Jumper Wires.
6. Laptop or PC

Connection Diagram:

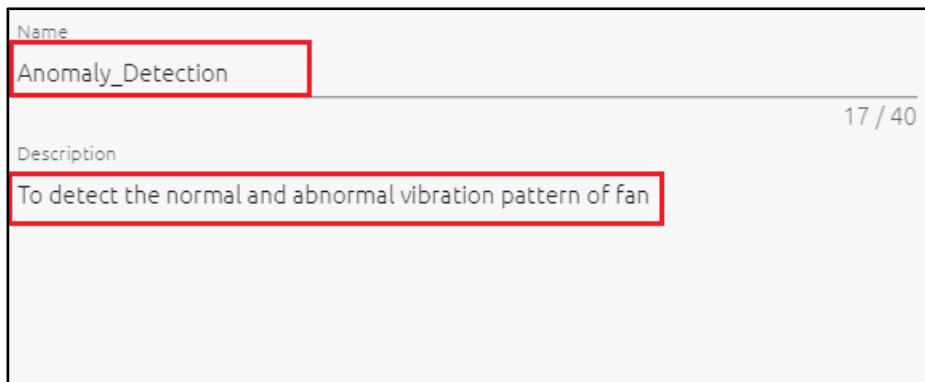


Steps Involved (NanoEdge AI Studio):

1. Open NanoEdge AI Studio.
2. Select **Anomaly Detection** project type and select **Create New Project**.



3. A new window will open. On the first step, **Project Settings**, name the project as **Anomaly Detection**. Under **Description** type “To detect the normal and abnormal vibration pattern of fan”.



4. Let **Max RAM** and **Max Flash** option remain as default.

Library Max RAM

32

KB

No Flash limit
☐
Limit Flash

64

KB

5. In **Target** section, select your proper STM32 Nucleo board. In **Sensor type** section, select the type of sensor you are working with. For this experiment you have to choose **Accelerometer 3- axes**.

Target Selection

Your favorite targets

B-L455H-OT01A


STM32 DEVELOPMENT BOARDS
147

MICROCONTROLLERS
557

ARDUINO BOARDS
19

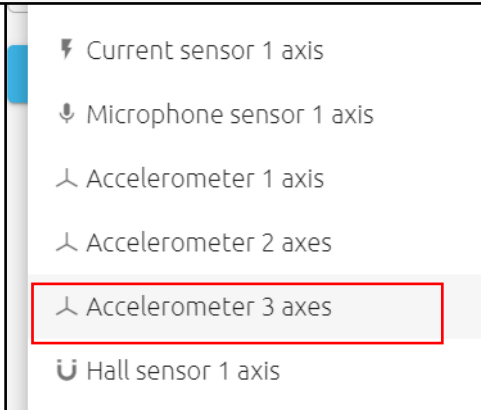
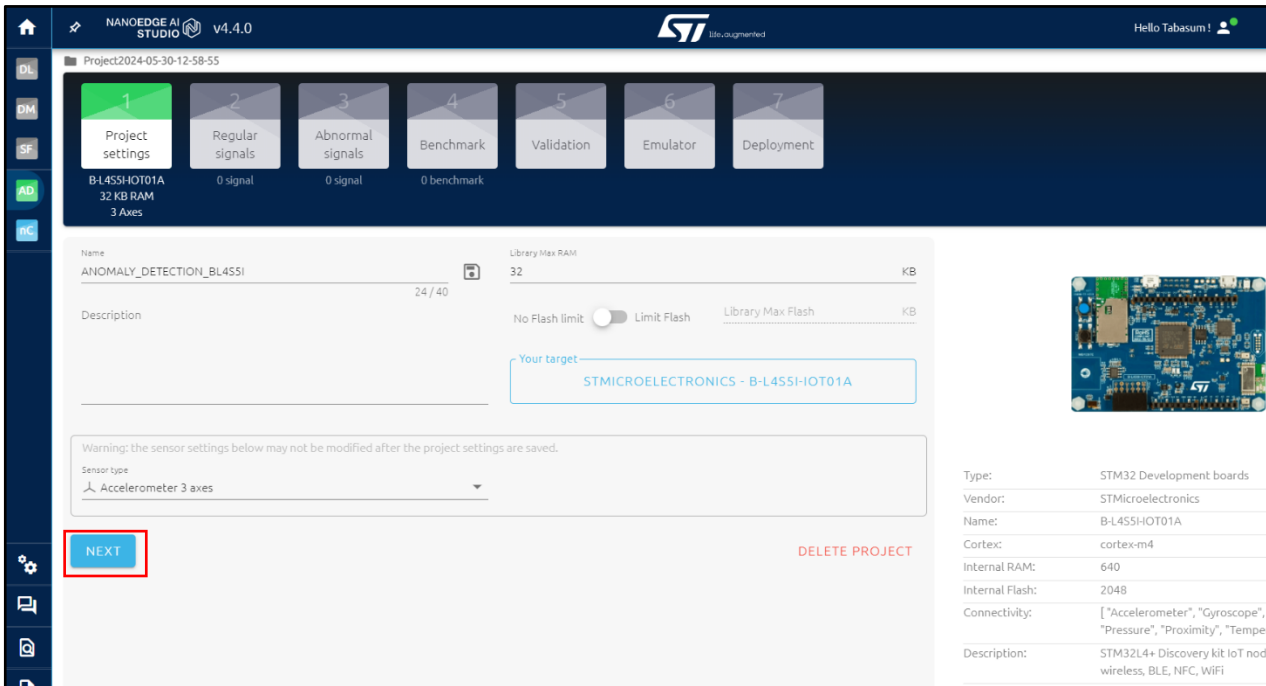
Series
STM32L4+ Series

Search Commercial Part N° or MCU Series

Overview	Commercial Part N°	MCU Series	Link	Favorite
	B-L455H-OT01A	STM32L455VIT6	Link	☒
	NUCLEO-L4P5ZG	STM32L4P5ZGT6	Link	☐
	NUCLEO-L4R5ZI	STM32L4R5ZIT6	Link	☐
	NUCLEO-L4R5ZI-P	STM32L4R5ZIT6P	Link	☐
	STEVAL-MKSBOX1V1	STM32L4R9ZIU6	Link	☐
	STEVAL-SMARTAG2	STM32L4P5CGU6	Link	☐
	STEVAL-STLINKV2	STM32L4R9ZIU6	Link	☐

CANCEL
CONFIRM

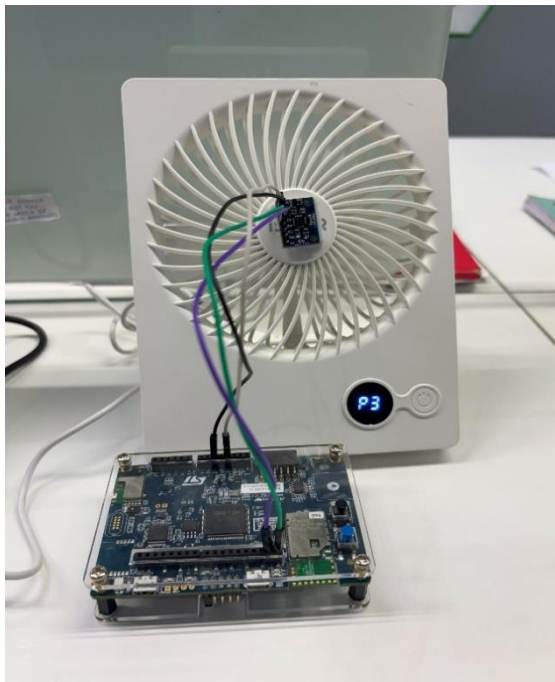
6. Click on **Save & Next** to move onto the next step.



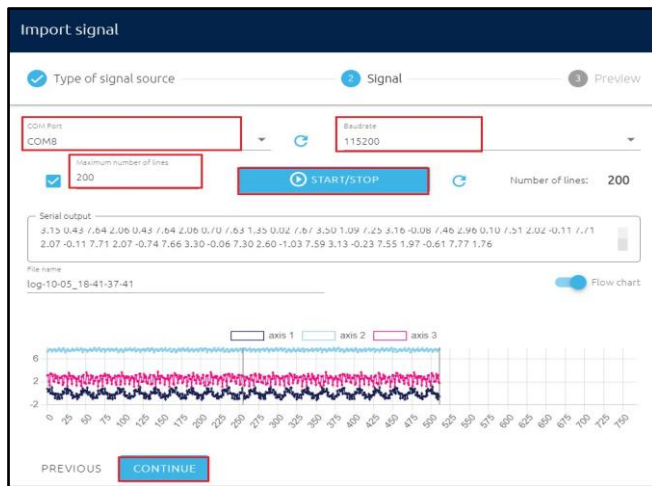
7. In the second step on NanoEdge AI Studio – **Regular Signals**, click on **Add Signals**. A window will open with different source options for you to import signals, select **From Serial (USB)** as we are collecting sensor data connected to microcontroller board, which in turn is connected to your PC through a Serial USB Cable.



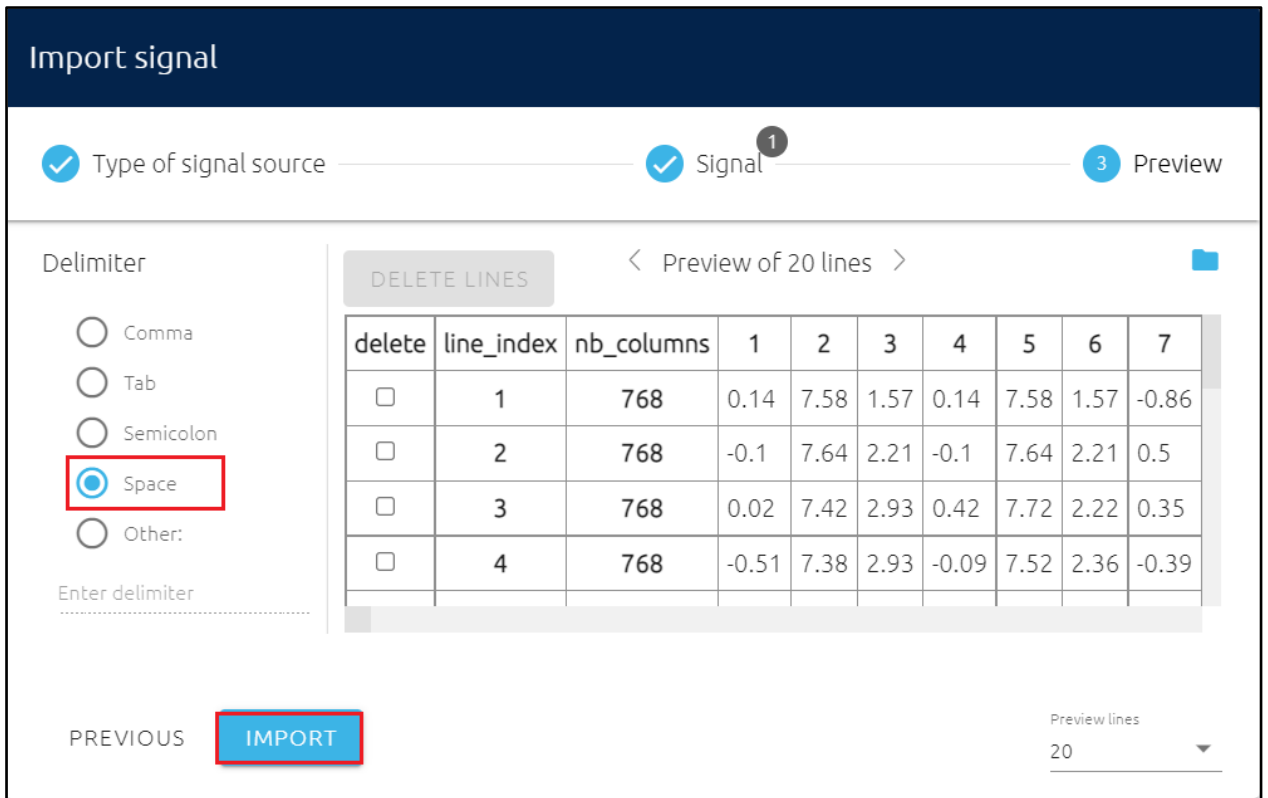
8. Before collecting signals, your **datalogger** code needs to be dumped into microcontroller and should be **disconnected** from serial console of stm32cubeIDE also you have to attach your MPU6050 sensor at the middle of the fan for collecting the vibration data from the fan.



9. Now you can add your signals. In **COM Port** select the correct COM port based on device manager info. Select **Baud rate** as 115200. In **Maximum number of lines** enter the number of lines of data you want to collect. In this case we will collect 200 lines of data. Now turn on the fan and start collecting the vibration data from it. Click on **Continue** to move onto importing the vibration samples.



10. In this window, select the **Delimiter** as **Space**. If any line of data has some corrupted values, an error message will be shown under the preview lines. You can select the particular line(s) and delete it. Select **Preview Lines** as 200 to check all the line of data you have collected, and click on **Import**.



Delimiter

☐ Comma

☐ Tab

☐ Semicolon

☒ Space

☐ Other:

Enter delimiter

DELETE LINES

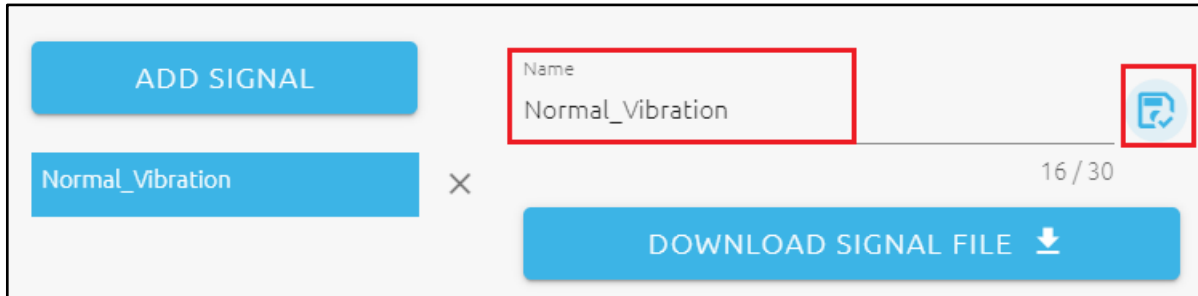
< Preview of 20 lines >

delete	line_index	nb_columns	1	2	3	4	5	6	7
☐	1	768	0.14	7.58	1.57	0.14	7.58	1.57	-0.86
☐	2	768	-0.1	7.64	2.21	-0.1	7.64	2.21	0.5
☐	3	768	0.02	7.42	2.93	0.42	7.72	2.22	0.35
☐	4	768	-0.51	7.38	2.93	-0.09	7.52	2.36	-0.39

PREVIOUS IMPORT

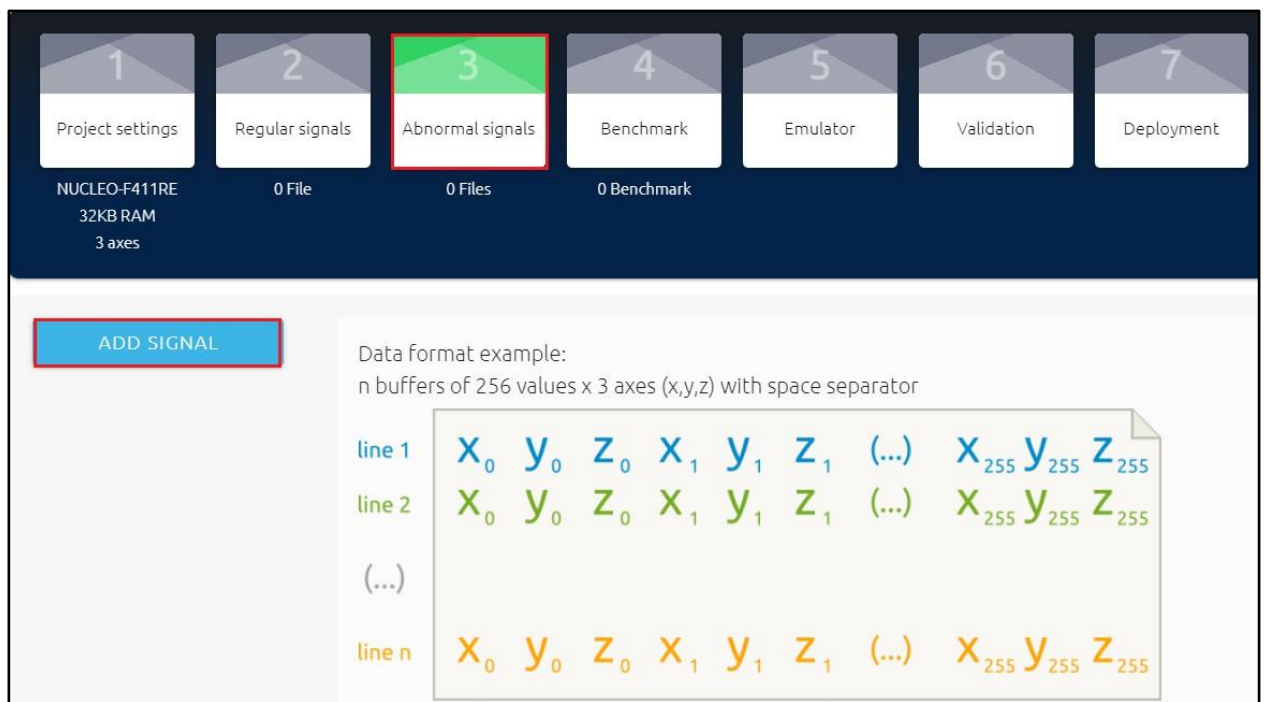
Preview lines
20

11. In the new window, rename the sample under **Name** as **Normal_Vibration** and click on the **Blue Save Icon** to save the renamed sample.



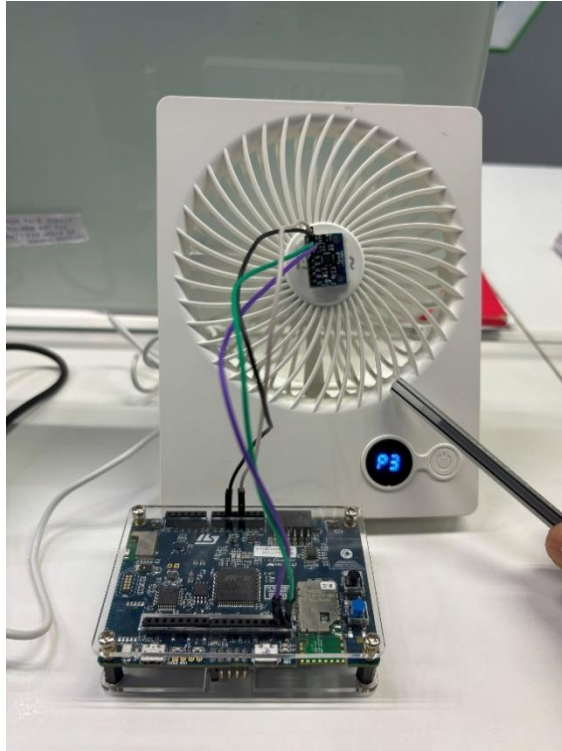
The screenshot shows a window titled 'ADD SIGNAL'. It contains a text input field labeled 'Name' with the value 'Normal_Vibration'. To the right of the input field is a blue square icon with a white document and a checkmark, which is highlighted with a red box. Below the input field is a blue button labeled 'DOWNLOAD SIGNAL FILE' with a download icon. The window also shows a list of signals with 'Normal_Vibration' selected, and a '16 / 30' indicator.

12. Now select **Abnormal Signals** and click on **Add Signal** again.

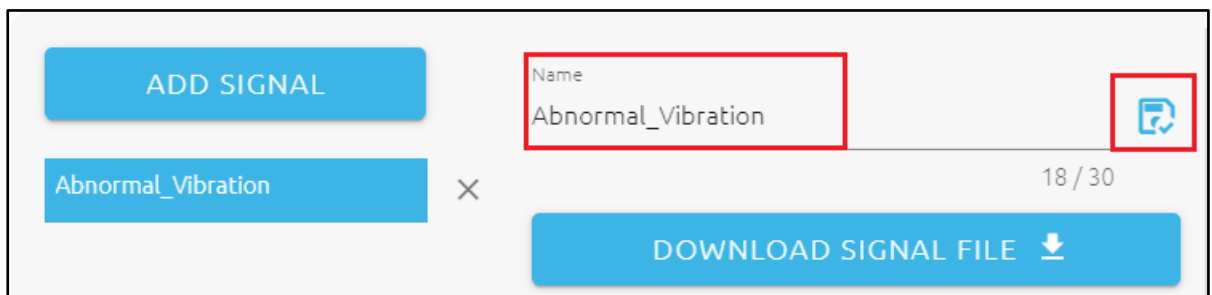


The screenshot shows a dashboard with seven tabs: 1. Project settings, 2. Regular signals, 3. Abnormal signals, 4. Benchmark, 5. Emulator, 6. Validation, and 7. Deployment. The 'Abnormal signals' tab is selected and highlighted with a red box. Below the tabs, there is a section for 'ADD SIGNAL' with a blue button highlighted by a red box. To the right of the button, there is a text area with a data format example: 'n buffers of 256 values x 3 axes (x,y,z) with space separator'. The example shows a table with columns for x, y, and z axes, and rows for line 1, line 2, and line n. The data is represented as $x_0, y_0, z_0, x_1, y_1, z_1, \dots, x_{255}, y_{255}, z_{255}$.

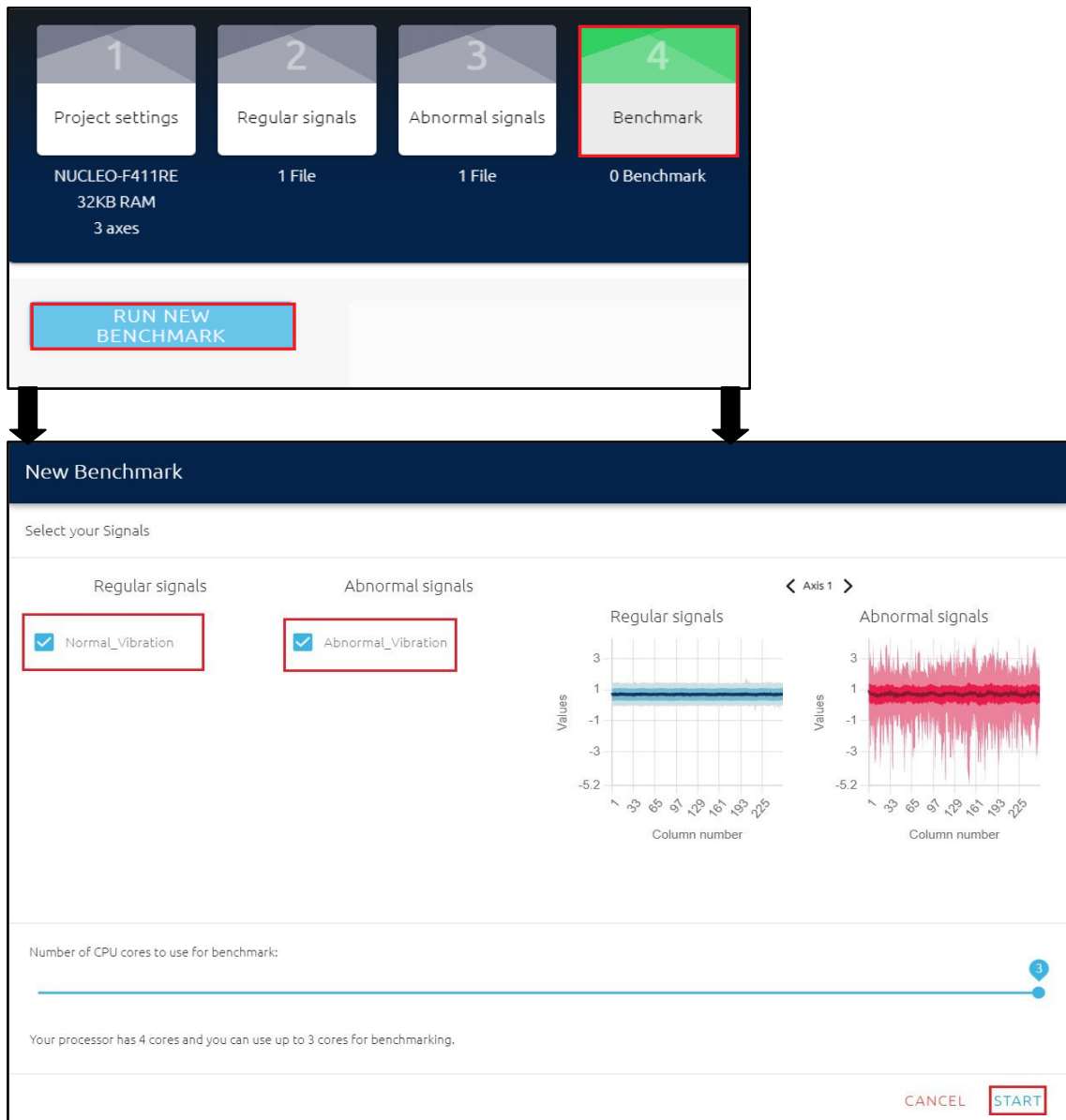
13. Turn on the fan, give some disturbances by your finger and start collecting data for the abnormal vibration. Collecting data process will be same as above.



14. Now turn off the fan and name the file as **Abnormal Vibration**.



15. In the fourth step of the NanoEdge AI Studio- **Benchmark**, click on **Run Benchmark**.
Next select **Regular Signals** and **Abnormal Signals** and click on **Start**.



1 Project settings
NUCLEO-F411RE
32KB RAM
3 axes

2 Regular signals
1 File

3 Abnormal signals
1 File

4 Benchmark
0 Benchmark

RUN NEW BENCHMARK

New Benchmark

Select your Signals

Regular signals

☒ Normal_Vibration

Abnormal signals

☒ Abnormal_Vibration

Regular signals

Abnormal signals

Axis 1

Values

Column number

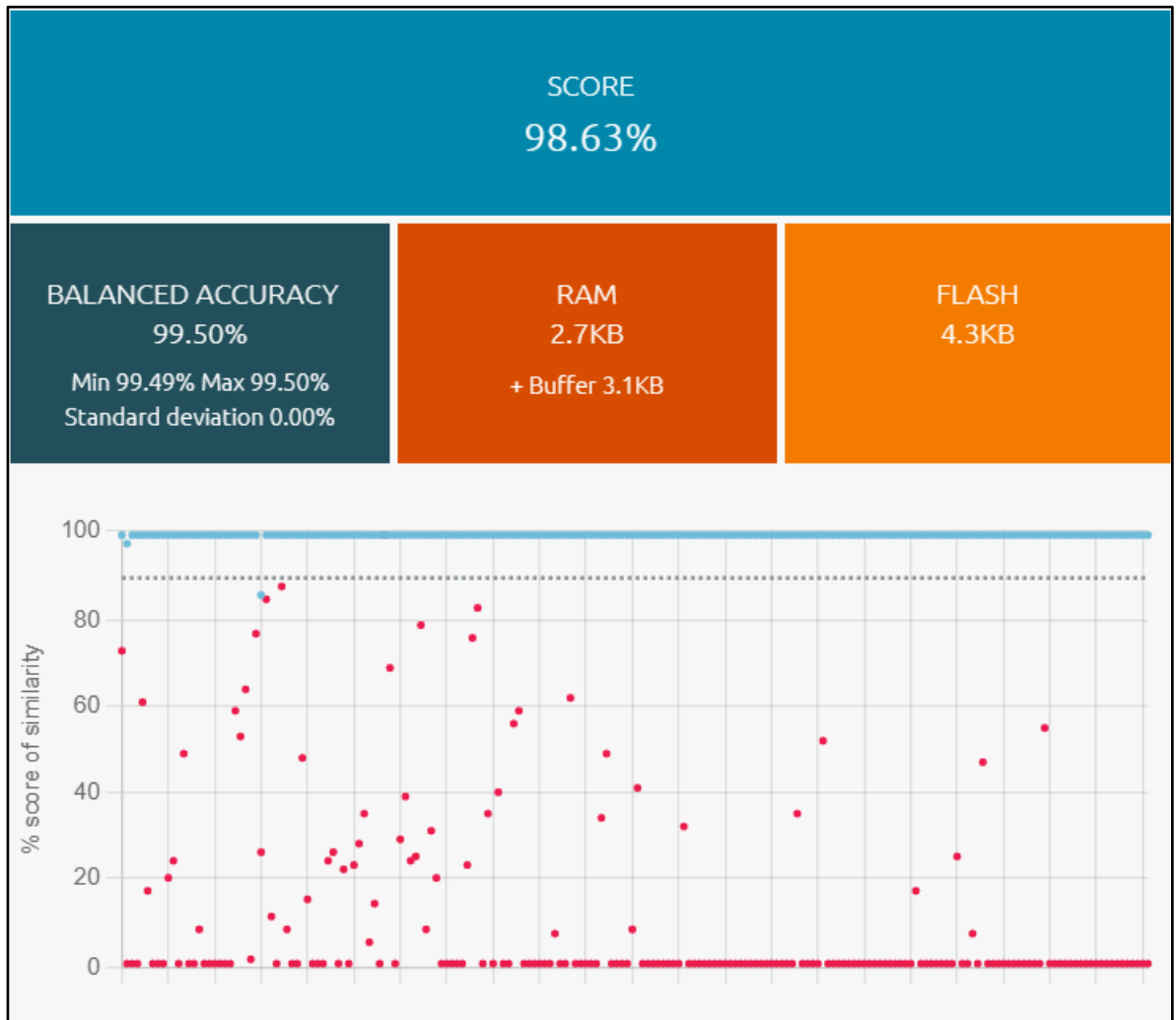
Number of CPU cores to use for benchmark:

3

Your processor has 4 cores and you can use up to 3 cores for benchmarking.

CANCEL **START**

16. After the Benchmark has reached a **Score** of above 95% stop the benchmark process.



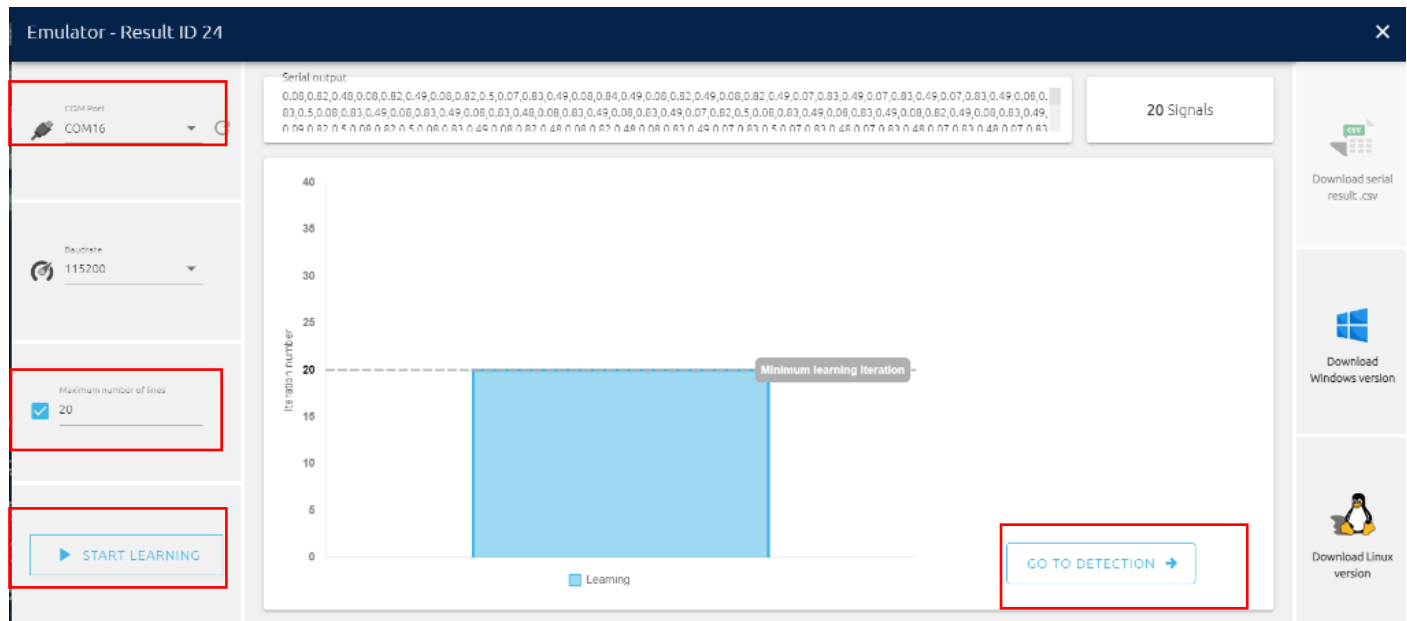
17. After the benchmark process is done, go to the next stage, **Validation**. Click on **Serial Emulator** to move on to test the library you have selected on the NanoEdge AI Studio.

Benchmark 2024-11-30 13:14-Bench										
+ NEW EXPERIMENT Please select at least one library to start a new experiment										
Result ID	Score	Balanced accuracy	RAM	Flash	Minimum learning iteration ⓘ	Execution time ⓘ	Model	Report	Serial emulator	Compilation lib
☐	24	91.20 %	94.47 %	95.8 KB	94.8 KB	20	3.0 ms	IPCA		
☐	20	79.73 %	83.78 %	2.8 KB	9.5 KB	20	1.9 ms	IPCA		
☐	22	71.30 %	84.16 %	5.5 KB	6.8 KB	20	1.9 ms	MML		

NOTE:The **IPCA model** (Independent Principal Component Analysis) is a variation of PCA (Principal Component Analysis) used to analyze and reduce the dimensionality of data. It assumes that the components are independent, rather than just uncorrelated, which allows for a more detailed separation of the underlying structure in the data, often used in unsupervised learning tasks.

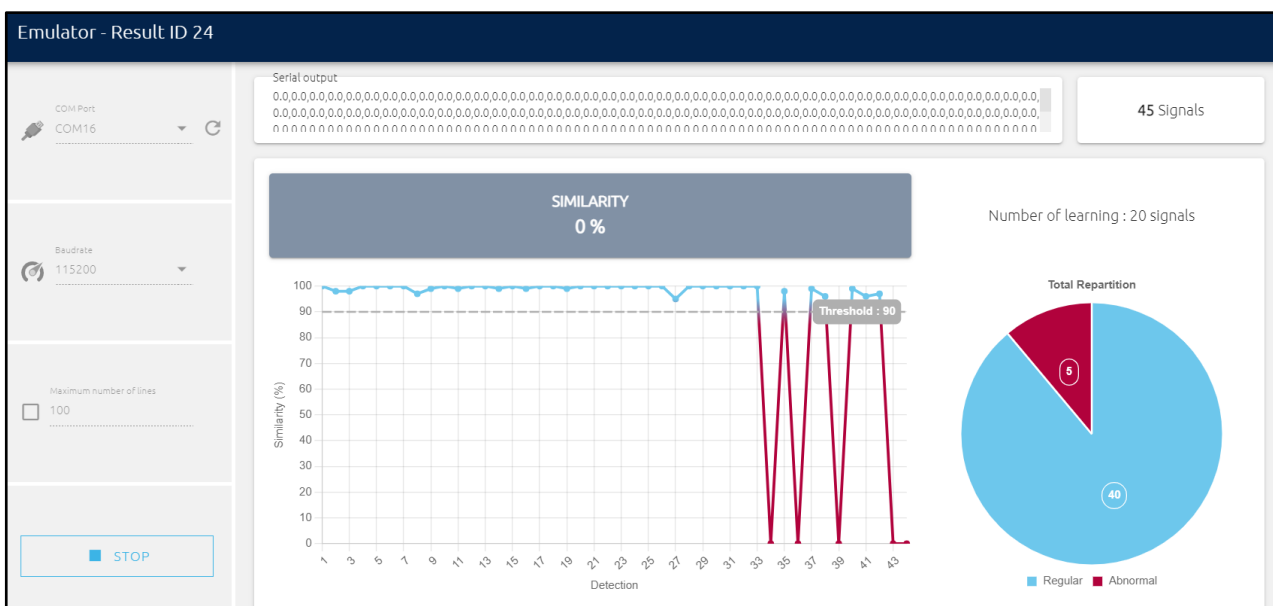
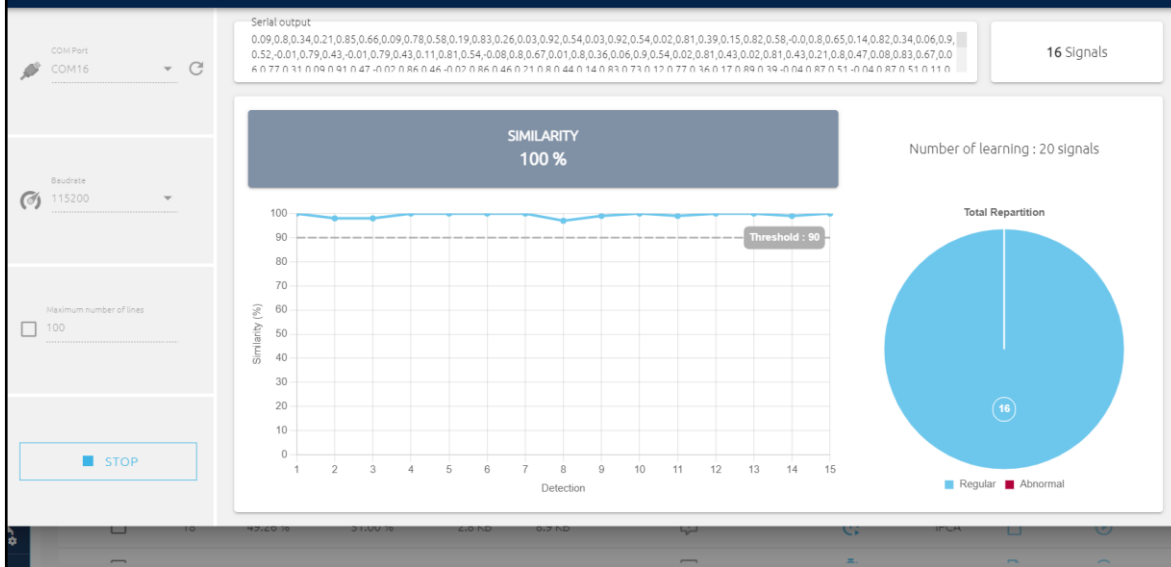
18. In the new window, select **From Serial (USB)**. Select **COM Port** according to info from Device Manager. Select Baud rate as 115200. Keep the **Maximum number of lines** as 20. Start the fan and let the ML Model learn the fan normal vibration for 40 cycles. Now select **GO TO DETECTION**.

NOTE: In anomaly detection with NanoEdge AI, **iterations** involve repeatedly training the

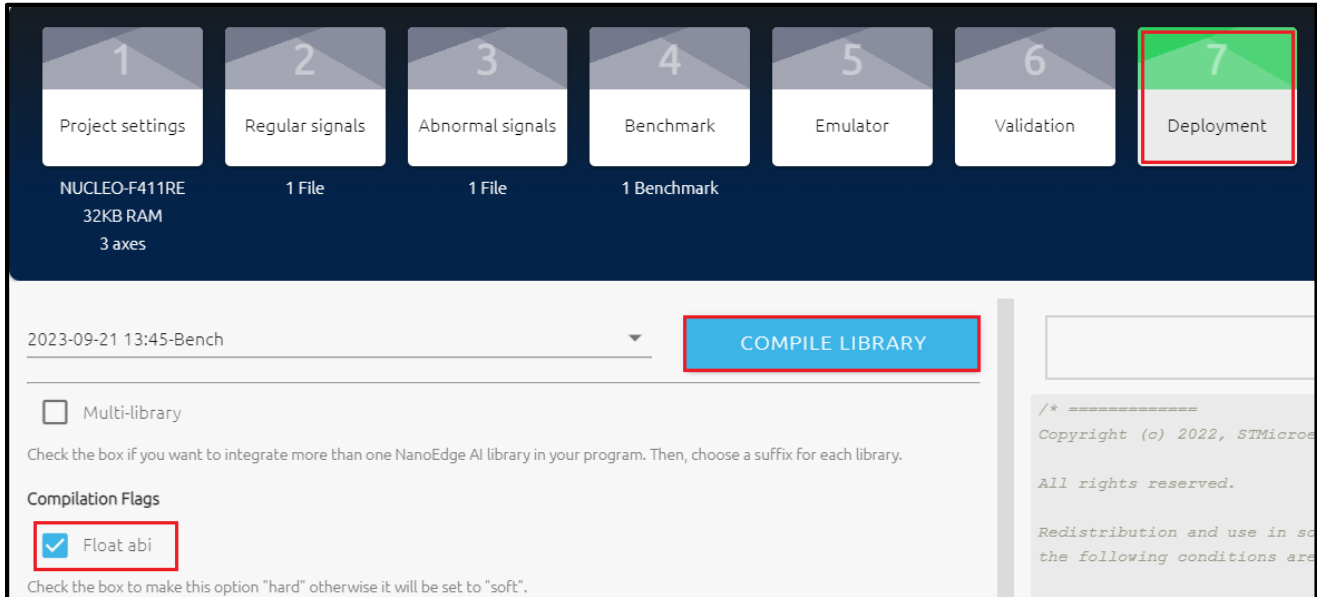


model to improve its accuracy in identifying anomalies by refining its detection capabilities with each cycle. Maximum number of lines should be double of your iterations.

19. In the Detection part, select From Serial (USB). Select COM Port according to info from Device Manager. Select **Baud rate** as 115200. Click on **START/STOP**. At this point of time your fan should be running in normal mode no need to give any disturbances So for normal vibration it will show **100% Similarity** and when there is some disturbances on the fan, the similarity score drops down.



20. Finally go to **Deployment** stage, select **Float abi** and click on **Compile Library**. Click on **Get Library**, and save your library on the PC



The screenshot displays the DigiToad Technologies web interface. At the top, there is a navigation bar with seven stages: 1 Project settings, 2 Regular signals, 3 Abnormal signals, 4 Benchmark, 5 Emulator, 6 Validation, and 7 Deployment. The Deployment stage is highlighted with a green background and a red border. Below the navigation bar, the main content area shows the project details: NUCLEO-F411RE, 32KB RAM, and 3 axes. A dropdown menu shows the date and time: 2023-09-21 13:45-Bench. A blue button labeled "COMPILE LIBRARY" is visible. Below this, there is a checkbox for "Multi-library" and a section for "Compilation Flags" where the "Float abi" option is checked. A red box highlights the "Float abi" checkbox. On the right side, there is a text area containing a copyright notice: "/* ===== Copyright (c) 2022, STMicroelectronics All rights reserved. Redistribution and use in source or binary form, without modification, are permitted provided that the following conditions are met: 1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer. 2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. 3. Neither the name of STMicroelectronics nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission. 4. This software is provided 'as is', without warranty of any kind, either express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall STMicroelectronics be liable for any damages, including but not limited to general, special, incidental, or consequential damages, arising out of the use of this software." The "COMPILE LIBRARY" button and the "Float abi" checkbox are highlighted with red boxes.

Note: All important steps and parts are highlighted with a red colour box for the proper understanding of the user. This document is meant for the use of education purpose only.

Lab 11c) :Anomaly Detection: Deployment of an ML based Anomaly Detection model into microcontroller using MPU 6050 Sensor

Lab Objective:

The Objective of this experiment is to interface a MPU6050 sensor to an STM32 microcontroller and deploy the Machine Learning Model built using the NanoEdge AI Studio into the microcontroller. This will give the microcontroller the ability to make a decision on the device itself based on classification on real time MPU6050 sensor data.

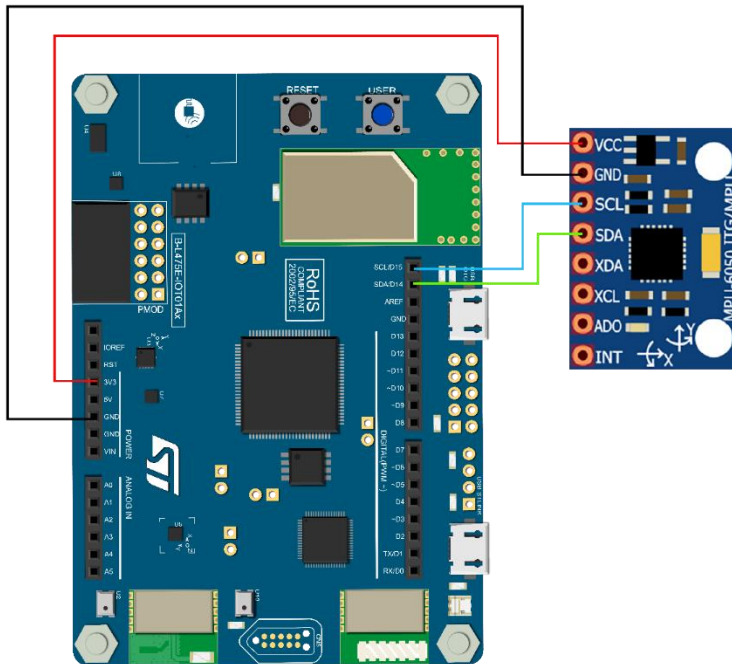
Lab Overview:

In this lab, we will interface an MPU6050 sensor with an STM32 microcontroller and deploy a machine learning model from NanoEdge AI Studio. This will enable the microcontroller to make real-time decisions based on the sensor data classification. This is the follow up of Mpu_datalogger project

Requirements:

1. STM32 Cube IDE software.
2. MPU6050 Sensor (I2C).
3. STM32 Microcontroller.
4. USB Cable for the microcontroller.
5. Jumper Wires.

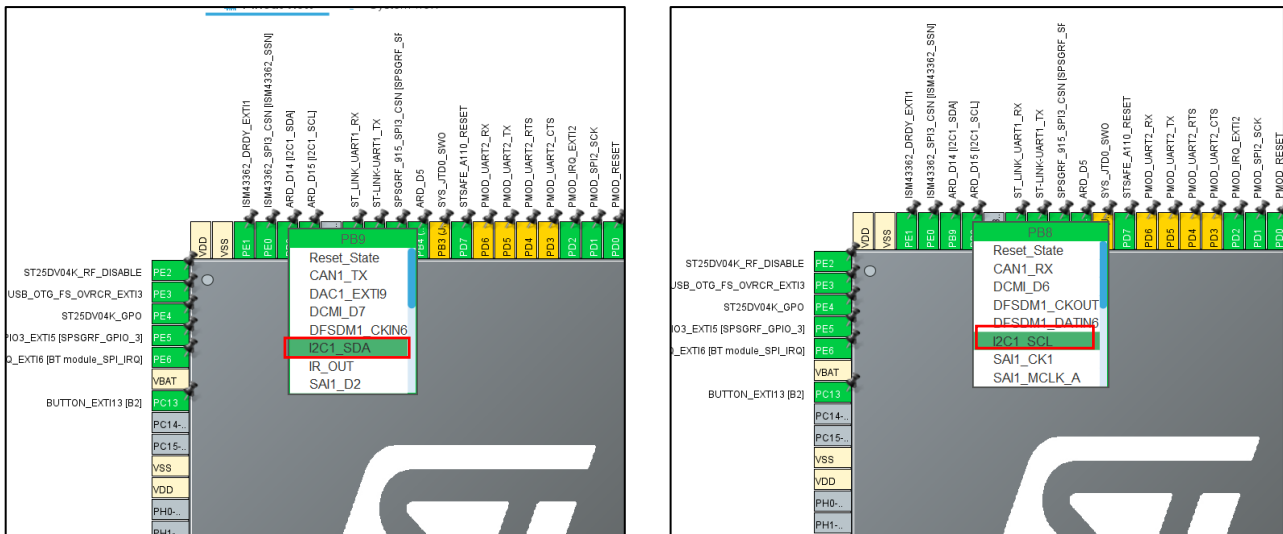
Experimental Setup:



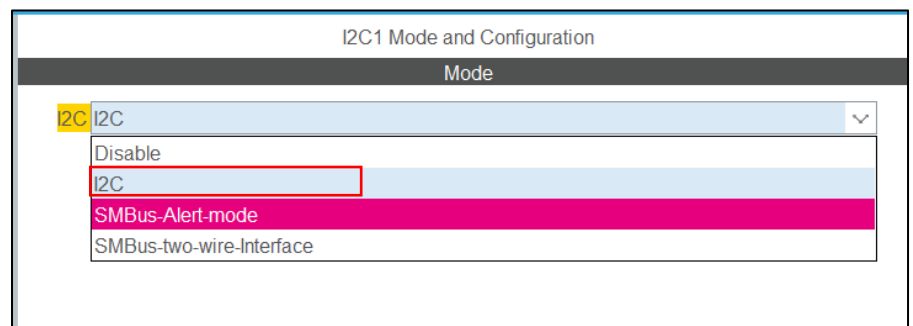
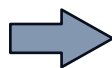
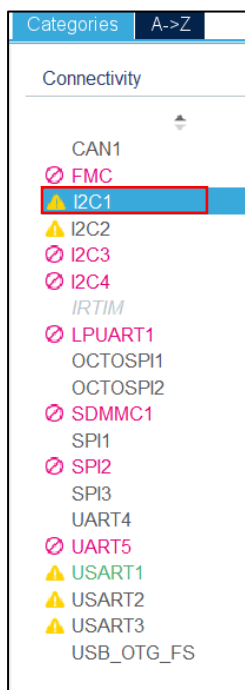
Steps Involved

- Click on **File→New→STM32 Project** to start your project on Cube IDE and name it as **ANOMALY_DETECTION**
- (Refer **Lab 11a** to create anew project on stm32cubeIDE)
- Then Go to IoC file and enable Pins for I2C communication.

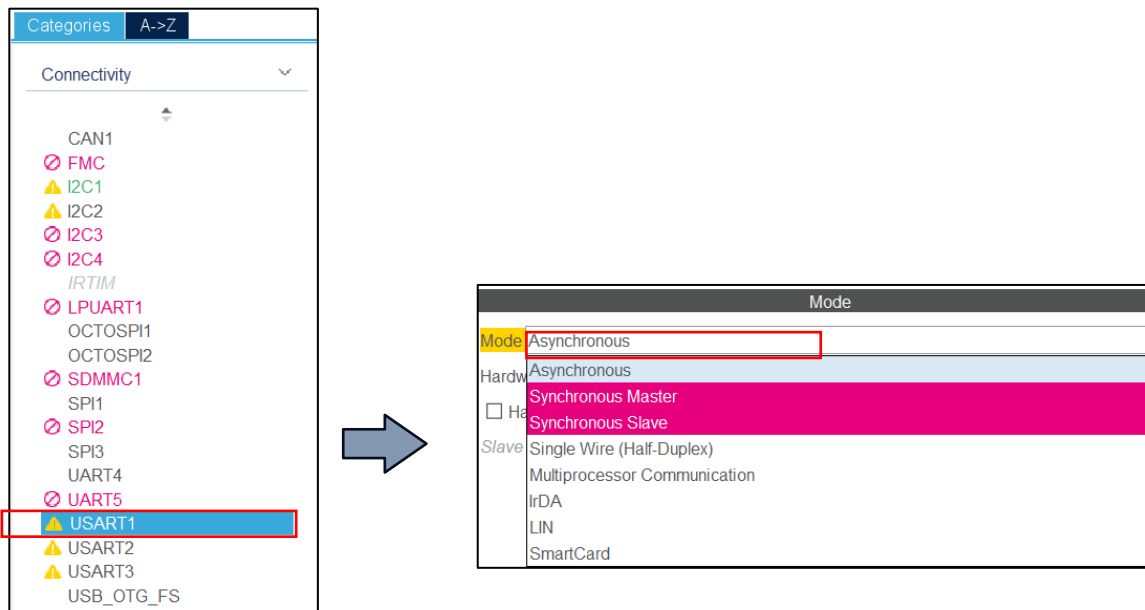
In the **Pinout & Configuration** tab, click on **PB8** pin and select it as an **I2C1_SCL** and **PB9** pin as an **I2C1_SDA**.



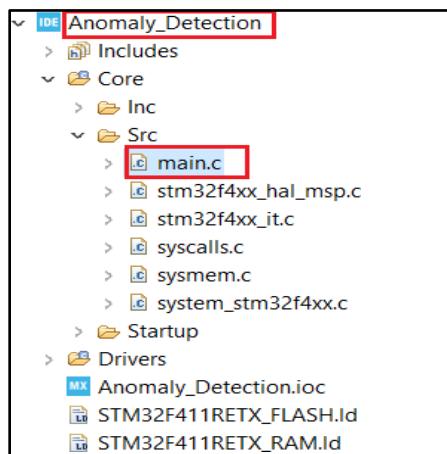
1. Next on the left-hand side under **Categories** → **Connectivity**, select I2C1 and enable it.



Next on the left-hand side under **Categories** → **Connectivity**, select USART1 and enable asynchronous mode.



2. Press **Ctrl+S** to generate your code. On the left-hand side of the Cube IDE, under **Project Explorer** go to the project you have created (For example, I have named my project as (Anomaly_Detection) **Anomaly_Detection** → **Core** → **Src** → **main.c** (double click to load the code).



3. Now open your project tree **Anomaly_Detection** → **Core** → **Inc**. Right click on your **Inc** folder and paste the **MPU.h** header file provided in the folder .
4. Then go to project and generate your **main.c** file Cube IDE automatically generates a code format based on the configurations you have done. Cube IDE uses HAL libraries. Below is the code snippets, please put your code in the appropriate places in the **main.c** file.

```

23  /* USER CODE BEGIN Includes */
24  #include "mpu.h"
25  #include "stdio.h"
26  #include "string.h"
27  #include "NanoEdgeAI.h"
28  /* USER CODE END Includes */
29
30  /* Private typedef -----*/
31  /* USER CODE BEGIN PTD */
32
33  /* USER CODE END PTD */
34
35  /* Private define -----*/
36  /* USER CODE BEGIN PD */
37  #define DATA_INPUT_USER 256
38  #define AXIS_NUMBER 3
39  #define NUMBER_LEARN 40
40  /* USER CODE END PD */
41
42  /* USER CODE BEGIN PV */
43
44  float mpu_buffer[DATA_INPUT_USER * AXIS_NUMBER] = { 0 };
45  uint8_t similarity = 0;
46
47  /* USER CODE END PV */
48
49  /* USER CODE BEGIN PFP */
50
51  void fill_mpu_buffer();
52  void Train(void);
53  void Inference(void);
54  /* USER CODE END PFP */
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104  /* USER CODE BEGIN 2 */
105  MPU6050_Init();
106
107  enum neai_state error_code = neai_anomalydetection_init();
108  if (error_code != NEAI_OK) {
109      printf("Application Init Not Done!!!");
110  }
111  else
112      printf("Application Init Done!!!");
113
114  HAL_Delay (1000); // wait for 1 sec
115  Train();
116  /* USER CODE END 2 */

```

```

119  /* USER CODE BEGIN WHILE */
120  while (1)
121  {
122      Inference();
123      /* USER CODE END WHILE */

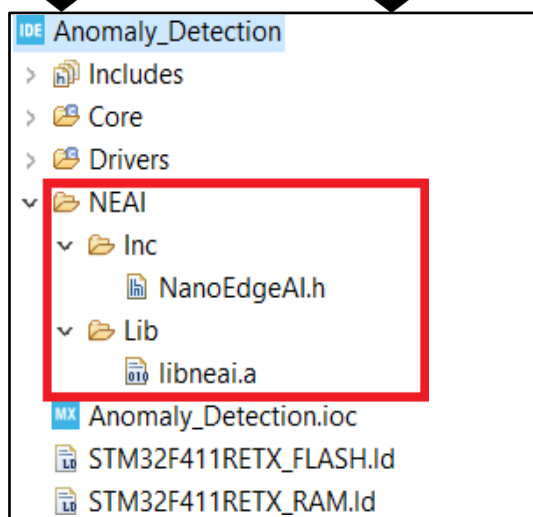
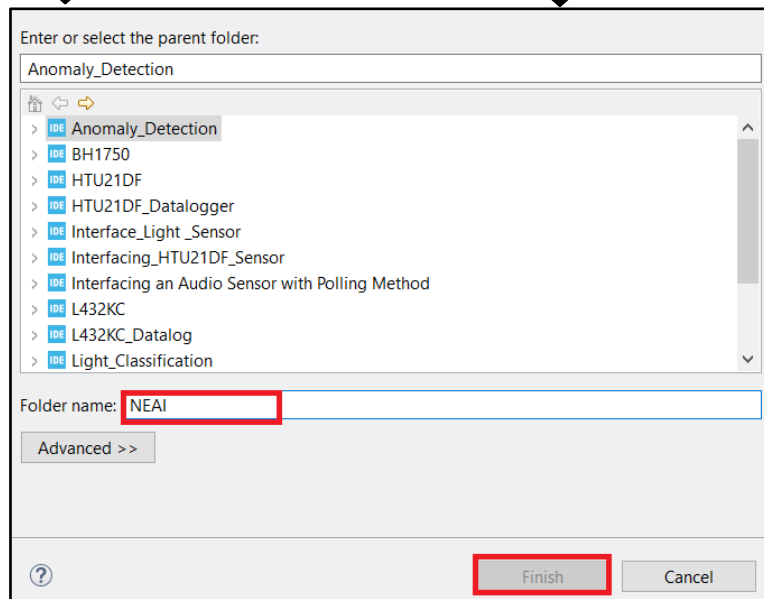
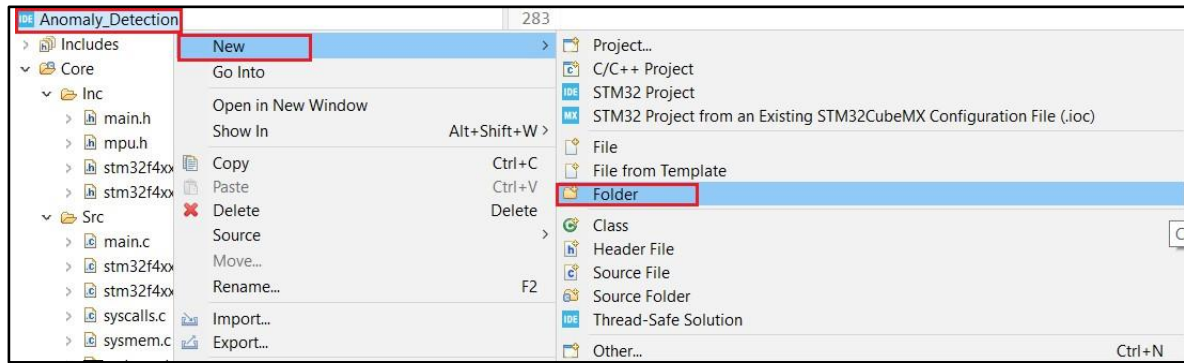
```

```

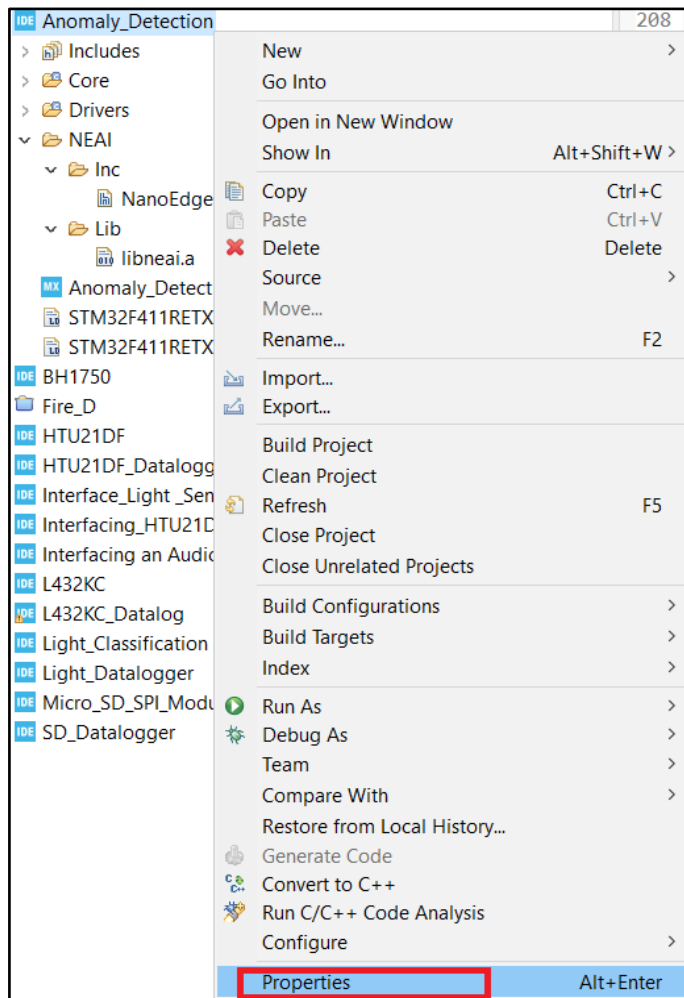
478 void fill_mpu_buffer() {
479     for (int i = 0; i < DATA_INPUT_USER; i++) {
480         MPU6050_Read_Accel(accel_data);
481         mpu_buffer[AXIS_NUMBER * i] = Ax;
482         mpu_buffer[AXIS_NUMBER * i + 1] = Ay;
483         mpu_buffer[AXIS_NUMBER * i + 2] = Az;
484     }
485 }
486
487 void Train(void)
488 {
489     for (int i = 0; i < NUMBER_LEARN; i++) {
490         fill_mpu_buffer();
491         neai_anomalydetection_learn(mpu_buffer);
492         printf("Training Cycle No: ");
493         printf("%d", i);
494         printf("\r\n");
495         HAL_Delay(200);
496     }
497     printf("Training Done");
498 }
499
500 void Inference(void)
501 {
502     fill_mpu_buffer();
503     neai_anomalydetection_detect(mpu_buffer, &similarity);
504     printf("Similarity Score is: ");
505     printf("%d", similarity);
506     printf("\r\n");
507 }
508
509 int __io_putchar(int ch){
510     HAL_UART_Transmit(&huart1, (uint8_t *) &ch, 1, HAL_MAX_DELAY);
511     return ch;
512 }
513
514 /* USER CODE END 4 */

```

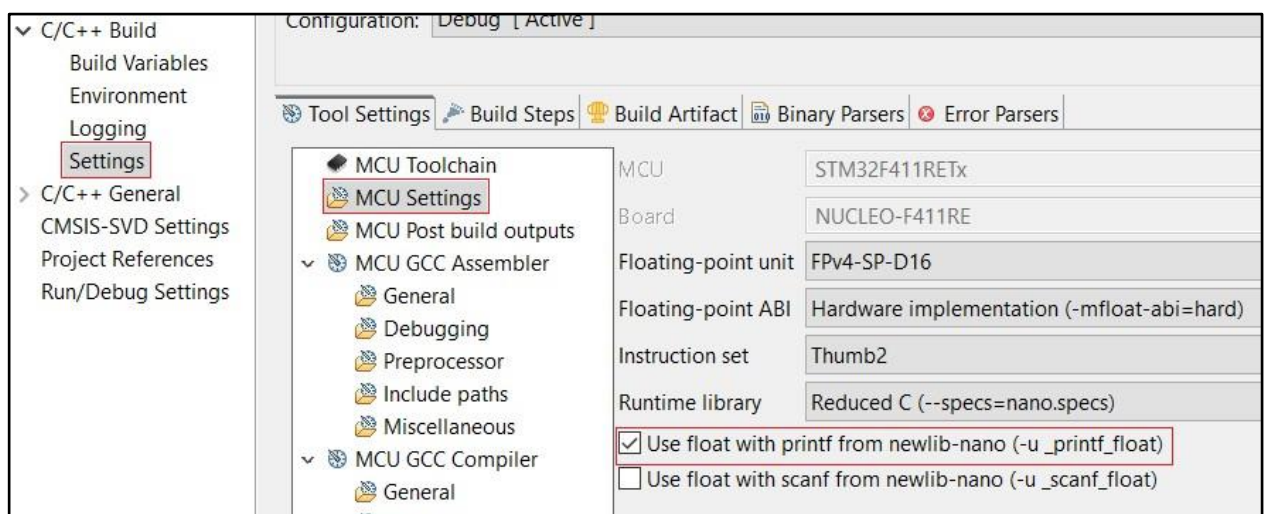

5. Right click on the **Anomaly_Detection** project and select **New→Folder**. Name the new folder as **NEAI** and click on **Finish**. You will be able to see a folder named **NEAI** inside of your **Anomaly_Detection** project. Now right click on this newly created NEAI folder and create two separate new folders and name them **Inc and Lib**.



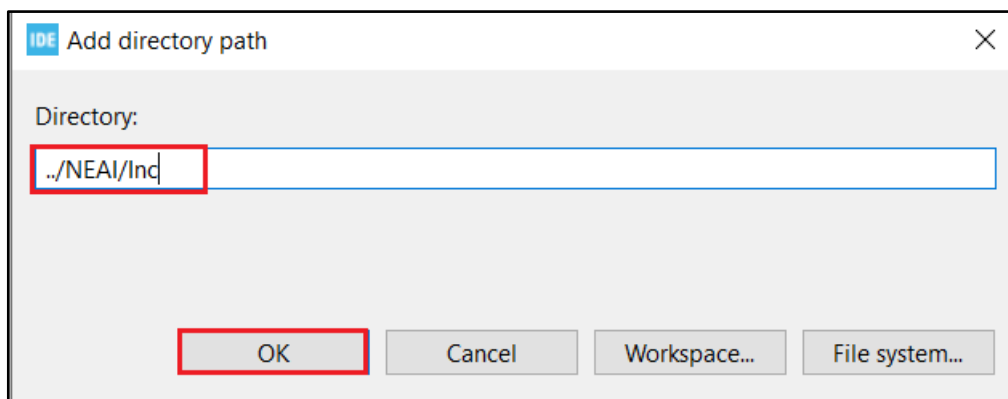
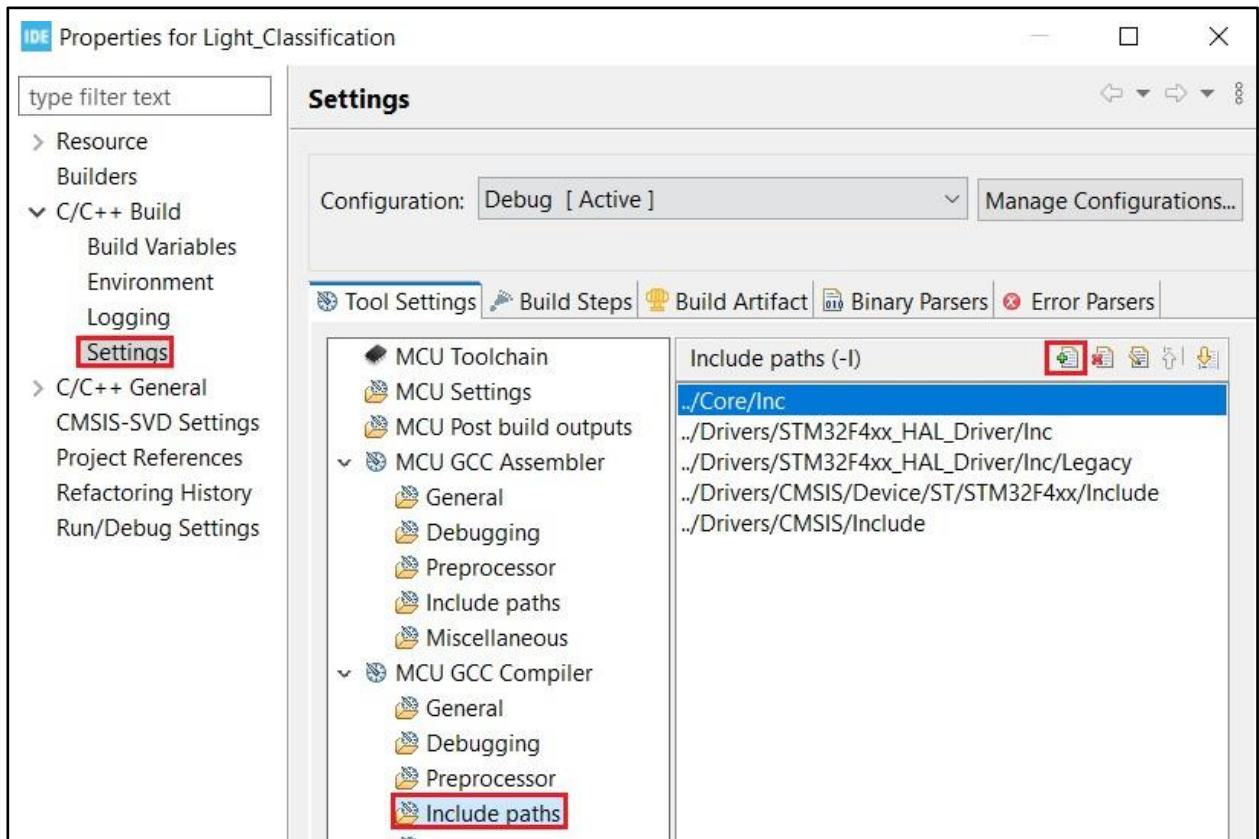
6. Now right click on your project tree, go to **Properties**.



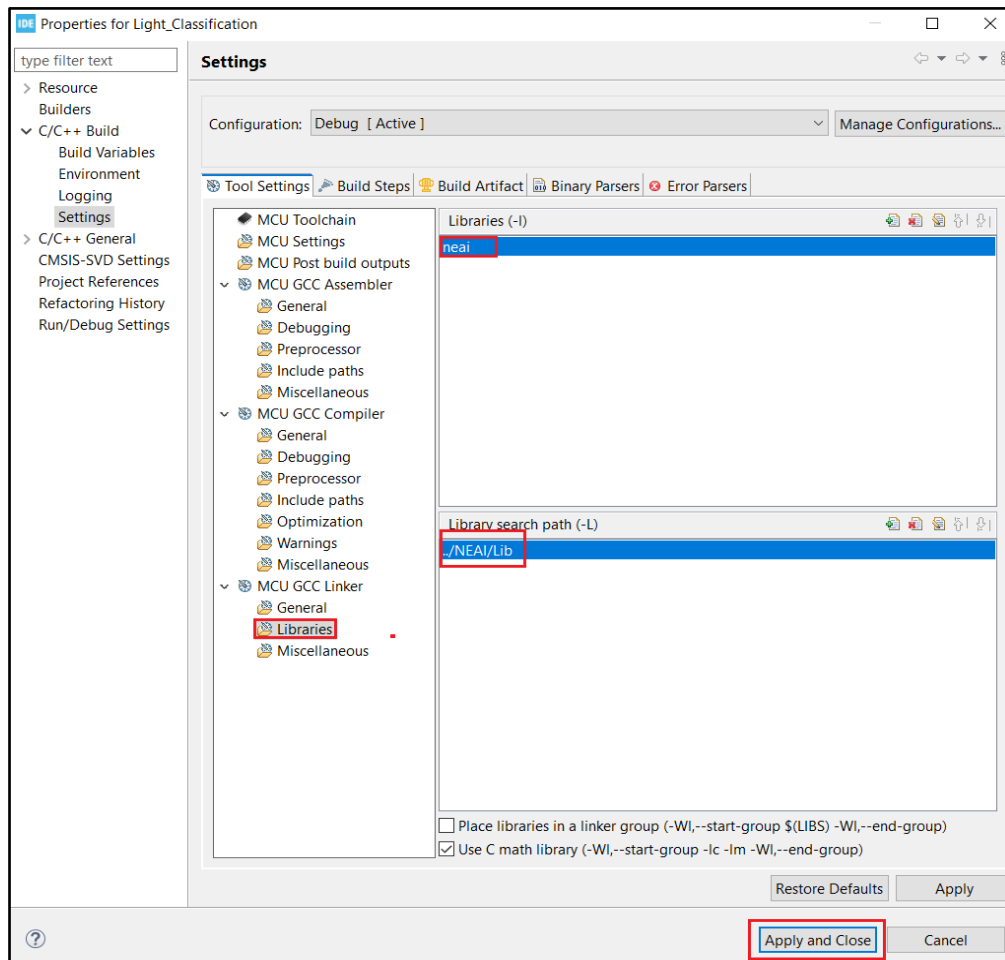
7. After that select **Settings**→**MCU Settings** and enable the **float printf**.



8. Next select **MCU GCC Compiler** → **Include paths** click on **Add** and in the new window select **Workspace** and select the **Inc** folder and click on **Ok**. The path of the **Inc** folder will be now added.



9. Next go to **MCU GCC Linker** → **Libraies**. In **Library search path (-L)** section, add the path of **Lib** folder similarly to the previous step of adding the **Inc** folder path. Next in the **Libraries(-l)** section click on **Add** and type **neai** then click on **Ok**. Then **Apply** and **Close**.



10. Now click on the **build** symbol  in the top left corner on your Cube IDE. If you have done everything correctly your code should be built without any errors.

```
20:31:22 **** Incremental Build of configuration Debug for project Anomaly_Detection ****
make -j4 all
arm-none-eabi-size  Anomaly_Detection.elf
  text  data  bss   dec   hex filename
 27112  308   7588  35008  88c0 Anomaly_Detection.elf
Finished building: default.size.stdout

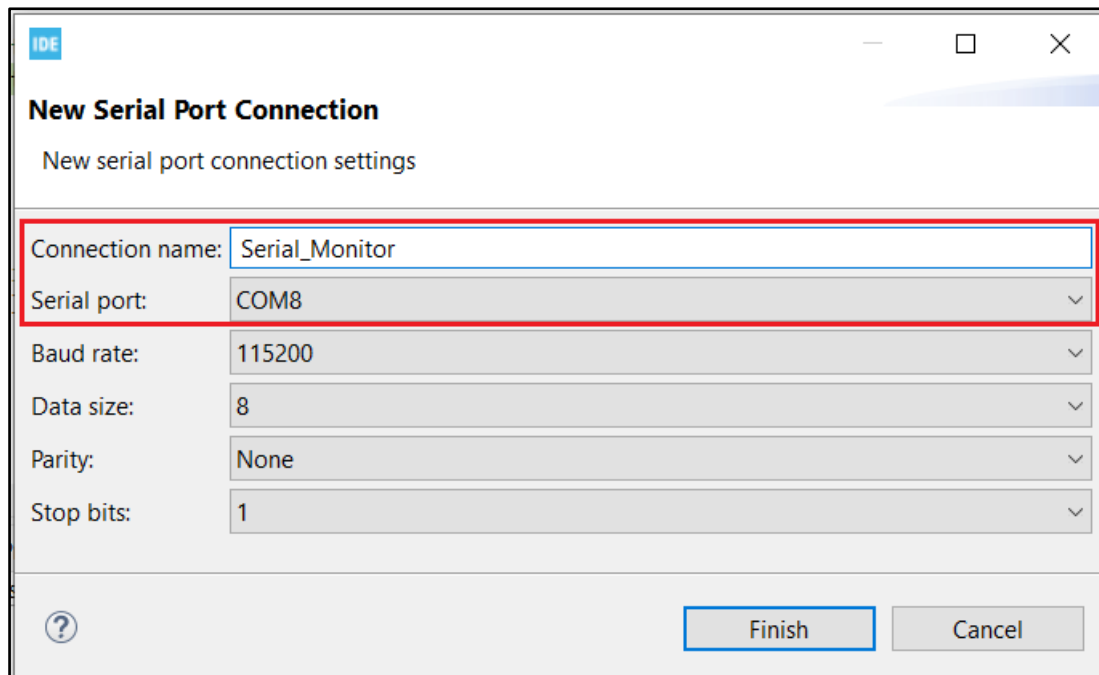
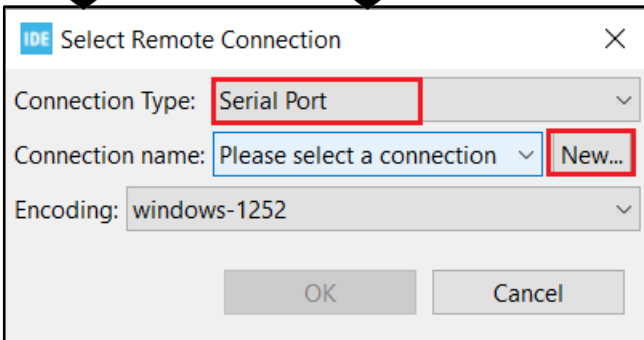
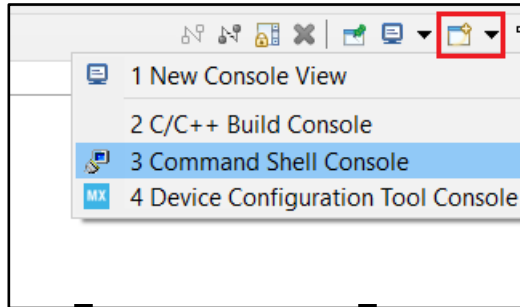
20:31:23 Build Finished. 0 errors, 0 warnings. (took 511ms)
```

11. Next connect your STM32 board with your audio sensor connect to it to your PC and click on



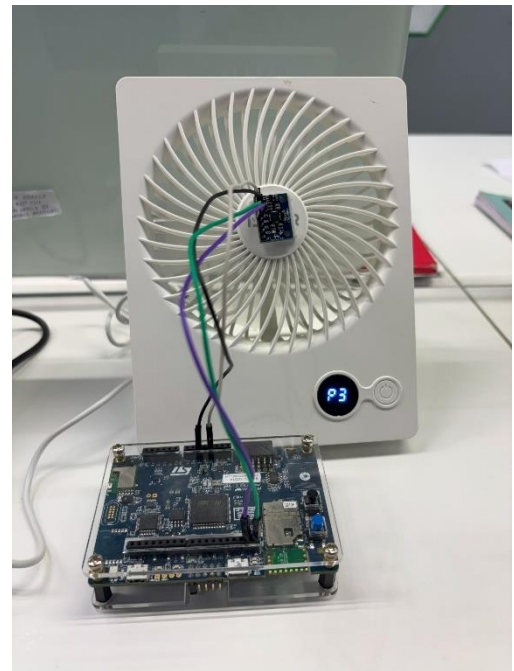
the **Debug** icon to start the Debugging process. An **Edit Configuration** window will open, click on **OK**, without making any changes.

12. In the debug mode, go to the bottom right hand side corner, click on open console. Select the **Connection Type** as **Serial Port**, then click on **New**. In the new window, in **Connection name** give some name to your new connection, and select the **Serial port** correctly. Then click on **Finish** and then **Ok**. A console with the given name will be opened at the bottom of your screen.



13. Click on the **Resume** icon  to run your code. You should be able to see the value of light sensor in the **Console**.

```
Application Init Done!!!Training Cycle No: 0
Training Cycle No: 1
Training Cycle No: 2
Training Cycle No: 3
Training Cycle No: 4
Training Cycle No: 5
Training Cycle No: 6
Training Cycle No: 7
Training Cycle No: 8
Training Cycle No: 9
Training Cycle No: 10
Training Cycle No: 11
Training Cycle No: 12
Training Cycle No: 13
Training Cycle No: 14
Training Cycle No: 15
Training Cycle No: 16
Training Cycle No: 17
Training Cycle No: 18
Training Cycle No: 19
Training Cycle No: 20
Training Cycle No: 21
```



```
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
Similarity Score is: 100
```

Normal Vibration

```
Similarity Score is: 25
Similarity Score is: 0
Similarity Score is: 63
Similarity Score is: 0
Similarity Score is: 41
Similarity Score is: 50
Similarity Score is: 0
Similarity Score is: 0
Similarity Score is: 40
Similarity Score is: 0
Similarity Score is: 0
Similarity Score is: 100
```

Abnormal Vibration

NOTE: while training the model your fan should be on normal vibration mode

- Before moving out of the debugging mode, click on **Disconnect**  and close the console then click on the **Terminate**  icon. You will be moved out of the debugging mode.

Lab 13: Local IoT Monitoring and Dashboard Creation using Node-RED for Real-Time Anomaly Detection

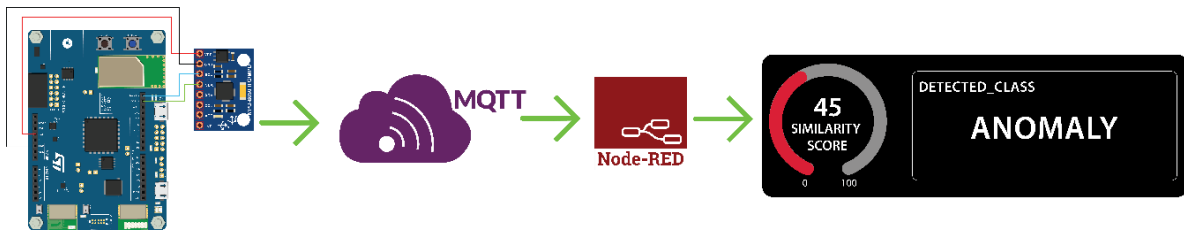
Lab Overview

In this lab, we will integrate IoT technology with an **anomaly detection application** that identifies abnormal vibration patterns from normal vibration patterns. The focus is on initializing **MQTT communication** to enable the STM32 board to send anomaly detection results to **Node-RED** for real-time monitoring and visualization. This lab builds upon the **Wi-Fi Initialization** and **Anomaly Detection Application** by adding cloud integration for remote data analysis and visualization.

Objective of the Lab

The objective of this lab is to integrate **IoT** with an **anomaly detection system** using **MQTT** to send **vibration classification results** from the **STM32 board** to **Node-RED** for **real-time monitoring and visualization**.

Experimental Setup



Pre-lab Requirements

Before proceeding with this lab, you must complete the following steps to ensure all prerequisites are in place for successful completion:

1. **Anomaly Detection Application Lab**
Ensure that you have successfully developed the anomaly detection application that distinguishes between normal and abnormal vibration patterns on the STM32 board. This provides the foundational classification data that will be sent to **Node-RED**.
2. **Wi-Fi Initialization Lab**
Complete the **Wi-Fi Initialization Lab** to establish Wi-Fi connectivity on your IoT device. This step ensures the device can connect to the network for data transfer.

Important Instruction:

For this lab, you should **not create separate projects** for the Anomaly Detection and Wi-Fi Initialization labs. Instead, integrate both functionalities into a single project. Start with the project from the Wi-Fi Initialization Lab and add the anomaly detection application code to it. Follow the steps in this manual to detect abnormal vibrations and send results to **Node-RED** via MQTT.

By the end of this lab, you should have a **single project** where:

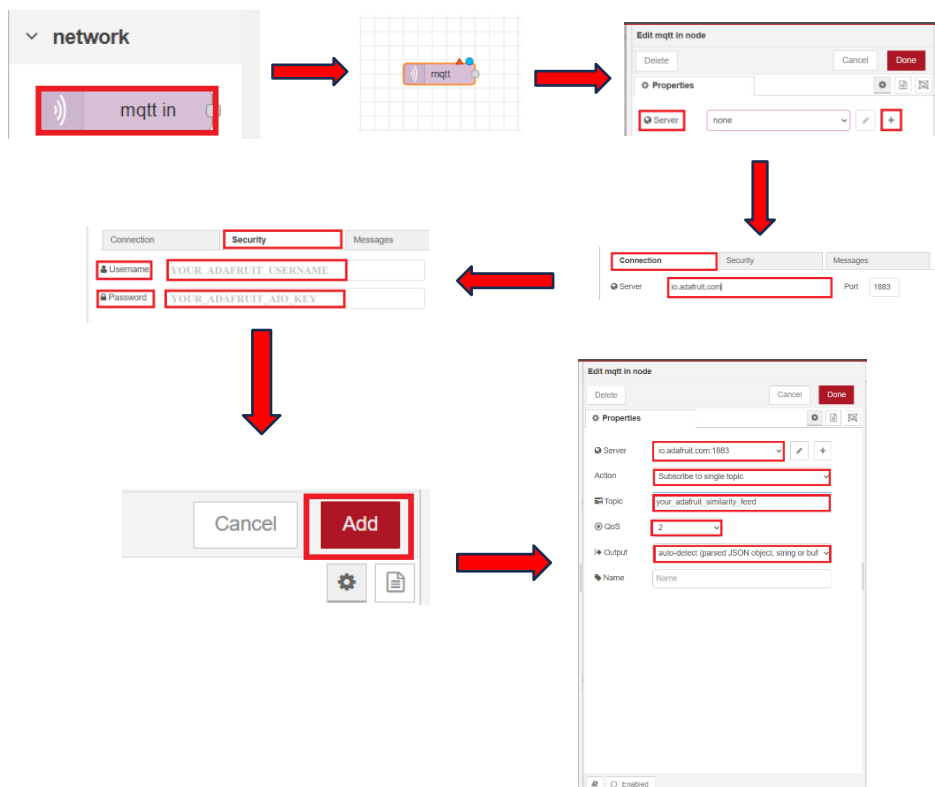
- The anomaly detection system identifies normal and abnormal vibration patterns.
- The **Wi-Fi module** is initialized and connects to the network.
- Results are transmitted to **Node-RED** via **MQTT** for real-time monitoring and visualization.

Steps Involved:

1. Add MQTT Node in Node-RED

- **Steps:**
 - a. Drag the **MQTT input** node into the Node-RED workspace.
 - b. Double-click on the MQTT node to open the configuration window.
 - c. Click on the **plus** icon near the **Server** field to add a new server.
 - d. In the new window, set the server to **io.adafruit.com** and the port number to **1883**. Leave other configuration settings as default.
 - e. Navigate to the **Security** tab. In the **Username** field, enter your **Adafruit username**, and in the **Password** field, provide your **AIO Key** from your Adafruit account. Then, click **Add**.
 - f. Ensure that the newly created server (io.adafruit.com) is selected in the **Server** dropdown.
 - g. In the **Action** dropdown, select **Subscribe to a single topic**. And in Topic field type the topic you get from Adafruit in the format **username/feeds/your_feed_name**.
 - h. Set the **QoS level** to **2** and select **Auto-detect** for the output format.
 - i. Enter a custom name for the MQTT node in the **Name** field and click **Done**.
 - **Purpose:** This step ensures that you start with a clean slate for integrating the new files provided for this lab.
- Note:** Similarly, repeat the same steps for subscribing to the anomaly topic.

- **Screenshot**



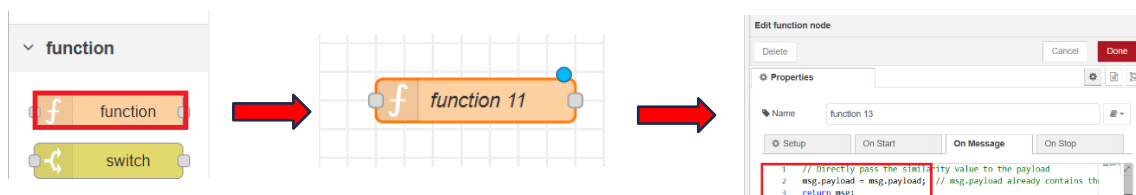
2. Add Function Node to Process Data Steps:

- Drag the **Function** node into the Node-RED workspace.
 - Double-click on the **Function** node to open the configuration window.
 - In the **Name** field, give a custom name for the function (e.g., "Parse Message").
- Navigate to the **On Message** tab and enter the following code:

```
javascript msg.payload = msg.payload; return msg;
```
 - Purpose:** This step prepares the **Function node** to process incoming data by allowing you to define custom logic. By setting up the **Function node** and assigning a name, you can organize the flow clearly. The code provided ensures the message payload remains unaltered for compatibility with subsequent nodes in the flow.

Note: Similarly, repeat the same steps for subscribing to the anomaly topic.

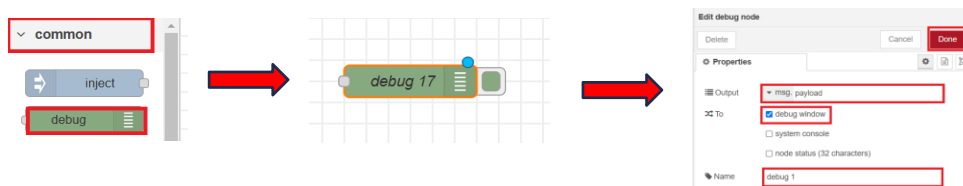
- **Screenshot:**



3. Add Debug Node to Verify Output

- **Steps:**
 - a. Drag the **Debug** node into the Node-RED workspace.
 - b. Double-click on the **Debug** node to open the configuration window.
 - c. In the **Output** field, select **msg.payload** to display the processed data in the debug window.
 - d. Make sure the **Debug Window** is selected to show the output in the debug pane.
 - e. In the **Name** field, you can provide a custom name for the debug node (e.g., "Display Temperature Data").
 - f. Click **Done** to save the configuration.
 - **Purpose:** This step allows you to verify that the incoming temperature data is being received and processed correctly. The data will be shown in the debug window, ensuring the MQTT messages are properly parsed and that the system is working as expected.
- Note:** Similarly, repeat the same steps for subscribing to the anomaly topic.

- **Screenshot:**

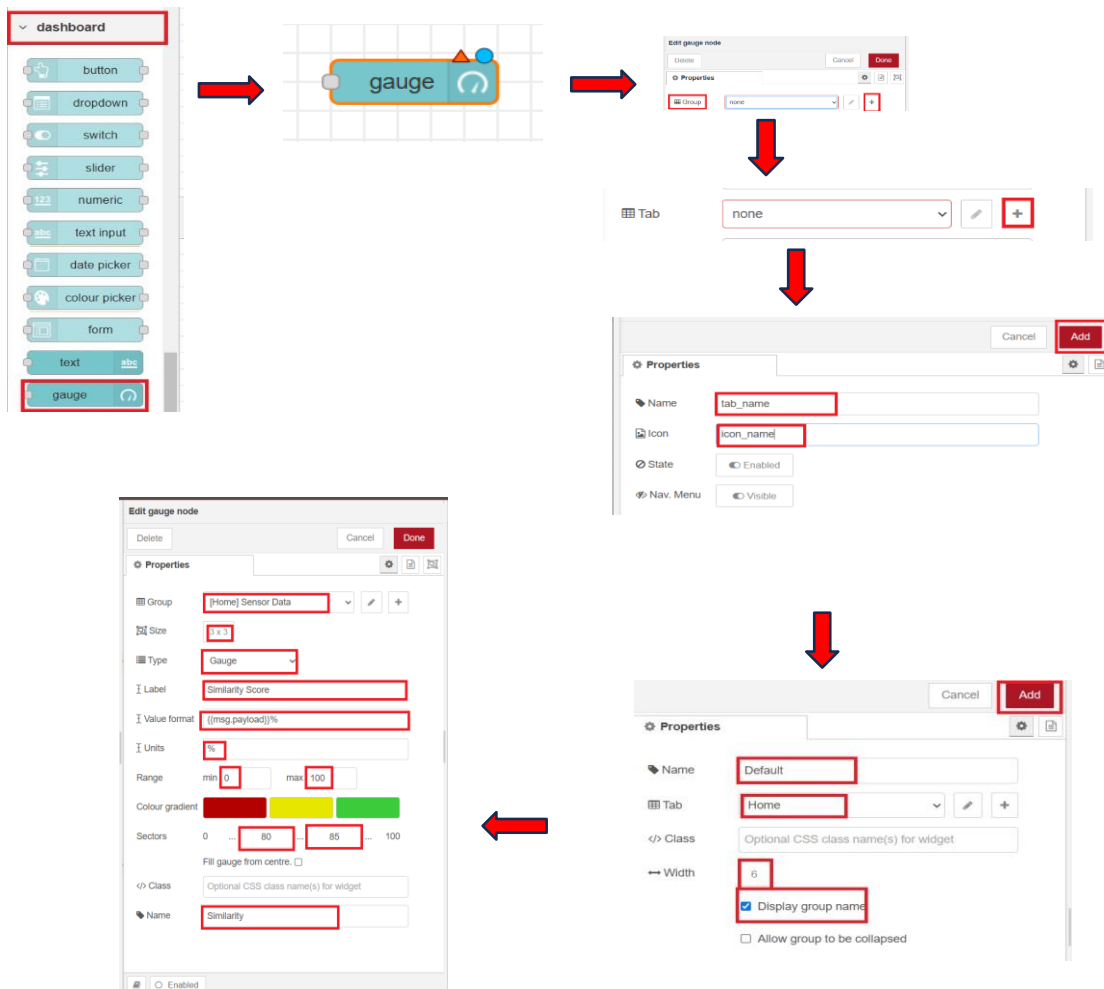


4. Add Gauge Node for Similarity Score Visualization Steps:

- a. Before proceeding, ensure you have downloaded the **Dashboard Package** mentioned in the Node-RED installation manual.
- b. Drag the **Gauge** node from the Dashboard section into the Node-RED workspace.
- c. Double-click the **Gauge** node to open the configuration window. In the **Group** field, click the + icon to create a new group.
- d. In the **Group Name** field, enter a name (e.g., "Anomaly Data Group"). In the **Tab** field, click the + icon to create a new tab. Add a name and an icon for your tab and click **Add**.
- e. After being redirected to the group configuration page, click **Add** and then **Next**. Ensure that the group you created is selected in the **Group** option of the gauge configuration
- f. In the **Size** section, select the desired gauge size (e.g., **4x4**). In the **Label** field, enter "Similarity_Score". In the **Value Format** field, enter `{{msg.payload}}%` to display the temperature in Celsius. In the **Unit** field, enter "Percentage".
- g. In the **Range** section, define the minimum and maximum values for your gauge (e.g., 0% to 100%)

- **Purpose:** This step configures a **gauge node** in the Node-RED dashboard to visualize the Similarity_Score data in real-time, with customizable range and warning highlights.

- **Screenshot:**



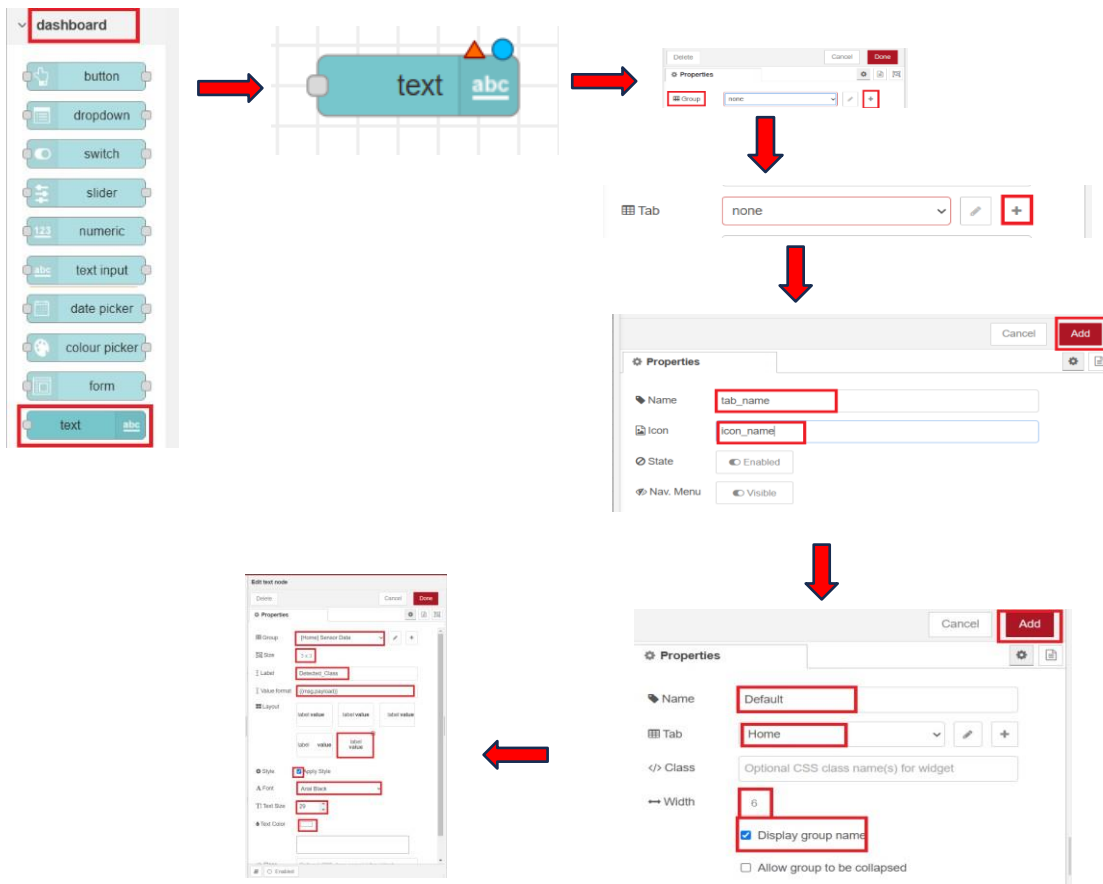
5. Add Text Node for Similarity Score Visualization Steps:

- Before proceeding, ensure you have downloaded the **Dashboard Package** mentioned in the Node-RED installation manual.
- Drag the **Text node** from the **Dashboard** section into the Node-RED workspace.
- Double-click the **Text node** to open the configuration window.
- In the **Group** field, select the **same group** used for the Similarity Score gauge (e.g., "Anomaly Data Group").
- In the **Label** field, enter "Detected Class" to display the detected vibration pattern. In the **Value Format** field, enter `{{msg.payload}}` to display the incoming classification result.
- Configure the **Size** section to select an appropriate size for the text display (e.g., 4x2).

Purpose

This step configures a **Text node** in the Node-RED dashboard to display the **Detected Class** in real time. By using the same group as the **Similarity Score**, the dashboard remains organized and allows for cohesive visualization of anomaly detection results

Screenshot:

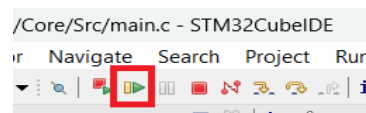
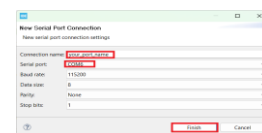
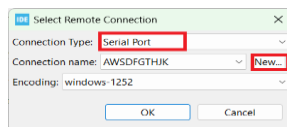
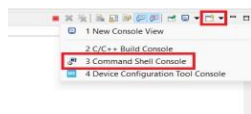
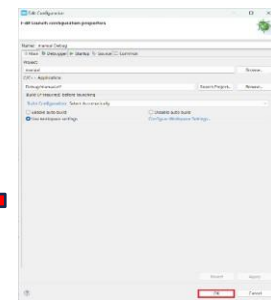
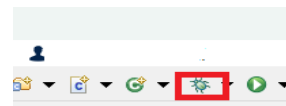
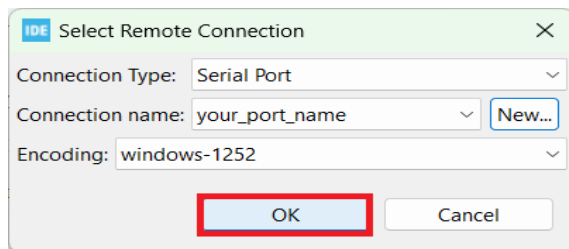


5. Run the Project in STM32CubeIDE Steps:

- Open the **Anomaly_Detection Integrated with IoT Project** in STM32CubeIDE.
- Ensure your Wi-Fi hotspot is turned on in your mobile device. Then, make sure that the SSID and password in your device configuration in main.c are set to match the credentials of the Wi-Fi network you want the device to connect to.
- Ensure that all MQTT configurations in main.c are correctly modified for your Adafruit account, and debug the code to verify no errors are present.
- Run the project by selecting Debug and then clicking Run Debug. Once it shows "Download Verified Successfully," proceed to create a serial monitor to view the data by selecting Command Shell Console.
- When you select Command Shell Console, choose Serial Port as the connection type. Click on New, provide an appropriate name for the console, and select the correct port. You can check the port

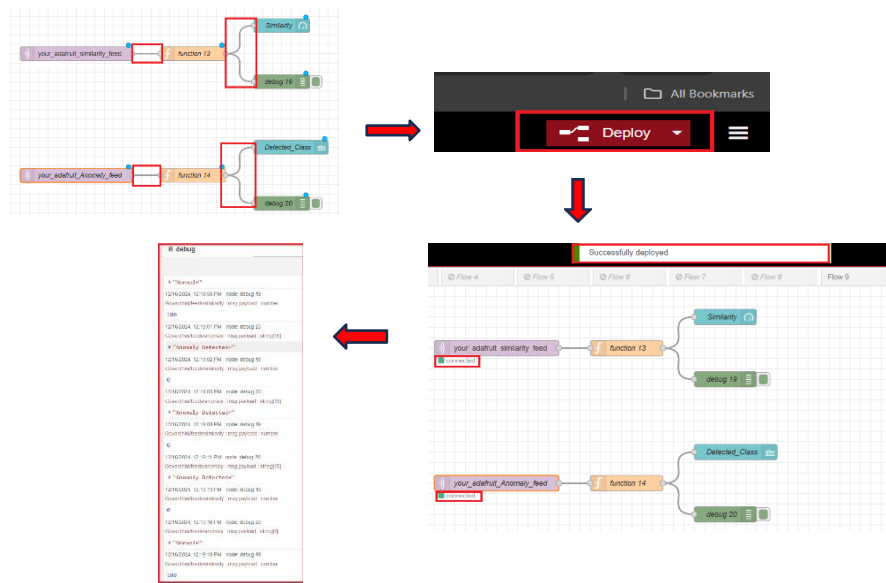
associated with ST-Link by going to Device Manager, expanding the Ports section, and identifying the port connected to ST-Link.

- f. Click Finish, then click OK to open the serial monitor. Next, in the navigation bar, select the Resume option. After resuming, the output will be displayed in the serial console. Ensure that the hotspot is turned on for the device with the configuration you provided in the previous step.
- **Purpose:** This step initializes the STM32 board, enabling it to collect sensor data from the MPU6050 sensor and send it to the **Adafruit MQTT broker**. Running the project ensures that the data from the sensor is transmitted to the cloud via MQTT, allowing Node-RED to retrieve and visualize the data.
- **Screenshot:**



6. Deploy and Verify Data in Node-RED Steps:

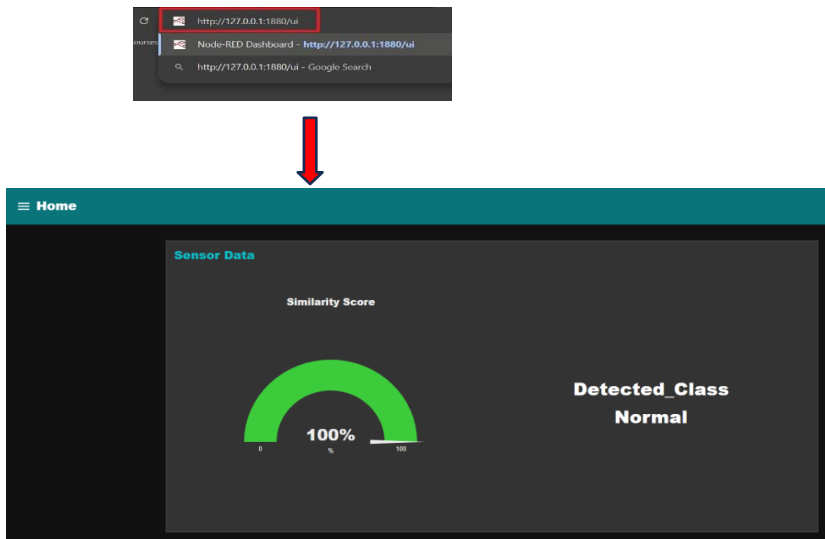
- Open **Node-RED** and ensure that the MQTT nodes are correctly configured to receive data. Before proceeding, verify that Adafruit feeds are receiving data.
 - Once verified, click **Deploy** in Node-RED to deploy the flow.
 - Navigate to the **Debug message node**. Once deployed, you will see the status near the MQTT-in node, showing text like "Connected."
 - In the **Debug panel**, you should now see the **Similarity_Score** and **Detected_Class** data being received and displayed.
- Purpose:** This step ensures that Node-RED is properly receiving sensor data from the MQTT broker and displaying it in the Debug panel, confirming that the data flow is working as expected.
 - Screenshot:**



7. Access the Dashboard in Node-RED Steps:

- After deploying the flow, open a new browser tab.
 - Enter the **local IP** address obtained from the Node-RED environment (e.g., 127.0.0.1:1880).
 - Append /ui to the IP address (e.g., 127.0.0.1:1880/ui) to access the Node-RED dashboard.
 - You will be able to see the **temperature** and **humidity** data displayed in the dashboard created using the **gauge nodes**.
- Purpose:** This step allows you to visualize the real-time sensor data on a web-based dashboard, providing an interactive display for monitoring temperature and humidity.

- **Screenshot:**



Final Observation:

- **Wi-Fi Initialization:** The **Wi-Fi module** was successfully **initialized** and connected to the network, enabling internet communication.
- **MQTT Client Initialization:** The **MQTT client** was initialized and configured with the **publish topics** Govardhini/feeds/anomaly and Govardhini/feeds/similarity for real-time data transfer.
- **Training and Detection:** The **training cycles** were successfully completed, and the **anomaly detection system** distinguished between "**Normal**" and abnormal vibration patterns.
- **Similarity Score:** The system calculated and published the **Similarity Score** as **100%**, indicating the vibration pattern matched normal behavior.
- **Real-Time MQTT Data Transmission:** The **Similarity Score** and **Detected Class** results (e.g., "Normal") were **successfully published** to the specified **MQTT topics**.
- **Node-RED Dashboard Visualization:** The **Similarity Score** was displayed on a **gauge node**, and the **Detected Class** was shown using a **text node** on the **Node-RED dashboard** in real time.
- **System Validation:** The entire system, including **Wi-Fi connectivity**, **MQTT communication**, and **data visualization** on Node-RED, functioned seamlessly for real-time anomaly detection.